Тех інформація, вона не фінальна і не остаточна, проаналізуй і продумай як краще

Це від Gemini AI:
Чат в режиме ИИ: який ШІ зараз краще всього для розробки ігор? гра по типу https://store.steampowered.com/app/413150/Stardew_Valley/?curator_clanid=41316928 веб версію треба зробити
Вы отправили 1 вкладка и сделали запрос "який ШІ зараз краще всього для розробки ігор? гра по типу https://store.steampowered.com/app/413150/Stardew_Valley/?curator_clanid=41316928 веб версію треба зробити"

> Stardew Valley on Steam
> який ШІ зараз краще всього для розробки ігор? гра по типу https://store.steampowered.com/app/413150/Stardew_Valley/?curator_clanid=41316928 веб версію треба зробити

Для розробки веб-версії гри типу Stardew Valley (2D тайтл із великою кількістю взаємопов'язаних систем: фермерство, інвентар, збереження, ШІ персонажів) найкраще використовувати комбінацію Claude 3.5 Sonnet (для написання коду та архітектури) та Midjourney / Recraft.ai (для генерації піксель-арту).

Оскільки ви плануєте саме веб-версію, гра розроблятиметься на JavaScript/TypeScript. У цій сфері мовні моделі (LLM) демонструють найкращі результати, оскільки мають величезну базу навчальних даних по веб-технологіях.

Нижче наведено детальний розбір найкращих ШІ-інструментів для кожного етапу створення вашої гри.

---

1. Написання коду та логіки гри (Найважливіше)

Створення симулятора життя вимагає складної логіки (генерація карти, зміна дня і нічі, сітка для посадки рослин). З цим найкраще впораються такі нейромережі:

- Claude 3.5 Sonnet (через веб-інтерфейс або в IDE Cursor):
    - Чому він: На сьогодні це абсолютний лідер у кодингу. Він чудово розуміє логіку ігор, алгоритми пошуку шляху (A*) для персонажів та вміє створювати складні структури даних для інвентарю чи збережень.
    - Перевага: Функція *Artifacts* у веб-версії Claude дозволяє запускати і тестувати невеликі шматки коду (наприклад, прототип механіки збору врожаю) прямо в браузері всередині чату.

- Редактор коду Cursor (на базі Claude/GPT-4o):
    - Це модифікований VS Code, у який глибоко інтегровано ШІ. Він може бачити весь ваш проект, автоматично виправляти помилки у кількох файлах одночасно та генерувати код на основі контексту всієї гри.
- ChatGPT (GPT-4o):
    - Гарний варіант для високорівневого планування, написання сценаріїв, діалогів для NPC та базових алгоритмів, але в чистому програмуванні він зараз трохи поступається Claude.

2. Генерація графіки (Pixel Art для Stardew Valley)

Stardew Valley відома своїм затишним піксельним стилем. ШІ вже непогано справляється з тайлсетами та спрайтами:

- Midjourney (v6):
    - Генерує неймовірно красивий та деталізований піксель-арт. За допомогою правильних промптів (наприклад: `pixel art tileset, top-down view, green grass, soil, RPG maker style --v 6.0`) можна отримати чудові заготовки для текстур землі, будинків та дерев.
- Recraft.ai:
    - Унікальний інструмент, який має окремий режим "Pixel art". Величезний плюс — він вміє генерувати зображення на прозорому тлі (без заднього плану), що критично важливо для ігрових об'єктів (іконки предметів, персонажі, інструменти).

3. Звуки та музика

Музика в Stardew Valley створює половину атмосфери. Для веб-гри можна використовувати:

- Suno AI або Udio: Для створення фонової розслаблюючої музики (Lo-Fi, оркестрова або акустична акустика в стилі кантрі).
- ElevenLabs: Якщо ви захочете додати озвучку або специфічні звукові ефекти (звук інструментів, вигуки персонажів).

---

З чого почати? Технологічний стек для веб-версії

Щоб ШІ допомагав вам ефективно, вам потрібно обрати правильний "рушій" або бібліотеку для браузера. Запитайте у Claude 3.5 Sonnet або Cursor зразок коду для однієї з цих технологій:

| Технологія / Бібліотека | Для чого підходить | Переваги для роботи з ШІ |
|---|---|---|
| Phaser 3 (JavaScript) | Найпопулярніший 2D рушій для браузера. | ШІ знає його ідеально. Легко робити сітку, анімацію спрайтів та фізику. |
| React + Canvas / PixiJS | Для чистішої структури інтерфейсів (меню, інвентар). | Зручно керувати станом гри (state), ШІ легко пише UI. |
| Kaboom.js | Спрощена бібліотека для швидких прототипів. | Код виходить коротким, ідеально для генерації перших механік за один промпт. |

План дій для старту з ШІ:

1. Завантажте редактор Cursor.
2. Відкрийте чат з Claude 3.5 Sonnet і напишіть: *"Я хочу створити веб-гру типу Stardew Valley на Phaser 3. Допоможи мені створити структуру проекту та базове вікно гри, де персонаж може ходити по траві за допомогою стрілочок"*.
3. Покроково нарощуйте функціонал (додайте сітку, потім саджання насіння, потім збереження). Не просіть ШІ зробити "всю гру в один клік" — розбивайте завдання на мікро-кроки.

Щоб я міг дати точніші рекомендації, підкажіть:

- Чи є у вас досвід у програмуванні (наприклад, знання JavaScript/HTML), чи ви хочете, щоб ШІ писав 100% коду "з нуля"?
- Який приблизно масштаб проекту планується (простий клікер-ферма для портфоліо чи повноцінна RPG з квестами та великим містом)?

В ответах искусственного интеллекта могут быть ошибки. Подробнее…

розпиши стек і всі тулзи максимально детально

Для створення вебверсії гри типу Stardew Valley вам знадобиться стек технологій, який забезпечує високу продуктивність (оскільки рендеринг сотень плиток землі, рослин та NPC вимагає ресурсів) та хорошу роботу з базою даних для збереження прогресу.

Нижче наведено максимально детальний розбір технологічного стека, архітектури та ШІ-інструментів, оптимізованих для такої розробки.

---

## 🛠️ 1. Технологічний стек (Frontend & Backend)

Оскільки гра працюватиме в браузері, оптимально використовувати комбінацію сучасного JavaScript/TypeScript-екосистеми.

### Frontend (Клієнтська частина гри)

- Мова програмування: TypeScript. Обов'язково для гри такого масштабу. У Stardew Valley є тисячі предметів, стадій росту рослин, типів риби та характеристик NPC. Строга типізація TypeScript вбереже вас від багів, коли «капуста» раптово отримає властивості «кирки». ШІ (особливо Claude) пише код на TypeScript навіть краще і чистіше, ніж на звичайному JS.
- Ігровий рушій: Phaser 3. Це золотий стандарт для HTML5 2D-ігор.
    - *Чому він:* Має вбудовану підтримку фізики (Arcade Physics), роботу з тайлсетами (картами з плиток), систему анімацій спрайтів, керування камерою та звуком. ШІ знає Phaser 3 "від і до".
    - *Альтернатива:* PixiJS (якщо хочете писати власну логіку гри, але потрібен надшвидкий рендеринг) або Babylon.js / Three.js (якщо захочете зробити 2.5D або псевдо-3D, як у *Graveyard Keeper*).
- Інтерфейс користувача (UI): React.js або Vue.js.
    - *Важливий лайфхак:* Не малюйте інвентар, меню налаштувань, крафт та діалоги всередині ігрового рушія Phaser — це довго і складно. Зробіть ігрове поле на Phaser, а поверх нього накладіть шар React для красивих HTML/CSS меню, інвентарю та магазинів.
- Збірка проекту: Vite. Найшвидший інструмент для збірки веб-додатків. Гра буде перезавантажуватися за мілісекунди під час внесення змін ШІ.

Backend & База даних (Серверна частина)

*Якщо гра сингплплеєрна, на старті можна обійтися без бекенду, зберігаючи все в браузері через `LocalStorage` або `IndexedDB`. Але якщо потрібна хмарна синхронізація, автоавторизація чи мультиплеєр:*

- Платформа: Node.js (NestJS або Express) + TypeScript.
- База даних: PostgreSQL або MongoDB. Для зберігання профілю гравця, його координат, стану кожної грядки на фермі (координати, тип рослини, стадія росту, чи полита) та вмісту скринь.
- Готове рішення (BaaS): Supabase або Firebase. Ідеально для розробки з ШІ. Вони дають готову базу даних, аутентифікацію користувачів та збереження файлів "із коробки", що економить місяці розробки.

---

🤖 2. ШІ-інструменти для розробки (AI Stack)

Щоб ефективно розробляти, вам знадобиться декілька різних ШІ, кожен з яких виконує свою роль.

Написання коду та архітектура

1. Cursor (IDE) + Claude 3.5 Sonnet:
    - *Як використовувати:* Замість звичайного VS Code встановіть Cursor. Завантажте туди свій проект. За допомогою функції `@Files` або `@Folder` ви можете змусити ШІ аналізувати весь код гри.
    - *Приклад завдання:* «Подивись на мій файл `Player.ts` та `Inventory.ts`. Напиши функцію, щоб при натисканні на грядку предмет з інвентарю (насіння) зникав, а на карті з'являвся об'єкт рослини».
2. v0.dev (від Vercel):
    - *Як використовувати:* Це ШІ для генерації UI. Використовуйте його, щоб згенерувати стильний інтерфейс інвентарю, вікна торговця П'єра чи меню налаштувань у стилі Pixel Art за допомогою Tailwind CSS, а потім просто скопіюйте код у свій React-додаток.

Графіка, Анімація та Спрайти (Pixel Art)

Генерація правильного піксель-арту для ігор — це найскладніша частина, бо графіка має бути розбита на однакові квадрати (зазвичай 16x16 або 32x32 пікселі).

1. Recraft.ai (або його API):
    - Найкращий ШІ для генерації ігрових ассетів на сьогодні. Оберіть режим Pixel Art. Ви можете генерувати об'єкти (наприклад, "іконка стиглого томату", "заіржавіла лійка") відразу на прозорому фоні (Alpha channel), що дозволяє відразу вставляти їх у гру.
2. Midjourney v6:
    - Ідеально для генерації великих фонів, екранів завантаження, портретів персонажів під час діалогів (як у Stardew Valley, коли з'являється обличчя NPC поруч із текстом).
3. ШІ-інструменти редагування (Adobe Firefly / Photoshop AI):
    - Потрібні для того, щоб "нарізати" згенеровані картинки на тайлсети (плитки) або виправляти артефакти нейромереж (ШІ іноді змазує пікселі).

Звук, Ефекти (SFX) та Музика

1. Suno 4.0 / Udio:
    - Для створення безкінечних затишних інструментальних треків (Country, Folk, Acoustic Guitar, Lo-Fi) для різних пір року в грі.
2. ElevenLabs (Sound Effects):
    - Має круту модель для генерації звуків за текстовим описом. Можна згенерувати: "звук удару сокири по дереву в 8-бітному стилі", "звук виливання води з лійки", "кроки по траві".

---

🛠️ 3. Допоміжні утиліти та інструменти розробника

Окрім коду та графіки, вам знадобляться специфічні ігрові утиліти, з якими ШІ також вміє взаємодіяти:

1. Tiled Map Editor (Програма для створення карт):
    - Ви не будете розставляти кожне дерево кодом. Ви завантажуєте свої піксельні текстури в програму Tiled, малюєте там карту ферми та міста мишкою, а програма видає вам файл `.json`. Цей `.json` файл ви згодом завантажуєте в Phaser 3. *ШІ чудово вміє писати парсери для JSON-карт із Tiled.*
2. Aseprite або Pixelorama:
    - Найкращі софти для редагування піксель-арту та створення покадрової анімації (як персонаж махає вудкою чи йде). Сюди ви будете завантажувати графіку від ШІ для фінальної обробки.
3. Git + GitHub:
    - Обов'язково для контролю версій. ШІ іноді може зламати проект або написати код, який заплутає архітектуру. Можливість "відкотитися" до робочої версії збереже вам нерви.

---

🚀 Покроковий план побудови архітектури через ШІ

Щоб проект не перетворився на хаос, згодуйте цей план вашому ШІ (Claude/Cursor) і рухайтеся строго по ньому:

1. Фаза 1: Прототип Grid-системи (Сітки). Навчіть Phaser ділити екран на клітинки 32x32. Напишіть логіку, яка визначає, на яку клітинку клікнув гравець.
2. Фаза 2: База даних рослин. Створіть JSON-структуру (Data Structure) для рослин. Наприклад: `id: "potato", growthStages: 4, timeToGrow: 4 days, sprite: "potato_sheet"`.
3. Фаза 3: Система часу (Time Manager). Створіть ігровий годинник. 1 секунда реального часу = 1 ігрова хвилина. Кожні 24 ігрові години — настає новий день. При настанні нового дня запускається функція перевірки грядок: "якщо грядка була полита -> стадія росту рослини +1".
4. Фаза 4: Інвентар. Зв'яжіть Phaser з React. Напишіть систему підняття предметів із землі та відображення їх у графічних слотах інвентарю.

Цей стек є максимально гнучким, безкоштовним для старту та має найбільшу підтримку з боку штучного інтелекту, що дозволить вам створити гру навіть самотужки.

В ответах искусственного интеллекта могут быть ошибки. Подробнее…

> • Чи є у вас досвід у програмуванні (наприклад, знання JavaScript/HTML), чи ви хочете, щоб ШІ писав 100% коду "з нуля"? - ні я не маю досвіду, все має робити ШІ • Який приблизно масштаб проекту планується (простий клікер-ферма для портфоліо чи повноцінна RPG з квестами та великим містом)? - РПГ з квестами

Створення повноцінної RPG з квестами типу Stardew Valley на 100% за допомогою ШІ без досвіду програмування — це амбітне, але вже цілком реальне завдання.

Головний секрет тут — не просити ШІ написати "всю гру відразу", бо він заплутається і видасть неробочий код. Ви будете виступати в ролі Головного Архітектора та Режисера, який керує Ші-програмістом, видаючи йому мікро-завдання.

Оскільки ви не знаєте коду, ми повністю міняємо підхід до розробки. Забудьте про складне налаштування серверів. Ваш стек має бути максимально візуальним та простим для запуску.

---

🚀 Стек для розробки "Без досвіду" (Zero-Code за допомогою ШІ)

Щоб ви могли бачити гру та запускати її в один клік, замість складних професійних фреймворків найкраще використати один із двох шляхів.

Варіант А: Стек "Cursor + HTML/JS" (Найкращий для веб-версії)

Ви не будете встановлювати бази даних чи складні сервери. Вся гра буде працювати у звичайному браузері (як Chrome), а ШІ писатиме код у спеціальному редакторі Cursor.

- Движок: HTML5 Canvas + чистий JavaScript (або проста бібліотека Kaboom.js). ШІ дуже легко пише для них код без багів.

- Збереження гри: `LocalStorage` (вбудована пам'ять браузера). ШІ зробить так, що гра автоматично зберігатиметься на комп'ютері гравця, коли він лягає спати в грі.
- Перевага: Ви просто відкриваєте файл `index.html` у браузері подвійним кліком і граєте.

Варіант Б: RPG Maker MV / MZ (Альтернатива, якщо веб-версія може бути в форматі HTML5)

Це спеціальна програма для створення 2D RPG (саме на ній починалися багато відомих ігор). Вона з самого початку має готову систему квестів, діалогів, пересування та інвентарю, а також експортує ігри у веб-версію (HTML5).

- Роль ШІ тут: Ви просите Claude писати спеціальні скрипти (плагіни) на JavaScript для RPG Maker, щоб додати туди механіку фермерства (посадку насіння, полив).

---

🤖 Як саме ви будете працювати з ШІ (Покрокова інструкція)

Крок 1: Підготовка робочого місця

1. Завантажте та встановіть редактор коду Cursor (це безкоштовно).
2. Створіть на комп'ютері порожню папку `MyStardewValley`. Відкрийте її через Cursor.
3. Відкрийте вбудований ШІ-чат у Cursor (комбінація клавіш `Ctrl + K` або `Ctrl + L`).

Крок 2: Перший промпт (Створення основи)

Напишіть ШІ перший запит. Скопіюйте цей текст:

> *"Я не вмію програмувати. Я хочу створити веб-гру типу Stardew Valley на чистому JavaScript та HTML5 Canvas. Створи для мене*

> *базову структуру проекту: файл index.html, style.css та script.js. Зробіть так, щоб у центрі екрана був зелений квадрат (гравець), яким я можу керувати кнопками WASD або стрілочками на клавіатурі. Напиши весь код повністю, без скорочень, і поясни, як мені його запустити."*

ШІ згенерує код і сам запропонує кнопку `Apply` (Застосувати), яка розставить код по файлах. Ви двічі клікнете на `index.html` і побачите свою першу мікро-гру в браузері.

Крок 3: Покрокове нарощування (Метод "Снігової кулі")

Ніколи не кажіть: "Додай фермерство і квести". Додавайте по одній деталі за раз.

- Запит 2: *"Тепер зроби так, щоб гравець не міг виходити за межі екрану. І додай на фон сітку з клітинок 32x32 пікселі."*
- Запит 3: *"Додай логіку: якщо я натискаю клавішу `E`, на тій клітинці, де стоїть гравець, земля змінює колір на коричневий (зорана грядка)."*
- Запит 4: *"Створи систему часу. Нехай у кутку екрану йде годинник. 1 хвилина в грі = 1 секунда в житті. Коли настає 24:00, день змінюється на наступний."*

---

🗺️ Як реалізувати RPG-елементи та Квести через ШІ

Оскільки ви хочете повноцінну RPG з містом та квестами, архітектуру системи квестів потрібно закласти правильно. Ші чудово вміє створювати так звані "Движки діалогів та квестів" через текстові формати (JSON).

Ви попросите Claude: *"Створи систему квестів для моєї гри. Квести мають зберігатися у вигляді тексту, щоб я міг сам їх писати"*. Ші зробить для вас систему, де ви зможете додавати квести простими словами:

json
```
{
```

```
  "quest_id": "first_meet",
  "title": "Знайомство з мером",
  "description": "Принеси меру 5 штук томатів",
  "reward_gold": 100,
  "is_completed": false
}
```

Используйте код с осторожностью.

Вам не доведеться програмувати логіку квесту. ШІ напише код, який зчитує цей текст, а ви будете просто вигадувати сюжети квестів, імена персонажів та діалоги українською мовою.

---

🎨 Що робити з графікою та звуком?

Оскільки ви не програміст і не художник, вашим найкращим другом стане сайт Itch.io.

1. Зайдіть на Itch.io у розділ "Free Game Assets" -> "Pixel Art".
2. Там є тисячі безкоштовних готових наборів (тайлсетів) у стилі Stardew Valley: готові дерева, будинки, персонажі з анімацією ходьби, іконки овочів.
3. Завантажте їх, закиньте в папку з грою і покажіть ШІ: *"Я завантажив картинку персонажа `player.png`. Перепиши код так, щоб замість зеленого квадрата по екрану ходила ця картинка"*.

Якщо якогось унікального предмета чи квестового персонажа не вистачатиме — генеруйте його через Recraft.ai (в режимі Pixel Art) з прозорим фоном.

Для старту нашої розробки пропоную зробити перший крок прямо зараз. Який варіант вам ближче:

1. Почати створювати чистий веб-проект через Cursor (код з нуля, повна свобода дій, гра запускається в браузері)?
2. Спробувати розібратися в програмі RPG Maker (простіший візуальний конструктор, де ШІ буде лише допомагати плагінами)?

Оберіть варіант, і я напишу для вас перший детальний промпт, який ви вставите в ШІ для запуску гри!

В ответах искусственного интеллекта могут быть ошибки. Подробнее…

> допоможи мені із технічним стеком і базою, архітектурою і розумінням ШІ що від нього треба. Я маю наброски ідеї і структури сценарію, але сценарій просто болванка-заготовка і мені потрібно все з нуля по факту. В мене Cloude вже є і він кодить мені сам сайт але на сайт я хочу додати гру і ось тут мені треба щоб ти продумав всі тулзи, архітектурні рішення, модернові підходи і технології. Все треба спочатку це буде згодувати Клоду щоб він знав і міг зрозуміти що від нього треба і міг розуміти команди і сам розписав гру в Джирі, я дав в Джиру вже доступ і він пише таски саам і сам іх робить, тестує, публікує. Сайт йде автономно і автоматизовано і як конвеєр побудований в мене там ворк флоу то я хочу щоб і гра так само йшла автономно і автоматизовано. В мене апп клод на макбук і я не можу в нього додати новий акаунт бо при перемиканні сессія вимикає роботу іншого чату то він каже поставити веб в хром версію, я не проти, але може ти правий що хкод може вирішити цю проблему чи то правда?

Це чудова та зріла автоматизація! Оскільки у вас вже побудований CI/CD-конвеєр (автономний воркфлоу), де Claude сам створює таски в Jira, пише код, тестує та деплоїть його на сайт, додавання гри має відбуватися за тими ж «рейками».

Нижче наведено повний технічний документ (Бріф-Архітектуру), який ви скопіюєте та згодуєте вашому Claude (у новий або поточний сесійний чат), щоб він зафіксував контекст, розбив гру на епіки/таски в Jira та почав автономну роботу.

Спочатку закриємо технічне питання щодо додатку та Xcode, а потім перейдемо до архітектури гри.

---

## 🔥 Питання про додаток Claude, Chrome та Xcode

Ваш Claude правий: десктопний додаток Claude на macOS має обмеження ізоляції сесій (при перемиканні акаунтів або довгих процесах потоки можуть заважати один одному або скидати токени).

- Чи врятує Xcode? Ні, Xcode тут ні до чого. Xcode — це середовище для розробки додатків під Apple (iOS/macOS). Він не вирішує проблему мультиакаунтів чи утримання сесій вебсайтів.
- Найкраще рішення: Використовувати Google Chrome з різними профілями користувачів. Створіть у Chrome окремий профіль для "Сайту" і окремий для "Гри". Кожен профіль Chrome — це повністю ізольований всесвіт (свої кукі, сесії, токени). Вони працюватимуть паралельно, автономно і ніколи не скинуть сесії один одного.

---

## 📜 АРХІТЕКТУРНИЙ БРІФ ДЛЯ CLAUDE (Скопіюйте текст нижче)

Скопіюйте весь текст нижче та відправте його вашому Claude, щоб запустити конвеєр розробки гри:

text

```
Контекст проекту: Розробка 2D RPG гри (тип Stardew Valley) як
вбудованого модуля для існуючого автономного сайту.
Твоя роль: Lead Game Developer / QA / DevOps. Ти працюєш
автономно через наш CI/CD конвеєр та інтеграцію з Jira.
Твоє завдання зараз: Проаналізувати цей технічний бріф,
розбити його на Епіки та Таски в Jira, узгодити архітектуру і
почати автономний цикл розробки, тестування та публікації.

1. МОДЕРНОВИЙ ТЕХНІЧНИЙ СТЕК (FRONTEND-ONLY ARCHITECTURE)
Оскільки розробка має бути максимально стабільною для
ШІ-генерації та інтегруватися в існуючий веб-сайт, ми
використовуємо моно-серверний/клієнтський підхід без складного
бекенду на старті.

- Движок: Phaser 3 (HTML5 Canvas / WebGL). Обов'язково
TypeScript версія для уникнення логічних помилок типізації при
масштабуванні RPG-систем.
```

- UI Шар (Інвентар, Діалоги, Квести, Магазини): React.js (або Vue.js, залежно від поточного стеку сайту) поверх Canvas-ігрового поля. Взаємодія між Phaser та React через Event Bus (Phaser.Events.EventEmitter).
- Побудова карти: Tiled Map Editor сумісний JSON формат. Карти генеруються або зчитуються як статичні ассети.
- Стан гри (State Management): Реактивне сховище (Zustand або Redux Toolkit), яке синхронізує стан інвентарю/квестів між движком Phaser та UI на React.
- Збереження прогресу: IndexedDB (через бібліотеку LocalForage) для локального автозбереження великих обсягів даних (стан кожної грядки, координати, світ). Хмарний бекенд підключається через готовий API/Firebase/Supabase ТІЛЬКИ для синхронізації сейвів.

2. МОДУЛЬНА АРХІТЕКТУРА ГРИ (SCALABLE & AI-FRIENDLY)
Гра повинна мати архітектуру "Data-Driven" (керування через дані). Код движка не повинен змінюватися, коли ми додаємо новий квест або рослину. Все керується через JSON-конфіги, які ти (Claude) зможеш легко редагувати та розширювати.

Структура модулів:
- TimeManager: Ігровий годинник (хвилини, години, дні, сезони). Подія "Новий день" тригерить оновлення всього світу (ріст рослин, спавн ресурсів).
- GridSystem (Сітка): Світ розбитий на клітинки (наприклад, 32x32 пікселі). Кожна клітинка має свій стан: зорана, полита, зайнята об'єктом.
- CropEngine: Розрахунок стадій росту рослин на основі даних з JSON-довідника рослин.
- QuestEngine: Движок квестів, що працює на кінцевих автоматах (Finite State Machine). Квест має стани: Locked -> Available -> Active -> ReadyToTurnIn -> Completed.
- DialogueEngine: Система діалогів з підтримкою віток вибору, яка може зчитувати ID квестів для активації умов (якщо квест активний -> показати діалог Б, інакше -> діалог А).

3. АВТОНОМНИЙ ВОРКФЛОУ ДЛЯ ТЕБЕ (CLAUDE) В JIRA
Ти маєш діяти за наступним алгоритмом:
Крок 1: Створи в Jira Епіки на основі архітектури (наприклад: [Epic] Core Game Engine, [Epic] Farming & Grid Mechanics, [Epic] Quest & Dialogue System).
Крок 2: Наповни Епіки атомарними тасками (Task). Кожна таска має містити чіткий критерій приймання (Acceptance Criteria).

```
Крок 3: Бери таску в роботу (In Progress), пиши ізольований
код, покривай його базовими Unit-тестами (Jest), тестуй
рендеринг у Phaser.
Крок 4: Роби деплой через наш конвеєр, перевіряй
працездатність білду, закривай таску (Done) і переходь до
наступної.

4. СТРУКТУРА СЦЕНАРІЮ ТА БОЛВАНКИ
Сценарій та квести мають імпортуватися через окремі файли
конфігурації (наприклад, quests.json, npc_dialogues.json). Я
надам тобі драфт сценарію, твое завдання — розпарсити його на
об'єкти квестів та діалогів, які зрозуміє твоя QuestSystem.

Підтвердь, що ти повністю зрозумів технічний стек (Phaser 3 +
TypeScript + React/Zustand + IndexedDB), архітектурні вимоги
Data-Driven та воркфлоу в Jira. Якщо все ок, створи першу
пачку тасок в Jira для розгортання архітектурного каркасу гри
(Boilerplate) та відпишись про готовність.
```

Используйте код с осторожностью.

---

Що відбудеться далі?

Коли ви відправите це вашому Claude:

1. Він зрозуміє архітектуру (поєднання Phaser 3 для графіки та руху + React для меню та квестів). Це найкращий сучасний підхід для веб-ігор, оскільки ШІ ідеально знає React для інтерфейсів.
2. Він не буде писати "кашу" в одному файлі, а створить чітку структуру (окремо годинник, окремо грядки, окремо квести).
3. Він самостійно піде в Jira, створить структуру тасок і почне конвеєрну роботу, так само як він зараз робить із сайтом.

Коли будете готові, дайте знати:

- Який фреймворк розробки сайту у вас зараз використовується (Next.js, чистому React, чи щось інше), щоб я скоригував сумісність гри з вашим поточним сайтом?
- Бажаєте, щоб я накинув структуру базового квесту та діалогу в JSON, щоб ви бачили, як передавати сценарій вашому роботу-Клоду?

В ответах искусственного интеллекта могут быть ошибки. Подробнее…

> Так, Next + React. Ось стек і архітектура — усе з репозиторію, не з пам'яті. Стек шар що саме фронт Next 15.5.27, React 19.3, Tailwind 4.3.3, TypeScript 5 мапи MapLibre GL 6.11 (WebGL, вантажиться dynamic, ssr: false) бек NestJS 12.1, TypeORM 1.1, pg 8.23, TypeScript 6 база PostgreSQL + PostGIS (postgis, uuid-ossp), 84 таблиці CMS Directus 12.5 — окремо, не в npm-workspaces тести Playwright (юніти + e2e + візуальні), Jest для API інфра Hetzner CPX32, systemd, Caddy, fail2ban, logrotate; Cloudflare лише DNS/проксі Монорепо на npm-workspaces: apps/api і apps/web; apps/cms стоїть збоку. Архітектура, у двох реченнях Дані ніколи не йдуть із фронта в базу напряму. Next ходить тільки в наш API, API — єдиний, хто говорить із Postgres. Directus редагує той самий Postgres, але сайт із нього не читає: це свідоме рішення (§4.1(a) угоди + стійкість до падіння ліцензії). Браузер → Next (SSG/ISR) → NestJS API → Postgres+PostGIS ↑ Directus (адмінка) Модулі API CI Конвеєр даних окремо від застосунку — scripts/ Деплой — скриптами

Чудово, тепер картина абсолютно чітка. Оскільки у вас використовується потужний стек Next.js 15 (React 19) на фронтенді та NestJS 12 + PostgreSQL (PostGIS) на бекенді з повним розділенням шарів розробки, інтегрувати гру потрібно за тими ж суворими архітектурними правилами.

Ніяких клієнтських збережень в `IndexedDB` як основного джерела істини. Оскільки у вас вже є 84 таблиці в Postgres і Directus як CMS, ми перетворюємо гру на повністю Data-Driven додаток.

Ось фінальний, адаптований під ваше монорепо технічний бріф, який ви згодуєте Claude.

---

# 📜 ФІНАЛЬНИЙ БРІФ ДЛЯ CLAUDE: ІНТЕГРАЦІЯ RPG У MONOREPO

Скопіюйте та відправте цей текст вашому Claude:

text

```
Контекст нашого монорепо (npm-workspaces):
- apps/web: Next 15.5.27, React 19.3, Tailwind 4.3.3,
TypeScript 5. Мапи: MapLibre GL 6.11 (dynamic, ssr: false).
- apps/api: NestJS 12.1, TypeORM 1.1, pg 8.23, TypeScript 6.
- База даних: PostgreSQL + PostGIS. Єдине джерело істини. Next
ходить тільки в NestJS API.
- apps/cms: Directus 12.5 (адмінка, що редагує ту саму БД
Postgres).
- Тести: Playwright та Jest. Деплой — автоматизований,
скриптами через CI/CD.

ТВОЄ ЗАВДАННЯ: Розробити 2D RPG гру (тип Stardew Valley) як
новий інтегрований модуль у цьому монорепо. Гра має бути
автономною за логікою, але повністю інтегрованою в наш
конвеєр.

1. АРХІТЕКТУРНІ РІШЕННЯ ДЛЯ ГРИ (DATA-DRIVEN)
Дотримуємося правила: "Дані ніколи не йдуть із фронта в базу
напряму". Браузер (Phaser/React) -> Next -> NestJS API ->
Postgres.

- Frontend (apps/web):
  * Рушій: Phaser 3 (або PixiJS) імпортується dynamic (ssr:
false) всередину Next.js сторінки, аналогічно до MapLibre.
  * UI Шар: Компоненти інтерфейсу (Інвентар, Вікно квестів,
Діалоги) пишуться на React 19 + Tailwind 4 та рендеряться
поверх Canvas.
  * State Management: Стан гри на клієнті синхронізується
через Zustand (або React Context).
- Backend (apps/api):
  * Створюється новий модуль `GameModule` у NestJS.
  * API обробляє: валідацію збережень, синхронізацію стану
ферми, перевірку виконання квестів, збереження позиції гравця.
- CMS (apps/cms):
  * Directus використовується як ігровий редактор (Game Design
Tool). Через нього ми будемо наповнювати базу: довідник рослин
(час росту, спрайти), квести, репліки NPC та товари в
```

магазинах. Сайт з Directus не читає — API NestJS буде єдиним вікном.

2. СТРУКТУРА ТАБЛИЦЬ У POSTGRES (TypeORM)
Тобі потрібно буде створити міграції для нових таблиць:
- `game_profiles` (id, user_id, current_gold, current_day, season, player_x, player_y)
- `game_inventory` (id, profile_id, item_id, quantity, slot_index)
- `game_farm_grid` (id, profile_id, coordinate_x, coordinate_y, is_tilled, is_watered, crop_id, crop_growth_stage, planted_at)
- `game_player_quests` (id, profile_id, quest_id, status ['active', 'completed'], current_progress)

3. ВОРКФЛОУ РОБОТИ В JIRA ТА КОНВЕЄР (CI/CD)
Дій автономно за нашим стандартним воркфлоу:
- Крок 1: Створи в Jira Епіки для гри: [Game] Architecture & DB Migrations, [Game] Core Engine (Phaser+Next.js), [Game] Inventory & React UI, [Game] Quest & Dialogue System.
- Крок 2: Розбий їх на атомарні таски із чіткими Acceptance Criteria.
- Крок 3: Бери таску в роботу, пиши TypeScript код, покривай API через Jest, а фронтенд/візуальну частину — через Playwright.
- Крок 4: Публікуй через скрипти деплою, закривай таску (Done) і переходь далі.

Підтвердь, що архітектурна схема зрозуміла. Створи перші таски в Jira для налаштування Boilerplate гри (інтеграція Phaser в Next.js 15 з ssr: false та створення базових TypeORM сутностей у NestJS). Чекаю на звіти в Jira.

Используйте код с осторожностью.

---

Чому цей підхід ідеальний для вашої системи?

1. Максимальна ізоляція: Гра запускається через `dynamic(() => import(...), { ssr: false })` так само, як і ваша поточна карта `MapLibre`. Вона не зламає серверний рендеринг (SSR) Next.js.
2. Контроль через Directus: Ви зможете зайти в адмінку Directus і просто текстом додати новий квест чи змінити ціну продажу картоплі в грі. NestJS API підхопить це з Postgres, і гра оновиться без переписування коду.
3. Робота для Playwright: Оскільки у вас налаштований Playwright, Claude зможе писати e2e тести типу: *"Емулювати клік на клітинку ферми -> Перевірити в базі даних, чи статус змінився на is_tilled"*. Це гарантує, що конвеєр не пропустить багів.

Ваш робочий конвеєр повністю готовий до масштабування гри. Можете згодовувати бріф Клоду в Хром-профілі!

Конвеєр налаштовано, тепер можна закласти фундамент для першої таски Клода. Підкажіть:

- Який напрямок сценарію (сюжету) або головний квест має бути першим (наприклад: знайомство з кимось, очищення ділянки тощо)? Я можу допомогти накидати структуру цього квесту в форматі, який Клод відразу перетворить на таски в Jira.

Це від ChatGPT AI:

CampLife: The Journey — технологія та архітектура вебверсії

Я б будував гру як повноцінну браузерну 2D-гру з локальними сценами, картою Європи, симуляцією побуту й подорожей, системою подій і довготривалим збереженням світу. Архітектура має дозволяти почати з невеликого прототипу, а потім розширювати гру без переписування основних систем.

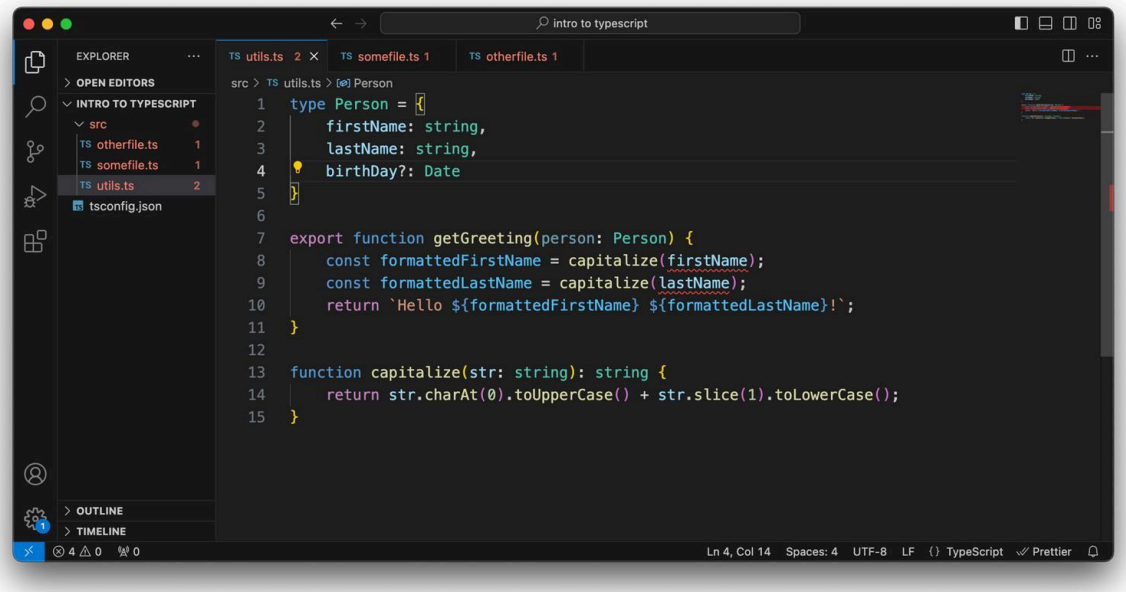
Ключовий принцип: ігровий світ, правила симуляції, сюжет і візуальне відображення мають бути розділені. Тоді можна буде додавати нові кемпінги, транспорт, професії, NPC, погодні ситуації та квести як дані й модулі, а не переробляти всю гру.

1. Технологічний стек

Ігровий рушій — Phaser

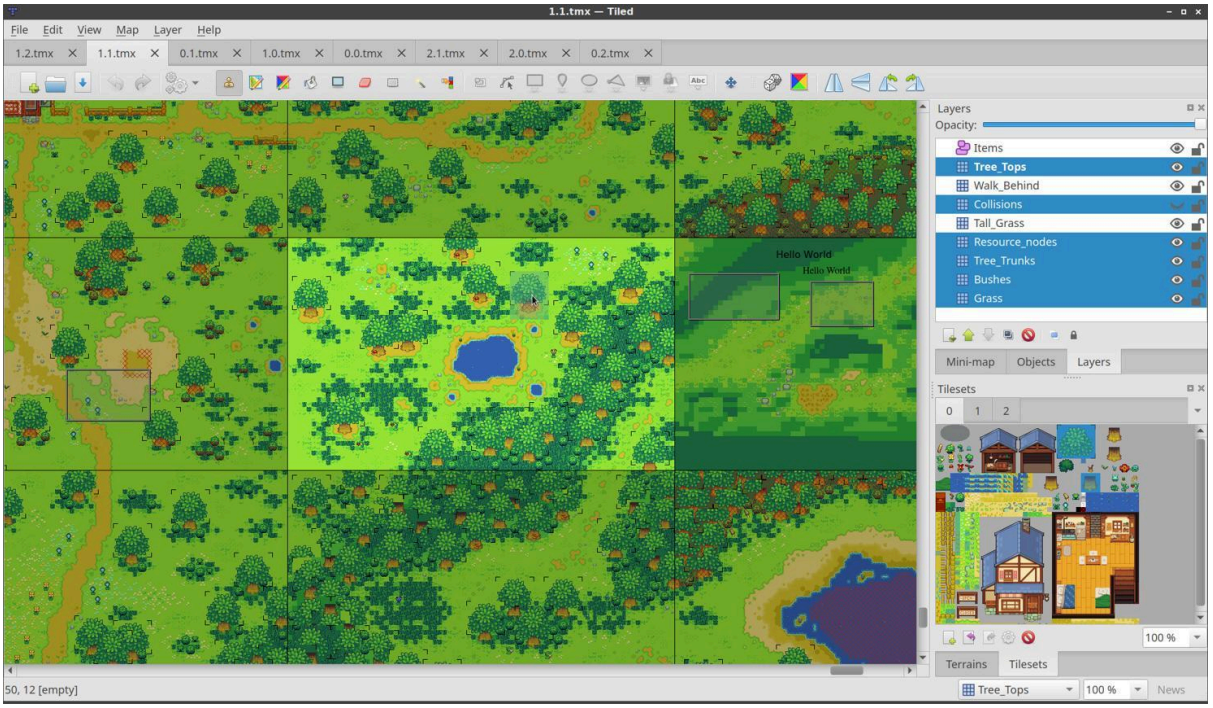
Рекомендую

Відповідає за 2D-сцени, спрайти, анімації, камеру, колізії, введення з клавіатури й миші та відображення ігрового світу. Добре підходить для браузерної піксельної гри.



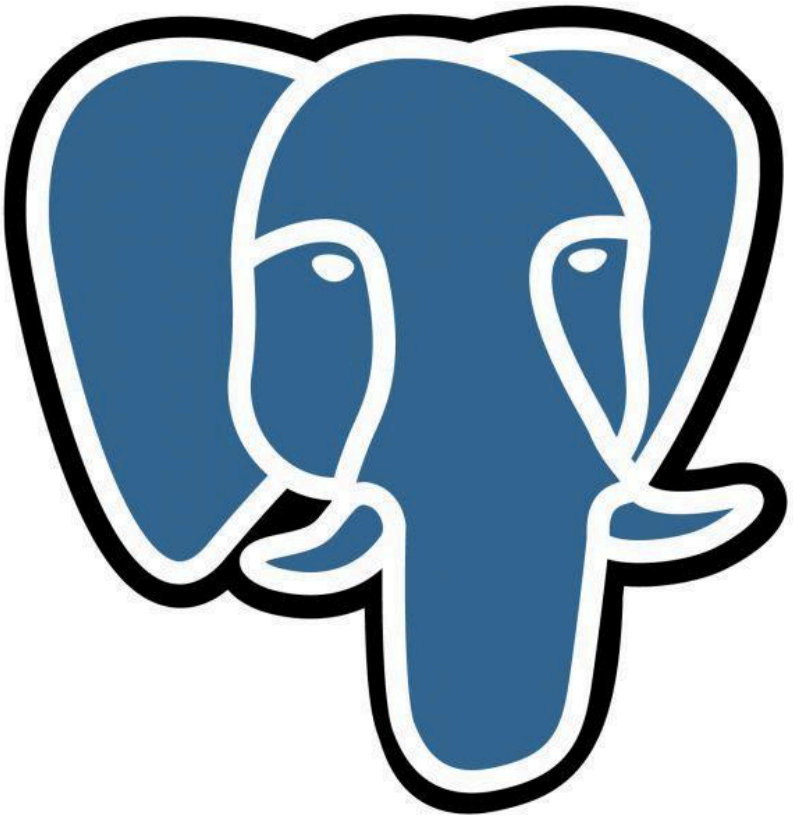

Мова — TypeScript

Ігрові сутності, інвентар, транспорт, NPC, події та фінанси матимуть чіткі типи. Це знижує ризик помилок у великій симуляції та полегшує роботу AI-розробника.




Редактор рівнів — Tiled

Для створення будинку, магазину, майстерні, кемпінгу, містечка, лісових стежок і транспортних вузлів без ручного кодування кожної плитки.



Серверна база — PostgreSQL

Для облікових записів, хмарних збережень, профілів світу, прогресу, резервних копій і майбутньої синхронізації між пристроями.

Збірка — Vite

Швидкий цикл розробки, пакування TypeScript і оптимізація ресурсів для браузера.

Додатково: HTML/CSS для меню й інтерфейсу, Web Audio API для звуку, Web Workers — лише для важких обчислень, а для тестування — автоматизовані unit-тести й браузерні сценарії.

Чому не робити всю гру на HTML-елементах? Бо рух персонажів, колізії, камера, анімація та багато інтерактивних об'єктів зручніше реалізовуються в ігровому рушії. А меню персонажа, планування маршруту, бюджет і склад можна зробити звичайними вебінтерфейсами поверх ігрового поля.

## 2. Загальна схема архітектури

Я пропоную модульний моноліт із чіткими межами між підсистемами, а не мікросервіси. Для гри, яку спочатку розробляє невелика команда або AI-розробник, це простіше тестувати, запускати й підтримувати. Серверні компоненти за потреби можна винести окремо пізніше.

Вебклієнт

Phaser + TypeScript · React для меню й панелей

Ігрове ядро

Стан світу · час · правила · події · завдання · симуляція

Простір

Європа, маршрути, локальні карти, колізії

Життя

Персонажі, сім'я, NPC, тварини, робота

Подорожі

Транспорт, погода, дороги, бронювання

Економіка

Бюджет, покупки, зарплати, ремонт, борги

Шар збереження та даних

IndexedDB → API сервера → PostgreSQL

Важливе уточнення: ці блоки — логічні модулі, а не обов'язково окремі сервери чи програми. На початку вони можуть працювати в одному клієнтському застосунку.

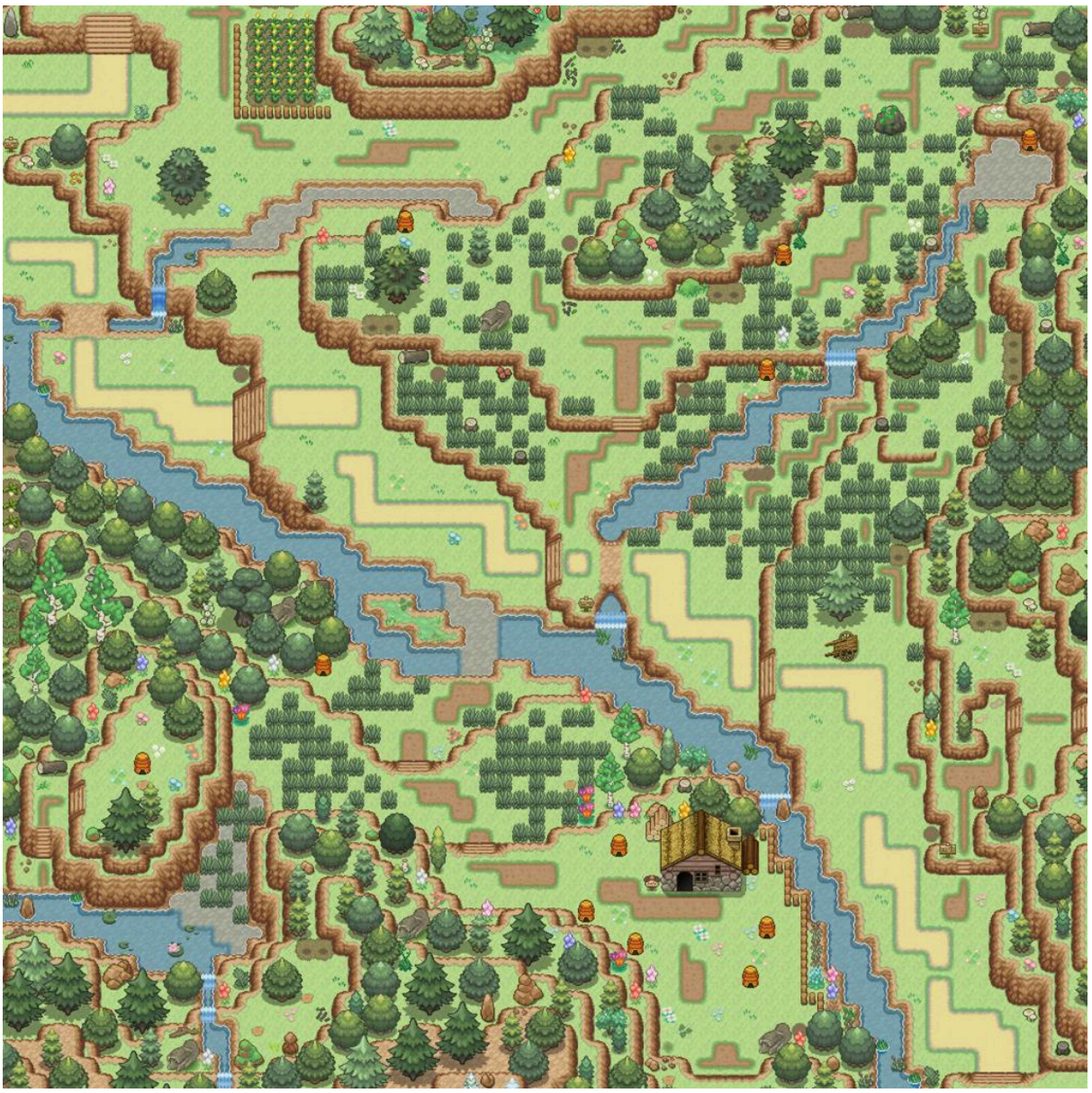
## 3. Три рівні ігрового світу

Не варто створювати одну величезну карту Європи з деталізацією кожної вулиці. Гравець переміщується між трьома масштабами, кожен з яких має власні правила.

Рівень 1 — карта Європи

Планування відпустки та вибір напрямку.

Тут гравець бачить країни, регіони, міста, пороми, залізничні сполучення, великі дороги й доступні кемпінги. Вибирає пункт призначення, дати, бюджет і приблизний маршрут. Карта показує орієнтовний час, вартість та обмеження.



Рівень 2 — маршрут

Дороги, відстані й рішення в дорозі.

Маршрут — це граф дорожніх сегментів із довжиною, типом дороги, обмеженнями, перепадом висоти, приблизною швидкістю та платними ділянками. Розрахунок залежить від транспорту, завантаження, погоди й навичок. Пропущений поворот або запізнення на пором можуть змінити весь день.

Рівень 3 — локальна сцена

Безпосереднє керування персонажем.

Кемпінг, магазин, дім, майстерня, містечко, вокзал, поромний термінал або стежка. Гравець вільно ходить, розмовляє, взаємодіє з речами, ставить намет, готує їжу, ремонтує велосипед і спілкується з NPC.

Для локальних карт доцільно використовувати тайли й об'єкти, створені в Tiled, а потім завантажувати їх у Phaser. Tiled підтримує експорт карт у JSON, що зручно для вебзастосунку.

Tiled 1.12.2 documentation
+1

Правило переходів: вихід із локальної сцени не створює новий світ. Він завантажує іншу локацію, зберігаючи дату, час, стан персонажів, інвентар, транспорт, бронювання, погоду та активні завдання.

4. Модулі ігрового ядра

Я б розділив ядро на такі незалежні модулі. Кожен має визначені вхідні дані, результати й тести.

| Модуль | Відповідальність |
| --- | --- |
| `WorldState` | Поточний світ, дата, час, активна локація, глобальні прапорці |
| `TimeSystem` | Ігровий годинник, тривалість дій, календар, сезони |
| `CharacterSystem` | Потреби, здоров'я, втома, навички, особисті цілі |
| `HouseholdSystem` | Сім'я, стосунки, розклад, потреби дітей і супутників |
| `PetSystem` | Годування, прогулянки, перевезення, доступність тварин |
| `InventorySystem` | Речі, вага, об'єми, стан, сумісність спорядження |
| `VehicleSystem` | Велосипеди, автомобілі, кемпери, паливо, вода, ремонт |
| `RouteSystem` | Маршрути, транспортні обмеження, ETA, пороми |
| `WeatherSystem` | Дощ, вітер, температура, видимість і вплив на подорож |
| `EconomySystem` | Зарплата, витрати, ціни, рахунки, борги, випадкові доходи |

| | |
|---|---|
| `ReservationSystem` | Бронювання кемпінгів, квитки, скасування, правила розміщення |
| `InteractionSystem` | Контекстні взаємодії з предметами, дверима, NPC |
| `NPCSystem` | Розклади, потреби, пам'ять, стосунки, особисті цілі |
| `QuestSystem` | Місії, цілі, передумови, розгалуження, винагороди |
| `EventSystem` | Побутові, позитивні, соціальні та дорожні події |
| `SaveSystem` | Збереження, завантаження, міграції форматів і резервні копії |
| `AudioSystem` | Музика, звуки оточення, дії й сигнали попереджень |

Модулі не повинні напряму змінювати внутрішні дані один одного. Наприклад, `VehicleSystem` не списує гроші самостійно — він формує запит на ремонт, а `EconomySystem` перевіряє ціну й проводить платіж. Так менше ризику отримати ремонт без списання грошей або подвійне списання.

Як системи взаємодіють

Припустімо, сім'я їде до кемпінгу:

1. `RouteSystem` розраховує маршрут і прогнозований час прибуття.
2. `VehicleSystem` обчислює витрату пального з урахуванням автомобіля та завантаження.
3. `WeatherSystem` передає дані про дощ і вітер.
4. `TimeSystem` враховує зупинки, швидкість і затримки.

5. `ReservationSystem` перевіряє час прибуття й правила заїзду.
6. `EconomySystem` рахує витрати на пальне, дороги, пором і проживання.
7. `EventSystem` може запропонувати цікаву зупинку або доречну проблему.
8. `SaveSystem` фіксує важливі зміни.

Подія не повинна просто випадково віднімати гроші. Вона має мати причину, зрозумілий контекст, можливі рішення та наслідки.

5. Архітектура часу й симуляції

Це один із найважливіших технічних аспектів. Якщо неправильно реалізувати час, розклади NPC, витрата пального, потреби, робота й події почнуть суперечити одне одному.

Пропоную розділити час на два механізми.

- Час кадру: рух персонажа, анімації, колізії та візуальні ефекти.
- Ігровий час: хвилини й години, які рухають потреби, роботу, транспорт, погоду, розклади та події.

Ігровий час не повинен автоматично йти з реальною швидкістю 1:1. Гравець зможе прискорювати час у дорозі, пропускати сон або чекати на послугу, але зупиняти його під час важливих рішень.

Приклад правил:

- Пересування локально: ігровий час іде поступово.
- Розмова: час може зупинитися або пройти коротким кроком.
- Готування: дія займає задану кількість ігрових хвилин.
- Сон: час переходить до ранку, потреби й світові події оновлюються.
- Поїздка: симуляція обчислює відрізки маршруту, зупинки й затримки.
- Відкриття меню: симуляція за замовчуванням призупиняється, якщо це не спеціальний екран планування подорожі.

Для точних розрахунків треба використовувати фіксовані кроки симуляції та накопичувач часу, а не залежати від частоти кадрів браузера. Рендеринг може працювати з різною частотою, але однакові ігрові дії повинні давати однакові результати.

Система подій

Усі системи публікують події у внутрішню чергу, наприклад:

type GameEvent =

```
  | {
      type: "VehicleFuelConsumed";
      vehicleId: string;
      liters: number;
    }
  | {
      type: "ReservationCheckInMissed";
      reservationId: string;
    }
  | {
      type: "NpcRelationshipChanged";
      npcId: string;
      delta: number;
      reason: string;
    };
```

Це приклади контрактів, не готова реалізація. Важливо, щоб типи подій були централізовані й перевірялися компілятором.

Для передбачуваних подій, як-от зарплата, дата бронювання чи закінчення терміну зберігання продукту, потрібен календар запланованих дій. Для випадкових історій потрібен окремий механізм вибору подій за умовами, вагами, історією та періодом повторення.

6. Персонажі, NPC і сюжет — через дані, а не через окремий код

Кожен NPC отримує незмінний унікальний `npcId`. Ім'я не може бути його ідентифікатором, адже ім'я теоретично може змінитися або повторитися в іншій культурі.

Приклад запису NPC:

```
interface NPCDefinition {
  npcId: string;
  name: string;
  age: number;
  appearanceId: string;
  biographyId: string;
  traits: string[];
  goals: string[];
  scheduleId: string;
  startingRelationships: Record<string, number>;
  dialogueTreeIds: string[];
  questIds: string[];
```

}

Потрібно розділити визначення NPC і його стан у конкретному проходженні. Визначення містить біографію, зовнішність і базові риси. Стан містить поточне місце, стосунки, виконані завдання, речі, обіцянки та спогади про гравця.

Так само квести та події зберігаються в JSON або іншому перевірюваному форматі даних. Автор контенту зможе створювати нові місії без написання нової системи для кожної з них.

7. Як реалізувати транспорт і реалістичні подорожі

Тут не можна обмежитися властивістю `speed: 50`. Транспорт повинен мати параметри, що реально впливають на подорож.

Автомобіль або кемпер

- Маса, допустиме завантаження, витрата пального.
- Обсяг бака й поточний рівень пального.
- Технічний стан двигуна, шин, гальм та акумулятора.
- Висота, ширина й довжина для обмежень маршрутів.
- Витрати на обслуговування, паркування та платні дороги.
- Вода, електрика, холодильник і санітарні системи для кемпера.

Велосипед

- Тип, маса, передавальні числа, стан шин і трансмісії.
- Маса багажу та розподіл навантаження.
- Ухил, покриття, вітер, дощ і фізична форма велосипедиста.
- Втома, комфорт сідла, запас води й необхідність зупинок.

Не потрібно симулювати повну фізику двигуна або велосипедної трансмісії. Для гри достатньо фізично правдоподібної моделі з перевірюваними формулами.

Наприклад, базова витрата пального:

$$L = \frac{D \cdot C}{100}$$

де $D$ — відстань у кілометрах, $C$ — витрата у літрах на 100 км. Потім результат коригується на завантаження, тип дороги, рельєф і погоду. Усі коефіцієнти мають бути конфігурованими й відкаліброваними тестами.

Маршрут краще представити графом: вузли — перехрестя, міста, кемпінги, заправки й пороми; ребра — ділянки доріг із характеристиками. Для першої версії можна створити власний компактний граф регіону, а не залежати від зовнішнього сервісу під час кожної поїздки.

8. Як працюватимуть збереження й офлайн-режим

Я б реалізував збереження у два етапи.

Етап 1 — локальна гра без обов'язкового акаунта.

- `IndexedDB` для збережень, історії та великих об'єктів даних.
- `localStorage` — лише для дрібних налаштувань, наприклад гучності.
- Автозбереження після важливих дій, переходів між сценами та за певними інтервалами.
- Щонайменше два локальні резервні слоти.
- Перевірка версії збереження й контрольоване відновлення після пошкодження.

Етап 2 — хмарні збереження.

- Облікові записи та API сервера.
- PostgreSQL для профілів і метаданих збережень.
- Версійовані записи збережень, резервні копії й механізм конфліктів синхронізації.
- Можливість грати локально без мережі, а потім синхронізувати прогрес.

Збереження повинно містити не тільки координати гравця, а й дату, активний маршрут, стан домогосподарства, транспорт, інвентар, фінанси, бронювання, NPC, прапорці сюжету та версію схеми даних.

Для вебверсії також варто передбачити PWA-режим із кешуванням ігрового застосунку та ресурсів. Але офлайн-доступ до гри не означає, що в офлайні доступні свіжі прогнози погоди, зовнішні карти чи синхронізація.

9. Зовнішні дані: що підключати, а що залишити внутрішнім

| Джерело або система | Застосування | Рішення |
| --- | --- | --- |

| | | |
|---|---|---|
| OpenStreetMap | Географія, дороги, місця, POI | Імпорт у власний набір даних після перевірки ліцензії та атрибуції |
| Open-Meteo | Історична погода й погодні сценарії | Підключати через окремий адаптер, зберігати кеш і дату даних |
| Власний каталог кемпінгів | Ціни, правила, послуги, доступність | Власна нормалізована база; реальні дані — лише з дозволених джерел |
| Власний каталог транспорту | Моделі, характеристики, витрата, вантажність | Версійовані довідники з джерелами й рівнем достовірності |
| Внутрішній рушій подій | Квести, поломки, знайомства, випадкові історії | Працює локально та не залежить від зовнішнього API |

Для погоди існує історичний API Open-Meteo, який може бути корисним для побудови правдоподібних сценаріїв. Але історична погода не є прогнозом на майбутнє; ігрова погода повинна мати власну модель, якщо гравець подорожує у вигаданому або майбутньому ігровому часі.

open-meteo.com

Реальні назви, координати, ціни та характеристики потрібно зберігати разом із джерелом, датою отримання, територією дії та статусом перевірки. Якщо дані застаріли, гра повинна вміти показати приблизну інформацію, а не видавати її за актуальну.

10. Структура проєкту для AI-розробника

Ось базова структура репозиторію, яку я рекомендую закласти відразу.

```
camplife/
├── apps/
│   ├── web/              # React, Vite, маршрутизація, меню
│   └── game/             # Phaser, сцени, камера, введення
├── packages/
│   ├── core/             # Стан світу, час, правила симуляції
│   ├── domain/           # Типи персонажів, транспорту, предметів
│   ├── systems/          # Економіка, NPC, погода, події, квести
│   ├── content/          # JSON-дані, діалоги, місії, каталоги
│   ├── maps/             # Локальні карти та граф маршрутів
│   ├── persistence/      # Збереження, міграції, імпорт/експорт
│   └── shared/           # Спільні типи, константи, утиліти
├── server/
│   ├── api/              # API для акаунтів і хмарних збережень
│   └── migrations/       # Міграції бази даних
├── tests/
│   ├── unit/
│   ├── integration/
│   └── gameplay/
├── docs/
│   ├── architecture.md
│   ├── game-design.md
│   ├── save-format.md
│   └── data-sources.md
└── package.json
```

Це цільова структура, а не вимога створювати всі папки на першому етапі. Якщо розробка починається з прототипу, можна почати з меншої кількості пакетів, але не змішувати логіку симуляції з кодом малювання.

11. Продуктивність браузера

Щоб гра працювала на звичайних ноутбуках, потрібні чіткі бюджети продуктивності.

- Симулювати детально лише активну локальну сцену й сутності поблизу гравця.
- Для віддалених NPC використовувати спрощені розклади й підсумкові стани, а не постійний рух кожного персонажа.
- Завантажувати текстури, звук і карти за потреби, а не все під час старту.

- Використовувати атласи спрайтів, обмежувати кількість активних ефектів і не перераховувати весь світ кожен кадр.
- Важкі обчислення маршрутів або великих наборів даних за потреби переносити у Web Worker.
- Передбачити зменшену якість ефектів для слабших пристроїв.

Рекомендована початкова ціль: стабільні 60 FPS на типовому сучасному настільному браузері, з адаптивним режимом для слабших систем. Це ціль тестування, а не гарантований результат до профілювання.

12. Тестування й захист від логічних помилок

Для симулятора особливо небезпечні не лише візуальні баги, а й суперечності правил.

Приклади автоматичних тестів:

- Гравець не може купити предмет, якщо не має достатньо коштів.
- Один і той самий рахунок не списується двічі.
- Автомобіль не може використати більше пального, ніж є в баку.
- Вантаж не перевищує допустиму масу без відповідних наслідків.
- Бронювання не можна використати після завершення його строку.
- Тварина не зникає з домогосподарства під час переходу між локаціями.
- Квест не активується повторно, якщо він одноразовий.
- Завантаження збереження відновлює ті самі важливі стани, що були під час збереження.
- Прискорення часу не змінює результат одного й того самого детермінованого розрахунку.
- NPC не може одночасно бути в двох локаціях.

Крім unit-тестів, потрібні сценарні тести: наприклад, «сім'я планує тижневу поїздку, бронює кемпінг, їде через платну дорогу, витрачає пальне, прибуває й заселяється». Такий сценарій повинен виконуватися від початку до кінця автоматично.

13. План реалізації

Фаза 1 — технічний прототип

Мета: перевірити основу.

Запуск у браузері, одна локальна карта, рух персонажа, камера, колізії, взаємодія з предметами, базовий інтерфейс і локальне збереження.

Фаза 2 — ігрове ядро

Мета: зробити гру системною.

Ігровий час, персонаж, потреби, гроші, інвентар, покупки, простий транспорт, NPC, квести та система подій.

Фаза 3 — перша подорож

Мета: завершений ігровий цикл.

Планування маршруту, перша крамниця, кемпінг, бронювання, погода, приготування їжі, транспортні витрати й одна сюжетна лінія.

Фаза 4 — розширення світу

Мета: контент і тривала кампанія.

Більше регіонів Європи, транспортних засобів, професій, домашніх тварин, NPC, місій, випадкових подій та реальних довідників.

Фаза 5 — хмарні функції

Мета: довготривале використання.

Акаунти, синхронізація, резервні копії, PWA, інтеграція зовнішніх даних і підготовка до можливої десктопної версії.

14. Що зафіксувати в технічному завданні

Я б затвердив такі архітектурні правила перед написанням основного коду:

1. TypeScript і Phaser — основа ігрової частини; React — для складних меню й інформаційних панелей, якщо це виправдано.
2. Симуляція, правила та контент не залежать від Phaser і не повинні потребувати браузерного рендерингу для тестування.
3. Усі важливі параметри балансу зберігаються в конфігураціях, а не розкидані по коду.
4. Події, квести, NPC, предмети й транспорт мають стабільні унікальні ID.
5. Стан світу є єдиним джерелом істини; інтерфейс лише показує його й надсилає команди.
6. Локальні збереження працюють до впровадження сервера.
7. Зовнішні дані підключаються через адаптери, щоб гра могла працювати й без них.
8. Кожна підсистема має тести та задокументовані контракти.

9. Новий контент не повинен вимагати переписування основних правил.
10. Жодна серйозна негативна подія не повинна виникати без причин і зрозумілих механік реагування.

Моя головна рекомендація: спочатку реалізувати невелику, але повністю працюючу подорож — від створення персонажа й покупки спорядження до маршруту, прибуття в кемпінг, ночівлі та повернення додому. Якщо цей цикл буде захопливим, логічним і стабільним, архітектура дозволить розширювати гру поступово, не перетворюючи її на нестабільний набір непов'язаних механік.

—----------

Це ми з ЧатГПТ думали і писали про гру ідеї і сценарій:

в мене є проєкт кемпінг вебсайт і я хочу створити інді гру по типу https://store.steampowered.com/app/413150/Stardew_Valley/?curator_clanid=41316928
Ти мені потрібен щоб написати сценарій, але він має копіювати справжнє життя людей і кемпінг пригоди і відпочинок. В мене є задумка але ти маєш її доповнити і записати ідельно для ШІ який буде її робити. Перевір всі знання в світі про життя любдей їх роботу і відпочинок і подорожі і болі і радощі з якими вони зіштовхуються в житті і в подорожах (є форуми де лди описують і блоги) тому ти все прочитай і переведи це в ігрові процси, але давай без хвороб і смертей бо гра має затягуцвати грати в неї. Впораєшся із таким?

Але реалістичну по сценарію де є лдина і він обирає роль і чи є в нього родина чи він одинак, який тип подорожей він обирає (вело, пішки, авто, авто з тентом, авто з причепом, авто автобудинком - пречепом, будинок на колесах). Тобто наприклад людина зайшла і написала я маю дружигну і сина, в мене авто і гра йому каже який бюджет доступний і першим його вистачить на палатку, далі буде таймінг коли людина обирає палатку і з пон по пт працює начебто на якийсь роботі і на початку нового міс йому прийшла зп і він готовий купити палатку і інше спорядження, далі в нього йдуть гроші на бензин і на оренду кемпінг місця, він їде в подорож і купує продукти, ставить авто проводить там час і повертається додому (в нього будуть расходи на ремонт обладнання і авто), він накопичує гроші і моде купити наприклад собі нову більшу палатку чи причеп який відкривається як палатка-будиночок. Далі він може їздити на довші відстані і на довший час бо він накопичив на відпустку гроші + локації наші реальні світові і реальний наш планер можна тули парсити. Далі можна продавати палатку і авто і інше обладнання таке як плитка, спальні місця тощо (дивись еквіпмент який притаманний до кожного виду і варіанту відпустки з різними видами кемперів і палаток). Також він з часом і довсідом може купити кемпер спочатку підтриманий але який буде потребувати ремонту і часу для ремонту + оновлення і апгрейди такі як соняшні панелі чи інше приладдя яке є в автофургоні, далі він може навіть заробляти гроші купуючи і реновуючи і апгерйд роблячи а іноді йти в мінус бо ніхто не купив зараз і є хтось хто пропонує манше грошей ніж він вклав (як в реальному житті). Далі гравець може купити великий автодом але він буде потребувати пройти курси водія для отримання категорії "С" бо фіргон занадто важкий. В дорозі будуть і поломки і пригоди і будинок буде ламатись і будуть дороги які закриті, і сход лавин і дощі і вітер і град (все реалістично взагалі притамано регіонам і порі року). Тобто в нас в грі буде ігровий час + будуть зміни клімату тощо. В ідеалі щоб все було побудовано так щоб повторювати життя кемпер користувачів і їх пригоди. Тобто кемпінг був забуканий але щось пішло не так і їх місце зайнято і їм треба знайти інший, подзвонити поїхати туди але ж це може бути високий сезон і все зайнято і вони ночцватимуть біля супермаркету. В дорозі запаси треба буде поповнювати і паливо і воду і їжу і зливати сіру і чорну воду, і газові балони поповнювати. Тобто я хочу прямо супер реалістично щоб було, так як справжнє життя в подорожі і приготування до неї і потім і наслідки по типу авто зламалося а грошей немає у гравця і він має повертатися додому чи брати кредит в ігровому своєму банку (тобто по типу що є кредитка на пару-тройку тис євро в нього), але там будуть відсотки як в справжнього банку і треба буде повертати і поповнювати. Також персонаж буде мати скіли, хтось каже що він в житті ІТ а хтось механік і може щось сам лагодити механічне але запчастини все одно потрібні, а хтось каже він електрик і може полагодити електрику чи робити апгрейд венів безкоштовно лише електричку частину, хтось вміє все по трішечки і чим довше він в грі тим вище скілл прокачується + поломки загортовують гравця і він отримує навички, виживання в лісу в палатці дикарем без кемпінгу окремі бонуси і там хижаки і людина може пораннення отримати чи загинути

якщо погано підготувався (теж як в справжньому житті виживання в дикому кемпінгу - тут також прив’язка до території де людина обирає кемпер привал робити), продумати треба ігрові моменти і я можу скласти тобі більш детальний ігровий сценарій але питання чи ти зможеш це зробити?

Так, я впораюся. І більше того — з твоєї задумки можна зробити не просто гру про кемпінг, а великий симулятор життя мандрівника, у якому гравець поступово переходить від дешевих вихідних у звичайній палатці до тривалих подорожей Європою, життя у вані або автодомі, ремонту кемперів, облаштування власного мобільного будинку та навіть невеликого бізнесу.

Я розумію, чого ти хочеш: не аркаду, де гравець натиснув кнопку «Поїхати в кемпінг» і через хвилину отримав нагороду. Ти хочеш, щоб кожна подорож була маленькою історією зі справжнього життя: підготовка, робота, бюджет, очікування, дорога, природа, побутові проблеми, несподіванки, радість від пригод і бажання наступного разу поїхати ще далі.

І все це має бути достатньо захопливим, щоб гравець повертався у гру щодня.

1. Якою я бачу твою гру

Етап 1. Перші пригоди

Палатка, недорогий автомобіль, вихідні, перші покупки.

Гравець має роботу, житло, обмежений бюджет і певні обов'язки. Він купує базове спорядження, планує першу поїздку, вчиться розкладати табір, готувати їжу та правильно розраховувати витрати.

Етап 2. Свобода та комфорт

Кращий намет, велосипед, обладнання, кемпери та довші маршрути.

З'являються складніші подорожі, сезонність, бронювання, витрати на технічне обслуговування, апгрейди, сімейні потреби та вибір між комфортом і пригодами.

Етап 3. Власний кемпер

Ремонт, переобладнання, сонячні панелі, автономність.

Гравець купує вживаний фургон, знаходить приховані дефекти, планує бюджет переобладнання, монтує меблі та електрику, а потім вирішує, залишити його собі чи продати.

Етап 4. Життя в дорозі

Великі автодоми, далекі країни, складна логістика.

Планування багатотижневих маршрутів, платні дороги, технічні обмеження, погодні ризики, пошук сервісів, витрати на кемпінги та поступове відкриття нових регіонів.

Це не обов'язково чотири лінійні рівні. Гравець може залишитися прихильником велотуризму чи палаток на сотні ігрових годин. Інший побудує цілий бізнес із ремонту та продажу кемперів. Третій житиме у фургоні й досліджуватиме світ.

Головний принцип: гра не повинна нав'язувати єдиний правильний спосіб подорожувати.

2. Так, я можу дослідити реальний досвід мандрівників

Але хочу одразу правильно визначити підхід: неможливо буквально прочитати весь інтернет. Натомість можна систематично зібрати репрезентативні історії, типові проблеми, практичні рекомендації та повторювані життєві ситуації з форумів, блогів, спільнот кемперів і велосипедистів, посібників з обслуговування техніки та офіційних джерел про погоду, дороги й кемпінги.

Для цього я б досліджував такі джерела:

Reddit — історії реальних людей

r/camping · r/CampingGear · r/vandwellers · r/overlanding

Пошук спорядження, невдалих поїздок, сімейних суперечок, бюджету, облаштування фургонів, життя без кемпінгів та неочікуваних витрат.

MotorhomeFun та Practical Motorhome

MotorhomeFun · Practical Motorhome

Технічні несправності, вибір уживаного автодому, вологість, опалення, водопостачання, зимова експлуатація та витрати на сервіс.

Bikepacking.com та Adventure Cycling Association

Bikepacking · Adventure Cycling

Втома, розподіл ваги, ремонт велосипеда, харчування в дорозі, вибір спорядження та планування маршрутів залежно від фізичної підготовки.

Офіційні джерела про реальний світ

Meteoalarm · Copernicus · OpenStreetMap · Eurostat

Погодні небезпеки, географія, дорожня мережа, сезонність туризму та фактичні відмінності між регіонами.

Я б не просто виписував історії. Я б перетворював кожну повторювану ситуацію на ігрову механіку.

Наприклад, гравець на форумі описує, як після дощової ночі його спальник залишився вологим, а наступного дня температура впала. У грі це не просто випадкове повідомлення: вологе спорядження займає місце, потребує сушіння, погіршує комфорт і може зіпсувати наступну ночівлю. Якщо гравець купив якісний тент, правильно організував табір і взяв запасний одяг, він має перевагу.

Так справжні життєві історії стають системами, а не декоративними подіями.

3. Найважливіше — ігровий цикл реального життя

Я бачу основний цикл так:

Робота та повсякденне життя

Зарплата, рахунки, сім'я, вільний час, навички

Планування та покупки

Бюджет, спорядження, бронювання, маршрут, прогноз погоди

Подорож та пригоди

Дорога, ночівля, їжа, природа, люди, проблеми та відкриття

Повернення та наслідки

Ремонт, прибирання, витрати, відновлення спорядження, спогади

Розвиток

Заощадження, нові навички, краще спорядження, довші подорожі

Цикл повторюється, але кожна наступна подорож залежить від попередніх рішень.

Приклад однієї ігрової історії

Початкові умови: персонаж працює п'ять днів на тиждень, має партнерку та сина, володіє старим легковим автомобілем.

- 1-й день місяця. Надходить зарплата. Гравець сплачує оренду житла, рахунки, витрати на сім'ю та автомобіль. Визначає бюджет відпочинку.
- 3-й день. Купує намет, два спальники, килимки, пальник і базове кухонне обладнання. Не все вдається придбати відразу, тому частину речей доводиться позичити або взяти дешевші.
- 8-й день. Бронює кемпінг біля озера на вихідні. Перевіряє прогноз, пальне, харчування та час у дорозі.
- 12-й день. Напередодні виїзду виявляється, що один із килимків пропускає повітря. Гравець може відремонтувати його, придбати новий або ризикнути.
- 13-й день. Родина вирушає в дорогу. На трасі ремонтні роботи, тому маршрут стає довшим, а витрати на пальне зростають.
- П'ятниця ввечері. Приїхавши, гравці дізнаються, що їхнє місце помилково віддали іншим відвідувачам. Потрібно телефонувати, шукати альтернативу, їхати до іншого кемпінгу або обирати дозволене місце для ночівлі.

- Субота. Дощ змінює плани. Можна залишитися під тентом, піти в сусіднє містечко, відвідати музей або скоротити маршрут прогулянки.
- Неділя. Родина повертається додому. Вологий намет потрібно висушити, пальник почистити, автомобіль заправити, а залишки їжі правильно зберегти.
- Кінець місяця. Гравець оцінює витрати, досвід і враження. Наступного місяця він уже знає, що потрібно придбати, а на чому можна заощадити.

Це лише одна історія. Інший персонаж може їхати сам на велосипеді, працювати віддалено, мати нерегулярний дохід або витратити всі заощадження на старий кемпер, який виявиться значно проблемнішим, ніж здавалося продавцеві.

І найголовніше: не кожна проблема повинна карати гравця. Деякі труднощі можуть створювати кумедні історії, знайомства, нові навички або альтернативні пригоди.

4. Які системи я закладу в повний сценарій

Персонаж, професія та родина

Самотній мандрівник, пара, родина з дітьми, друзі. Вік і потреби дітей, вільний час, графіки роботи, характер, звички, навички та вподобання.

Час, робота та економіка

Зарплата, оплачувана відпустка, вихідні, регулярні рахунки, сезонні витрати, заощадження, кредит, відсотки, ремонт і фінансові резерви. Економіка має враховувати не лише гроші, а й час.

Транспорт та способи подорожувати

Пішки, велосипедом, автомобілем із палаткою, звичайним наметом, причепом-палаткою, житловим причепом, кемперваном або великим автодомом. Для кожного типу — власні обмеження, витрати, спорядження, комфорт і складність експлуатації.

Спорядження та інвентар

Намети, спальники, матраци, пальники, холодильники, вода, газ, акумулятори, сонячні панелі, інструменти, меблі, запасні частини. Вага, об'єм, сумісність, зношення, вологість і термін служби.

Реальна географія та маршрути

Реальні країни й регіони, дорожня мережа, рельєф, сезонні обмеження, платні дороги, пороми, правила ночівлі, природні території, кемпінги та сервіси. Дані карт можна отримувати з реальних джерел, але ігровий світ має працювати навіть без інтернету.

Погода, пори року та природні події

Дощ, спека, холод, вітер, град, сніг, повені, пожежні обмеження, перекриття доріг і небезпечні гірські умови — з урахуванням регіону, висоти, сезону та прогнозу. Події мають бути правдоподібними, а не випадковими карами.

Ремонт, модернізація та торгівля

Діагностика дефектів, пошук запчастин, вартість роботи, відмінності між самостійним ремонтом і сервісом. Купівля вживаних машин, ризики прихованих пошкоджень, оновлення та перепродаж із прибутком або збитком.

Емоції, стосунки та спогади

Радість від першої ночівлі, гордість за ремонт, втома після довгої дороги, розбіжності в родині, нові знайомства, улюблені місця, фотографії, колекція спогадів і бажання повернутися в певний регіон.

Окремо я розроблю систему професійних навичок. Механік матиме перевагу в роботі з двигуном і ходовою частиною, електрик — в електриці, IT-фахівець — в електроніці, цифрових системах і віддаленій роботі. Але жодна професія не повинна робити персонажа універсальним майстром.

З досвідом гравець зможе освоювати нові компетенції, а кожна поломка даватиме можливість навчитися чогось нового. При цьому для складних ремонтів усе одно можуть знадобитися інструменти, запчастини, обладнання та допомога спеціаліста.

Щодо дикого кемпінгу, я б зробив окрему систему з навичками орієнтування, розпалювання вогню там, де це дозволено, організації безпечного табору, пошуку води, оцінки погоди та взаємодії з дикою природою. Небезпеки й серйозні наслідки можуть існувати, але не обов'язково робити смерть центральною механікою. Можна зосередитися на травмах без фатальних наслідків, евакуації, втраті спорядження, фінансових витратах і вимушеному завершенні походу.

5. Що зробить гру справді захопливою, а не виснажливою

Тут є тонкий момент. Ти хочеш реалізму, але якщо гравець повинен щодня оплачувати кожен бутерброд, вручну перевіряти кожен болт і дивитися, як

десять хвилин ігрового часу нічого не відбувається, гра перетвориться на роботу.

Тому я пропоную три рівні симуляції:

- Детальний рівень: важливі рішення — бюджет, покупки, ремонт, маршрут, вибір місця, облаштування табору.
- Автоматичний рівень: повторювані процеси — регулярні платежі, звичайне обслуговування, повсякденні витрати.
- Подієвий рівень: цікаві ситуації — зламаний насос, несподіване знайомство, дощ, переповнений кемпінг, пропозиція купити старий фургон.

Гравець може обирати складність. Для тих, хто любить глибоку симуляцію, будуть докладні фінанси й технічні системи. Для тих, хто хоче насамперед подорожувати, — більше автоматизації.

Ще одна принципова річ: зарплата не повинна автоматично надходити на початку кожного місяця незалежно від професії. Гравець може мати погодинну оплату, фіксовану зарплату, сезонну роботу, власну справу чи віддалений дохід. Це додасть реалістичності й різноманіття.

6. Як я підготую сценарій для ШІ, який створюватиме гру

Я пропоную не писати один гігантський промпт, у якому змішані всі механіки. Для такого проєкту краще створити єдину дизайн-документацію гри (Game Design Document, GDD) і набір окремих специфікацій, які ШІ зможе реалізовувати послідовно.

Документ 1. Бачення гри та правила світу

Жанр, аудиторія, стиль, основні принципи, що вважається реалістичним, а що заборонено спрощувати.

Документ 2. Ігрові системи та взаємозв'язки

Час, календар, гроші, професії, сім'я, покупки, транспорт, спорядження, погода, маршрути, ремонт, навички й випадкові події.

Документ 3. Дані, економіка та симуляція

Структури даних для речей і транспортних засобів, формули зношення, бюджети, ціни, графіки, залежності систем і правила генерації подій.

Документ 4. Світ і контент

Реальні регіони, кемпінги, маршрути, магазини, ремонтні майстерні, типи доріг, сезонні умови та життєві сценарії.

Документ 5. Технічні завдання для ШІ-розробника

Архітектура, порядок реалізації, критерії приймання, тести й заборона створювати фальшиві функції, які існують лише в інтерфейсі.

Це дасть можливість розробляти гру поетапно: спочатку робочий цикл життя та подорожей, потім спорядження й транспорт, потім світ, економіку, ремонт і більш складні симуляції. Кожна нова система буде інтегруватися з попередніми, а не існувати окремо.

7. Тепер хочу почути саме твою задумку

Ти вже маєш важливі ідеї. Я не хочу забирати в тебе авторство або вигадувати іншу гру замість тієї, яку ти уявляєш. Моя роль — розвинути твою концепцію, знайти прогалини, додати реалістичні механіки та перетворити все на чітке технічне завдання для ШІ.

Відповідай поки лише на одне запитання:

Який саме результат ти хочеш отримати на першому етапі?

1. Що має бути результатом нашої роботи?

Повна концепція гри та великий GDD — від початку життя персонажа до складної симуляції кемперів.
Сценарій першої ігрової версії, яку ШІ зможе відразу почати програмувати.
Спочатку повна концепція, потім покрокова реалізація робочої гри.

2. Який візуальний стиль ти уявляєш?

2D pixel art, як у Stardew Valley.
2D з виглядом зверху, але з більш реалістичними деталями.
3D-світ із реалістичними автомобілями, кемперами та природою.
Поки не вирішив — спочатку механіки та сценарій.

3. Де має працювати гра?

ПК — окрема гра для Windows/Linux/macOS.
У браузері, можливо як продовження мого кемпінг-вебсайту.
Спочатку вебверсія, з перспективою випуску в Steam.
Ще не вирішив.
Продовжити розробку концепції

Можеш також просто написати свою задумку наступним повідомленням, навіть якщо це буде довгий, неструктурований текст. Я розберу його на механіки, покажу, чого бракує, і поступово ми створимо специфікацію, за якою ШІ зможе будувати не просто гарну картинку, а цілісну гру з працюючим світом, економікою та справжнім відчуттям життя в дорозі.

Мої відповіді щодо кемпінг-гри: Результат: Повна концепція та GDD Стиль: 2D pixel art Платформа: Браузер Продовжуй розробку концепції з урахуванням моїх відповідей та допоможи мені доповнити мою власну задумку.

Чудово. Тоді ми розробляємо не просто гру про кемпінг, а повноцінний 2D pixel-art симулятор життя, роботи, подорожей і розвитку власного кемпінгового стилю — браузерну гру з глибокими системами, реальними географічними локаціями та історіями, які виникають із рішень самого гравця.

Я пропоную робочу назву CampLife: The Journey. Це лише кодова назва — її можна буде змінити, коли визначимо характер і унікальність гри.

Нижче починаю формувати першу версію нашого Game Design Document (GDD). Це буде основа, яку ми розширюватимемо твоїми ідеями, дослідженнями реального досвіду мандрівників і конкретними правилами для ШІ-розробника.

GDD v0.1 — концепція гри

games-stats.com
2D Pixel Art
Browser Game

Life Simulation
Camping & Travel
Open-ended Progression

1. Бачення гри

Основна фантазія гравця: жити власним життям, заробляти гроші, планувати відпочинок, відкривати світ, розвивати навички й поступово отримувати більше свободи для подорожей.

Гравець починає зі звичайного життя. У нього є робота або інше джерело доходу, житло, регулярні витрати, певна кількість вільного часу та особисті вподобання. Він може бути одинаком, мати партнера, дітей або подорожувати з друзями.

На початку доступні скромні варіанти відпочинку. Згодом гравець накопичує гроші, купує спорядження, обирає транспорт, навчається ремонтувати речі, досліджує нові регіони й може перетворити хобі на власний бізнес.

Гра не має обов'язкового фіналу. У ній можна провести сотні ігрових днів, змінюючи стиль життя.

Чим наша гра відрізнятиметься від Stardew Valley

| Stardew Valley | Наша гра |
| --- | --- |
| Фермерство та життя в містечку | Робота, побут і подорожі реального типу |
| Ферма — головний актив | Транспорт, спорядження, заощадження та кемпери |
| Основна діяльність прив'язана до господарства | Мандрівки, маршрути, кемпінги та ремонт |
| Фантазійна економіка й світ | Світ, натхненний реальними країнами та правилами |

| Розвиток господарства | Розвиток особистості, спорядження й мобільності |
| --- | --- |

Ми можемо запозичити відчуття затишку, піксельну естетику, приємний темп і задоволення від поступового розвитку, але не копіюватимемо персонажів, контент чи конкретні системи гри.

2. Структура світу та головного ігрового інтерфейсу

Гра повинна мати два взаємопов'язані режими: повсякденне життя та подорож.

Режим повсякденного життя

steambase.io

Тут гравець керує життям персонажа:

- працює, отримує зарплату, шукає іншу роботу або виконує віддалені замовлення;
- оплачує житло, комунальні послуги, харчування, транспорт і сімейні потреби;
- планує вихідні та відпустку;
- переглядає календар, бюджет і прогноз погоди;
- купує, продає, зберігає й ремонтує спорядження;
- готує автомобіль або велосипед до поїздки;
- розвиває навички й відносини з близькими.

Не потрібно змушувати гравця вручну виконувати кожну побутову дію. Наприклад, звичайний робочий день можна симулювати автоматично, а гравець втручається лише тоді, коли має важливе рішення: взяти понаднормову роботу, заощадити на відпочинку або піти раніше, щоб підготувати спорядження.

Режим подорожі

store.playstation.com

Тут гравець безпосередньо досліджує світ, переміщується, облаштовує табір, взаємодіє з предметами й іншими людьми.

Основні екрани:

- карта світу та планувальник маршруту;
- сцена дороги або стежки;
- кемпінг із власним місцем;
- внутрішній простір намету, причепа чи кемпера;
- магазини, майстерні та сервісні станції;
- журнал подорожі з фотографіями, витратами й спогадами.

Перехід між режимами не повинен бути суто декоративним. Якщо гравець не перевірив запас пального вдома, це може стати проблемою в дорозі. Якщо він не висушив намет після повернення, наступного разу спорядження буде в гіршому стані.

3. Створення персонажа

На старті гравець не просто вибирає ім'я. Він визначає життєву ситуацію, яка впливатиме на весь подальший процес.

Приклад початкового профілю

Алекс, 32 роки

Працює механіком у майстерні, живе з партнеркою та 6-річним сином.

- Доходи: стабільна щомісячна зарплата.
- Транспорт: старий універсал.
- Спорядження: базовий рюкзак, ліхтарик, кілька туристичних речей.
- Навички: механіка — початковий професійний рівень; кемпінг — новачок.
- Мета: регулярно їздити на вихідні й поступово придбати кемпер.

Життєві параметри

Фінансова безпека

Середня

Вільний час

Обмежений

Досвід кемпінгу

Низький

Потрібно підтримати різні стартові ситуації:

| Тип персонажа | Перевага | Обмеження |
| --- | --- | --- |
| Одинак із невеликим доходом | Менше обов'язкових витрат на інших людей | Обмежений бюджет, усе робить сам |
| Пара без дітей | Можна розподіляти витрати й обов'язки | Потрібно узгоджувати вподобання |
| Родина з дітьми | Спільні активності та довгострокові цілі | Більше речей, потреб і планування |
| Досвідчений мандрівник | Початкові навички та знання | Може мати старе або спеціалізоване спорядження |
| Віддалений працівник | Може працювати в частині подорожей | Потрібні інтернет, електрика й робочий час |

| | | |
|---|---|---|
| Самозайнята людина | Гнучкіший графік | Нестабільні доходи й замовлення |

Професія, сім'я та характер не повинні блокувати контент. Вони змінюють вартість, складність і доступні можливості.

Наприклад, віддалений працівник може провести два тижні у фургоні, але повинен враховувати мобільний зв'язок, заряджання ноутбука й робочі зустрічі. Механік може заощадити на певних ремонтах, але не обов'язково матиме потрібний інструмент або запчастини.

4. Ігровий час, календар і життєвий баланс

Це одна з найважливіших систем усієї гри.

Я пропоную, щоб один ігровий світ працював за власним календарем, незалежним від реального часу гравця. Тоді людина може відкрити браузер сьогодні, а наступного разу продовжити з того самого ігрового дня.

Початкові налаштування для тестування:

| Параметр | Пропозиція |
|---|---|
| Ігровий рік | 365 днів |
| Тривалість робочого дня | Визначається професією |
| Робочий графік | Понеділок–п'ятниця за замовчуванням |
| Типова коротка подорож | 1–3 ігрові дні |
| Тривала відпустка | 7–21 ігровий день |

| | |
|---|---|
| Пори року | Залежно від регіону |
| Перебіг часу | Прискорений, із паузою під час планування |

Це не остаточні числа — їх потрібно буде перевірити під час тестування.

Час має бути ресурсом нарівні з грошима

Уявімо, що персонаж має €1 200 вільних коштів. Він може дозволити собі спорядження для довгого походу, але якщо працює до п'ятниці ввечері, а місце призначення розташоване за 700 км, він не зможе повноцінно відпочити за одні вихідні.

Тоді з'являються справжні рішення:

- обрати ближчий кемпінг;
- виїхати вночі після роботи;
- взяти додатковий день відпустки;
- розділити маршрут на дві подорожі;
- працювати віддалено;
- накопичити гроші й поїхати на тиждень наступного місяця.

Час у дорозі залежить від виду транспорту, відстані, рельєфу, дорожніх умов і зупинок. Час підготовки — від досвіду, кількості спорядження та його стану.

Важливе правило: гра не повинна карати за те, що гравець не заходив у неї кілька днів реального часу. Поки він офлайн, не повинні раптово накопичуватися борги, псуватися продукти або ламатися обладнання. Симуляція просувається за ігровим календарем, а не за часом, протягом якого вкладка браузера була закрита.

5. Економіка: від першої палатки до власного кемпера

Гроші мають бути не просто числом у кутку екрана. Вони повинні визначати, які можливості гравець може собі дозволити.

Структура особистого бюджету

Доходи:

- зарплата;

- понаднормова робота;
- віддалені замовлення;
- сезонна робота;
- продаж власних речей;
- прибуток від ремонту та перепродажу кемперів;
- інші професійні можливості.

Витрати:

- житло й комунальні послуги;
- харчування та повсякденні потреби;
- транспорт і пальне;
- спорядження;
- бронювання та плата за ночівлю;
- технічне обслуговування;
- ремонти;
- страховки, податки й дозволи там, де вони застосовні;
- кредитні платежі та відсотки.

Фінансовий ризик повинен бути зрозумілим

Наприклад, гравець купує старий кемпер за €8 000. Після огляду виявляється, що потрібні нові шини, ремонт гальм, усунення протікання та часткова заміна електрики.

Він може:

1. Вкласти додаткові гроші й отримати надійніший транспорт.
2. Виконати доступну частину робіт самостійно.
3. Відкласти дорогі апгрейди.
4. Продати кемпер як є, можливо, зі збитком.
5. Взяти кредит, якщо доступний ліміт і платоспроможність це дозволяють.

Прибутковий перепродаж не повинен бути гарантованим. На ціну впливатимуть попит, сезон, стан техніки, якість ремонту, репутація продавця та пропозиції конкурентів.

Банківська система

Для першої версії я пропоную просту, але реалістичну модель:

- дебетовий баланс;
- кредитний ліміт;
- річна відсоткова ставка;
- мінімальний регулярний платіж;

- графік погашення;
- комісії, якщо передбачені умовами кредиту;
- наслідки прострочень;
- можливість дострокового погашення.

Банк не повинен бути нескінченним джерелом грошей. Доступний кредит залежить від доходів, поточних боргів і фінансової історії персонажа.

6. Система спорядження: кожна річ має призначення

У грі не повинно бути просто «слабкого намету» й «сильного намету». Різні речі мають бути корисними в різних ситуаціях.

Наприклад, легкий туристичний намет чудовий для велосипедиста, але не обов'язково найкращий для родини з дитиною, яка проводить тиждень біля озера.

Каталог спорядження

Піший та велосипедний туризм

Намет, рюкзак, спальник, килимок, фільтр для води, аптечка першої допомоги, дощовик, ліхтар, туристичний пальник, ремонтний набір велосипеда.

Головні обмеження: вага, об'єм, витривалість, запас води та залежність від погоди.

Автомобіль і звичайна палатка

Великий намет, матраци, складані меблі, холодильник, кухонне обладнання, тент, ящики для зберігання, зарядні пристрої.

Головні обмеження: місткість багажника, час встановлення табору, паливо та місце для паркування.

Причіп-палатка

Складаний житловий модуль, спальні місця, тент, система зберігання, опори, з'єднання з автомобілем і обладнання для встановлення.

Головні обмеження: сумісність із тягачем, маса, стан зчіпного пристрою та час монтажу.

Кемперван

Ліжко, кухонний блок, холодильник, бак чистої води, помпа, акумулятор, освітлення, вентиляція, опалення, сонячні панелі.

Головні обмеження: обсяг води, ємність акумулятора, технічний стан, вентиляція та обмежений простір.

Великий автодім

Стаціонарні спальні місця, кухня, холодильник, туалет, душ, баки чистої та стічної води, опалення, електросистема, газове обладнання, допоміжні батареї.

Головні обмеження: габарити, маса, витрати на пальне, технічне обслуговування та водійські вимоги конкретної країни.

Кожен предмет матиме характеристики, що впливають на симуляцію:

- маса та займаний об'єм;
- ціна покупки й продажу;
- стан і зношення;
- ремонтопридатність;
- сумісність з іншими предметами;
- споживання енергії або витратних матеріалів;
- комфорт і надійність;
- вплив на підготовку, встановлення табору та час прибирання.

У перспективі гравець зможе порівнювати спорядження за реальними сценаріями. Наприклад, обрати легший спальник для гірського маршруту або тепліший для холодного сезону.

7. Подорожі, кемпінги та непередбачувані ситуації

Це має бути серце гри. Проте події повинні виникати з логіки світу, а не з генератора випадкових неприємностей.

Система бронювання

У кожного кемпінгу будуть:

- кількість місць;
- доступні типи розміщення;
- ціни та правила бронювання;
- сезонний попит;
- доступ до електрики, душу, води та зливу;
- правила щодо тварин, шуму, багаття й транспортних засобів;
- години заїзду та виїзду;
- ризик скасування, помилки бронювання або зміни доступності.

Якщо гравець приїхав і його місце зайняте, він може зв'язатися з адміністрацією, вимагати альтернативу, шукати інший кемпінг або знайти дозволену альтернативу для ночівлі.

Ночівля на стоянці біля супермаркету не повинна бути універсальним рішенням. Гра має перевіряти, чи дозволене там перебування вночі, чи безпечно це та чи можна залишити транспорт.

Приклад системи подій

| Подія | Причина | Можливі наслідки |
|---|---|---|
| Кемпінг переповнений | Високий сезон, помилка бронювання | Інший маршрут, компенсація або додаткові витрати |
| Дощ затопив частину табору | Опади, рельєф, невдале розміщення | Вологе спорядження, прибирання, зміна планів |
| Автомобіль не заводиться | Зношення, несправність або розряджений акумулятор | Діагностика, ремонт, евакуація |

| | | |
|---|---|---|
| Немає води | Недостатній запас або несправна помпа | Пошук водопостачання, зміна маршруту |
| Перекрита дорога | Ремонт, погодні умови, місцеві обмеження | Об'їзд, додатковий час і пальне |
| Сусіди шумлять уночі | Поведінка інших відвідувачів | Розмова, зміна місця або гірший відпочинок |
| Знайомство з мандрівником | Спільний інтерес або випадкова зустріч | Поради, нові контакти, спільна поїздка |

Кожна подія повинна мати умови появи, імовірність, можливі реакції гравця та наслідки. Для небезпечних подій — зрозумілі попередження, якщо вони реалістично могли бути відомі заздалегідь.

8. Ремонт і переобладнання кемперів

Я пропоную зробити цю систему окремою великою гілкою прогресії.

finaloutpostgame.com

Кожен транспортний засіб має технічний паспорт:

- двигун, трансмісія та ходова частина;
- гальма й шини;
- електрична система;
- водопостачання та стоки;
- опалення й вентиляція;
- кузов, ізоляція та захист від вологи;
- житловий модуль і меблі.

Під час купівлі вживаного кемпера гравець бачить не обов'язково всі дефекти. Досвід, огляд, перевірка документів і діагностика допомагають виявляти приховані проблеми.

Далі працює послідовність:

1. Огляд і діагностика.
2. Розрахунок бюджету та пошук деталей.
3. Вибір між власним ремонтом і майстернею.
4. Виконання робіт із витратою часу.
5. Перевірка результату.
6. Експлуатація, подальше обслуговування або продаж.

Навички мають відкривати можливості, але не скасовувати витрат. Навіть механік купує запчастини, а електрик — кабелі, запобіжники, обладнання та інструменти. Складні роботи можуть потребувати спеціального обладнання або кваліфікованої майстерні.

Для великих автодомів система повинна враховувати не тільки масу самого транспортного засобу, а й допустиму повну масу, навантаження на осі, причепи, місцеві правила та відповідну категорію посвідчення водія. Вимоги категорії C не можна універсально застосовувати до всіх великих кемперів: це залежить від країни та характеристик автомобіля.

9. Реальна географія без залежності від інтернету

Тут я бачу важливу перевагу твого наявного кемпінг-вебсайту: його дані потенційно можуть стати основою каталогу місць для гри.

Проте я б розділив дві системи:

Реальна база локацій: країни, дороги, кемпінги, сервіси, водойми, гірські райони та міста.

Ігрова симуляція: ціни, доступність, випадкові події, стан дороги в ігровому світі, потреби персонажа й можливість відвідування.

Для географічних даних можна розглядати OpenStreetMap. Його дані доступні за умовами відкритої ліцензії, але необхідно дотримуватися вимог атрибуції та ліцензування. Стандартні сервери картографічних плит мають окремі обмеження на використання й масове завантаження, тому для комерційного проєкту потрібно вибрати відповідного постачальника або власний сервіс.

OpenStreetMap Foundation
+1

Для погодних систем можна використовувати дані прогнозу й історичної погоди від Open-Meteo. Його історичні набори містять дані про температуру, опади, вітер та інші показники, що можуть допомогти створити регіонально обґрунтовані сезонні моделі.

open-meteo.com

Але я не рекомендую вимагати живих даних для кожної ігрової події. Гра повинна мати локальну базу контенту, а погодні та географічні API використовувати для оновлення, планування й додаткової точності. Якщо сервіс тимчасово недоступний, симуляція має продовжувати працювати.

10. Технічна основа браузерної гри

Для 2D-гри в браузері я пропоную розглянути Phaser із TypeScript. Це фреймворк для браузерних 2D-ігор із підтримкою рендерингу через WebGL і Canvas. Він добре відповідає нашій платформі та піксельному стилю.

Phaser Help
+1

Архітектуру варто розділити на чотири частини:

Ігровий клієнт

Pixel art, анімації, меню, карта, інтерфейс, взаємодія

Симуляційне ядро

Час, гроші, персонажі, інвентар, погода, транспорт, події

Сервер і база даних

Облікові записи, збереження прогресу, перевірка транзакцій, резервні копії

Зовнішні джерела

Географія, кемпінги, погода та інші дані

Якщо гра спочатку буде однокористувацькою, можна почати з локального збереження та простої серверної частини. Але якщо гравець має мати змогу зайти з іншого пристрою й продовжити гру, потрібні обліковий запис і серверне збереження. Важливі покупки, продажі й банківські операції потрібно перевіряти на сервері, а не довіряти браузеру.

11. Що має увійти до першої робочої версії

Ми не будемо намагатися реалізувати весь світ відразу. Потрібно довести до робочого стану невеликий фрагмент гри, у якому вже є повний цикл життя й подорожі.

MVP — перша версія

Рекомендований порядок розробки

Виконано: 0 із 8

Скинути

Створення персонажа

Одинак або родина, професія, стартові гроші, житло та графік роботи

Календар і зарплата

Робочі дні, вихідні, регулярні витрати та планування відпустки

Магазин спорядження

Купівля, продаж, інвентар, вага та місткість

Автомобіль і намет

Пальне, пробіг, базовий стан, встановлення та прибирання табору

Три тестові локації

Недорогий кемпінг, популярне місце біля озера та природна зона з обмеженнями

Перша подорож

Планування, дорога, бронювання, погода, ночівля та повернення

Ремонт і розвиток

Зношення речей, прості поломки, навички та заощадження

Збереження прогресу

Завантаження гри після закриття браузера, відновлення календаря та інвентарю

Критерій готовності MVP простий: гравець повинен мати можливість почати звичайне життя, отримати дохід, підготуватися до поїздки, вирушити в подорож, зіткнутися хоча б з однією ситуацією, що потребує рішення, повернутися, оплатити наслідки та почати планувати наступну поїздку.

12. Принципи, які я пропоную закріпити в GDD

1. Реалізм має створювати цікаві рішення, а не зайву рутину.
2. Жодна професія чи модель транспорту не повинна бути єдиним правильним вибором.
3. Важливі події мають мати зрозумілі причини й наслідки.
4. Фінансові втрати повинні бути відчутними, але гра має пропонувати шляхи відновлення.
5. Погода, транспорт, спорядження й регіональні правила повинні взаємодіяти.
6. Персонаж має розвиватися через досвід, а не лише через накопичення грошей.
7. Гра не повинна вимагати входу щодня за реальним календарем.
8. Основний акцент — пригоди, свобода, затишок і задоволення від поступового розвитку, без хвороб і смертей як центральної механіки.

9. Реальні дані мають бути відокремлені від ігрових припущень, щоб їх можна було оновлювати.
10. Кожна механіка, передана ШІ-розробнику, повинна мати конкретні правила, умови, результати та тести.

3. Що мені потрібно від тебе для наступної версії

Тепер ми маємо основу GDD. Наступним кроком я хочу визначити кілька фундаментальних речей, перш ніж деталізувати сотні предметів, економічні формули та події.

1. Географія світу

Починаємо з Європи, потім додаємо інші континенти.
Одразу закладаємо весь світ, але спочатку деталізуємо Європу.
Вигаданий світ, натхненний реальною географією.

2. Як гравець бачитиме подорож?

Усе в одному вигляді зверху, включно з дорогою й кемпінгом.
Карта для маршруту, а кемпінг і побут — окремі pixel-art сцени.
Бічний вигляд із детальними інтер'єрами кемпера та намету.

3. Наскільки глибоким має бути виживання?

Комфортний кемпінг: лише важливі потреби й небезпеки.
Реалістичне виживання: запаси, температура, вода, погода, навички та ризики.
Обидва режими — гравець обирає складність.

4. Чи потрібні інші реальні гравці?

Тільки одиночна гра з NPC та симуляцією інших мандрівників.
Кооперативні подорожі з друзями.
Спочатку одиночна гра, але архітектура допускає кооператив у майбутньому.

5. Ти зазначав, що не хочеш хвороб і смертей. Який рівень наслідків залишаємо?

Без смертей і серйозних травм — лише фінансові втрати, поломки та невдалі пригоди.
Можливі нефатальні травми, рятувальні операції та вимушене завершення подорожі.
Усе залежить від обраного режиму складності.
Зафіксувати рішення та продовжити GDD

Після цього я зможу перейти до наступного великого розділу: повної економічної моделі, каталогу транспорту й спорядження, системи навичок, формул зношення, алгоритму генерації подій та формату даних для ШІ-розробника. Саме там наша задумка почне перетворюватися з концепції на детальну специфікацію справжньої гри.

Уточнення до GDD кемпінг-гри: Географія: Спочатку Європа Візуальна структура: Єдиний вигляд зверху Виживання: Реалістичне виживання Мультиплеєр: Одиночна гра Наслідки небезпек: Можливі нефатальні травми й рятувальні операції Продовжуй GDD з урахуванням цих рішень.

CampLife: The Journey — GDD v0.2

Частина II. Світ, симуляція, економіка та правила ігрового процесу

Фіксуємо твої рішення як основні вимоги проєкту:

Географія — Європа

Реальні країни, дороги, природні регіони, кемпінги та місцеві правила.

Єдиний вигляд зверху

Персонажі, автомобілі, палатки, кемпери, дороги та природа представлені в єдиному 2D pixel-art світі.

Реалістичне виживання

Вода, харчування, температура, втома, погода, спорядження, навички та безпека.

Одиночна гра

Інші люди представлені NPC зі своїми звичками, планами та поведінкою.

Нефатальні наслідки

Можливі травми, евакуація, рятувальні операції, витрати та дострокове завершення подорожі. Смерті не є частиною ігрової системи.

Ключове рішення для всієї архітектури: це симулятор, де всі системи впливають одна на одну. Погода впливає на дорогу, дорога — на автомобіль і бюджет, бюджет — на спорядження, спорядження — на безпеку, а безпека — на можливість продовжити подорож.

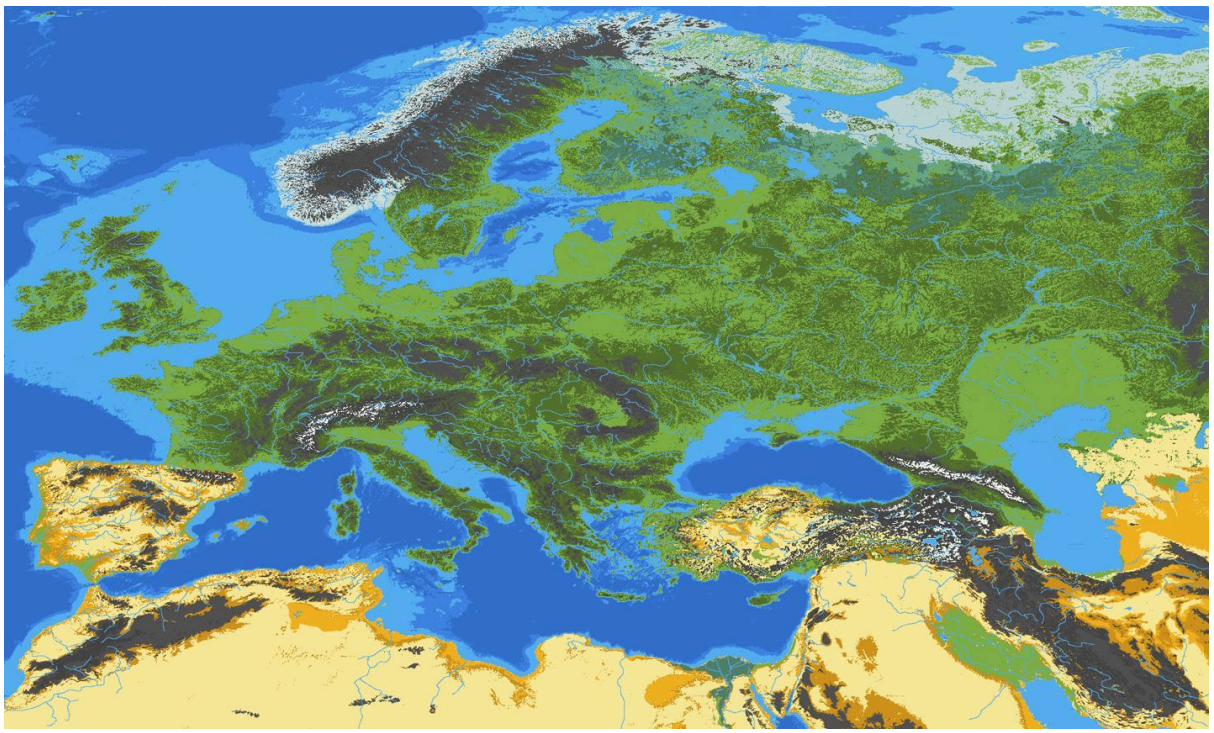
Водночас ми не симулюємо кожен фізичний процес на рівні молекул. Ми створюємо достатньо точну модель, щоб гравець міг приймати правдоподібні рішення.

14. Єдиний світ зверху: як він працюватиме

kauzz.itch.io

Усі активні сцени використовують єдину систему координат і вигляд зверху. Не буде окремого тривимірного режиму для автомобіля або окремого виду збоку для життя у фургоні.

Три масштаби світу



Масштаб 1. Карта Європи

Гравець бачить країни, міста, дороги, природні регіони та точки інтересу. Планує маршрут, порівнює відстані, час у дорозі, платні дороги, пороми, погоду й доступні місця ночівлі.

Масштаб 2. Дорога та маршрут

Гравець бачить автомобіль або персонажа на локальній карті. Дороги мають різні обмеження, покриття, ухили та ризики. Для піших і велосипедних маршрутів діють власні правила швидкості, витривалості й доступності.

Масштаб 3. Локальна сцена

Кемпінг, лісова стоянка, майстерня, заправка, магазин або двір будинку. Тут персонаж ходить, взаємодіє з предметами, ставить намет, підключає обладнання, готує їжу та обслуговує транспорт.

У всіх трьох масштабах використовується той самий напрямок візуального дизайну, але деталізація різна. Не потрібно промальовувати кожне дерево на карті Європи: регіональна карта показує ліс як територію, а локальна сцена — окремі дерева, стежки та місця для табору.

Важливе технічне правило

Не можна перетворювати всю Європу на одну гігантську деталізовану pixel-art карту. Це буде надто дорого для браузера й незручно для навігації.

Натомість:

- глобальна карта використовує спрощені географічні дані;
- дорожній маршрут — граф доріг із вузлами та сегментами;
- локальні сцени — окремі карти з тайлами;
- об'єкти завантажуються за потреби;

- переходи між сценами зберігають позицію, стан персонажа, транспорт і час.

Це дозволить розширювати світ без необхідності перемальовувати його цілком.

15. Реалістична система виживання

Це один із ключових розділів гри. Ми маємо зробити виживання правдоподібним, але не перетворити його на постійне керування шкалами.

Персонаж має фізичні потреби, які змінюються залежно від погоди, активності, спорядження та умов ночівлі.

15.1. Параметри персонажа

| Параметр | Що на нього впливає | Ігровий результат |
|---|---|---|
| Голод | Час, фізичні навантаження, запас їжі | Менше енергії, погіршення самопочуття |
| Спрага | Температура, активність, доступ до води | Падіння працездатності, небезпечне зневоднення |
| Втома | Дорога, ходьба, робота, недосипання | Повільніші дії, гірша концентрація |
| Температурний комфорт | Погода, одяг, спальник, укриття | Дискомфорт і ризик переохолодження чи перегрівання |
| Мокрість | Дощ, бризки, конденсат, мокрий одяг | Втрата теплового комфорту, потреба в сушінні |

| | | |
|---|---|---|
| Стрес | Невизначеність, конфлікти, проблеми | Впливає на настрій та оцінку подорожі |
| Фізична форма | Тренування, відпочинок, активність | Витривалість і швидкість відновлення |
| Травма | Падіння, небезпечна погода, інциденти | Тимчасові обмеження та потреба в допомозі |

Ці параметри не повинні працювати незалежно. Наприклад, втома після складного походу посилюється, якщо персонаж погано спав, мало їв і ніс важкий рюкзак.

Правило температури

Температура тіла не повинна залежати лише від показника повітря. Система враховує:

- температуру та вітер;
- опади й вологість одягу;
- інтенсивність фізичної активності;
- ізоляцію одягу та спальника;
- наявність укриття;
- можливість висушитися й зігрітися.

Звідси виникають цікаві рішення. Дорогий спальник не допоможе повною мірою, якщо він мокрий. Теплий одяг не замінює надійного укриття під час тривалої негоди. А фізична активність, яка зігріває на початку маршруту, може призвести до сильного охолодження під час зупинки.

15.2. Як уникнути нескінченного мікроменеджменту

Я пропоную три стани кожної потреби:

- Норма: усе добре, гравець може займатися іншими справами.
- Попередження: потрібно звернути увагу на проблему.
- Критичний стан: потрібна негайна дія або припинення небезпечної активності.

Голод не повинен вимагати натискати кнопку «Їсти» кожні кілька хвилин. Гравець планує харчування на день, бере запаси, а гра моделює споживання за часом та активністю.

Так само запас води потрібно контролювати перед походом, а не відстежувати кожну краплю вручну.

### 15.3. Система травм та рятувальних операцій

Смерті в грі немає. Натомість небезпечна ситуація може завершитися травмою, допомогою інших людей, евакуацією або втратою частини спорядження.

Приклади:

- розтягнення гомілкостопа на кам'янистій стежці;
- легка травма під час падіння велосипеда;
- перегрівання під час походу в спеку;
- вимушена зупинка через сильний вітер у горах;
- автомобіль застряг на складній дорозі, і потрібна допомога;
- пошкодження спорядження після негоди.

Результат залежить від підготовки, складності місцевості, навичок і швидкості реакції гравця.

Приклад: велосипедна подорож у горах

Персонаж вирушає на складний маршрут без достатнього запасу води. Погода стає спекотною, а велосипедист втомлюється. Гра показує попередження про погіршення стану.

1. Зупинитися, відпочити й оцінити запас води.
2. Скоротити маршрут і спуститися до найближчого населеного пункту.
3. Якщо стан небезпечний — викликати допомогу.

Виклик рятувальників може коштувати грошей, потребувати страхового покриття або спричинити втрату частини відпустки. Персонаж відновлюється, а гравець отримує урок, який впливає на майбутні рішення.

Важливий принцип: гра не повинна приховувати серйозні ризики за випадковим кидком кубика. Якщо небезпека була передбачуваною, гравець має отримати можливість помітити її заздалегідь.

## 16. Система навичок і розвитку персонажа

Навички поділяються на професійні, туристичні, технічні та соціальні.

Кемпінг і побут

Встановлення палатки, організація табору, приготування їжі, пакування, догляд за спорядженням.

Механіка та електрика

Діагностика, обслуговування автомобіля, ремонт велосипеда, електросистеми кемпера, встановлення обладнання.

Орієнтування та безпека

Навігація, планування маршрутів, оцінка погоди, організація запасів, реагування на небезпеки.

Робота та фінанси

Віддалена робота, переговори, пошук вигідних покупок, оцінювання вартості вживаного обладнання, керування власною справою.

16.1. Як навички прокачуються

Досвід нараховується за успішно виконані дії, навчання та практику. Повторення однієї простої дії не повинно безмежно підвищувати рівень.

Наприклад:

- перша самостійна установка палатки дає значний навчальний ефект;
- наступні установки поступово скорочують час;
- складні умови дають більше досвіду, якщо дія завершилася успішно;
- помилки можуть відкривати нові знання, але не повинні винагороджувати небезпечну поведінку;
- деякі спеціалізовані роботи потребують навчання, інструментів або сертифікації.

Професія визначає стартові навички, а не остаточні можливості персонажа.

Механік може навчитися працювати з сонячними панелями, але спочатку робитиме це повільніше за досвідченого електрика. IT-фахівець може розвинути навички ремонту електроніки, а досвідчений турист — краще оцінювати ризики на маршруті.

### 16.2. Система втоми від дій

Навички не повинні скасовувати фізичні обмеження. Навіть досвідчений мандрівник потребує відпочинку, а професійний механік не може безкінечно ремонтувати автомобіль без перерв.

Тому кожна дія матиме:

- базовий час виконання;
- необхідний рівень навички;
- потрібні інструменти;
- витрату енергії;
- імовірність помилки;
- вплив стану персонажа.

Це дозволить будувати цікаві рішення: відремонтувати несправність самостійно, витративши пів дня, або заплатити майстерні й продовжити відпустку.

## 17. Економічна модель: правила, за якими працюватиме світ

Усі ціни нижче — початкові проєктні припущення в євро, а не твердження про поточні ринкові ціни. Їх потрібно буде калібрувати на основі конкретних країн, років, типів обладнання та реальних пропозицій.

### 17.1. Бюджет подорожі

Перед виїздом гравець бачить попередній кошторис:

$$B_{trip} = C_{fuel} + C_{stay} + C_{food} + C_{road} + C_{other}$$

Де:

- $C_{fuel}$ — пальне;
- $C_{stay}$ — ночівлі;
- $C_{food}$ — харчування;
- $C_{road}$ — платні дороги, пороми та парковки;
- $C_{other}$ — інші заплановані витрати.

Вартість пального оцінюється за відстанню, витратою автомобіля та ціною пального в регіоні:

$$C_{fuel} = \frac{D \cdot F}{100} \cdot P$$

Тут $D$ — відстань у кілометрах, $F$ — витрата в літрах на 100 км, $P$ — ціна одного літра.

Наприклад, якщо автомобіль витрачає 7 літрів на 100 км, маршрут становить 400 км, а пальне коштує €1,80 за літр, прогнозована вартість становить €50,40. Це лише ілюстрація формули.

Реальні витрати можуть бути вищими через затори, завантаження автомобіля, гори, об'їзди та роботу двигуна на холостому ходу.

17.2. Фінансова безпека

Гра повинна показувати не тільки поточний баланс, а й те, скільки грошей залишиться після обов'язкових витрат.

$$B_{free} = B_{cash} - C_{essential} - C_{committed}$$

Де $B_{free}$ — вільний бюджет, $B_{cash}$ — наявні гроші, $C_{essential}$ — необхідні побутові витрати, а $C_{committed}$ — уже заплановані платежі.

Перед дорогою гравець може отримати повідомлення:

> Після оплати пального, кемпінгу та харчування у вас залишиться €85. Ремонт автомобіля в цю суму не входить. Рекомендується залишити резерв.

Це не заборона їхати. Це інформація, на основі якої гравець ухвалює рішення.

17.3. Динамічний ринок вживаних кемперів

Вартість транспортного засобу визначається не одним випадковим числом.

На неї впливають:

- вік, пробіг і технічний стан;
- історія обслуговування;
- вологість і пошкодження кузова;
- стан житлового обладнання;
- якість переобладнання;

- сезонний попит;
- регіональний ринок;
- документально підтверджені роботи.

Для внутрішнього моделювання можна почати з формули:

$$V_{market} = V_{base} \cdot K_{age} \cdot K_{condition} \cdot K_{market}$$

Коефіцієнти тут — внутрішні параметри симуляції, які треба калібрувати, а не універсальні реальні закони ціноутворення.

Після оцінювання ринку гравець виставляє свою ціну. NPC покупці мають власні бюджети й вимоги, тому пропозиції можуть бути нижчими за очікування продавця.

17.4. Вартість часу

Усі великі дії повинні мати ціну в часі.

| Дія | Що визначає тривалість |
|---|---|
| Зібрати намет | Модель намету, навички, погода |
| Підготувати автомобіль | Стан, перелік перевірок, досвід |
| Приготувати їжу | Рецепт, обладнання, підготовка продуктів |
| Відремонтувати велосипед | Тип поломки, інструменти, навички |
| Переобладнати фургон | Обсяг робіт, деталі, досвід |
| Знайти інше місце ночівлі | Доступність, зв'язок, відстань |

Це створює компроміс між грошима та часом. Гравець може виконати роботу самостійно, але витратити на неї частину відпустки.

18. NPC та поведінка інших людей

Для реалістичного світу недостатньо випадкових персонажів, які стоять біля наметів і повторюють однакові фрази.

NPC повинні мати власні цілі, потреби, розклад і поведінку.

Інші туристи

Родини з дітьми, пари, одиночні мандрівники, велосипедисти, досвідчені кемпери. Вони мають різні бюджети, рівні підготовки, звички й потреби.

Працівники кемпінгів та сервісів

Адміністратори, механіки, продавці, працівники заправок. Їхня доступність залежить від графіка роботи, місця розташування та сезону.

Випадкові знайомства

Інші люди можуть поділитися маршрутом, порадити сервіс, запропонувати запчастину, запросити на спільну прогулянку або розповісти про приховану локацію.

Як працюватиме поведінка NPC

Кожен NPC отримує:

- поточну мету;
- розклад;
- бюджет;
- спорядження та транспорт;
- особисті вподобання;
- ставлення до ризику;
- набір можливих дій.

Наприклад, сім'я бронює місце біля озера, але через дощ змінює плани. Інші туристи можуть виїхати раніше, звільнивши місце. Адміністратор може запропонувати альтернативний майданчик. Механік може бути зайнятий до наступного ранку.

Такі події повинні виникати з поведінки світу, а не тільки з написаних вручну сценаріїв.

Для браузерної гри не потрібно симулювати кожного туриста в Європі постійно. Далекі NPC працюють у спрощеному режимі, а детальна поведінка активується поблизу гравця.

19. Технічна специфікація для ШІ-розробника

Тепер важливо перетворити концепцію на структуру даних, яку можна реалізувати в коді.

Я пропоную використовувати TypeScript для ігрової логіки, Phaser для відображення 2D-світу та окрему систему збереження. Phaser підтримує тайлові карти й поділ гри на сцени, що відповідає нашій архітектурі.

Phaser Help
+1

19.1. Основні сутності

| Сутність | Що зберігає |
| --- | --- |
| `PlayerCharacter` | Особисті дані персонажа, параметри, навички |
| `Household` | Партнер, діти, сімейні потреби та бюджет |
| `Employment` | Професія, графік, дохід, відпустка |
| `BankAccount` | Баланс, кредит, платежі, відсотки |
| `Inventory` | Предмети, кількість, стан і місце зберігання |
| `Vehicle` | Транспорт, пробіг, стан вузлів, обладнання |
| `Trip` | Маршрут, бюджет, дати, учасники, статус |
| `Location` | Географія, тип місця, правила, доступні послуги |
| `WeatherState` | Поточні умови, прогноз, сезонні характеристики |
| `Incident` | Подія, причина, наслідки, варіанти реакції |

| | |
|---|---|
| `SkillProfile` | Рівні навичок, досвід і розблоковані можливості |
| `WorldCalendar` | Дата, час, сезон, розклад симуляції |

19.2. Приклад формату даних спорядження

Нижче — приклад структури, а не готовий ігровий каталог.

```
type CampingItem = {
  id: string;
  name: string;
  category: string;

  purchasePrice: number;
  resaleValue: number;

  weightKg: number;
  volumeLiters: number;

  condition: number;
  durability: number;

  waterproofRating?: number;
  temperatureRatingC?: number;

  compatibleWith: string[];
  requiredSkills: string[];
};
```

Ця структура дозволить описати предмети різного призначення: від легкого намету до холодильника, газового пальника чи акумулятора.

Однак реальні предмети потребуватимуть різних характеристик. Наприклад, для акумулятора важлива ємність у ват-годинах, для пальника — витрата газу, для намету — місткість, водостійкість і час встановлення.

Тому в наступній технічній ітерації характеристики потрібно буде розділити на типізовані набори, а не залишати всі предмети з однаковими полями.

19.3. Приклад правил для ігрової події

```
type IncidentDefinition = {
  id: string;
  category: "weather" | "vehicle" | "camping" | "social";

  preconditions: string[];
  probabilityRules: string[];

  warningAvailable: boolean;
  severity: "minor" | "moderate" | "serious";

  possibleActions: string[];
  consequences: string[];
};
```

Подія «автомобіль не заводиться» може перевіряти заряд акумулятора, стан стартера, температуру, історію обслуговування та інші чинники.

Генератор подій не повинен вигадувати довільні несправності без причин. Він має працювати на основі стану транспортного засобу й умов експлуатації.

### 19.4. Збереження світу

Збереження повинно включати не тільки гроші та інвентар, а й:

- дату та час;
- позицію персонажа;
- активний маршрут;
- стан транспортних засобів;
- заброньовані місця;
- поточні події;
- стан спорядження;
- борги й платежі;
- прогрес навичок;
- важливі рішення та історію подорожей.

Система збереження повинна мати версію формату. Це потрібно, щоб майбутні оновлення гри могли додавати нові характеристики без втрати старих збережень.

Для браузера також необхідні автоматичне збереження, обробка помилок і можливість відновлення після аварійного закриття вкладки.

## 20. Дослідження реального життя: як перевірятимемо достовірність

Щоб гра справді відчувалася життєвою, нам потрібен не лише список механік, а й процес перевірки того, чи відповідають вони реальному досвіду.

Пропоную створити базу життєвих ситуацій. Кожен запис описує реальну проблему, її причини, способи розв'язання та ігрову реалізацію.

Наприклад:

Приклад запису в базі ситуацій

Технічна проблема

Ситуація: у кемпері перестала працювати водяна помпа.

Можливі причини: розряджений акумулятор, запобіжник, несправність помпи, порожній бак або проблема з проводкою.

Що робить реальний мандрівник: перевіряє рівень води, живлення, запобіжник і доступні інструкції; за потреби звертається до сервісу.

Ігрові умови: стан обладнання, навички електрики, наявність інструментів і запчастин.

Варіанти гравця: діагностика, ремонт, купівля деталі, звернення до майстерні або тимчасове використання альтернативного джерела води.

Наслідки: витрата часу, грошей і запасів води; можливість навчитися ремонту.

Джерела для перевірки: посібники виробників обладнання, технічні інструкції та реальні історії власників кемперів.

Для географії та погодних умов можна використовувати відкриті дані з відповідними ліцензіями. Наприклад, OpenStreetMap надає географічні дані, але їхнє публічне використання вимагає дотримання умов ODbL та правил атрибуції.

OpenStreetMap Wiki
+1

Історичні погодні дані Open-Meteo можуть допомогти моделювати сезонні умови, але їх не слід сприймати як гарантію того, що конкретна подія відбудеться в реальному майбутньому.

open-meteo.com
+1

Для кожної важливої механіки варто вести такі поля:

- джерело та дата перевірки;
- регіон, до якого стосується інформація;
- підтверджені факти;
- припущення для спрощення;
- формула або правила симуляції;
- тестові сценарії;
- критерії правдоподібності.

Це особливо важливо для правил ночівлі, водійських категорій, страховок, рятувальних служб і природних небезпек, оскільки вони відрізняються між країнами.

21. План створення повної гри

Я б розділив розробку на шість великих етапів.

01

Завершення GDD

Зафіксувати механіки, економіку, типи транспорту, правила світу, основні сценарії та критерії якості.

Результат: повна концепція та специфікація.

02

Прототип симуляції

Календар, персонаж, робота, зарплата, бюджет, інвентар і перша подорож.

Результат: гра вже має працюючий цикл життя.

03

Світ і pixel-art

Рух персонажа, карти зверху, кемпінги, транспорт, взаємодія з предметами та NPC.

Результат: світ стає візуально інтерактивним.

04

Реалізм та пригоди

Погода, виживання, бронювання, поломки, травми, ремонти й наслідки рішень.

Результат: подорожі стають системною симуляцією.

05

Розширення світу

Реальні регіони Європи, різні типи кемпінгів, нові транспортні засоби, магазини й майстерні.

Результат: більше маршрутів та стилів гри.

06

Баланс і полірування

Перевірка економіки, виправлення помилок, оптимізація браузерної продуктивності, балансування складності та зручності.

Результат: стабільна й захоплива гра.

22. Який розділ GDD розробляємо наступним

Зараз ми вже визначили загальну архітектуру світу, виживання, навичок, економіки та технічних даних. Наступний великий крок — перейти до детальної специфікації контенту, щоб кожна система мала конкретні характеристики та правила.

Я рекомендую наступним створити повний каталог транспорту й спорядження: від туристичного рюкзака та бюджетної палатки до причепів, кемперванів і великих автодомів. Для кожної категорії визначимо початкові ціни, місткість, масу, сумісність, термін служби, ремонт, апгрейди, витрати на експлуатацію й умови відкриття.

Після цього ми зможемо пов'язати каталог із бюджетом, навичками, професіями, сезонністю та першими ігровими сценаріями.

Але є ще одна важлива річ, яку я хочу з'ясувати перед тим, як зафіксувати структуру предметів:

Ти хочеш, щоб гравець міг вільно переміщатися по всій локальній карті та вручну виконувати дії — наприклад, підійти до намету, взяти предмет, встановити його, підійти до автомобіля та завантажити речі — чи основна взаємодія має відбуватися через меню та інвентар, а вільне пересування буде лише частиною гри?

Це рішення суттєво вплине на архітектуру браузерної гри, кількість необхідних анімацій і те, як ми реалізуємо симуляцію побуту.

Але хочу одразу правильно визначити підхід: неможливо буквально прочитати весь інтернет. Натомість можна систематично зібрати репрезентативні історії, типові проблеми, практичні рекомендації та повторювані життєві ситуації з форумів, блогів, спільнот кемперів і велосипедистів, посібників з обслуговування техніки та офіційних джерел про погоду, дороги й кемпінги.

- саме так, досліди і детально розпиши сценарії які можливі бо гра багатошарова і не має бути сумною і "плоскою за сценарієм", натоміст має мати багато пригод, в нас є справжні данні про реальні авто і обладнання і кемпери і кемпенги по всій ЄС а також дороги, це все і тобі доступно тому пиши сценарій та сценарії і події і має завжди бути вибір якийсь, як в житті, обрав вело то тобі і вага важлива і нахил і то і інше впливає на якість відпочинку і дороги а також на авто не скрізь можна проїхати, тому справжні пригоди які можуть обернутись розвилкою в ігровому світі, я повернув не туди і проїхав поворот і тут зайва витрата часу і пального і я запізнився на паром і все, тепер -1 день відпочинку і додаткові витрати, таких пригод може бути багато і погода теж впливає і стан авто бо ізнос є і не якісно підготовлений транспорт чи продукти чи не спланував маршрут і немає чи питної води чи фікалії переповнені і воняє в авто))) тоді йдуть мінус бали чи карма страждає, тобто гра вчить як бути в кемпінг відпочинку і готує людей і вчить правильно планувати свій відпочинок в залежності від обраного персонажу і авто і пригод і подорожі.

Я б не просто виписував історії. Я б перетворював кожну повторювану ситуацію на ігрову механіку.

- саме так і треба

Початкові умови: персонаж працює п'ять днів на тиждень, має партнерку та сина, володіє старим легковим автомобілем.

- а може є і собака! і не всі кемпінги приймають з тваринами!!!

І найголовніше: не кожна проблема повинна карати гравця. Деякі труднощі можуть створювати кумедні історії, знайомства, нові навички або альтернативні пригоди.

- саме так, гра не карає, вона затягує своєю реалістичністю подій і має дарувати радість і щастя, люди можуть будувати справжні подорожі і гра як мотиватор і планувальник, то люди будуть мати уявлення як це жити в подорожі і отримувати задоволення, тобто провів відпустку = скіли вгору, здоров'є і карма вгору а поки працюєш і готуєшся то втома від життя і роботи і карма і здоров' просідає і то є мотиватор їхати раз на рік чи десять, тут вже гравець має сам вирішувати який сценарій по його нраву. Але точно пару разів на рік треба мати подорожі не менше ніж по пару тижнів, як в справжньому житті, є дитина = вихідні і праздники і шкільні канікули тільки доступні і якщо він працює то теж 2 рази на рік по 2 тижні має відпустку + праздничні і вихідні може мати для подорожей, але якщо лбдина обрала ІТ чи когось хто персонаж має можливість працювати віддалено то він може хоч жити в фургоні, але в нього менше тоді буде грошей бо жити в фургоні значно дорожче за правилами життя.

Тут є тонкий момент. Ти хочеш реалізму, але якщо гравець повинен щодня оплачувати кожен бутерброд, вручну перевіряти кожен болт і дивитися, як десять хвилин ігрового часу нічого не відбувається, гра перетвориться на роботу.

- ні, в жодному разі гра не має стати роботою, гра відпочинок мозоку так би мовити і має затягувати в ігровий процесс, тобто вчити і затягувати, все як в іграх має бути, бо від класного сценарія і виконнаня є ігри від яких неможливо відірватися і ось саме ти маєш написати крутий сценарій і продумати всі тонкощі і нюанси, тому твори!!!

3. Де має працювати гра?

- тут насправді я б хотів в браузері і потім перенести на ПК якщо гра зайде людям.

Ти хочеш, щоб гравець міг вільно переміщатися по всій локальній карті та вручну виконувати дії — наприклад, підійти до намету, взяти предмет, встановити його, підійти до автомобіля та завантажити речі — чи основна взаємодія має відбуватися через меню та інвентар, а вільне пересування буде лише частиною гри?

- тут буде вільне переміщення але і фізіка теж, але в межах локації, магазин, чи табір чи фургон, не треба переускладнювати але треба міссії і цілі які люди будуть досягати в ігрі і локаціях. Тому треба в сценарії додати цікавих моментів

Продумай і пиши сценарій.

CampLife: The Journey

Великий сценарний документ — від буднів до пригод, які хочеться переживати знову

Я бачу цю гру не просто як симулятор кемпінгу, а як гру про свободу, маленькі перемоги, сім'ю, пригоди та мистецтво подорожувати. Гравець починає звичайне життя, мріє про наступну мандрівку, готується до неї, долає несподівані ситуації, відкриває красиві місця й повертається додому не лише з фотографіями, а й із досвідом, навичками та бажанням вирушити знову.

Головний принцип: реалістичні правила створюють пригоди

ок, то ти можеш написати сценарій? повний із персонажами і всіма локаціями і пригодами і міссіями? Прямо все продумати і розписати логічно всі ці події?

... знайти безпечне рішення, купити запас, скоригувати маршрут або познайомитися з іншими мандрівниками. А якщо підготувався добре — отримаєш більше свободи, спокою та можливостей.

Так, я можу розробити повний сценарій гри — з персонажами, родинами, локаціями, місіями, подіями, розгалуженнями, наслідками та логікою їх виникнення. Але це має бути не одна лінійна історія на кшталт «поїхав — зламався — відремонтувався». Це має бути система, яка здатна породжувати тисячі різних історій залежно від рішень гравця.

Тому я будуватиму сценарій на трьох рівнях:

- Авторські історії: персонажі, сюжетні арки, цікаві знайомства, сімейні ситуації, особливі завдання та пам'ятні пригоди.
- Системні пригоди: погода, стан транспорту, вага багажу, рельєф, вода, харчування, бронювання, дороги, пороми, тварини та гроші. Події виникають через конкретні умови, а не просто випадково.
- Вільна подорож: гравець сам обирає транспорт, напрямок, стиль відпочинку, ризик, бюджет і темп. Гра реагує на цей вибір, але не змушує проходити одну й ту саму історію.

Нижче починаю створювати сценарну біблію CampLife: The Journey — документ, на основі якого згодом можна розробляти гру, а не лише обговорювати її ідею.

1. Основа світу та сценарію

store.epicgames.com

Візуальний напрям: затишна піксельна Європа, невеликі живі локації, впізнавані предмети й пригоди, що виникають із повсякденних рішень.

Робоча назва: CampLife: The Journey Жанр: пригодницький симулятор життя, подорожей і кемпінгу. Формат: одиночна браузерна гра з перспективою перенесення на ПК. Візуальний стиль: 2D pixel art, єдина перспектива зверху. Структура: вільне пересування в невеликих локаціях, а великі подорожі — на карті доріг і маршрутів.

Центральна ідея

Гравець не проходить одну історію від початку до кінця. Він створює власну історію життя мандрівника.

Один гравець може жити в місті, працювати в офісі, мати дружину, дитину, собаку й старе авто. Інший — бути самотнім велосипедистом, який подорожує з наметом. Третій — IT-фахівцем на віддаленій роботі, який живе у фургоні. Четвертий — механіком, що купує старі кемпери, ремонтує їх і продає.

Усі вони живуть в одному світі. Їхні історії відрізняються не тільки діалогами, а й тим, які можливості їм доступні.

П'ять цілей, які утримують інтерес

1. Мрія: побачити нові місця, купити кращий транспорт, навчити дитину плавати, пройти гірський маршрут, зустріти світанок біля моря.
2. Підготовка: заробити гроші, зібрати спорядження, вибрати маршрут, врахувати погоду, забронювати місце, підготувати домашніх і тварин.
3. Пригода: зіткнутися з несподіванкою, знайти альтернативу, познайомитися з людьми, відкрити приховану можливість.
4. Нагорода: відпочити, отримати нові навички, покращити здоров'я й настрій, зміцнити стосунки, відкрити нові маршрути.
5. Нова мрія: після повернення гравець уже знає більше, має нові цілі й починає планувати наступну подорож.

Це не система, де треба постійно перемагати. Це система, де хочеться побачити, що буде, якщо наступного разу підготуватися інакше.

2. Як працює життя персонажа

Гра починається не з намету, а з життя персонажа. Саме його повсякденність визначає, які пригоди він зможе пережити.

Стартові архетипи

1. Родина вихідного дня

Партнерка або партнер, дитина, собака, звичайна робота й уживаний легковик. Найдоступніший старт.

Сильні сторони: сімейні бонуси, спільні спогади, прості короткі подорожі. Виклики: шкільні канікули, дитячий багаж, дозволені для собак кемпінги та обмежений бюджет.

2. Веломандрівник

Самостійний персонаж або пара, велосипед, сумки й намет.

Сильні сторони: дешеве пересування, вузькі дороги, доступ до маршрутів, недоступних автомобілям. Виклики: вага, підйоми, вітер, дощ, втома, проколи та пошук місця для ночівлі.

3. Віддалений працівник

IT-фахівець або інша людина, чия робота дозволяє працювати дистанційно. Починає з автомобіля чи невеликого фургона.

Сильні сторони: гнучкий календар і довгі подорожі. Виклики: інтернет, електрика, тиша для дзвінків, робочі дедлайни, витрати на стоянки та обслуговування житла на колесах.

4. Майстер-мандрівник

Механік, електрик або універсал із практичними навичками.

Сильні сторони: може самостійно діагностувати несправності, економити на ремонті й відновлювати старі кемпери. Виклики: вартість деталей, час, приховані дефекти та ризик невигідної покупки.

Це не чотири окремі кампанії. Це різні початкові умови для одного великого світу. У процесі гри велосипедист може придбати авто, родина — пересісти у фургон, а віддалений працівник — змінити професію.

Родина та домашні обов'язки

Якщо в персонажа є дитина, календар враховує навчання, вихідні, свята й шкільні канікули. Звичайна сімейна відпустка планується з урахуванням роботи дорослих та доступних днів відпочинку.

Якщо персонаж працює за наймом, він має оплачувані відпустки відповідно до обраного ігрового профілю. Довга подорож посеред робочого тижня може вимагати відпустки або коштувати втраченого заробітку. Самозайнятий і віддалений працівник мають більше свободи, але не нескінченні гроші й не безмежний вільний час.

Після кількох добре спланованих подорожей гравець має відчувати різницю: робота дає ресурси для життя, а подорожі допомагають відновлюватися після роботи.

Не вводимо обов'язковий лічильник «життя стало поганим». Натомість тривала перевтома поступово знижує настрій, мотивацію та продуктивність. Відпочинок, природа, сон, активність і спілкування допомагають відновлюватися.

Собака — повноцінний учасник пригод

Собака не просто косметичний супутник. У нього є характер, витривалість, звички, потреба у воді, їжі, прогулянках і відпочинку.

Можливі особливості:

- собака любить плавати, але боїться грози;
- може загавкати вночі й розбудити сусідів;
- знаходить стежку або привертає увагу до дикої тварини;
- втомлюється раніше за господаря на спекотному маршруті;
- може не дозволяти комфортно залишатися в деяких локаціях;
- потребує повідця, прибирання за собою та безпечного місця в транспорті.

Перед бронюванням гравець бачить, чи приймає кемпінг собак, чи є обмеження, додаткова плата або спеціальні зони. Це не вигадане обмеження: правила щодо тварин дійсно відрізняються між кемпінгами, а деякі майданчики мають окремі зони для гостей із собаками.

Euro Camping
+1

Важливо: гра не повинна змушувати гравця відмовлятися від собаки. Вона дає йому змогу планувати подорож, у якій комфортно всім.

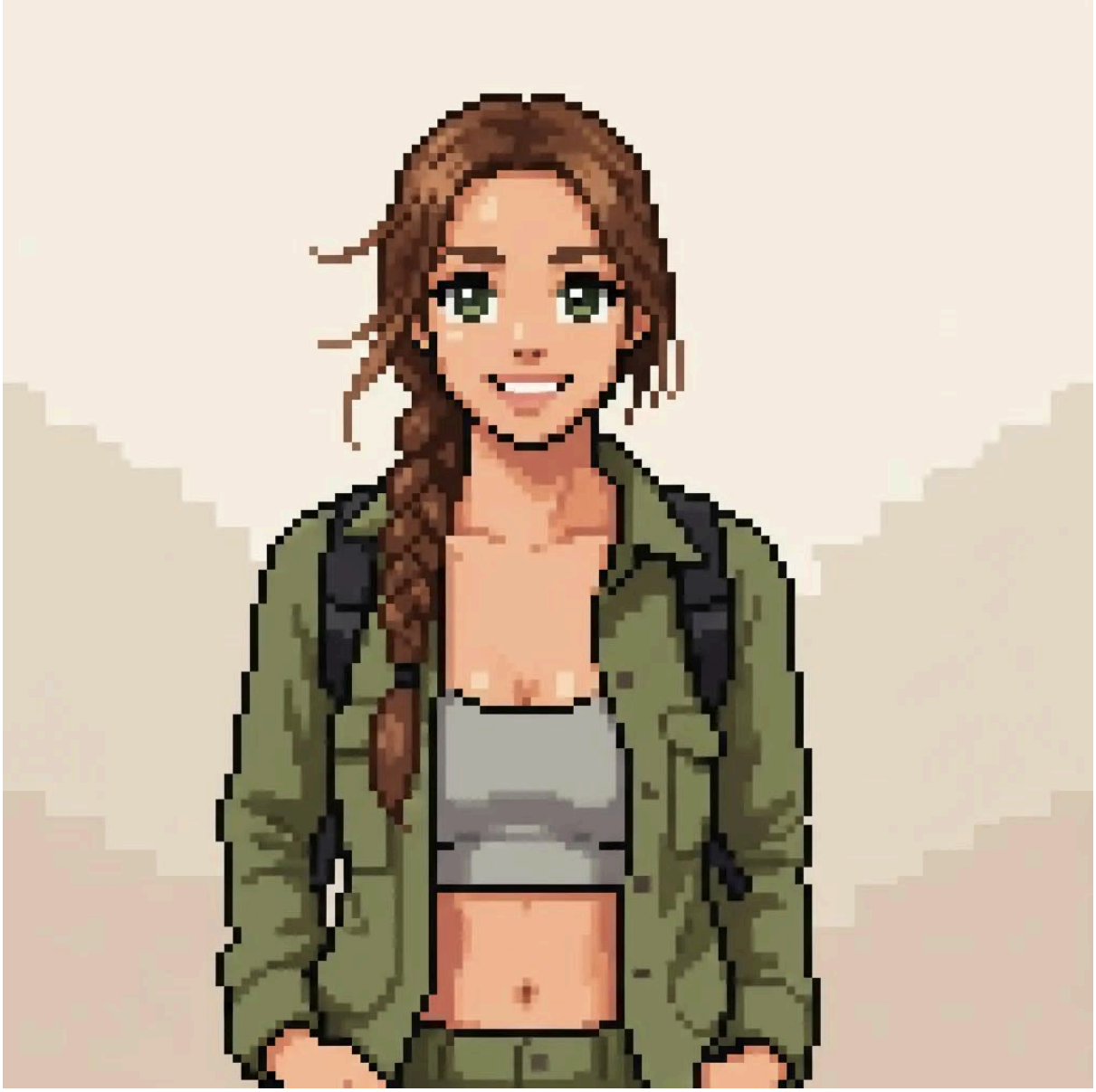
3. Основні персонажі першої сюжетної кампанії

Перша кампанія називається «Перша справжня відпустка». Вона знайомить гравця з головними системами, але не закриває світ після завершення.

Алекс — головний герой за замовчуванням

34 роки, працює п'ять днів на тиждень. Має старий сімейний автомобіль. Хоче показати дитині, що світ значно більший за школу, роботу й торговий центр.

Особиста ціль: уперше організувати самостійну двотижневу подорож без туристичного пакета.



Міра — партнерка

Практична, любить природу, але не хоче перетворювати відпустку на нескінченний список справ. Вона може підтримувати економний варіант із наметом або переконати гравця заплатити за зручніше житло.

Особиста ціль: хоча б кілька днів не поспішати й не вирішувати робочі проблеми.

Лео — син

Допитливий, любить карти, мости, потяги та комах. Не завжди хоче йти туди, куди вирішили дорослі.

Особиста ціль: побачити справжній водоспад і знайти п'ять різних видів метеликів. Якщо дорослі включать його цілі в маршрут, сімейний настрій покращуватиметься.

Бруно — сімейний собака

Енергійний, дружній, іноді впертий. Має власні потреби й характер. У дощ любить калюжі, а після прогулянки може занести бруд просто в чистий фургон.

Особиста ціль: не сюжетне завдання, а система поведінки — прогулянки, гра, спокій, вода, безпечне перевезення та довіра до господарів.

Інгрід — досвідчена мандрівниця

Живе в невеликому кемпері частину року. Не видає гравцю готову інструкцію, а ставить запитання: «Ти перевірив прогноз на перевалі?» або «А де плануєш злити сіру воду?»

Роль у грі: наставниця, що відкриває навчальні місії, корисні звички й альтернативні способи вирішувати проблеми.

Томас — механік

Власник маленької майстерні. Не ремонтує автомобіль безкоштовно, але пояснює, за що саме бере гроші. Якщо гравець виявляє інтерес, може навчити базової діагностики.

Роль у грі: транспорт, технічний стан, запчастини, ремонт, купівля вживаних кемперів.

Ноа — адміністратор кемпінгу

Знає місцеві маршрути, завантаженість, правила тиші, місця для собак і найближчі сервісні точки. Іноді має вільне місце, коли онлайн-бронювання вже закрите, але гарантій немає.

Роль у грі: знайомство з правилами кемпінгу, бронюваннями, людьми та локальними подіями.

Ці персонажі — ядро першої історії. Пізніше до них додаються велосипедисти, сезонні працівники, інші родини, власники кемперів, рятувальники, майстри, фермери та мандрівники з різних країн.

Ключове правило: NPC повинні мати власні плани. Інгрід може виїхати раніше через погоду, Ноа може закінчити зміну, а велосипедист, з яким гравець познайомився в перший день, — несподівано зустрітися з ним через кілька тижнів в іншій країні.

4. Географія: перший ігровий регіон

Я пропоную почати не з усієї Європи одночасно, а з повноцінного регіону, який можна дослідити до дрібниць, а потім розширювати світ.

Перший регіон — Бельгія, Арденни, долина Маасу та маршрути в напрямку Люксембургу, Нідерландів і Франції. Це зручна стартова зона: є міста, ліси,

пагорби, річки, веломаршрути, кемпінги, дороги різних типів і транспортні розв'язки.

Це ігрові локації, натхненні реальними регіонами, а не обіцянка точного відтворення кожного будинку чи дороги. Реальні дороги, кемпінги, обмеження та погодні дані можна поступово підключати до симуляції. Наприклад, EuroVelo має 17 офіційних далеких веломаршрутів, а Open-Meteo надає історичні погодні дані для моделювання типових сезонних умов.

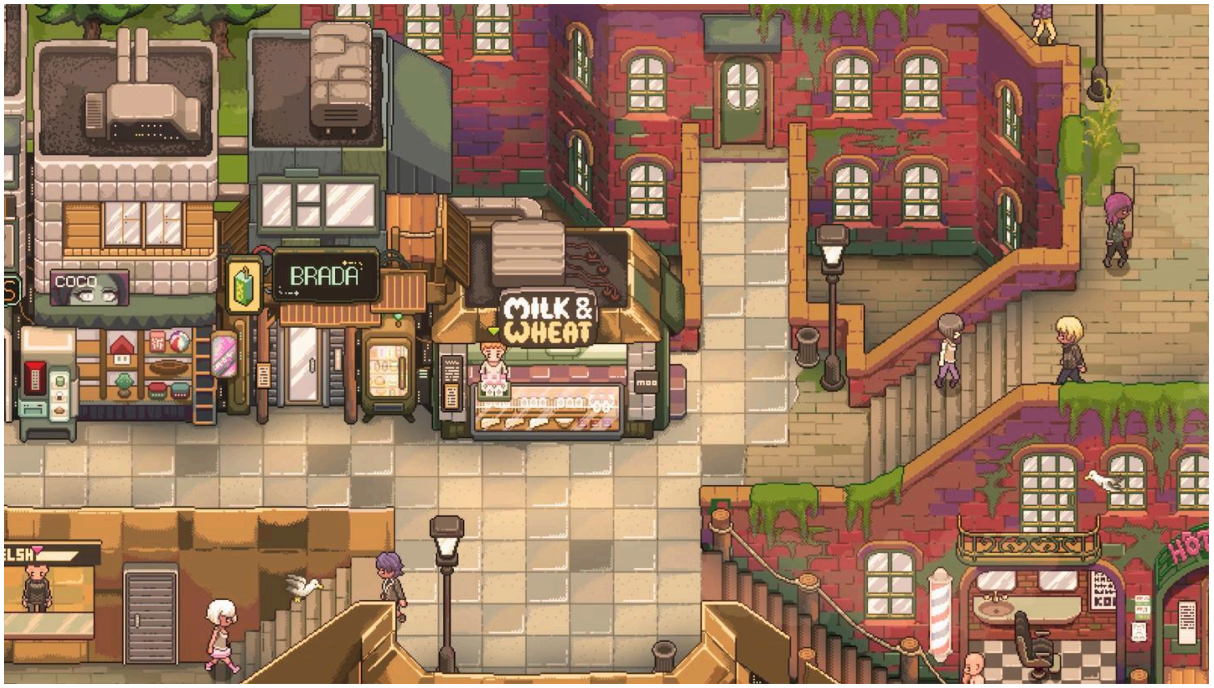
EuroVelo
+1
4,8
4,1
4,1
4,9
Use two fingers to move the map
Use ⌘ + scroll to zoom the map
АнтверпенTourist attraction
Намюр4,8•Haute French restaurantОткрыто
Дінан4,1•Tourist attractionОткрыто
Сен-Юбер4,1•Catholic church
Люксембург4,9•Historical landmark

Основні локації та їхні функції

Локація 01. Дім

Стартова база персонажа. Тут можна ходити між кімнатами, взаємодіяти з речами, складати спорядження, перевіряти календар, готувати їжу, прати одяг і ремонтувати дрібні предмети.

Місії: «Перший бюджет», «Що беремо із собою?», «Знайди старий ліхтар», «Склади список для Бруно».



Локація 02. Містечко

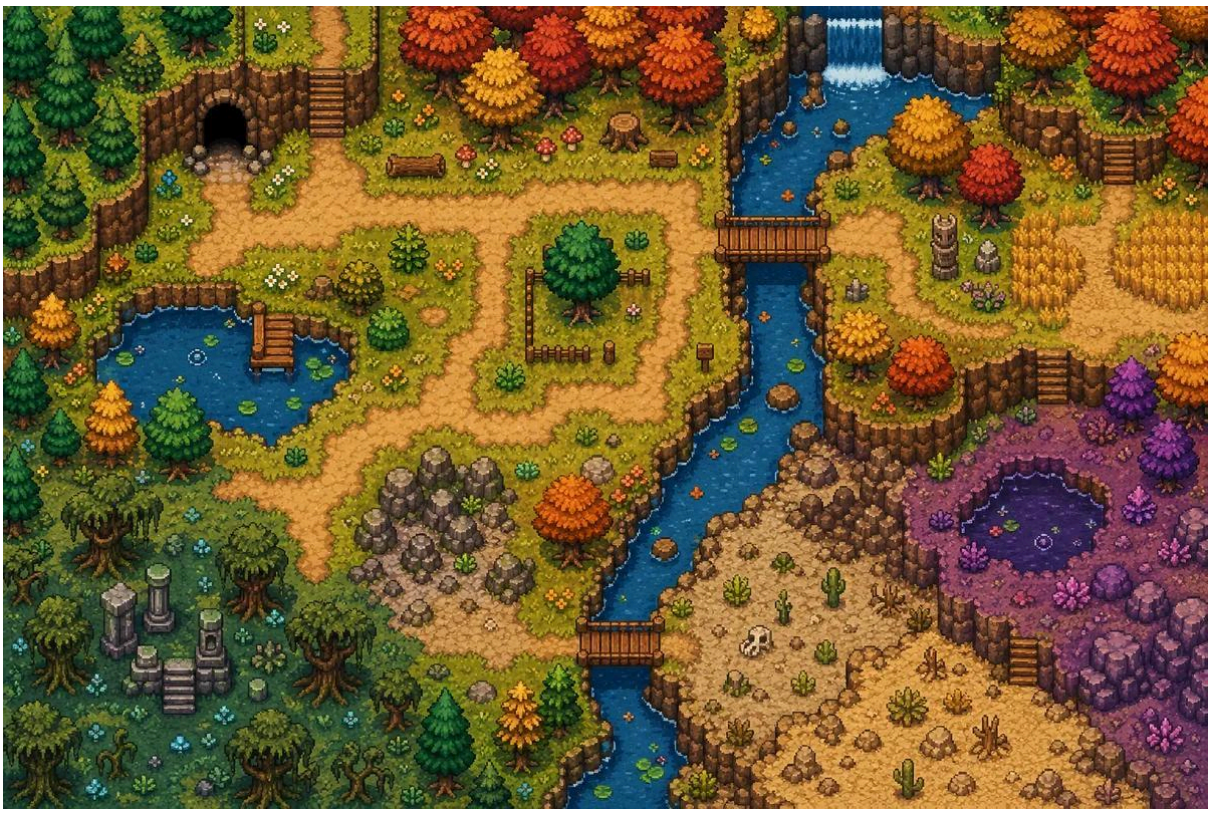
Магазин продуктів, супермаркет спорядження, велосипедна майстерня, автосервіс, аптека, банкомат, пошта та залізнична станція.

Місії: знайти дешевшу деталь, забрати замовлення, купити запасну камеру, дізнатися про ремонт, продати зайве спорядження.

Локація 03. Сімейний кемпінг

Рецепція, душові, туалети, місце для миття посуду, точки води, контейнери для сміття, майданчик, стежки й різні типи місць для наметів та автомобілів.

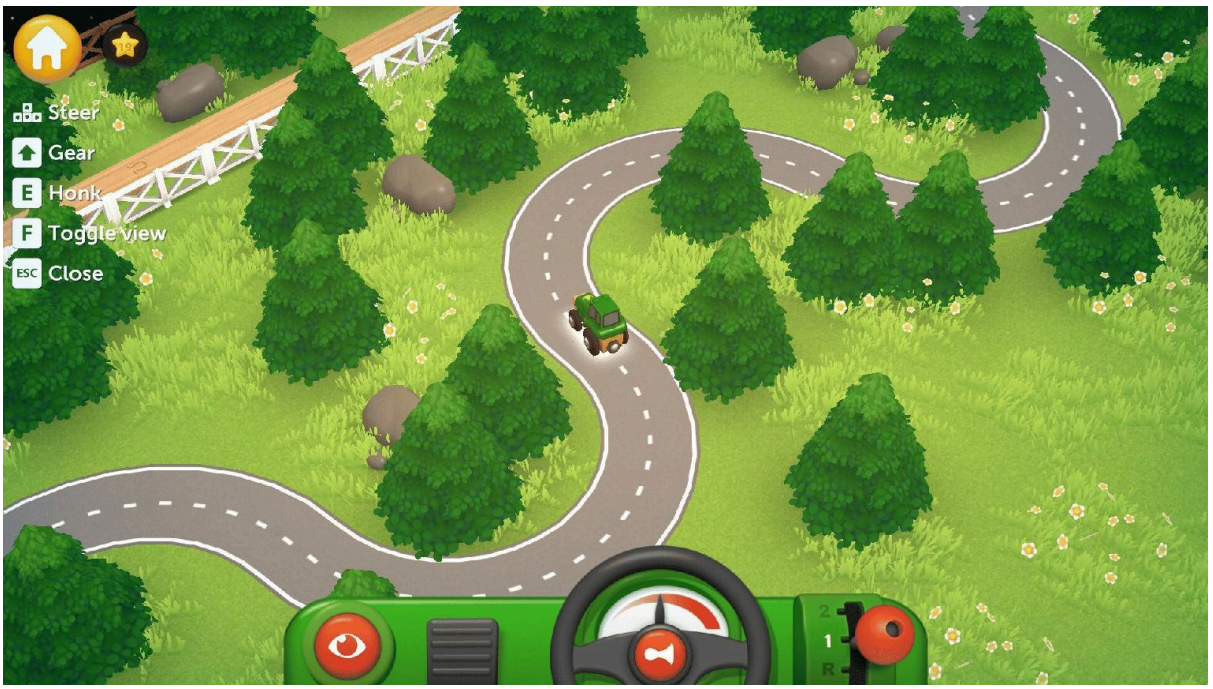
Місії: зареєструватися, знайти правильне місце, познайомитися із сусідами, дотримуватися тихих годин, знайти майданчик для собаки.



Локація 04. Ліс і річкова долина

Пішохідні маршрути, оглядові майданчики, мости, місця для пікніка, круті підйоми, слизькі стежки й невеликі природні пам'ятки.

Місії: пройти маршрут, знайти водоспад, сфотографувати тварин, повернутися до темряви, допомогти іншому туристу.




Локація 05. Дорога і перевал

Перехрестя, дорожні знаки, ремонтні роботи, вузькі мости, підйоми, спуски, заправки та майданчики для зупинки.

Місії: обрати маршрут за висотою й витратами, знайти об'їзд, дістатися заправки, безпечно подолати спуск.

Локація 06. Порт і переправа

Квитковий автомат, черга, зона очікування, посадка, інформаційне табло, кафе та альтернативні маршрути.

Місії: встигнути на пором, змінити квиток, знайти ночівлю після пропущеного рейсу, допомогти іншій родині розібратися з бронюванням.

Локація 07. Майстерня

Діагностика, запчастини, шини, акумулятори, електрика, обладнання кемперів, кошторис ремонту.

Місії: визначити причину несправності, обрати між ремонтом і заміною, знайти вживану деталь, перевірити кемпер перед купівлею.

Усі локації повинні мати предмети, з якими можна взаємодіяти, але не кожен предмет потребує окремої складної анімації. Гравець ходить, відкриває двері, бере речі, ставить намет, сідає в транспорт, готує їжу й виконує ключові дії. Складні розрахунки — фінанси, інвентар, карта, бронювання та діагностика — працюють через зрозумілі меню.

5. Перша кампанія: «Перша справжня відпустка»

Тривалість кампанії — приблизно 10–14 ігрових днів подорожі плюс підготовка вдома. Це не обов'язковий таймер: досвідчений гравець може змінити маршрут, а вільний режим дозволить грати без завершення кампанії.

Акт I. Мрія стає планом

Місія 1 — «Ще один робочий тиждень»

Алекс приходить додому втомлений. Міра показує календар: через кілька тижнів можна взяти відпустку. Лео хоче побачити водоспад, а Бруно тягне повідець до дверей.

Гравець обирає стиль подорожі:

- економний: намет і прості кемпінги;
- комфортний: більше зручностей і бронювань;
- пригодницький: менше заздалегідь визначених зупинок, більше свободи.

Результат: формується початкова мета, бюджет і бажаний рівень комфорту.

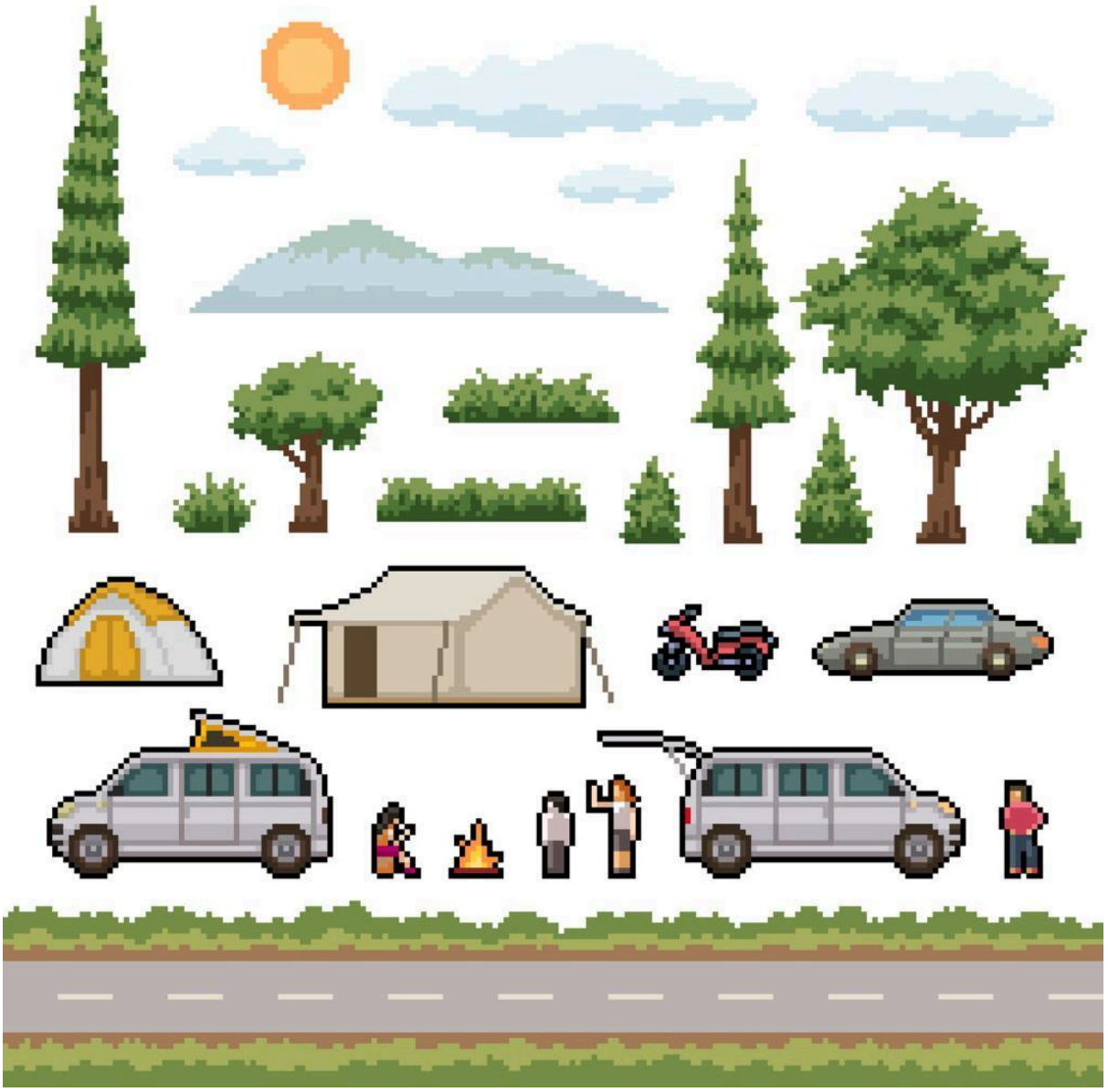
Місія 2 — «Усі хочуть різного»

Міра хоче спокійний кемпінг із душем. Лео хоче водоспад. Алекс хоче заощадити. Для Бруно потрібне місце, де дозволені собаки.

Гравець може:

- скласти маршрут, який враховує всі побажання;
- обрати дешевшу базу й робити одноденні виїзди;
- витратити більше, але зменшити переїзди;
- домовитися, що кожен обирає одну головну активність.

Прихована система: задоволення кожного члена родини. Немає єдиного правильного вибору, але постійне ігнорування чиїхось потреб впливатиме на атмосферу.



Місія 3 — «Старе авто, великі плани»

Гравець перевіряє автомобіль. У нього є зношені шини, обмежений багажник і невідома історія обслуговування.

Варіанти: поїхати на діагностику, самостійно перевірити доступні компоненти або відкласти частину покупок заради ремонту.

Якщо гравець нехтує підготовкою, це не означає, що поломка обов'язково станеться. Але ймовірність проблем у відповідних умовах буде вищою.

Нагорода: знання про стан автомобіля, список необхідних речей і перший досвід планування.

Акт II. Вирушаємо в дорогу

Місія 4 — «Багажник не безмежний»

Гравець збирає спорядження в реальній локальній сцені: відкриває багажник, переносить речі, складає їх у доступні місця.

Кожна річ має вагу та об'єм. Важкі предмети, розташування вантажу й допустиме навантаження впливають на автомобіль. Якщо взяти все «про всяк випадок», доведеться залишити щось удома або змінити план.

Особливі вибори:

- взяти великий намет і більше комфорту;
- обрати легше спорядження й звільнити місце;
- придбати компактний туристичний комплект;
- забути щось важливе й вирішувати проблему вже в дорозі.

Місія 5 — «Поворот, якого не помітили»

На карті є коротший маршрут через містечко та довший, але простіший для водіння. Гравець дивиться на загальну відстань, проте не перевіряє дорожні роботи.

На під'їзді до перехрестя знак змушує змінити напрямок. Якщо гравець проїжджає потрібний поворот, він витрачає додатковий час і пальне.

Далі виникає розвилка:

- повернутися тим самим шляхом;
- продовжити до наступного перехрестя;
- обрати альтернативну дорогу;
- зупинитися й переглянути маршрут.

Якщо через затримку гравець пропускає забронюваний пором, починається наступний ланцюжок пригод.

Місія 6 — «Пором уже пішов»

bexraehart.com

Гравець прибуває після завершення посадки. Це не просто повідомлення «місія провалена». Порт стає повноцінною локацією з новими можливостями.

Вибір:

1. Перебронювати квиток. Можливі додаткові витрати та тривале очікування.
2. Залишитися на ніч. Потрібно знайти місце для ночівлі й оплатити його.
3. Обрати інший маршрут. Можливо, доведеться витратити більше пального, але зберегти частину відпустки.
4. Змінити план подорожі. Замість далекої цілі дослідити регіон поблизу.

Якщо сім'я втомилася, іноді найкраще рішення — зупинитися. Якщо всі в гарному настрої, затримка може перетворитися на вечірню прогулянку портом, вечерю в маленькому кафе та знайомство з іншими мандрівниками.

Сюжетна нагорода: Лео знаходить інформаційну табличку про кораблі, а Ноа, адміністратор майбутнього кемпінгу, згодом виявляється людиною, яка також пропустила свій рейс багато років тому. Ця історія відкривається лише за певного варіанта подій.

Акт III. Кемпінг — це теж пригода

Місія 7 — «Місце є, але не для всіх»

Гравець приїжджає до кемпінгу. Вільне місце є, але не кожна ділянка підходить для великого автомобіля, причепа або родини із собакою.

На рецепції потрібно уточнити:

- чи підходить розмір ділянки;
- чи дозволені собаки;
- чи є підключення електрики;
- де можна набрати питну воду;
- де спорожнити бак сірої води й туалет;
- коли дозволено заїжджати й виїжджати;
- які правила тиші та вогню.

Ці правила не слід робити однаковими для всієї Європи: конкретний кемпінг повинен мати власні параметри. Наприклад, реальні кемпінги можуть обмежувати розмір автомобіля, вимагати бронювання, пропонувати електрику за доплату та визначати окремі місця для різних типів житла.

Un camping en pleine nature
+2

Місія 8 — «Постав табір до дощу»

Гравець бачить прогноз: дощ очікується ввечері. Треба вирішити, що робити першочергово.

- Поставити намет на відповідній рівній ділянці.
- Спершу розвантажити автомобіль, але ризикнути намочити речі.
- Допомогти дитині облаштувати спальне місце.
- Перевірити, чи правильно закріплені відтяжки та чи не стоїть намет у місці, де збирається вода.

Якщо все зроблено добре, родина проводить затишний вечір під дощем. Якщо ні — з'являються мокрий одяг, конденсат, поганий сон і потреба сушити речі наступного дня.

Це важлива різниця: гра показує зв'язок між підготовкою та комфортом, але не вимагає ідеального виконання кожної дії.

Місія 9 — «Дивний запах у кемпері»

Ця подія доступна, якщо гравець має кемпер або моторхоум із туалетом і баками.

Усередині з'являється неприємний запах. Причина не показується одразу. Гравець може оглянути систему, перевірити індикатор заповнення, вентиляцію та записи про попереднє обслуговування.

Можливі причини:

- бак туалету переповнений;
- давно не проводилося очищення;
- неправильно встановлено або закрито з'єднання;
- забруднено ущільнення;
- проблему створює вентиляція.

Рішення: знайти сервісну точку, спорожнити бак у призначеному місці, очистити систему або звернутися до майстра, якщо причина технічна.

Якщо гравець відкладає проблему, комфорт у салоні падає, NPC можуть реагувати на ситуацію, а наступна ніч стає менш приємною. Але немає довільного штрафу за «погану карму»: наслідок має пояснюватися поведінкою та обставинами.

Місія 10 — «Сусіди з іншої країни»

На сусідній ділянці зупинилася пара велосипедистів. Вони планували довгий маршрут, але через сильний зустрічний вітер прибули пізно.

Гравець може:

- запропонувати допомогу з насосом;
- обмінятися маршрутами;
- запросити на спільну вечерю;
- просто привітатися й відпочивати окремо.

Якщо відносини розвиваються, велосипедисти пізніше можуть повідомити про красиву стежку, попередити про складний підйом або запросити на місцевий захід.

Це не обов'язкова дружба, а можливість, що виникає з поведінки.

Акт IV. Погода змінює плани

Місія 11 — «Гроза над долиною»

Гравець планує піший маршрут до водоспаду. Зранку погода нормальна, але прогноз і попередження вказують на ймовірну грозу.

Можливі рішення:

- піти раніше й повернутися до погіршення погоди;
- обрати коротший маршрут;
- перенести прогулянку та відвідати містечко;
- вирушити попри попередження.

Рішення впливає на ризик, витрати часу та настрій родини. Небезпечні умови повинні мати зрозумілі сигнали, а не виникати як несправедливий сюрприз.

Місія 12 — «Загублена стежка»

Після дощу одна з лісових стежок стає слизькою, а частину маршруту перекриває дерево. Гравець може повернутися, обійти перешкоду позначеною стежкою або звернутися по допомогу, якщо персонаж травмувався.

Можливі наслідки — затримка, втома, мокре спорядження, витрати на транспорт або допомогу. Серйозні травми не призводять до смерті: гра пропонує зупинитися, надати доступну допомогу й організувати евакуацію, якщо вона потрібна.

Несподівана позитивна гілка: через зміну маршруту сім'я потрапляє на невеликий місцевий фестиваль. Лео бере участь у пошуку природних об'єктів, Міра відпочиває, а Алекс знайомиться з місцевим майстром, який пізніше допоможе з ремонтом автомобіля.

Акт V. Повернення, яке не є кінцем

Місія 13 — «Останній вечір»

Гравець вирішує, чи залишитися ще на день, чи повернутися додому вчасно. Потрібно врахувати залишок бюджету, стан автомобіля, робочі обов'язки, погоду та бажання родини.

Місія 14 — «Розпакуй не лише речі»

Удома гравець:

- сушить намет і одяг;
- очищує спорядження;
- перевіряє автомобіль;
- записує витрати;

- відновлює запаси;
- переглядає фотографії та спогади.

Якщо все залишити мокрим або брудним, наступна подорож почнеться зі зношеним спорядженням чи додатковими витратами. Якщо добре обслужити речі, наступна поїздка буде легшою.

Місія 15 — «Наступна мрія»

У щоденнику подорожі гравець бачить не оцінку «добре/погано», а історію своїх рішень:

- що вдалося;
- які навички зросли;
- скільки коштувала подорож;
- які місця сподобалися кожному;
- що варто змінити наступного разу;
- які нові маршрути відкрилися.

Після цього відкриваються нові цілі: велотур, подорож уздовж річки, перший кемпер, зимовий кемпінг, ремонт старого фургона або тривале життя в дорозі.

6. Система пригод: щоб світ не повторювався

Сюжетні місії створюють основу. Але справжню довговічність гри забезпечує система подій, що комбінуються між собою.

Кожна подія має містити:

- умови запуску — місце, погода, стан транспорту, час, інвентар, історія персонажа;
- попередження — якщо проблему можна було передбачити;
- кілька рішень — щонайменше два, бажано три або більше;
- наслідки — час, гроші, комфорт, навички, стосунки або нові можливості;
- подальші гілки — ситуація може продовжитися в іншій локації або через кілька днів.

Приклади подій, що комбінуються

| Подія | Умови | Варіанти гравця |
| --- | --- | --- |

| | | |
|---|---|---|
| Прокол шини велосипеда | Зношена шина, гостре сміття, погана поверхня | Ремонт, запасна камера, майстерня, громадський транспорт |
| Перегрів двигуна | Технічний стан, навантаження, спека, підйом | Безпечно зупинитися, дати охолонути, викликати допомогу |
| Сильний зустрічний вітер | Відкритий маршрут, прогноз, тип транспорту | Скоротити етап, змінити напрямок, перечекати |
| Немає питної води | Недостатній запас, спека, довга прогулянка | Купити воду, знайти перевірену точку, змінити маршрут |
| Переповнений бак | Тип транспорту, використання туалету, затримка обслуговування | Знайти сервіс, використати інший туалет, змінити ночівлю |
| Кемпінг не приймає собак | Правила конкретного місця | Інший кемпінг, дозволена ділянка, альтернативне житло |
| Немає вільних місць | Високий сезон, популярна локація, відсутність бронювання | Телефонувати, шукати альтернативу, змінити регіон |
| Закритий гірський маршрут | Сезон, погода, локальне попередження | Об'їзд, інший маршрут, ночівля поруч |

| | | |
|---|---|---|
| Розрядився акумулятор | Вік батареї, навантаження електроприладів | Діагностика, зарядка, допомога, скорочення споживання |
| Собака злякалася грози | Характер тварини, укриття, інтенсивність грози | Знайти спокійне місце, заспокоїти, змінити плани |

Велосипедна механіка повинна враховувати не лише кілометри: вага багажу, набір висоти, круті спуски та поверхня впливають на зусилля, швидкість і комфорт. Європейська федерація велосипедистів окремо наголошує на впливі ухилів на витрати зусиль і безпеку руху.

ECF
+1

Приклад одного ланцюжка з кількома гілками

Початкова подія

Дощ посилюється, а до кемпінгу ще 35 км.

А. Продовжити

Швидше прибуття, але більше втоми й ризику на складній дорозі.

Б. Зупинитися

Додаткові витрати, зате відпочинок і безпечніший темп.

В. Змінити маршрут

Довша дорога, але можна знайти укриття чи сервіс.

Г. Знайти допомогу

Знайомство з місцевими, нова інформація або несподівана ночівля.

Кожна гілка створює нові можливості. Не всі наслідки позитивні, але жоден варіант не повинен бути автоматичним кінцем пригоди.

7. Карма, здоров'я та задоволення від подорожі

Я б не робив карму магічним числом, яке падає за будь-яку помилку. Натомість варто розділити кілька систем:

- Самопочуття: втома, сон, стрес, фізична активність і відновлення.
- Настрій: наскільки персонаж задоволений подорожжю, чи має час на улюблені заняття.
- Стосунки: чи враховує гравець потреби сім'ї, чи допомагає іншим, чи виконує обіцянки.
- Репутація: як його знають інші мандрівники, майстри та працівники кемпінгів.
- Навички: планування, навігація, приготування їжі, ремонт, кемпінг, фінансова грамотність.

Здоров'я та настрій зазвичай поліпшуються від якісного відпочинку, але не обов'язково після кожної поїздки. Виснажлива мандрівка може дати чудові спогади й нові навички, але вимагати кількох днів відновлення.

«Карма» може бути необов'язковою назвою для системи добрих вчинків: допоміг іншому мандрівнику — пізніше він допоможе тобі; залишив після себе безлад — сусіди менше довірятимуть; дотримався правил і допоміг новачку — здобув хорошу репутацію.

Так гра навчає не моралізаторством, а реакцією світу.

8. Як забезпечити багато пригод без нескінченного ручного сценарію

Щоб гра не стала набором повторюваних випадкових повідомлень, потрібно створити бібліотеку сценарних модулів.

Кожна подія зберігається як окрема структура з умовами, діалогами, варіантами вибору, наслідками та посиланнями на інші події. Сценарна система перевіряє, що зараз відбувається у світі, і вибирає доречні ситуації.

Наприклад, «поломка автомобіля» не має запускатися довільно. Вона залежить від технічного стану, навантаження, дороги, температури, обслуговування й попередніх проблем. Якщо автомобіль справний і не було причин для поломки, гра не повинна ламати його тільки для створення драми.

Але коли подія виникла, вона може розвиватися різними шляхами:

- ремонт у майстерні;
- самостійна діагностика;
- покупка деталі;
- допомога іншого мандрівника;
- зміна маршруту;
- нове знайомство;
- виявлення давньої прихованої несправності.

Тому один тип проблеми може породити десятки різних історій.

Принципи генерації подій

1. Не повторювати однакові події надто часто.
2. Враховувати біографію персонажа: механік отримує інші можливості, ніж новачок.
3. Враховувати транспорт: велосипедист не має проблем із двигуном, а власник кемпера — із запасами води й відходами.
4. Враховувати географію: гори, узбережжя, міста, ліси та порти мають різні ризики.
5. Враховувати сезон: зимові умови не повинні випадково виникати влітку.
6. Пам'ятати історію: пропущений пором, знайомства, обіцянки й раніше знайдені місця впливають на майбутнє.
7. Залишати простір для спокою: не кожен день має містити проблему. Іноді найкраща подія — красивий захід сонця, вечеря біля намету або тихий ранок біля озера.

9. Реальні дані та межа між реальністю і вигадкою

Твоя ідея використати справжні автомобілі, обладнання, кемпінги та дороги цілком підходить для такої архітектури. Але кожен набір даних потрібно перевіряти на актуальність, повноту та ліцензійні умови.

Для картографічної основи можна дослідити OpenStreetMap: його дані охоплюють дороги, стежки, заклади й багато інших об'єктів. Для публічного використання потрібно виконувати вимоги ліцензії ODbL та належним чином зазначати джерело.

OpenStreetMap Foundation

+1

Для погоди можна використовувати історичні дані та прогнози, щоб симуляція спиралася на правдоподібні умови, а не на випадкові дощі.

open-meteo.com

Для кожного реального кемпінгу бажано зберігати такі поля:

- координати та доступні під'їзди;
- типи дозволених транспортних засобів;
- максимальні розміри, якщо відомі;
- правила щодо собак;
- вода, електрика, душ, туалет і спорожнення баків;
- ціни, сезонність, години заїзду та виїзду;
- доступність бронювання та дата останньої перевірки.

Важливе розділення: реальні факти використовуються як основа правил, а сюжетні персонажі, діалоги, місії й вигадані події створюють ігровий досвід. Якщо інформація про конкретне місце невідома або застаріла, гра має показувати невизначеність, а не вигадувати гарантовані умови.

10. Що має входити до повної сценарної біблії

Цей документ задає основу світу й першої кампанії. Щоб передати проєкт розробнику без критичних прогалин, повну біблію треба деталізувати в окремих взаємопов'язаних розділах:

А. Персонажі та стосунки

Біографії, характер, цілі, діалоги, розклад дня, уподобання, реакції на рішення гравця, умови появи й розвитку стосунків.

Б. Локації та географічні правила

Регіони, маршрути, місця для ночівлі, дороги, обмеження транспортних засобів, сезонність, доступні послуги й точки інтересу.

В. Сценарні гілки

Умови запуску, діалоги, вибори, наслідки, відкладені результати, альтернативні завершення та зв'язки між подіями.

Г. Транспорт та спорядження

Моделі, характеристики, сумісність, вантажопідйомність, знос, обслуговування, поломки, ремонт, ринок уживаних речей.

Ґ. Події та життєві ситуації

Погода, дороги, кемпінги, тварини, харчування, вода, відходи, соціальні ситуації, небезпечні умови, допомога й порятунок.

Д. Календар та економіка

Робочі графіки, відпустки, школа, свята, доходи, витрати, ціни, сезонний попит, фінансові цілі та розвиток персонажа.

Кожна місія в остаточній версії повинна мати власний ID, вступну умову, список учасників, сцену, діалоги, доступні рішення, результати кожного рішення, умови повторного запуску, нагороди та зв'язки з іншими місіями. Це дасть змогу не лише написати сценарій, а й реалізувати його як систему, яку можна перевіряти та розширювати.

Моя рекомендація: першою повністю довести до готового стану кампанію «Перша справжня відпустка», усі її діалоги та гілки, а паралельно створити універсальний механізм подій. Коли перша кампанія стане цікавою й працюватиме від початку до кінця, можна додавати нові професії, типи транспорту, країни, регіони та тисячі комбінацій пригод, не переписуючи гру з нуля.

перспективу і візуал я показав на грі прикладі, але можемо зробити краще якщо технічні будуть можливості, але там вид зверху під нахилом.

П'ять цілей, які утримують інтерес

=- їх буде треба значно більше, бо гра має бути цікавою і різноманітною за сюжетом і кожен має знайти щось своє

Гра починається не з намету, а з життя персонажа. Саме його повсякденність визначає, які пригоди він зможе пережити.

- насправді гра починається із створення персонажу і його гендеру, віку, базових скілів, роботи, статків, тобто треба щоб люди наповнили свій профайл але не можна ставити багато вільних грошей бо сенс гри відразу можна втратити, бо я поставлю 1млн вільних і куплю відразу самий крутий кемпервен і все! Тут сенс в тому що ми маємо дати базові навички обрати і дати всім однакові умови для старту, так, щоб всі мали

приблизно однакові вільні гроші і базові витрати, а кожен може заробляти в грі купуючи - лагодити може і пробувати продавати речі, а також зарплата буде нараховуватися.

Веломандрівник

- короткі переїзди і обмежений

Родина вихідного дня

- я не зрозумів, він що розлучений і тільки на вихідних зберається з родиною? він чи треба нового персонажу, тий хто має пару але чи живуть чи не живуть разом, а також уяви того хто самотній але бере в подорож друзів.

Майстер-мандрівник

- а також йому потрібен і певно і іншим купити набори інструментів + на роботі він втомлюється і може бути не в захваті працювати, тому треба мати на увазі якщо він не бізнесмен який сам купує і ремонтує ті самі кемпери то він має основну роботу як механік.

Це не чотири окремі кампанії. Це різні початкові умови для одного великого світу. У процесі гри велосипедист може придбати авто, родина — пересісти у фургон, а віддалений працівник — змінити професію.

- продумай більше умов-персонажей бо як на мене то занадто мало, хтось мое не знайти свій

Собака не просто косметичний супутник. У нього є характер, витривалість, звички, потреба у воді, їжі, прогулянках і відпочинку.

- тоді якщо прям так, то можна обрати мала, середня чи велика собака), щоб і запаси їжи ічас на прогулянки і догляд був різним. Як в житті)))

Важливо: гра не повинна змушувати гравця відмовлятися від собаки. Вона дає йому змогу планувати подорож, у якій комфортно всім.

- ні, відмовитись неможна, якщо обрав що є то все, то вже твій супутник і тягар, просто менше кемпенгів доступні і витрати зростають, але собака + к кармі і здоров’ю бо вона допомагає релаксувати і гуляти пішки і дарує радість

Алекс — головний герой за замовчуванням

- George - його звати так 42 роки

Міра — партнерка

- Victoria його дружина 42 роки

Лео — син

- Matthew 15 років їх син

Бруно — сімейний собака

- Міа їх кішка маленька Шотландьска віслоуха 5 років

Додавай більше персонажів!

Ноа — адміністратор кемпінгу

- не треба вуха як у тварин чи інші не притаманні людям речі, в грі тільки люди а не істоти видумані.

NPC

- так їх має бути під кожну історію багато і унікальних, тобто не треба копіювати одну і ту саму NPC для різних локацій за іменем чи історією чи виглядом.

Основні локації та їхні функції

- тут треба добре все проаналізувати і продумати і дати все детально

Всю сюжетну лінію треба продумати до дрібниць і так, щоб логіка була ідеальною як і механіка і персонажи і код і все взагалі.

Перші місії повинні залучати гравців а не випалювати їх, треба завлікати цікавостями і інтерактивом, не можна відразу проблеми їм видавати як поворот не туди чи погода не ок, чи грошей обмаль чи важко речі зібрати.

Приклади подій, що комбінуються

- занадито мало цікавого, проаналізуй ще і додай ще потенційні реалістичні і можна приємні а не тільки погані, бо може лотерею виграв, спадок отримав, бонус на роботі, друзі планували відпустку і в них не получається і вони подарували місце в кемпінгу чи обладнання купив зі знижкою чи друзі своє старе подорували, але і погані додай бо щось потенційно бональне може бути що просто вирішити але треба час чи

гроші. Подивись як в інших іграх таких як наша буває і також дивись на що люди писали в пригодах на авто і з кемпінгом кемпером.

Щоб гра не стала набором повторюваних випадкових повідомлень, потрібно створити бібліотеку сценарних модулів.

- продумай все до дрібниць і випиши, бо від тебе треба сценарій який буде детальним, логічним, цікавим.

10. Що має входити до повної сценарної біблії

- проаналізуй і зроби

CampLife: The Journey — сценарна біблія v0.3

Персонажі, життєві сценарії, світ, місії та система пригод

Тепер ми уточнили кілька принципових речей, які суттєво змінюють дизайн гри. Я закладу їх не просто в опис, а в саму структуру майбутньої гри та технічного завдання.

По-перше, візуал. Орієнтир — не плаский вигляд суворо згори, а ізометрично-подібна перспектива: камера дивиться згори під кутом, видно верх предметів і їхні бокові поверхні, персонажі ходять між об'єктами, а локації мають глибину. Для 2D pixel art це реалістично зробити в браузері. Варто порівняти два варіанти — tilemap із перспективою 3/4 та ізометричну сітку — і обрати той, що дасть найкраще поєднання читабельності, анімації та вартості виробництва. Не потрібно одразу будувати складний 3D-світ.

По-друге, персонаж створюється самим гравцем. Ніякого обов'язкового Алекса, готової родини чи нав'язаного професійного сценарію. Гравець визначає вік, гендер, побутову ситуацію, професію, навички, стосунки, дітей, домашніх тварин та стартовий транспорт. При цьому початкові фінанси збалансовані: різні біографії дають різні можливості, але ніхто не може просто створити персонажа з мільйоном євро й одразу придбати найкращий моторхоум.

По-третє, це не гра про суцільні проблеми. Перші години мають бути приємними: цікаві відкриття, вибір спорядження, знайомства, красиві місця, маленькі перемоги й відчуття, що світ відкривається. Несправності та труднощі з'являються поступово, коли гравець уже розуміє правила й має інструменти для рішень.

По-четверте, NPC — це конкретні люди. Не один і той самий продавець, механік або турист під різними іменами. Кожен важливий NPC матиме свою біографію,

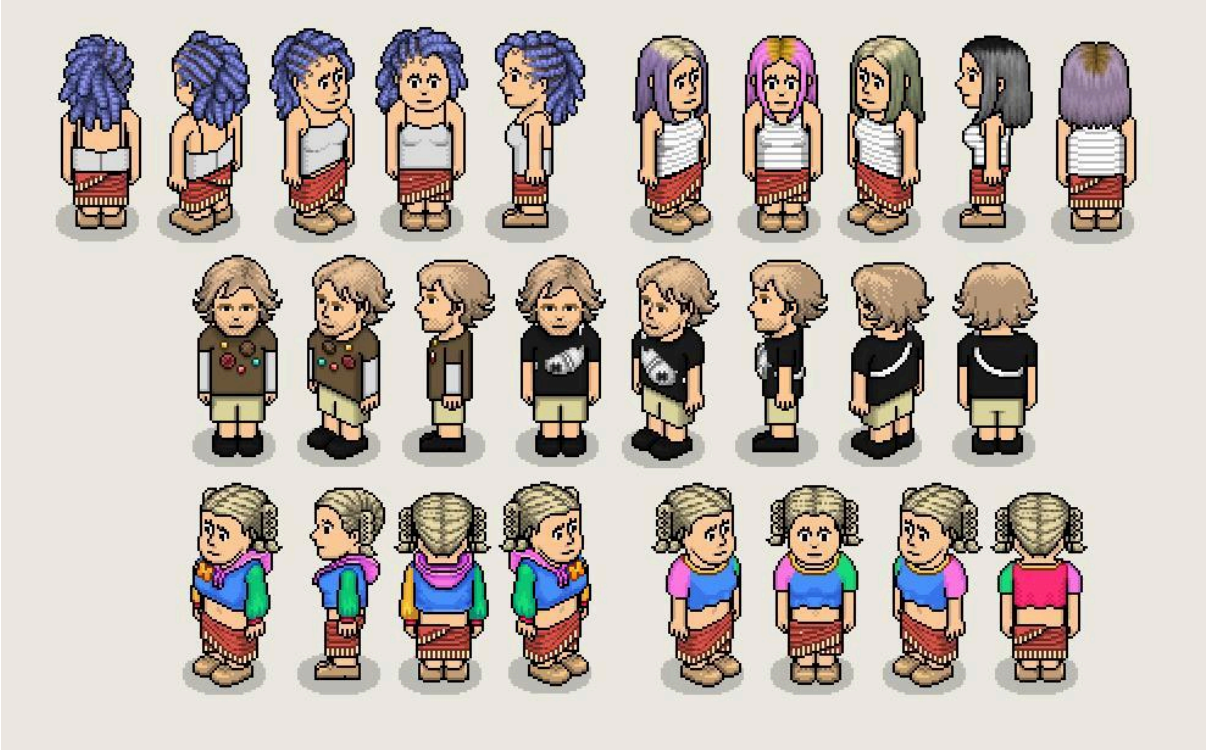
зовнішність, розклад, характер, цілі, зв'язки й історію. Другорядних мешканців можна генерувати за шаблонами, але з унікальними комбінаціями ознак і збереженням їхньої ідентичності.

Нижче я розширюю дизайн від основи персонажа до місій, географії та системної бібліотеки пригод.

1. Створення персонажа: головна точка входу в гру

Гра повинна починатися з редактора персонажа, а не з готового героя. Проте редактор не має бути анкетою на 40 хвилин. Я пропоную зробити його інтерактивним: гравець створює людину, бачить її майбутнє житло, бюджет, доступний транспорт і короткий опис перших можливостей.

Етапи створення



Крок 1. Зовнішність та особистість

Ім'я, прізвище, гендер, вік, зовнішність, зачіска, одяг, стиль поведінки. Зовнішність не повинна визначати доступні професії або якість персонажа.

Крок 2. Професія та графік

Гравець обирає основну роботу, рівень досвіду, робочі години, віддалений або очний формат, тип доходу та можливість додаткових підробітків.

Крок 3. Побут і стосунки

Самотній, у стосунках, одружений, живе з партнером або окремо; діти та їхній вік; друзі, які можуть приєднуватися до поїздок; домашні тварини.

Крок 4. Стартові ресурси

Житло, накопичення, транспорт, спорядження, інструменти, кредити та регулярні витрати. Усі варіанти проходять через систему балансу, а не дають безкоштовну перевагу.

Крок 5. Особисті цілі

Гравець обирає декілька бажань: побачити море, навчитися ремонтувати велосипед, купити фургон, подорожувати з друзями, провести більше часу з родиною, відвідати національні парки або жити в дорозі.

Баланс стартового бюджету

Я рекомендую не давати кожному однакову суму готівки буквально. Краще забезпечити приблизно однаковий рівень стартової життєздатності.

Наприклад, один персонаж має автомобіль, але майже не має накопичень. Інший не має автомобіля, зате має велосипед, намет і більше грошей. Третій заробляє краще, але має високу оренду й кредит. Четвертий має інструменти, але мусить витрачати вільний час на додаткову роботу.

Умовна стартова модель для балансування — не реальні ринкові ціни:

| Показник | Стартове правило |
| --- | --- |
| Вільні кошти | Обмежені, достатні для невеликої першої подорожі |
| Основний дохід | Відповідає професії й зайнятості |
| Житлові витрати | Враховують оренду, кредит або утримання житла |
| Транспорт | Не обов'язково є; може бути вживаним |
| Спорядження | Базовий набір або можливість придбати його |

| | |
|---|---|
| Інструменти | Залежать від професії та попереднього досвіду |
| Борги | Не обов'язкові; якщо є, вони показуються до підтвердження |
| Доступ до дорогих транспортних засобів | Потребує накопичень, кредиту або вигідної угоди |

Щоб уникнути експлойтів, стартові профілі мають проходити автоматичний тест: чи може кожен персонаж покрити базові витрати, заробити гроші й організувати доступну першу подорож без необхідності багато годин працювати.

Базові навички

Пропоную вісім напрямів, кожен із яких впливає на рішення, але не закриває гру для новачка:

1. Планування й навігація.
2. Кемпінг і облаштування табору.
3. Механіка й обслуговування транспорту.
4. Електрика й автономне живлення.
5. Приготування їжі та організація запасів.
6. Фінансова грамотність і торгівля.
7. Фізична витривалість і безпека.
8. Комунікація й соціальні навички.

Навички мають початковий рівень, досвід, темп розвитку та практичні обмеження. Людина з високим рівнем механіки краще діагностує проблему, але не може без відповідного інструмента безпечно виконати будь-який ремонт.

2. Більше життєвих сценаріїв: не класи, а комбінації

Попередні чотири архетипи були надто вузькими. Вони не повинні бути жорсткими класами. Замість цього створимо матрицю біографій, яка дозволяє комбінувати роботу, стосунки, транспорт, фінанси, навички й цілі.

01. Самостійний дослідник

Живе сам, працює на звичайній роботі, має небагато обов'язків. Може почати з велосипеда, громадського транспорту або старого авто.

Історії: перший самостійний похід, знайомства в кемпінгах, подорожі без чіткого плану, поступова купівля спорядження.

02. Пара, що живе разом

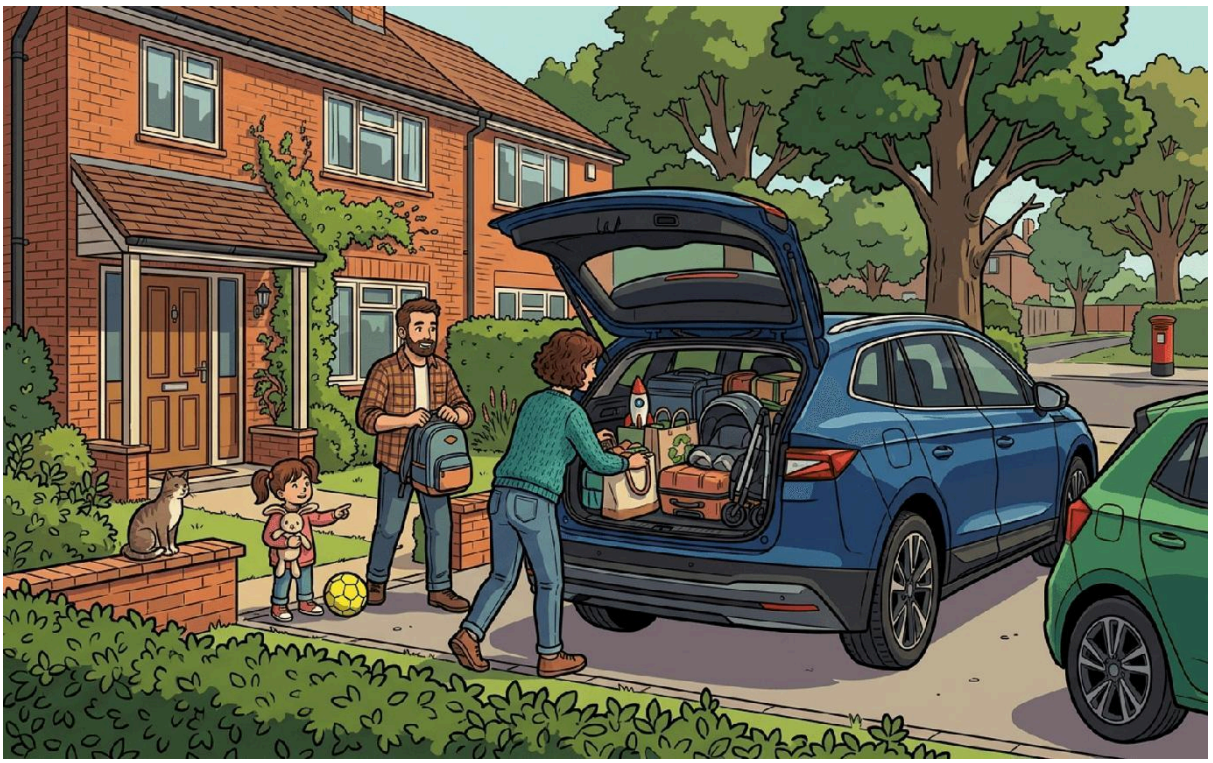
Двоє дорослих із різними вподобаннями, доходами й графіками. Один хоче активні походи, інший — спокійний відпочинок.

Історії: компроміси, спільні цілі, несподівані подарунки, романтичні вечори, розподіл бюджету.

03. Самотній персонаж із компанією друзів

Живе сам, але може запрошувати друзів у подорожі. У кожного друга свої гроші, час, потреби та вподобання.

Історії: хтось запізнився, друг позичив намет, компанія змінила маршрут, друзі сперечаються щодо бюджету або спільно купують спорядження.



04. Сім'я з підлітком

Джордж, 42 роки; Вікторія, 42 роки; Метью, 15 років; кішка Мія, шотландська висловуха, 5 років.

Історії: інтереси підлітка, потреба в особистому просторі, спільні активності, догляд за кішкою, вибір житла, де тварину дозволено тримати.

05. Найманий механік

Працює в майстерні, має технічні навички, але після зміни втомлюється. Йому потрібні власні інструменти для домашніх проєктів, а не лише професійні знання.

Історії: ремонт у вихідний, пошук дешевих деталей, дилема між відпочинком і додатковим заробітком, купівля старого кемпера з прихованим дефектом.

06. Самозайнятий майстер і торговець

Купує техніку, ремонтує, продає або здає в оренду. Його бізнес залежить від попиту, якості роботи, капіталу й часу.

Історії: вигідна покупка, приховані дефекти, невдалий ремонт, клієнтська суперечка, прибутковий сезон.

07. Віддалений працівник

Працює онлайн, може жити в кемпері або подорожувати між робочими днями. Має більше географічної свободи, але потрібні електрика, інтернет, місце для роботи й резервний план.

Історії: відеодзвінок під час грози, розряджена батарея, слабкий зв'язок, робота з бібліотеки чи кафе, спонтанна подорож після завершення проєкту.

08. Працівник зі змінним графіком

Медик, працівник сервісу, рятувальник або інший спеціаліст, який працює позмінно.

Історії: кілька вільних днів поспіль, заміна зміни, складнощі з бронюванням, короткі поїздки в будні, несподіваний виклик на роботу.

09. Активна пара на пенсії

Має більше вільного часу, але враховує бюджет, фізичну витривалість, комфорт і технічний стан транспорту.

Історії: довгі міжсезонні поїздки, зустрічі зі старими друзями, нові хобі, спокійні маршрути, відновлення старого автомобіля.

10. Студент із малим бюджетом

Обмежені гроші, навчальний календар, підробітки й бажання побачити світ.

Історії: знижки на спорядження, волонтерські заходи, недорогі маршрути, спільні поїздки, обмін навичками.

11. Один із батьків із дитиною

Поїздки залежать від шкільного календаря, бюджету, підтримки родичів і доступності зручних маршрутів.

Історії: перша ніч у наметі, планування без перевантаження, нові друзі дитини, зміна плану через втому.

12. Велотурист

Не обмежується короткими переїздами: його дальність залежить від фізичної форми, велосипеда, рельєфу, ваги, погоди й ночівель. На старті доступні короткі етапи, а з досвідом — багатоденні маршрути.

Історії: ремонт у дорозі, попутний вітер, складний перевал, пошук місця для намету, зустрічі з іншими велотуристами.

Це лише дванадцять основних сюжетних профілів. У фінальній системі гравець не обирає один із дванадцяти готових класів: він збирає власну біографію з сумісних компонентів.

Як не дати гравцю створити надто сильного персонажа

Використовуємо систему стартових балів.

Гравець має умовний ліміт на стартові переваги. Висока зарплата коштує балів, так само як і цінне майно, професійні інструменти та рідкісні навички. Але вибір

може бути нерівномірним: досвідчений механік із малим бюджетом цілком нормальний, як і багатший офісний працівник із низькими практичними навичками.

Родина, тварини, борги та житлові витрати не повинні бути просто штрафами за вибір. Вони відкривають нові сюжетні можливості й додають змісту життю персонажа.

Після вибору гра показує стартовий баланс: дохід − обов'язкові витрати = вільний залишок. Якщо бюджет негативний уже до початку гри, гравець має побачити це й підтвердити такий виклик або повернутися до налаштувань.

3. Тварини: не декорація, а частина сім'ї

Ти уточнив, що якщо гравець обрав домашню тварину, вона залишається з ним. Тоді тварина — постійний член домогосподарства, а не предмет, який можна прибрати з інвентаря заради зручності.

Для собаки обираються розмір і темперамент. Для кота — характер, звички, чутливість до транспорту й потреба у спокійному просторі. Кожна тварина має потреби, які можна задовольняти без щохвилинного мікроменеджменту.

| Параметр | Маленька собака | Середня собака | Велика собака |
|---|---|---|---|
| Корм | Менше запасів | Середні запаси | Більше запасів |
| Прогулянки | Коротші або частіші залежно від характеру | Збалансовані | Можуть потребувати більше часу й активності |
| Місце в авто | Менше | Середнє | Більше |
| Спорядження | Компактне | Середнє | Міцніше та більшого розміру |

| | | | |
|---|---|---|---|
| Потреба у воді | Залежить від спеки й активності | Залежить від спеки й активності | Зазвичай більше за однакових умов |
| Вплив на маршрут | Доступність місць і температура | Доступність, активність, температура | Доступність, простір, навантаження |

Розмір не повинен автоматично визначати темперамент чи витривалість: маленька собака може бути дуже активною, а велика — спокійною. Механіка враховує індивідуальні характеристики.

Для кота Мії важливі переноска, безпечне перевезення, вода, корм, лоток, спокій і можливість сховатися. Довгі прогулянки для неї не є обов'язковою механікою, як для собаки.

Під час міжнародних поїздок гра може навчати готувати документи для тварин, перевіряти актуальні вимоги та умови перевезення. Для реальних подорожей із собаками й котами в ЄС існують вимоги до ідентифікації, вакцинації та документів; конкретні правила залежать від маршруту.

Your Europe

+1

Нагорода за тварину — не просто бонус до «карми». Вона дає більше прогулянок, нові соціальні контакти, емоційну прив'язаність, приємні події та власні історії. Водночас тварина потребує часу, грошей і планування. Саме цей баланс робить її справжнім супутником.

4. Перші 90 хвилин гри: зацікавити, а не виснажити

Це критично важлива частина. Якщо гравець щойно створив персонажа, не можна відразу змушувати його ремонтувати автомобіль, рахувати кожен грам продуктів і боротися з поганою погодою.

Перші 90 хвилин мають створити відчуття: «Це моє життя, мої речі, моя подорож — і я хочу побачити, що буде далі».

0–15 хв

Етап 1. Моє нове життя

Гравець створює персонажа й бачить його житло. Можна пройтися кімнатами, змінити одяг, оглянути балкон, погладити тварину, відкрити календар і подивитися на стару карту в шафі.

Перша ціль проста: вибрати, куди хотілося б поїхати. Немає загроз, боргів, обов'язкових ремонтів або термінових завдань.

15–30 хв

Етап 2. Мрія має адресу

Гравець відкриває карту й обирає першу цікаву локацію. Може переглянути фотографії, доступні активності, кемпінги, приблизний бюджет і час у дорозі.

Відкривається проста мета: організувати одноденну або вихідну поїздку. Гра пропонує кілька підходящих варіантів, не нав'язуючи один.

30–45 хв

Етап 3. Магазин відкриттів

Гравець відвідує магазин спорядження. Тут можна роздивитися предмети, порівняти вагу, об'єм, ціну й умови використання.

Продавець радить базовий комплект відповідно до вибраного способу подорожі. Можна придбати річ, орендувати її або поки що нічого не купувати.

45–70 хв

Етап 4. Перша подорож

Гравець прибуває до невеликого, безпечного й доступного кемпінгу. Він може пройтися територією, поставити намет, знайти питну воду, облаштувати місце відпочинку, приготувати просту страву й поспілкуватися з іншими людьми.

Погода сприятлива. Основні механіки працюють, але ще не вимагають складного планування.

70–90 хв

Етап 5. Перша історія, яку хочеться запам'ятати

Гравець знайомиться з іншим туристом, знаходить коротку стежку до гарного краєвиду або бере участь у невеликій активності кемпінгу.

Увечері він отримує перший запис у щоденник, досвід за нові дії та кілька ідей для наступної поїздки.

Перша подія має бути приємною

Наприклад, сусід по кемпінгу залишає оголошення: він повертається додому й хоче віддати туристичний столик, який більше не використовує.

Гравець може забрати його безкоштовно, відмовитися або порадити іншому туристу. Столик стає частиною інвентаря, а історія знайомства записується у щоденник.

Так гравець отримує не випадковий подарунок із неба, а результат соціальної взаємодії.

Поступове введення складності

- Перші поїздки — комфортні, безпечні, з простими цілями.
- Після кількох поїздок — варіативна погода, планування запасів, вибір типу дороги.

- Далі — бронювання, технічний знос, складніші маршрути, віддалені локації та непередбачувані події.
- У пізній грі — тривалі подорожі, життя у фургоні, купівля й відновлення кемперів, сезонні маршрути, складні погодні умови й багатоступеневі сюжетні арки.

Складність зростає через відкриття нових можливостей, а не через постійне погіршення умов.

5. Як зробити NPC справді унікальними

Тут я пропоную запозичити не готових персонажів з інших ігор, а принципи симуляції соціального світу. У The Sims люди цікаві завдяки потребам, звичкам, стосункам і реакціям на середовище; у RimWorld історії виникають, коли характери, навички й події взаємодіють. Для CampLife потрібен м'який варіант: пригоди й соціальна пам'ять без постійних катастроф.

GameDesign.gg

+1

Кожен важливий NPC має власну картку

| Поле | Що зберігаємо |
| --- | --- |
| Ідентичність | Унікальний ID, ім'я, прізвище, вік |
| Зовнішність | Обличчя, волосся, одяг, статура, палітра кольорів |
| Біографія | Місце проживання, професія, попередній досвід |
| Характер | Товариськість, терпіння, організованість, авантюрність |

| | |
|---|---|
| Навички | Ремонт, велосипед, кулінарія, планування, спілкування |
| Поточні цілі | Дістатися місця, відпочити, знайти деталь, зустріти друзів |
| Розклад | Робота, сон, покупки, переміщення, відпочинок |
| Майно | Транспорт, спорядження, доступні ресурси |
| Стосунки | Родина, друзі, знайомі, конфлікти, борги й обіцянки |
| Пам'ять | Що NPC пережив із гравцем, що йому подарували, чим допомогли |
| Подальші гілки | Які місії або зустрічі можуть виникнути пізніше |

NPC не повинен забувати, що вчора гравець позичив йому насос. Якщо персонажі домовилися зустрітися в іншому кемпінгу, це повинно зберігатися в історії світу.

Перша група унікальних NPC

Це персонажі першого регіону, які можуть з'являтися у відповідних місцях. У кожного власна роль у сюжеті.

Elise Vermeulen — власниця сімейного кемпінгу

51 рік, багато років керує кемпінгом. Любить порядок, але цінує гостей, які цікавляться місцевістю.

Історія: планує невелике оновлення кемпінгу й шукає відгуки про нову зону для велосипедистів. Гравець може допомогти протестувати її.

Nora De Smet — велотуристка

28 років, подорожує велосипедом, фотографує дороги й невеликі містечка. Добре знає маршрути, але не любить надмірно важкий багаж.

Історія: збирає добірку гарних веломаршрутів і може запросити гравця в коротку спільну поїздку.

Luc Bernard — велосипедний механік

46 років, працює у невеликій майстерні. Любить якісні інструменти й не схвалює непотрібну заміну деталей.

Історія: пропонує навчальне завдання з налаштування гальм і вибору шин для різних поверхонь.

Ingrid Mertens — досвідчена мандрівниця

63 роки, багато років їздить Європою. Вона не всезнаюча: іноді сама потребує допомоги або змінює плани.

Історія: ділиться старими маршрутними нотатками, які відкривають альтернативні стежки та місця для відпочинку.

Milan Peeters — віддалений працівник

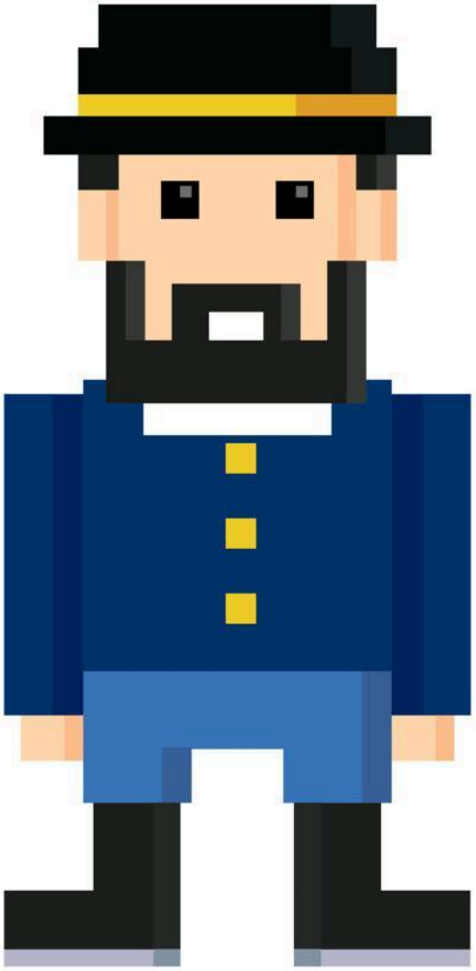
31 рік, працює онлайн із кемпера. Планує перейти на сонячне живлення, але поки не знає, як оптимізувати енергоспоживання.

Історія: гравець може допомогти порівняти варіанти батареї, сонячних панелей і потреб у електриці.

Sarah Lambert — місцева екскурсоводка

37 років, проводить прогулянки природними маршрутами. Знає місцеві історії, але її тури залежать від сезону та погоди.

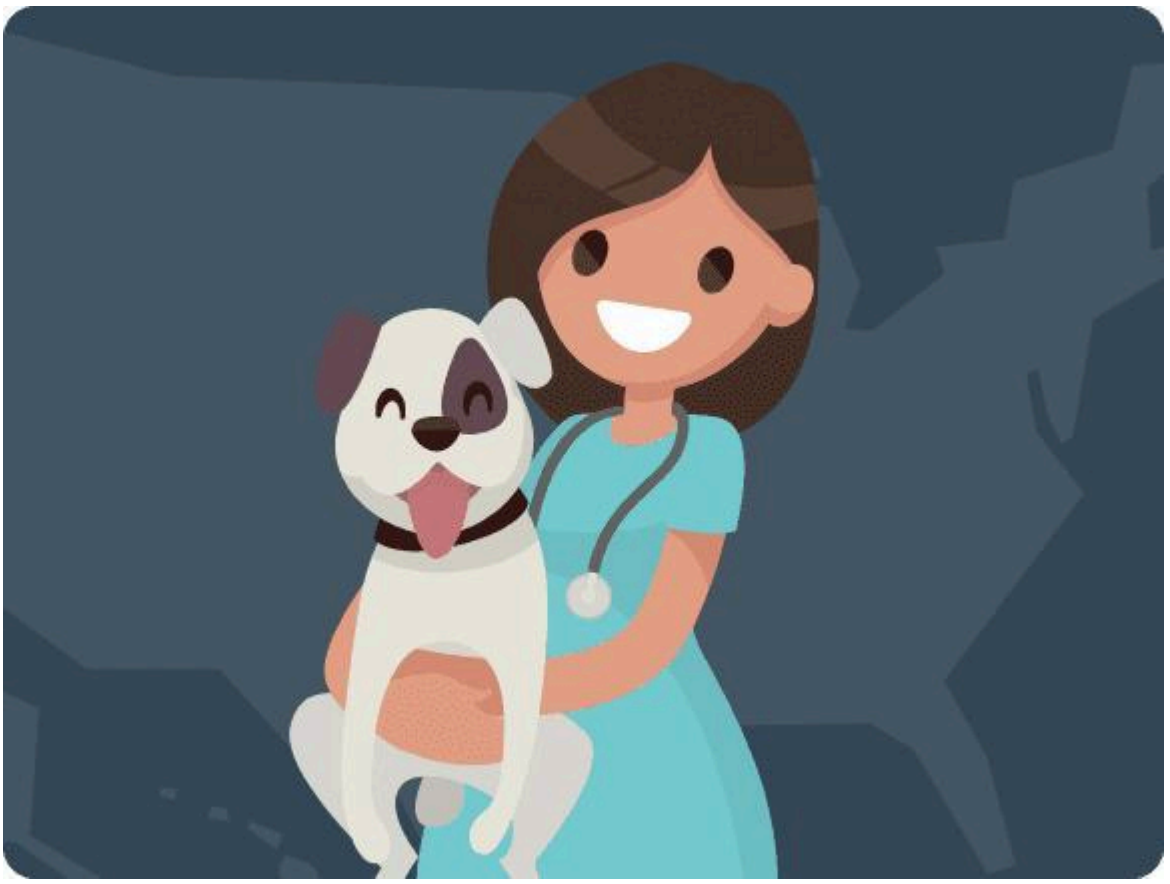
Історія: може запросити на екскурсію, де гравець дізнається про природу регіону та відкриє нові точки інтересу.



Thomas Legrand — працівник поромного термінала

44 роки, працює за змінним графіком. Знає процедури посадки та правила перевезення різних типів транспорту.

Історія: навчає гравця розраховувати час прибуття, перевіряти квитки й враховувати обмеження транспортного засобу.



Claire Dubois — ветеринарка

39 років, консультує власників тварин. Її місія — не драматична хвороба, а практична підготовка тварини до поїздки.

Історія: допомагає скласти список корму, документів, переноски, води та засобів догляду для Мії або собаки гравця.

Це лише перша вісімка. Для кожного регіону потрібно створити власний пул NPC, а для кожної великої сюжетної арки — окремих персонажів, яких не копіюємо в іншому місці під новим ім'ям.

Технічне правило: унікальність гарантується не тільки ім'ям, а й постійним `npcId`, зовнішністю, біографією, набором зв'язків і історією. Генератор випадкових NPC може використовувати повторно базові анімації та одяг, але не повинен створювати враження, що та сама людина раптово працює в різних країнах без пояснення.

6. Розширена система локацій

Локації потрібно проектувати як набори функцій, а не як красиві фони. Кожна має власні інтерактивні предмети, NPC, правила, доступні послуги, сюжетні події та переходи до інших місць.

01. Житло персонажа

Кімнати, кухня, ванна, спальня, комора, балкон, гараж або підвал залежно від житла.

Інтеракції: сон, приготування їжі, прання, зберігання речей, планування маршруту, спілкування з родиною, догляд за твариною.

Унікальні місії: перестановка меблів, пошук старого спорядження, підготовка до сімейної події, створення туристичної полиці.

02. Магазин спорядження

Намети, спальники, рюкзаки, ліхтарі, пальники, посуд, туристичний одяг, інструменти, аксесуари для тварин.

Інтеракції: порівняння характеристик, примірка, консультація, купівля, повернення товару за правилами магазину, знижки та сезонні пропозиції.

Унікальні місії: вибрати спорядження під конкретний маршрут, знайти сумісний адаптер, отримати знижку за комплект.

03. Автосервіс і майстерня кемперів

Підйомник, діагностичне обладнання, склад запчастин, робочі столи, зона огляду кемпера.

Інтеракції: діагностика, кошторис, ремонт, перевірка перед купівлею, встановлення додаткового обладнання.

Унікальні місії: знайти джерело витоку води, перевірити старий кемпер, відновити електросистему, оцінити ремонт перед продажем.

04. Інфраструктура кемпінгу

Рецепція, санітарні блоки, душові, пральня, кухня, точки питної води, спорожнення баків, сміттєві майданчики, електричні підключення.

Інтеракції: бронювання, реєстрація, прання, поповнення води, очищення баків, оплата додаткових послуг.

Унікальні місії: допомогти знайти загублену річ, розібратися з бронюванням, взяти участь у невеликому заході кемпінгу.

05. Житлова зона кемпінгу

Ділянки різного розміру, місця для наметів, караванів, кемперів, будиночки, загальні зони відпочинку.

Інтеракції: встановлення житла, розміщення меблів, приготування їжі, відпочинок, спілкування, ігри та активності.

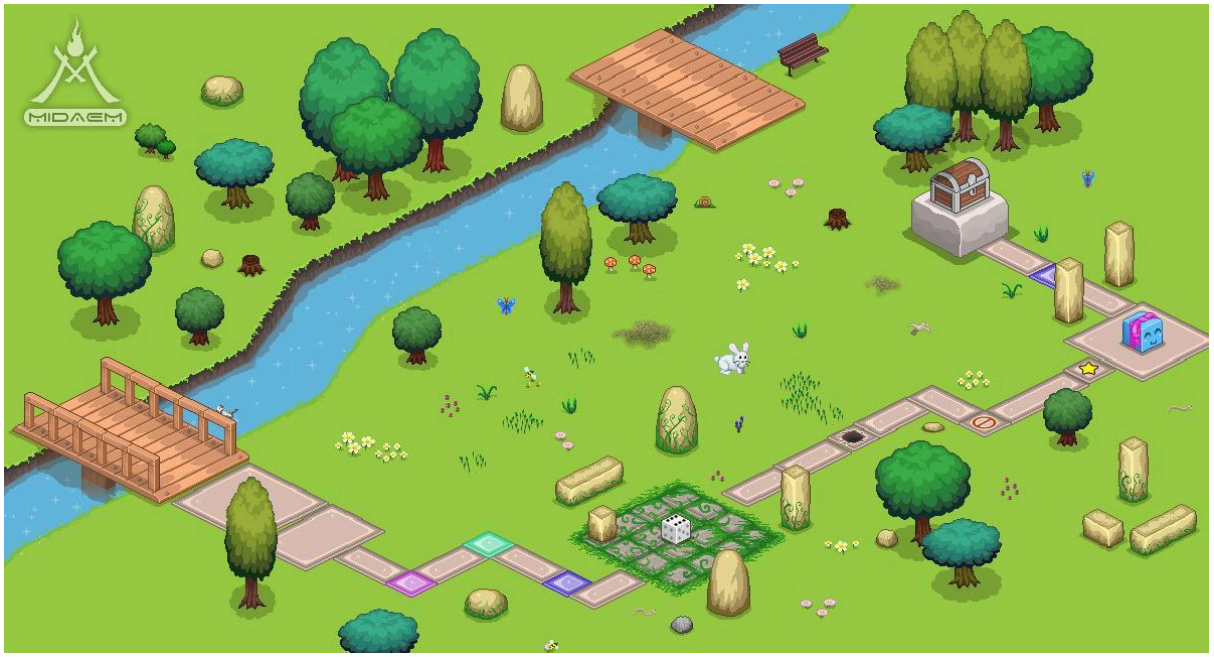
Унікальні місії: організувати комфортний табір, допомогти сусідам, знайти оптимальне місце з урахуванням вітру, тіні та доступу до зручностей.

06. Містечко та громадські послуги

Продуктовий магазин, пекарня, кафе, ринок, бібліотека, пошта, туристичний інформаційний центр, громадський транспорт.

Інтеракції: купівля продуктів, пошук інформації, робота з ноутбуком, зустрічі, локальні заходи.

Унікальні місії: знайти місцевий ярмарок, допомогти організатору заходу, відшукати потрібну деталь, дізнатися про маловідомий маршрут.



07. Природні маршрути

Лісові стежки, річки, оглядові майданчики, водоспади, пагорби, пляжі, луки та природні парки.

Інтеракції: прогулянки, фотографування, спостереження за природою, пікніки, пошук маркованих маршрутів.

Унікальні місії: знайти оглядовий майданчик, пройти маршрут певної складності, сфотографувати рідкісну природну особливість, допомогти загубленому туристу.

08. Транспортні вузли

Порти, вокзали, автобусні станції, паркінги, заправки, зарядні станції.

Інтеракції: квитки, очікування, заправлення, заряджання, пошук альтернативного маршруту, зустрічі з NPC.

Унікальні місії: встигнути на відправлення, допомогти знайти потрібну платформу, вирішити проблему з квитком, організувати альтернативний переїзд.

09. Ринок вживаних автомобілів і кемперів

Майданчик продажу, офіс дилера, зона огляду, документи, інструменти для перевірки.

Інтеракції: огляд, тестова поїздка, торг, перевірка документів, купівля, продаж, оцінка вартості ремонту.

Унікальні місії: знайти вигідну пропозицію, виявити прихований дефект, домовитися про ціну, придбати транспорт для відновлення.

10. Притулки й альтернативне житло

Гірські притулки, прості гостьові будинки, хостели, мотелі та інші місця, де можна перечекати негоду або відновитися.

Інтеракції: бронювання, ночівля, сушіння речей, отримання місцевої інформації, знайомство з іншими мандрівниками.

Унікальні місії: знайти останнє вільне місце, допомогти іншому туристу, змінити маршрут через закриття стежки.

Ці десять категорій — лише основа. У кожному регіоні повинні бути свої варіанти: міські кемпінги, лісові майданчики, узбережжя, гірські стоянки, фермерські місця для ночівлі, спеціальні велокемпінги, зони для караванів і сервісні станції. Вони мають відрізнятися не тільки текстурами, а й правилами, доступними послугами, NPC і типами пригод.

7. Велика бібліотека пригод: позитивні, нейтральні й складні події

Я переглянув приклади обговорень велотуристів, власників кемперів і матеріали про системне оповідання в іграх. Повторюються кілька важливих мотивів: вітер може змінити цілий велодень; невдала поверхня й навантаження змінюють темп; кемпінг може виявитися переповненим або не відповідати очікуванням; технічна проблема не завжди має очевидну причину. Водночас гравці цінують знайомства, відкриття маршрутів і несподівану допомогу.

Reddit

+3

Це не означає, що кожна реальна історія повинна стати окремою місією. Потрібно виділяти ситуацію, причину, вибір і наслідки, а потім комбінувати їх.

Каталог подій v0.3

Нижче — початковий каталог із 48 типів подій. Це не 48 повністю написаних місій, а 48 сімейств, кожне з яких може мати власні діалоги, умови та декілька розв'язок.

A. Приємні несподіванки та соціальні можливості

| ID | Подія | Можливий розвиток |
|---|---|---|
| POS-001 | Подарунок від друзів | Отримати старий намет, велосипедні сумки або кухонний набір |
| POS-002 | Розпродаж спорядження | Придбати якісну річ дешевше, але перевірити стан і сумісність |
| POS-003 | Бонус на роботі | Отримати додаткові кошти й вирішити, чи витрачати їх на подорож |
| POS-004 | Несподіваний спадок | Отримати помірну суму або річ, пов'язану з історією родича |
| POS-005 | Виграш у лотерею | Рідкісна подія; винагорода може бути невеликою, без руйнування балансу |
| POS-006 | Безкоштовне місце | Друг передає бронювання, яким не може скористатися, якщо правила дозволяють |
| POS-007 | Знайомство в кемпінгу | Обмін маршрутами, спільна вечеря або нове запрошення |
| POS-008 | Знайдена оглядова стежка | Відкрити красиву локацію, зробити фотографії й отримати запис у щоденнику |
| POS-009 | Попутний вітер | Скоротити час велопереїзду й витрати сил |

| | | |
|---|---|---|
| POS-010 | Допомога незнайомця | Отримати інструмент, пораду, підвезення або контакт майстра |
| POS-011 | Місцевий фестиваль | Додаткова активність, знайомства, сувеніри й регіональні страви |
| POS-012 | Щедрий сезонний врожай | Купити дешеві місцеві продукти на ринку або фермі |

Спадок і виграш мають бути рідкісними й логічними. Вони не повинні з'являтися як регулярний спосіб заробітку. Якщо гравець отримав велику суму, це відкриває вибір — поліпшити спорядження, зберегти гроші, погасити кредит або інвестувати в транспорт. Гра не повинна автоматично перетворювати його на багатія.

B. Повсякденні дрібниці, що створюють атмосферу

| ID | Подія | Можливий розвиток |
|---|---|---|
| LIFE-001 | У магазині немає потрібного продукту | Замінити його, відвідати інший магазин або змінити страву |
| LIFE-002 | Забули зарядний кабель | Купити заміну, позичити, обмежити споживання енергії |
| LIFE-003 | Мокрий одяг | Знайти сушильну кімнату, розвісити речі або придбати сухий комплект |
| LIFE-004 | Черга в душ | Почекати, скористатися іншим блоком або змінити розклад |

| | | |
|---|---|---|
| LIFE-005 | Сусіди готують цікаву страву | Поспілкуватися, обмінятися рецептом або залишитися наодинці |
| LIFE-006 | Дитина хоче затриматися на майданчику | Змінити графік, домовитися про час або перенести іншу активність |
| LIFE-007 | Знайдена стара фотографія | Відкрити спогад персонажа чи новий особистий сюжет |
| LIFE-008 | Невдалий кулінарний експеримент | Зіпсувати частину продуктів, навчитися новому рецепту |
| LIFE-009 | Загублений ключ | Перевірити останні місця, звернутися до адміністратора, використати запасний |
| LIFE-010 | Неочікувано гарний захід сонця | Отримати спогад, фото, позитивну реакцію супутників |
| LIFE-011 | Місцевий ринок закривається раніше | Змінити план покупок або прийти наступного дня |
| LIFE-012 | Пральня тимчасово недоступна | Вибрати іншу послугу або перерозподілити час |

Такі події важливі не менше за поломки. Вони роблять світ живим, але не змушують гравця постійно вирішувати кризи.

C. Велосипедні пригоди

| ID | Подія | Причини та вибір |
|---|---|---|
| BIKE-001 | Прокол шини | Ремонт, запасна камера, майстерня |
| BIKE-002 | Ланцюг зіскочив | Відновити на місці, використати інструмент або звернутися по допомогу |
| BIKE-003 | Складний підйом | Знизити темп, пройти пішки, змінити маршрут |
| BIKE-004 | Сильний зустрічний вітер | Скоротити етап, перечекати або знайти захищену дорогу |
| BIKE-005 | Гальма працюють гірше | Зупинитися й оглянути систему, відвідати майстерню |
| BIKE-006 | Дорога переходить у сипкий ґрунт | Продовжити обережно, розвернутися або знайти об'їзд |
| BIKE-007 | Неправильно розподілений багаж | Перепакувати речі, зменшити навантаження |
| BIKE-008 | Сідло натирає | Змінити налаштування, зробити перерву, придбати відповідне спорядження |
| BIKE-009 | Злива під час переїзду | Використати дощовик, знайти укриття, змінити план |

| | | |
|---|---|---|
| BIKE-010 | Знайдений веломаршрут | Відкрити альтернативний шлях і нові точки інтересу |
| BIKE-011 | Інший велосипедист потребує допомоги | Поділитися інструментом або разом дістатися майстерні |
| BIKE-012 | Велосипед не підходить для маршруту | Перейти на коротший шлях, змінити шини або орендувати інший велосипед |

Важливо моделювати не тільки швидкість, а й набір висоти, поверхню, вагу багажу, фізичну форму та напрямок вітру. Результат повинен бути зрозумілим: важкий велосипед на крутому підйомі потребує більше зусиль, а не раптом отримує випадковий штраф.

D. Автомобілі, каравани й кемпери

| ID | Подія | Причини та вибір |
|---|---|---|
| VEH-001 | Низький тиск у шині | Перевірити, підкачати, ремонтувати |
| VEH-002 | Розряджений акумулятор | Зарядити, отримати допомогу, замінити батарею |
| VEH-003 | Перегрів двигуна | Безпечно зупинитися, дати охолонути, викликати сервіс |
| VEH-004 | Невдалий під'їзд до кемпінгу | Перевірити габарити, обрати інший в'їзд |

| | | |
|---|---|---|
| VEH-005 | Перевантажений автомобіль | Перепакувати багаж і перевірити допустиму масу |
| VEH-006 | Погана витрата пального | Перевірити навантаження, тиск шин, технічний стан |
| VEH-007 | Несправність холодильника | Перевірити живлення, вентиляцію й налаштування; за потреби ремонт |
| VEH-008 | Розряджена побутова батарея | Скоротити споживання, зарядити від сумісного джерела |
| VEH-009 | Витік води | Знайти джерело, перекрити воду, виконати ремонт |
| VEH-010 | Неприємний запах у салоні | Перевірити туалет, баки, зливи, вентиляцію та ущільнення |
| VEH-011 | Переповнений бак сірої води | Знайти дозволену сервісну точку |
| VEH-012 | Прихований дефект старого кемпера | Оцінити ремонт, торгуватися, відмовитися від купівлі або відновити техніку |

Події повинні враховувати конкретну конструкцію транспортного засобу. Якщо в моделі немає побутової батареї або стаціонарного туалету, відповідні події не повинні запускатися.

Е. Бронювання, маршрут і погода

| ID | Подія | Причини та вибір |
|---|---|---|
| TRIP-001 | Кемпінг переповнений | Інше місце, альтернативна ночівля, зміна регіону |
| TRIP-002 | Пропущений пором | Наступний рейс, інший маршрут, ночівля |
| TRIP-003 | Дорожні роботи | Об'їзд, додатковий час, зміна зупинки |
| TRIP-004 | Закритий гірський маршрут | Альтернативна стежка, відпочинок, інший регіон |
| TRIP-005 | Раптова гроза | Укриття, скорочення маршруту, перенесення активності |
| TRIP-006 | Спека | Додаткова вода, ранній старт, тінь, коротший етап |
| TRIP-007 | Сильний вітер | Зміна маршруту або часу виїзду |
| TRIP-008 | Пошкоджене бронювання | Перевірка підтвердження, зв'язок із рецепцією, альтернативне місце |
| TRIP-009 | Неправильна оцінка часу | Перерахунок етапу, скорочення активностей |

| | | |
|---|---|---|
| TRIP-010 | Дорога не підходить великому транспорту | Об'їзд за габаритами й обмеженнями |
| TRIP-011 | Затримка на кордоні або переправі | Перевірка документів, резерв часу, зміна плану |
| TRIP-012 | Несподівано вільна стоянка | Перевірка правил і послуг, можливість змінити план |

F. Соціальні, сімейні та особисті події

| ID | Подія | Причини та вибір |
|---|---|---|
| SOC-001 | Друг не може поїхати | Запросити іншого, подорожувати самому, змінити бронювання |
| SOC-002 | Родина хоче різного | Компроміс, розділення активностей, зміна маршруту |
| SOC-003 | Супутник втомився | Відпочинок, коротша прогулянка, перенесення плану |
| SOC-004 | Новий знайомий запрошує на подію | Приєднатися, відмовитися, домовитися на інший день |
| SOC-005 | Сусіди потребують інструмента | Допомогти, позичити або порадити майстерню |

| | | |
|---|---|---|
| SOC-006 | Обіцянка іншому NPC | Виконати, перенести з поясненням або втратити частину довіри |
| SOC-007 | Родич дарує стару річ | Відремонтувати, використовувати, продати або зберегти як пам'ять |
| SOC-008 | Місцевий волонтерський захід | Допомогти, познайомитися з людьми, отримати нові навички |
| SOC-009 | Супутник хоче провести день інакше | Узгодити цілі або дозволити окрему активність |
| SOC-010 | Знайомство переходить у дружбу | Відкриваються спільні поїздки й довгострокові сюжетні гілки |
| SOC-011 | Подарунок на день народження | Підготувати сюрприз, вибрати активність, вплинути на стосунки |
| SOC-012 | NPC переїжджає або змінює роботу | Зустрітися востаннє, підтримувати контакт, відкрити нову історію |

G. Тварини, спорядження й побут

| ID | Подія | Причини та вибір |
|---|---|---|
| PET-001 | Кемпінг не приймає тварин | Інший кемпінг, дозволена зона або альтернативне житло |

| | | |
|---|---|---|
| PET-002 | Собака втомилася | Відпочинок, вода, скорочення маршруту |
| PET-003 | Кіт нервує в дорозі | Спокійне місце, перевірка переноски, пауза |
| PET-004 | Закінчився корм | Купити сумісний корм, змінити план покупок |
| PET-005 | Немає потрібних документів тварини | Перевірити правила, змінити маршрут до готовності документів |
| PET-006 | Загубився повідець або миска | Придбати заміну, знайти предмет, позичити |
| GEAR-001 | Зламалася блискавка намету | Тимчасовий ремонт або заміна |
| GEAR-002 | Протікає дощовик | Ремонт, просочення, покупка нового |
| GEAR-003 | Пошкодився пальник | Діагностика, безпечна заміна, інша їжа |
| GEAR-004 | Не вистачає питної води | Купити, знайти перевірене джерело, змінити маршрут |
| GEAR-005 | Продукти швидко псуються | Використати першими, змінити меню, безпечно утилізувати |

| | | |
|---|---|---|
| GEAR-006 | Загублений ліхтар | Знайти, купити, позичити або використати запасний |
| GEAR-007 | Виявлено корисну знижку | Купити зараз або зберегти бюджет |
| GEAR-008 | Знайдено безкоштовний предмет | Перевірити стан, сумісність і реальну користь |

Цей каталог — перша версія бібліотеки. Для повноцінної гри кожна сім'я подій потребуватиме варіантів діалогів, декількох результатів, перевірок передумов і способів повторного запуску. Саме це перетворить перелік подій на сценарну систему.

8. Як перетворити події на історії, а не випадкові повідомлення

Найкраща модель для CampLife — поєднати авторські сюжетні арки з системними пригодами. Саме такий поділ дозволяє одночасно мати цілі, які ведуть гравця вперед, і свободу, завдяки якій кожна подорож відрізняється. Цей підхід відповідає загальним принципам системного оповідання в симуляторах: історія виникає через взаємодію характерів, ресурсів, світу та рішень.

GameDesign.gg

+1

Приклад: історія старого кемпера

Початок — оголошення про продаж

Гравець знаходить старий кемпер за привабливою ціною. Продавець стверджує, що транспорт потребує лише дрібних робіт.

Гілка А: перевірити

Витратити гроші на огляд, виявити проблеми й торгуватися.

Гілка B: купити одразу

Зекономити на перевірці, але прийняти невизначеність.

Гілка C: відмовитися

Зберегти гроші й продовжити пошук іншого транспорту.

Гілка D: домовитися

Запропонувати ремонт або обмін на інше майно, якщо продавець зацікавлений.

Продовження

Стан кемпера визначає доступні ремонти. Гравець може працювати над ним у вільний час, платити майстерні, шукати вживані деталі або поступово відновлювати транспорт. Відремонтований кемпер відкриває нові маршрути й може стати частиною наступної сюжетної арки.

Одна покупка може створити десятки варіантів історії. Важливо, щоб наслідки залишалися логічними й довготривалими, а не зникали після зміни локації.

Система пам'яті світу

Гра повинна зберігати важливі факти:

- де гравець був;
- які рішення прийняв;
- кому допоміг;
- які обіцянки дав;
- які предмети придбав;
- що відремонтував;
- які маршрути відкрив;
- які місця сподобалися його супутникам;
- які проблеми виникли через підготовку або її відсутність.

Ці дані використовуються для наступних місій. Наприклад, NPC, якому гравець допоміг у першій поїздці, може через кілька місяців надіслати повідомлення про вигідну пропозицію кемпера.

9. Технічна основа сценарної біблії

Щоб усе це було реалізовано без хаосу, сценарні дані повинні зберігатися окремо від коду руху, економіки й фізики.

Приклад структури однієї події:

```
type GameEvent = {
  id: string;
  title: string;
  category: string;
  locationTags: string[];
  requiredConditions: string[];
  excludedConditions: string[];
  warningLevel: "none" | "advisory" | "urgent";
  choices: {
    id: string;
    text: string;
    requirements?: string[];
    consequences: {
      timeMinutes?: number;
      moneyChange?: number;
      skillChanges?: Record<string, number>;
      relationshipChanges?: Record<string, number>;
      unlockEventIds?: string[];
    };
  }[];
  cooldownDays: number;
  repeatable: boolean;
```

```
  storyFlagsSet?: string[];

};
```

Це ілюстративний контракт даних, а не повна реалізація. Для реального коду потрібно буде замінити текстові умови на перевірювані правила, додати локалізацію, перевірку конфліктів, версіонування та автоматичні тести.

Правила, без яких система розвалиться

1. Подія не може запуститися, якщо не виконані її передумови.
2. Подія не може пропонувати дію, недоступну в поточній ситуації.
3. Витрати, час, зміни стану та нагороди мають бути явно описані.
4. Події, які вже відбулися, не повинні повторюватися без причини.
5. Важливі сюжетні факти зберігаються між локаціями й після перезапуску гри.
6. Одна подія не повинна змінювати той самий ресурс двічі через повторну обробку.
7. Після будь-якого вибору гравець повинен розуміти, що змінилося й чому.
8. Рідкісні позитивні події, як-от спадок або великий виграш, мають бути збалансовані й не ламати економіку.
9. Небезпечні ситуації повинні мати зрозумілі передумови й можливість безпечного рішення.
10. Якщо гра не знає фактичного правила реальної локації, вона не повинна видавати припущення за підтверджений факт.

10. Як довести сценарій до повного виробничого документа

Наступний крок — не вигадувати сотні подій навмання, а перетворити цей каталог на перевірену бібліотеку з конкретними сценаріями.

Для кожної події потрібно створити окрему картку, яка містить:

- унікальний ID і назву;
- тип події та її сюжетну роль;
- місце й умови запуску;
- персонажів, які можуть брати участь;
- передісторію та вступний діалог;
- усі варіанти вибору;
- точні вимоги до кожного варіанта;
- безпосередні та відкладені наслідки;
- позитивні, нейтральні й негативні результати;
- нагороди, витрати та зміни навичок;
- зв'язки з іншими подіями;

- умови повторного запуску;
- правила завершення;
- тестові випадки для перевірки коду.

Пріоритет виробництва

| Етап | Результат |
|---|---|
| 1. Створення персонажа | Редактор, професії, родина, тварини, стартові ресурси |
| 2. Перші 90 хвилин | Повністю готовий вступ без нав'язаної кризи |
| 3. Перша кампанія | Усі місії, діалоги, вибори й результати від початку до завершення |
| 4. Сценарна система | Умови, пам'ять світу, ланцюжки подій, запобігання повторенням |
| 5. Бібліотека пригод | Розширені сценарії за транспортом, погодою, побутом, соціальними ситуаціями |
| 6. Географія та реальні дані | Регіони, маршрути, кемпінги, транспортні обмеження, джерела й дати перевірки |
| 7. Баланс і тестування | Економіка, темп гри, доступність, перевірка всіх гілок і збережень |

Для браузерної версії я рекомендую 2D рушій на кшталт Phaser із перспективою 3/4, локальними tilemap-сценами та окремою моделлю світу. Важливо, щоб сценарна бібліотека не залежала від конкретного рендерингу: це полегшить майбутній перенос на ПК.

Підсумок: тепер у нас є значно ширша основа: система створення персонажа, дванадцять життєвих профілів, перші унікальні NPC, десять типів локацій, план м'якого вступу, 48 сімейств пригод і схема даних для їх реалізації. Це ще не завершена сценарна біблія — для неї потрібно детально написати кожну місію, кожен діалог і всі гілки. Але тепер ми маємо структуру, яка дозволяє робити це послідовно, логічно й без перетворення гри на хаотичний список випадкових подій.

проаналізуй чи нічого ти не забув? чи все детально і логічно і розписано і сценарій і промпти тощо! Якщо що то пиши далі я не проти щоб ти продовжив розписувати гру далі але з розумом і детальним аналізом і вивіряно бо в нас є вище багато чого вже як стовбур але він голий без гілок і листя так би мовити то ти думай ширше і пропонуй розвиток сценарію і дії і розписуй далі

Я переглянув попередній план як єдиний проєкт: від задуму гри до архітектури, перших 90 хвилин, сценарію, промптів для AI-кодера та перевірки результату.

Висновок: основа правильна, але попередній комплект промптів ще не достатньо повний, щоб безпечно віддати його AI-розробнику й очікувати цілісну гру. Він добре описує перші технічні кроки, але місцями перескакує від одного працюючого екрана до іншого, не визначивши всіх правил, які мають їх об'єднувати.

Твоя метафора зі стовбуром дуже влучна. У нас є стовбур: технологія, системи, загальний сюжет, світ і принципи гри. Тепер потрібні гілки — детальні сценарії, правила поведінки світу, характери, діалоги, розгалуження, а також листя — конкретні предмети, реакції NPC, маленькі відкриття, візуальні деталі й десятки взаємопов'язаних ситуацій.

Я не став би зараз просто додавати ще двадцять промптів на код. Спочатку потрібно закрити прогалини в проєктуванні, щоб AI-кодер не почав самостійно вигадувати правила, які суперечать нашим попереднім рішенням.

1. Що вже продумано, а чого бракує

### Вже закладено добре

- Технологічна основа: TypeScript, Phaser, вебінтерфейс.
- Три масштаби світу: Європа, маршрут, локальна сцена.
- Створення персонажа, домогосподарство, тварини.
- Інвентар, бюджет, транспорт, погода, бронювання.
- Унікальні NPC, події, квести, збереження.
- Поступова розробка замість спроби написати все одразу.
- Позитивний вступ без ранніх покарань.

### Потрібно допрацювати до активної розробки

- Єдину специфікацію ігрових правил і стану світу.
- Повний сценарій першої подорожі з діалогами та розгалуженнями.
- Порядок місій, їхні передумови, провали й повторне проходження.
- Поведінку NPC поза діалогом і пам'ять про вчинки гравця.
- Конкретні правила часу, грошей, потреб, транспорту й інвентарю.
- Межі між Phaser, UI, симуляцією та збереженням.
- Контракти даних, схему тестування та захист від несумісних змін.
- Повний перелік ресурсів, станів анімацій і художніх вимог.

### Що потрібно додати як окремі документи

- Біблію світу та сценарію.
- Специфікацію першого ігрового циклу.
- Словник термінів та єдині ID сутностей.
- Технічні контракти між модулями.
- Реєстр контенту й графічних ресурсів.
- План тестів і критерії готовності кожного етапу.
- Інструкції для AI-кодера щодо роботи з контекстом, Git і виправлення помилок.

## 2. Найважливіші логічні прогалини

### 2.1. Не визначено, що саме означає «готовий етап»

Наприклад, AI може створити магазин, у якому працює кнопка покупки, але не врахує вагу предметів, обмеження багажника, вже наявне спорядження чи те, що персонаж має кота.

Формально магазин працює, але з погляду гри — система ще не завершена.

Тому треба розділяти три рівні готовності:

- Технічний: код запускається, типи й тести проходять.
- Функціональний: гравець може виконати дію від початку до кінця.
- Ігровий: результат логічний, зрозумілий, цікавий і не суперечить іншим системам.

Етап не можна вважати повністю завершеним лише на першому рівні.

2.2. Не описано повну причинно-наслідкову модель

У грі має бути ланцюг на кшталт:

*гравець обрав велосипед → придбав багажні сумки → завантажив надто багато речей → підйом став складнішим → довелося зробити додаткову зупинку → прибуття змістилося → змінилася доступність часу на кемпінгу.*

Але це не повинно автоматично перетворюватися на покарання. Гравець може зменшити багаж, змінити маршрут, відпочити або обрати інший транспорт.

Потрібно прописати не тільки самі параметри, а й те, які системи можуть їх змінювати, які наслідки дозволені й як гравець дізнається про ризик.

2.3. Сюжет поки що описаний переважно як список подій

Назва події — це ще не сценарій. Для кожної важливої події потрібні:

- причина появи;
- місце й час;
- персонажі, які беруть участь;
- початковий стан;
- репліки;
- вибір гравця;
- умови доступних варіантів;
- витрачений час;
- зміни грошей, предметів і стосунків;
- наслідки для майбутніх місій;
- спосіб завершення та захист від випадкового повтору.

Саме це перетворює каталог подій на живу гру.

2.4. Відсутній повний сценарій першої подорожі

У нас є початок, магазин і перший кемпінг, але ще не повністю визначені:

- підготовка вдома;
- вибір між наметом і кемпером;
- пакування та перевірка спорядження;
- відправлення;
- прибуття;
- заселення;
- перший вечір;
- ніч;
- ранок;
- знайомство з місцевістю;
- маленька пригода;
- повернення;
- підсумок витрат;
- пам'ять про поїздку й нові цілі.

Це має стати першою завершеною сюжетною дугою, а не просто переходом між двома картами.

2.5. Не вистачає правил для складних ситуацій

Наприклад:

- Що буде, якщо гравець не купить спальник?
- Якщо забуде корм для тварини?
- Якщо не вистачить грошей на кемпінг?
- Якщо прибуде після закриття рецепції?
- Якщо вирішить не виконувати місію?
- Якщо закриє браузер посеред діалогу?
- Якщо дві події хочуть відбутися одночасно?
- Якщо гравець повернеться до NPC після завершення його завдання?

Такі ситуації потрібно опрацьовувати не всі відразу, але архітектура повинна дозволяти їх коректно обробляти.

3. Як я пропоную перебудувати весь процес

Замість одного довгого списку завдань для AI-кодера створимо п'ять рівнів специфікації.

1

Правила проєкту

Що за гра, які принципи не можна порушувати, які технології використовуються, як працюють час, стан світу й дані.

2

Сценарна біблія

Персонажі, їхні біографії, локації, сюжетні арки, місії, діалоги, вибори та наслідки.

3

Специфікація першої версії

Які саме функції реалізуємо, які дані вони використовують, що має бути видно гравцю та як перевіряється результат.

4

Промпти на реалізацію

Маленькі завдання в правильній послідовності. Кожне посилається на затверджені правила, а не винаходить їх заново.

5

Перевірка й розширення

Автотести, ручне проходження, виправлення, балансування та поступове додавання нових історій.

Тепер доповню попередній комплект конкретними документами й промптами. Вони закривають найважливіші прогалини до того, як ми почнемо додавати сотні механік.

Частина 8. Новий документ — правила, які AI не має права вигадувати самостійно

CampLife: The Journey — Core Rules v1.0

1. Призначення

Цей документ є джерелом істини для всіх AI-агентів, які розробляють гру.

Якщо промпт на окрему функцію суперечить цьому документу, не змінюй базові правила мовчки. Вкажи на конфлікт і запропонуй узгоджене рішення.

2. Ігрова фантазія

Гравець створює власного персонажа й організовує життя, відпочинок і подорожі. Він може подорожувати самостійно, із партнером, друзями, дітьми та домашніми тваринами.

Головні цілі гри:

- відкривати нові місця;
- планувати подорожі;
- облаштовувати кемпінг;
- знайомитися з людьми;
- накопичувати досвід і навички;
- приймати побутові та фінансові рішення;
- створювати особисті історії;
- повертатися додому й планувати наступну поїздку.

Гра не є симулятором виживання з неминучою смертю або постійним покаранням. Серйозні ризики мають бути зрозумілими, а більшість проблем повинні мати кілька способів вирішення.

3. Незмінні принципи

1. Гравець не зобов'язаний виконувати всі місії.
2. Негативні події не повинні з'являтися без правдоподібної причини.
3. Перед серйозним ризиком гравець повинен отримати доступне попередження, якщо ризик можна було передбачити.
4. Кожна важлива дія має зрозумілий наслідок.
5. Позитивні події є повноцінною частиною симуляції.
6. Вибір гравця може змінювати майбутні діалоги, стосунки, доступні завдання й можливості.
7. NPC мають унікальні ідентифікатори, біографії та історії.
8. Домашні тварини, вибрані гравцем, залишаються частиною домогосподарства.
9. Меню не повинні підміняти локальні сцени там, де гравець має вільно пересуватися.
10. Реальні дані не можна видавати за актуальні, якщо їхня актуальність не перевірена.
11. Жодна нова функція не повинна непомітно руйнувати вже збережені проходження.

12. Навчальний вступ має бути доброзичливим і не повинен починатися з серйозної аварії, небезпечної погоди або бюджетної кризи.

## 4. Єдине джерело стану

Ігровий стан не можна зберігати лише у візуальних компонентах.

Стан містить:

- ідентифікатор проходження;
- версію схеми збереження;
- календар і поточний час;
- активну локацію та позицію персонажа;
- персонажа й домогосподарство;
- інвентар і спорядження;
- гроші, доходи, борги та витрати;
- транспорт і його технічний стан;
- маршрут і бронювання;
- потреби, навички й здоров'я;
- NPC, стосунки та пам'ять;
- активні й завершені місії;
- прапорці сюжету;
- історію важливих подій.

UI читає стан і відправляє команди. Системи перевіряють умови й застосовують зміни.

## 5. Команди й події

Команда — це намір гравця, наприклад BuyItem, StartTrip, TalkToNpc або CompleteTask.

Подія — підтверджений факт, наприклад ItemPurchased, TripStarted або NpcRelationshipChanged.

Команда не повинна вважатися успішною, поки відповідна система не перевірила її умови.

Неможливі дії мають повертати зрозумілий результат відмови, а не тихо змінювати стан.

## 6. Час

- Рух і анімація використовують час кадру.
- Ігрові дії використовують ігровий час.
- Довгі дії мають задану тривалість.
- Відкриття критичного меню призупиняє симуляцію, якщо інше не визначено явно.
- Перехід між локаціями не скидає календар і стан світу.
- Заплановані події не повинні виконуватися двічі через повторне завантаження.
- Час поїздки залежить від маршруту, транспорту, зупинок та умов.

## 7. Економіка

- Усі ціни зберігаються в одному каталозі даних.
- Кожна операція має однозначний результат.
- Покупка не може списати гроші двічі через повторний клік.
- Суми в інтерфейсі мають збігатися з фактичними змінами балансу.
- Початкові ресурси залежать від стартового профілю.
- Початкові профілі мають бути приблизно співставними за можливістю розпочати цікаву гру, але не обов'язково мати однакові гроші.
- Борги, подарунки, премії та випадкові доходи повинні бути явними ігровими операціями.

## 8. Сюжет і місії

Кожна місія має унікальний ID, умови запуску, цілі, можливі вибори, результати та умови завершення.

Місії не повинні залежати від конкретного порядку кліків, якщо це не передбачено сценарієм.

Гравець може відкласти більшість побутових і необов'язкових завдань. Система повинна підтримувати невиконані, завершені, провалені та недоступні стани, якщо вони потрібні конкретній місії.

9. Сумісність

Усі збереження мають версію схеми. Зміни типів і даних повинні супроводжуватися міграціями або явним рішенням про несумісність.

Новий модуль не має права самостійно змінювати правила інших модулів без узгодження контракту.

10. Рішення, яких ще немає

Якщо точне значення не визначено, не вигадуй складну систему. Обери найпростішу узгоджену реалізацію, познач її як тимчасову й додай у список відкритих рішень.

Не вигадуй реальні ціни, технічні характеристики, дорожні обмеження або факти про реальні кемпінги.

11. Критерій готовності

Функція вважається завершеною, коли:

- її поведінка відповідає специфікації;
- стан змінюється правильно;
- результат можна побачити в грі;
- помилкові дії обробляються;
- тести проходять;
- збереження працює, якщо функція змінює довготривалий стан;
- немає непояснених регресій.

Частина 9. Перша сюжетна арка — не просто вступ, а маленька історія

Я хочу змінити один момент у попередньому сценарії: перша подорож не повинна зводитися до «знайшов лист → купив речі → приїхав → забрав столик». Це функціонально, але не створює особливої історії.

Краще зробити її історією про те, як давнє бажання нарешті стає справжньою подорожжю. Гравець має самостійно ухвалювати рішення, а NPC — не просто видавати квести, а мати власні цілі.

Сценарій «Перша ніч поза домом»

Глава 1. Мрія на столі

Орієнтовно 10–15 хвилин

Персонаж знаходить старий блокнот, у якому колись записав місця, які хотів відвідати. Він може прочитати нотатки, оглянути спорядження та обговорити поїздку з членами домогосподарства.

Глава 2. Чого нам бракує?

Орієнтовно 10–20 хвилин

Гравець перевіряє наявні речі, обирає, що взяти із собою, і вирішує, чи купувати нове спорядження. Продавець допомагає підібрати речі під конкретний спосіб подорожі, а не нав'язує найдорожчі.

Глава 3. Дорога починається

Орієнтовно 5–15 хвилин

Перед від'їздом гравець перевіряє маршрут, погоду, спорядження й час заїзду. Перший маршрут простий, але дає змогу побачити, як відстань, зупинки та обраний транспорт впливають на час і витрати.

Глава 4. Нове місце

Орієнтовно 15–20 хвилин

Elise Vermeulen зустрічає гравця. Можна обрати місце, розпитати про правила, воду, душові, сміття та найближчу прогулянкову стежку. Тут гра навчає побуту кемпінгу через дії.

Глава 5. Маленька історія

Орієнтовно 10–15 хвилин

Гравець знайомиться з іншим відпочивальником, допомагає вирішити дрібну ситуацію або ділиться річчю. Виникає перша необов'язкова історія, яку можна згадати в наступній поїздці.

Глава 6. Ранок і повернення

Орієнтовно 10–15 хвилин

Гравець прибирає місце, збирає речі, перевіряє витрати та повертається додому. Після поїздки з'являється запис у щоденнику, нова пам'ятка про маршрут і кілька можливих цілей для майбутнього.

Це орієнтир тривалості, а не жорсткий таймер. Якщо гравець захоче поговорити з усіма, дослідити місцевість або змінити план, гра не повинна його квапити.

Чому саме така побудова

У першій подорожі ми непомітно навчаємо шести систем:

1. Взаємодія з предметами.
2. Планування й бюджет.
3. Інвентар і спорядження.
4. Транспорт і час.
5. Кемпінгова інфраструктура.
6. NPC, діалоги й наслідки рішень.

Але кожна система одразу дає щось цікаве: знахідку, нове місце, знайомство, спільну історію або запис у щоденнику. Гравець не повинен відчувати, що проходить підручник.

Приклад справжнього розгалуження

У кемпінгу гравець бачить складаний столик біля спільної зони.

Elise: «Схоже, наші гості залишили його після від'їзду. Я ще перевіряю, чи хтось за ним повернеться».

Гравець має три варіанти.

- Запропонувати допомогти знайти власника. Витрачається трохи часу. Elise запам'ятовує вчинок; згодом це може відкрити іншу соціальну подію.
- Запитати, чи можна тимчасово скористатися столиком. Якщо правила кемпінгу дозволяють, гравець може користуватися ним, не забираючи назавжди.
- Не втручатися. Жодного штрафу немає; гравець продовжує облаштовувати місце.

Якщо власник згодом повернеться, NPC пам'ятає, чи допоміг гравець, і може згадати цю ситуацію. Так з'являється історія з продовженням, а не одноразова нагорода за натискання кнопки.

Частина 10. Промпти, яких бракувало

Ці промпти не замінюють попередні 0–12. Вони доповнюють їх, щоб AI-кодер спочатку отримав чіткі специфікації, а потім реалізовував їх послідовно.

PROMPT 13 — Аудит специфікації перед розробкою

Ти працюєш над CampLife: The Journey.

Перед продовженням розробки перевір усі наявні документи, типи, код і рішення. Не починай створювати нові великі системи.

Підготуй реєстр специфікації з такими розділами:

1. Основні ігрові правила.
2. Системи та їхні відповідальності.
3. Моделі даних.

4. Сценарні події й місії.
5. UI та взаємодії.
6. Локації й переходи між ними.
7. Збереження й версії даних.
8. Художні ресурси.
9. Тести й критерії готовності.
10. Невирішені питання.

Для кожного пункту вкажи:

- ID;
- опис;
- статус: не визначено, заплановано, реалізовано, протестовано;
- залежності;
- джерело вимоги;
- ризик у разі пропуску.

Знайди:

- суперечності між документами;
- функції, які згадані, але не мають специфікації;
- дані, що зберігаються у кількох місцях;
- невизначені правила переходу між системами;
- сценарії без умов завершення;
- кнопки й дії без визначеної поведінки;
- відсутні негативні та граничні тести.

Не вигадуй відсутні правила. Для кожної прогалини запропонуй мінімальне рішення, але познач його як пропозицію, що потребує затвердження.

Після звіту зупинись. Не змінюй код.

---

PROMPT 14 — Повна специфікація першої сюжетної арки

Створи повну сценарну специфікацію першої подорожі CampLife: The Journey під назвою «Перша ніч поза домом».

Не реалізовуй код. Спочатку створи сценарні дані й документацію.

Арка включає:

1. Початок удома.
2. Знайомство з блокнотом і мрією про подорож.
3. Перевірку наявного спорядження.
4. Вибір маршруту й тривалості.
5. Покупки та підготовку.
6. Відправлення.
7. Прибуття в кемпінг.
8. Реєстрацію на рецепції.
9. Облаштування місця.
10. Першу соціальну взаємодію.
11. Вечір і ніч.
12. Ранок.
13. Збирання речей.
14. Повернення.
15. Підсумок подорожі й нові цілі.

Для кожного етапу визнач:

- унікальний ID;
- локацію;
- передумови;
- початковий стан;
- доступні дії;
- повні тексти діалогів;
- варіанти вибору;
- перевірки умов;
- зміни стану;
- витрати часу й грошей;
- предмети, які отримуються або використовуються;
- наслідки для NPC;
- наступні можливі етапи;
- умови завершення;
- спосіб відновлення після завантаження збереження.

Обов'язково передбач:

- гравець може відмовитися від необов'язкової дії;
- гравець може оглянути локацію до виконання цілі;
- гравець може мати різне спорядження;
- гравець може подорожувати різними типами транспорту;
- гравець може мати тварину або не мати її;
- відсутній предмет не спричиняє м'якої блокади всієї гри;
- помилковий вибір має зрозумілий результат;
- одноразові події не повторюються випадково;
- перша арка має позитивне завершення, навіть якщо гравець не виконав усі необов'язкові завдання.

Зроби граф переходів між етапами. Перевір, чи кожна гілка має шлях до завершення або зрозуміле повернення до основного сценарію.

Не додавай події, які неможливо пояснити станом світу. Не вводь нових NPC без ID та короткої біографії.

Результат повинен бути достатньо детальним, щоб інший розробник міг реалізувати сценарій без самостійного вигадування діалогів і наслідків.

---

PROMPT 15 — Схема даних і контракти між системами

Проаналізуй реалізовані модулі CampLife: The Journey та затверджену специфікацію.

Створи єдині типізовані контракти для:

- WorldState;
- Character;
- Household;
- Pet;
- Inventory;
- ItemDefinition;
- Vehicle;
- TripPlan;

- LocationDefinition;
- NPCDefinition;
- NPCState;
- QuestDefinition;
- QuestState;
- EventDefinition;
- EventState;
- Reservation;
- SaveGame.

Для кожної сутності визнач:

- унікальний ID;
- обов'язкові поля;
- необов'язкові поля;
- допустимі значення;
- правила валідації;
- зв'язки з іншими сутностями;
- правила збереження;
- версію схеми, якщо вона необхідна.

Розділи статичні визначення та стан проходження. Наприклад, ItemDefinition описує, що таке спальник, а інвентар містить кількість, стан і місце зберігання конкретного екземпляра.

Не допускай, щоб гроші, інвентар або прогрес квестів зберігалися як незалежні суперечливі копії в різних UI-компонентах.

Визнач командні контракти для критичних дій:

- створення персонажа;
- покупки;
- використання предмета;
- початку подорожі;
- прибуття;
- початку діалогу;
- завершення квесту;
- переходу між локаціями;

- збереження та завантаження.

Для кожної команди визнач успішний результат і причини відмови.

Додай тести на валідацію та некоректні дані. Не переписуй увесь проєкт, якщо можна додати контракти поступово.

---

PROMPT 16 — Повноцінна система місій і розгалужень

Реалізуй універсальну систему квестів на основі затверджених даних.

Система повинна підтримувати:

- передумови;
- необов'язкові цілі;
- обов'язкові цілі;
- кілька шляхів завершення;
- відкладене виконання;
- залежність від предметів, часу, місця та стосунків;
- нагороди;
- зміни стосунків;
- відкриття наступних місій;
- одноразові та повторювані місії;
- збереження прогресу;
- зрозумілі повідомлення про неможливість виконати дію.

Розділи визначення місії та стан конкретного проходження.

Не прив'язуй квести до назв UI-кнопок або конкретних класів Phaser. Система повинна працювати з ігровими командами й даними.

Не створюй нову окрему реалізацію для кожного квесту. Нові місії мають додаватися через перевірювані дані.

Почни з трьох місій:

1. Перевірити спорядження.
2. Прибути до кемпінгу й заселитися.
3. Познайомитися з NPC та завершити першу соціальну подію.

Критерії:

- кожна місія має коректні стани;
- прогрес зберігається;
- необов'язкові цілі не блокують завершення;
- повторне виконання працює лише за визначеними правилами;
- неможливі переходи відхиляються;
- всі переходи перевіряються тестами.

---

PROMPT 17 — Перевірка цілісності сюжету

Проведи сценарний аудит першої сюжетної арки.

Не змінюй сюжет автоматично. Спочатку знайди проблеми.

Перевір:

1. Чи всі NPC, локації, предмети й місії мають унікальні ID.
2. Чи є діалоги для всіх обов'язкових сюжетних взаємодій.
3. Чи має кожна місія шлях до завершення.
4. Чи не вимагає місія предмета, який неможливо отримати.
5. Чи не блокують необов'язкові дії основний сюжет.
6. Чи враховуються різні домогосподарства й тварини.
7. Чи не виникають суперечливі зміни грошей або інвентарю.
8. Чи не можуть одночасно відбутися взаємовиключні події.
9. Чи пам'ятають NPC ключові попередні взаємодії.
10. Чи зрозумілі наслідки вибору.
11. Чи може гравець повернутися після відхилення від основного маршруту.
12. Чи зберігається прогрес після перезавантаження.
13. Чи відповідають результати обіцянкам у діалогах.
14. Чи немає необґрунтованих покарань у навчальній арці.

15. Чи є позитивне завершення для гравців із різними стилями проходження.

Для кожної проблеми наведи приклад послідовності дій, яка її спричиняє, і запропонуй мінімальне виправлення.

Завершуй звіт таблицею: проблема, серйозність, причина, пропоноване рішення, необхідні тести.

Не додавай нові місії, поки аудит не завершений.

---

PROMPT 18 — Контроль регресій після змін

Перед завершенням кожного нового етапу порівняй поточний стан проєкту з останньою стабільною версією.

Перевір:

- чи не зникли раніше працюючі функції;
- чи не змінилися неузгоджено типи;
- чи не зламалися переходи між сценами;
- чи не дублюються слухачі подій;
- чи не створюються кілька копій систем;
- чи не губляться дані при переході між UI та Phaser;
- чи не порушена логіка часу;
- чи не змінюються гроші та інвентар без відповідної команди;
- чи не виникають повторні квести й одноразові події;
- чи сумісні старі збереження з поточною схемою.

Запусти наявні тести та додай відсутні тести для виявлених ризиків.

Не позначай етап як завершений, якщо:

- збірка не проходить;
- є критичні помилки;

- збереження пошкоджується;
- основний ігровий цикл не працює.

Якщо Git доступний, не видаляй історію та не переписуй попередні коміти. Не виконуй небезпечні операції з файлами без явного погодження.

Після перевірки підготуй короткий звіт про регресії й зупинись.

Частина 11. Ще одна критична річ — AI-кодер забуває контекст

Навіть хороший AI-кодер може після кількох десятків змін забути початкові домовленості або почати робити другу версію вже наявної системи.

Тому проєкту потрібна не тільки документація, а й постійний контрольний документ стану розробки.

PROMPT 19 — Оновлення контексту проєкту

Після завершення поточного етапу онови файл docs/PROJECT_STATUS.md.

Документ має містити:

1. Мета проєкту

Короткий опис CampLife: The Journey та її незмінних принципів.

2. Поточна стабільна версія

- Останній перевірений етап.
- Дата або ідентифікатор коміту, якщо доступний.
- Команди запуску й перевірки.
- Фактичний стан збірки.

3. Що реально працює

Перелічуй тільки перевірені функції.

4. Що реалізовано, але не перевірено

Не змішуй цей перелік із готовими функціями.

5. Відомі проблеми

Для кожної проблеми вкажи серйозність, умови виникнення та статус.

6. Важливі технічні рішення

Записуй затверджені архітектурні рішення та причини їх вибору.

7. Дані й контракти

Вкажи, де визначені основні типи, моделі та формати збереження.

8. Сценарний прогрес

Вкажи реалізовані місії, їхні ID та відомі прогалини.

9. Наступний етап

Опиши тільки найближче завдання, його передумови та критерії приймання.

10. Відкриті питання

Не вирішуй їх самостійно без необхідності. Вказуй, яке рішення потрібне.

Не копіюй сюди весь код і всю документацію. Це короткий індекс і контрольна точка для відновлення контексту.

Не позначай незавершену функцію як готову лише тому, що для неї існує файл.

Після оновлення перевір, що документ не суперечить коду й останньому звіту про тести.

Практично я б тримав у корені проєкту ще три документи:

- `docs/PROJECT_STATUS.md` — де ми зараз.
- `docs/DECISIONS.md` — чому ухвалили саме такі технічні рішення.
- `docs/CONTENT_REGISTRY.md` — усі NPC, місії, предмети, локації й події з їхніми ID та статусом готовності.

Це запобігатиме ситуації, коли AI вигадує другого NPC з тією самою біографією, створює дубль місії або забуває, що певний предмет уже має встановлену поведінку.

Частина 12. Як зробити NPC справді живими

У попередній архітектурі ми згадували пам'ять NPC, але недостатньо конкретно описали, що саме персонаж пам'ятає.

Наприклад, Elise має пам'ятати не всі натискання кнопок, а значущі факти:

- гравець уперше приїхав до її кемпінгу;
- чемно попросив пояснити правила;
- допоміг знайти власника забутої речі;
- залишив місце чистим;
- пообіцяв розповісти, як пройшла прогулянка;
- повернувся через кілька тижнів.

Ці факти змінюють репліки, ставлення й доступні історії.

Водночас немає потреби симулювати кожен спогад кожного NPC постійно. Для першої версії достатньо зберігати важливі сюжетні факти, стосунки й кілька останніх значущих взаємодій.

PROMPT 20 — Пам'ять і поведінка NPC

Спроєктуй і реалізуй базову систему NPC для CampLife: The Journey.

Не використовуй генеративну модель під час кожної розмови. Діалоги й результати мають бути передбачуваними, контрольованими даними гри та доступними офлайн.

Для NPC розділи:

1. Статичне визначення: ім'я, зовнішність, біографія, характер, цілі.
2. Поточний стан: місце, доступність, активна дія.
3. Стосунки: рівень довіри, знайомства, конфлікти або дружба.
4. Пам'ять: важливі факти й завершені взаємодії.
5. Розклад: де NPC зазвичай перебуває в різний ігровий час.
6. Сюжетний стан: які місії, обіцянки та події пов'язані з NPC.

Кожен NPC повинен мати незмінний унікальний npcId.

Пам'ять повинна містити типізовані факти, наприклад:

- PlayerMetNpc;
- PlayerHelpedNpc;
- PlayerMadePromise;
- PlayerKeptPromise;
- PlayerBrokePromise;
- SharedEventExperienced;
- NpcQuestCompleted.

Не зберігай кожне незначне натискання як постійний спогад.

Діалог може залежати від:

- попереднього знайомства;
- часу доби;
- поточного місця;
- виконаних місій;
- наявних предметів;

- стосунків;
- важливих спогадів.

Якщо кілька умов підходять одночасно, має бути визначене правило пріоритету, щоб NPC не показував випадкові суперечливі репліки.

Почни з Elise Vermeulen і першої зустрічі в кемпінгу.

Створи:

- граф діалогу;
- не менше трьох варіантів відповіді;
- відповідні наслідки;
- збереження пам'яті;
- реакцію на повторну зустріч;
- тести на одноразові й повторювані взаємодії.

Не додавай дублікати персонажа з іншим ім'ям для обходу системи пам'яті.

Частина 13. Графіка теж потребує технічного завдання

Тут є ще одна прогалина попереднього плану. Ми казали «потім замінимо тимчасові спрайти на піксель-арт», але не визначили, скільки саме ресурсів потрібно і як гарантувати, що вони матимуть однаковий стиль.

Якщо не визначити це заздалегідь, AI може створити будинок у перспективі 3/4, персонажа в іншому масштабі, дерево з іншим напрямком світла й тайли, які не стикуються.

Тому спочатку потрібен невеликий художній стандарт.

PROMPT 21 — Художня специфікація першого набору ресурсів

Не створюй одразу всю графіку гри. Спочатку підготуй технічну специфікацію для першого набору ресурсів.

Загальний стиль:

- затишна 2D-піксельна гра;
- перспектива 3/4;
- європейський будинок і невеликий двір;
- природні, стримані кольори;
- виразні силуети;
- читабельні об'єкти;
- єдиний масштаб персонажів і предметів;
- відсутність випадкового змішування перспектив.

Визнач:

1. Базовий розмір тайла.
2. Висоту й ширину персонажа.
3. Розмір і формат спрайтів.
4. Палітру та напрямок світла.
5. Правила прозорості.
6. Структуру атласів.
7. Назви файлів.
8. Точки колізій.
9. Опорні точки персонажів і предметів.
10. Напрямки анімацій.
11. Правила масштабування без розмиття пікселів.
12. Формати експорту.

Підготуй перелік першого набору ресурсів.

Персонаж:

- idle у чотирьох напрямках;
- ходьба у чотирьох напрямках;
- базовий силует і варіанти зовнішності, якщо вони підтримуються.

Дім:

- стіни;
- підлога;

- двері;
- вікна;
- стіл;
- стілець;
- шафа;
- коробка зі спорядженням.

Двір:

- трава;
- ґрунтова доріжка;
- паркан;
- ворота;
- дерево;
- кущ;
- поштовий ящик;
- велосипед;
- зовнішній стіл.

Інтерфейс:

- піктограми інвентарю;
- підказки взаємодії;
- базові піктограми предметів;
- панелі й кнопки.

Для кожного ресурсу вкажи розміри, формат, спосіб використання, колізію та статус.

Спочатку підготуй специфікацію й перевір, чи можна скласти з цих ресурсів першу сцену. Не генеруй сотні файлів до затвердження стандарту.

Тимчасові геометричні об'єкти мають залишатися доступними, щоб можна було тестувати гру навіть без фінальної графіки.

Після затвердження специфікації реалізуй один невеликий узгоджений набір і перевір його в браузері.

І важливий принцип: графічний ресурс і логіка об'єкта — різні речі. Наприклад, зображення намету саме по собі не означає, що в ньому можна ночувати. Намет має бути пов'язаний із визначенням предмета, правилами встановлення, місцем на карті, анімацією та взаємодією.

Частина 14. Повна матриця тестування першої подорожі

Коли перша подорож запрацює, не можна перевіряти тільки ідеальний сценарій. Потрібно пройти її кількома різними способами.

| Сценарій перевірки | Очікувана поведінка |
| --- | --- |
| Гравець купив усе потрібне | Поїздка проходить без проблем, пов'язаних із відсутнім спорядженням |
| Гравець не купив додаткові речі | Гра дозволяє продовжити, якщо базові умови виконано |
| Гравець має кота | У підготовці враховуються переноска, корм, вода та правила кемпінгу |
| Гравець має собаку | Враховуються вода, потреби в прогулянці та доступність тварин |
| Гравець подорожує велосипедом | Не списується автомобільне пальне |
| Гравець подорожує автомобілем | Витрата пального та бюджет розраховуються послідовно |

| | |
|---|---|
| Гравець відмовився від соціальної місії | Основна подорож продовжується |
| Гравець двічі натиснув покупку | Не відбувається подвійного списання за одну операцію |
| Гравець зберігся посеред подорожі | Після завантаження відновлюється той самий стан |
| Гравець повторно поговорив із Elise | Діалог відповідає історії попередніх зустрічей |
| Гравець забув необов'язковий предмет | Гра пояснює обмеження й пропонує можливий вихід, якщо він є |
| Гравець повернувся додому | Витрати, предмети й завершені події збережені |
| Гравець відкрив меню під час руху | Рух і симуляція призупиняються відповідно до правил |

Цю матрицю варто перетворити на автоматизовані тести там, де це можливо, і на покрокові браузерні сценарії для решти.

Частина 15. У якому порядку тепер усе виконувати

Я б трохи змінив початкову послідовність. Спочатку затверджуємо правила, потім створюємо вертикальний зріз гри — маленьку, але завершену подорож. Після цього розширюємо світ.

План робіт v1.1

Послідовність від задуму до першої повноцінної ігрової арки.

1. Узгодження документації. Core Rules, архітектура, контракти даних і реєстр відкритих рішень.
2. Мінімальний вебпрототип. Запуск, Phaser, карта, рух, камера та колізії.
3. Взаємодії й UI. Предмети, підказки, діалоги та панелі.
4. Створення персонажа. Домогосподарство, стартові ресурси, транспорт і тварини.
5. Симуляція. Час, гроші, інвентар, покупки та базові потреби.
6. Сценарій першої подорожі. Усі глави, діалоги, вибори, розгалуження й наслідки.
7. Кемпінг і NPC. Рух, взаємодії, інфраструктура, пам'ять і соціальна історія.
8. Збереження. Відновлення стану та тестування повторного завантаження.
9. Художній прохід. Узгоджені тайли, персонажі, анімації, предмети й інтерфейс.
10. Комплексне тестування. Усі варіанти першої подорожі, регресії й виправлення.
11. Розширення. Нові місії, кемпінги, професії, транспорт, регіони та складніші події.

Ще три рішення, які я рекомендую закріпити

Перше — не робити генеративний ШІ обов'язковою частиною ігрового світу. На першому етапі сценарії, NPC і події повинні працювати на звичайному коді та даних. Це дешевше, передбачуваніше, простіше тестувати й дозволяє грати офлайн. AI-кодер допомагає створювати гру, але сама гра не повинна залежати від нього.

Друге — не створювати повну Європу до перевірки першої подорожі. Спочатку достатньо одного вигаданого регіону з домом, магазином, дорогою, містечком і кемпінгом. Коли правила працюватимуть, можна підключати реальну географію, дані доріг, погоду та каталоги місць.

Третє — не плутати кількість контенту з глибиною гри. Краще мати вісім NPC, які пам'ятають гравця, мають власні цілі та повертаються в наступних історіях, ніж сто NPC, які лише повторюють стандартні репліки.

З чого починати на практиці

Я б не надсилав AI-кодеру всі 21 промпт поспіль. Спочатку дай йому Промпт 0, потім створи й затвердь `Core Rules v1.0`, а після цього переходь до Промпту 1. Сценарні документи й технічні контракти повинні зберігатися у файлах проєкту, щоб AI міг звертатися до них на кожному етапі.

І ще одне: у попередньому промпті створення персонажа вікові обмеження були сформульовані надто вузько. Вік і сімейні обставини мають бути частиною повноцінного редактора персонажа, а не довільним обмеженням. Водночас першу версію можна почати з кількох підтримуваних вікових груп і розширювати їх поступово.

Підсумок: тепер у нас є не лише початковий сценарій і набір команд для AI, а й правила, за якими він має працювати, спосіб зберігати контекст, модель NPC-пам'яті, специфікація сюжетної арки, художній стандарт і контрольна матриця. Це значно міцніша основа для послідовної розробки.

Наступним великим документом має бути повна сценарна біблія першої подорожі: конкретні репліки, ID усіх місій, усі варіанти відповідей, передумови, наслідки, переходи та відновлення після помилок — уже не загальний план, а готова специфікація для реалізації.

ось тепер я бачу твою силу, то продумай і допиши все в чому ми слабкі і чого в нас не вистачає, бо для цього ти і потрібен, але глибоко і широко думай при описі і не поспішай бо тут логіка не має впасти бо як ти написав може так і бути зроблено = а погано написано = баг і гра зламається. Пиши як бачиш ти все це бо в мене є ідея і ти маєш її зробити сценарієм і правилами і промптами і

взагалі довести до ідеалу бо зараз все сире і ріно те що тільки треба мені було тобі дати я дав, далі ти підтримай мене і розвивай всю сюжетну лінію, але не забувай про проблеми які в житті бувають, радості які в житті людей бувають бо робити вигадану гру я не хочу, я хочу реалістичну гру і з урахування життя і фізики і правил і тощо, це як повчальна гра туриста кемпер подорожуючого і це квест має бути, нече ти сам є не гравець а той хто в грі проживає всі ці моменти ігрові які притаманні життю людей із цим хоббі-ритмом-стилем життя, але ця гра може відкривати нові горизонти тим хто не мав такого досвіду. Ти слушно пишеш про все що є і чого нема і що треба, то роби. Треба щоб люди заходили в свій аккаунт і могли зберігати ігровий прогрес і не втрачали його. Не можна порушувати права, правила, акти, закони тощо, гра не має нічого незаконного мати, протизаконне заборонено, насилля і порно теж.

Я зрозумів головне: ми будуємо не просто гру про кемпінг, а цілий симулятор туристичного життя з сюжетом, квестами, фізикою, економікою, людськими стосунками й навчанням через власний досвід. Гравець має не читати підручник про подорожі, а проживати їх: планувати маршрут, радіти першому вдалому виїзду, знайомитися з людьми, припускатися помилок, знаходити вихід, відкривати нові місця й повертатися додому з історіями.

І ти правильно наголосив на найважливішому: погано сформульоване правило може перетворитися на баг, а суперечність у правилах — зламати цілу систему гри. Тому я розвиватиму не тільки сюжет, а й логіку, структуру даних, взаємодію систем, збереження прогресу, перевірки та послідовність розробки.

Ми не будемо намагатися вгадати все з першого разу. Ми побудуємо специфікацію так, щоб кожне рішення мало причину, кожна механіка — чіткі правила, а кожен етап розробки — перевірюваний результат. Це значно надійніше, ніж написати великий красивий сценарій, а потім виявити, що його неможливо коректно реалізувати.

1. Якою має стати гра

Живий туристичний світ

Гравець рухається локаціями, заходить до магазинів, облаштовує табір, користується душем, водою, пральнею, ремонтує спорядження, спілкується з людьми й досліджує околиці. Світ не повинен бути лише набором меню.

Реалістичні подорожі

Відстань, рельєф, вітер, дощ, маса багажу, витрата пального, запас води, стан велосипеда чи автомобіля, пороми, бронювання й дорожні обмеження впливають на рішення гравця.

Сюжет, який народжується з життя

Унікальні NPC, сімейні плани, дружба, обіцянки, випадкові знайомства, допомога незнайомців, маленькі конфлікти, несподівані подарунки й довготривалі наслідки вчинків.

Навчання без перетворення гри на покарання

Гравець поступово розуміє, як планувати справжню мандрівку. Небезпеки попереджаються завчасно, помилки мають зрозумілі причини, а більшість проблем можна вирішити кількома способами.

Це візуальні орієнтири, а не остаточний арт-дизайн. Наш напрям — 2D pixel art із виглядом згори під кутом, виразними персонажами, затишними локаціями та зрозумілими інтерактивними об'єктами.

П'ять принципів, які не можна втратити

1. Життя, а не покарання. Реалізм створює цікаві рішення, а не нескінченні штрафи, голод і поломки.
2. Свобода, а не одна правильна історія. Гравець сам обирає стиль життя, транспорт, супутників, бюджет і напрямок розвитку.

3. Причина перед наслідком. Серйозний наслідок повинен мати зрозумілу причину, а небезпечна ситуація — адекватне попередження, якщо її можна передбачити.
4. Пам'ять світу. NPC, квести, покупки, домовленості, подорожі та події не забуваються після переходу між локаціями чи перезапуску гри.
5. Правдивість. Якщо правило залежить від країни, сезону, типу транспортного засобу або місця, гра не повинна видавати одну універсальну відповідь за абсолютну істину.

2. Головна архітектура: спочатку правила світу, потім контент

Я бачу одну небезпеку, яку потрібно усунути ще до написання квестів: якщо окремо описувати погоду, транспорт, гроші, інвентар, NPC і календар, кожна система може працювати сама по собі, але разом вони дадуть суперечності.

Наприклад, гравець заплатив за кемпінг, пішов спати, зберігся, закрив браузер і зайшов наступного дня. Якщо час продовжився, але бронювання, оплата, положення автомобіля або стан персонажа не збереглися, світ стає нелогічним.

Тому вводимо єдину модель стану світу. Це не просто технічна структура — це договір, якого мають дотримуватися всі системи.

2.1. З чого складається світ

WorldState — стан світу

Єдине джерело правди про поточну гру

Персонажі й сім'я

Вік, потреби, навички, стосунки, домашні улюбленці

Економіка

Гроші, доходи, борги, витрати, резерви, зобов'язання

Подорожі

Маршрут, транспорт, час, бронювання, місце перебування

Навколишній світ

Погода, дороги, об'єкти, доступність послуг

Сюжет і пам'ять

Квести, вибори, факти про NPC, наслідки подій

Речі та спорядження

Кількість, стан, маса, сумісність, місце зберігання

Валідація та збереження

Перевіряє правила, фіксує зміни, відновлює прогрес

Важливий принцип: інтерфейс не повинен самостійно змінювати гроші, час або інвентар. Він надсилає команду, наприклад «купити пальник». Ігрова логіка перевіряє ціну, наявність товару, гроші й місце в інвентарі, після чого виконує операцію або пояснює, чому вона неможлива.

Так само квест не повинен сам вигадувати, чи є в гравця гроші. Він запитує актуальний стан економіки.

2.2. Подія, команда та наслідок — це різні речі

Цю відмінність обов'язково потрібно зафіксувати в технічному завданні.

- Команда: гравець хоче купити газовий пальник.
- Перевірка: магазин працює, товар є, грошей вистачає, покупка дозволена.
- Зміна стану: гроші зменшуються, пальник додається до інвентарю.
- Подія: `ITEM_PURCHASED`.
- Реакція інших систем: бюджет оновлюється, завдання підготовки до походу перевіряється, журнал подій отримує запис.

Якщо гравець двічі натиснув кнопку, одна покупка не повинна виконатися двічі через подвійний клік. Якщо операція перервалася, не можна списати гроші без видачі товару.

Те саме правило застосовується до бронювань, ремонту, продажу речей, винагород, квестів і витрат на пальне.

2.3. Час — це ресурс усіх систем

Ми не повинні мати окремий незалежний час у кожному квесті.

Єдині ігрові дата й час впливають на:

- робочі зміни, вихідні, відпустки та навчання дітей;
- тривалість руху, відпочинку, приготування їжі й ремонту;
- розклад NPC, роботу магазинів і час заселення;
- погоду та прогноз;
- пороми, громадський транспорт, бронювання;
- втому, сон, стан продуктів і домашніх тварин;
- тривалість подій та дедлайни.

Але не кожна дія має просувати час. Відкриття меню, перегляд карти або читання журналу не повинні випадково витрачати ігрові години. Це має бути визначено явно.

Правило: час просувається через визначені ігрові дії, подорожі та симуляційні кроки, а не через довільні затримки інтерфейсу.

3. Реалізм, який можна перевірити

Наша мета — не симулювати кожну молекулу повітря. Ми моделюємо ті аспекти реального життя, які впливають на рішення гравця.

3.1. Транспорт і фізика

Велосипед

Маса велосипедиста й багажу, передатні числа, ухил, покриття, зустрічний вітер, опір коченню, втома, стан шин, гальм і ланцюга. Розрахунок має давати правдоподібний час, а не абсолютно точний спортивний прогноз.

Автомобіль і кемпер

Маса, габарити, витрата пального, технічний стан, шини, дорожні обмеження, допустима висота й маса, місця для паркування, заправки та можливості ремонту. Не кожна дорога підходить великому транспортному засобу.

Спорядження та вантаж

Кожен предмет має масу, об'єм, стан і призначення. Великий запас речей додає комфорту, але може збільшити навантаження, займати місце й ускладнювати пересування.

Приклад моделі часу для велосипедної поїздки

Для першої версії можна використати базову швидкість на рівній дорозі, а потім коригувати її коефіцієнтами:

$$v_{еф} = v_0 \cdot K_{ухил} \cdot K_{вітер} \cdot K_{покриття} \cdot K_{вантажу} \cdot K_{втоми}$$

$\cdot K$

ухил

$\cdot K$

вітер

$\cdot K$

покриття

$\cdot K$

вантажу

$\cdot K$

втоми

Це початкова ігрова модель, а не універсальний фізичний закон. Коефіцієнти потрібно калібрувати на тестових маршрутах, обмежувати фізично правдоподібними межами й перевіряти на крайніх значеннях.

Час маршруту:

t=dvеф+tзупинок

$$t = \frac{d}{v_{еф}} + t_{зупинок}$$

Якщо ефективна швидкість дорівнює нулю або від'ємна, система не повинна ділити на нуль чи зависати: вона має позначити маршрут як непрохідний або запропонувати інший спосіб руху.

Що ще потрібно врахувати

- Ухил дороги впливає на швидкість по-різному під час підйому та спуску.
- Вітер має напрямок відносно руху, а не лише параметр «сильний/слабкий».
- Дощ впливає на зчеплення, видимість, комфорт і висихання речей; ефект залежить від покриття та умов.
- Маса кемпера впливає на розгін, гальмування, витрату пального й доступність доріг.

- Несправність не повинна виникати лише тому, що гра вирішила покарати гравця. Її ймовірність залежить від стану, навантаження, зношення, умов та обслуговування.
- Небезпечні маневри, перевантаження та погані умови мають зрозумілі попередження.
- Якщо реалістична фізика конфліктує з керованістю гри, ми обираємо спрощену, але послідовну модель і чесно навчаємо її правилам.

### 3.2. Вода, їжа, гігієна й побут кемпера

Це одна з найважливіших відмінностей нашої гри від звичайної пригоди.

Кемпер — це не просто машина, в якій можна спати. Він має системи з обмеженими ресурсами:

- чиста вода та її витрата;
- стічна вода й необхідність спорожнення резервуара;
- туалет і його обслуговування;
- холодильник та енергоспоживання;
- акумулятор житлового відсіку;
- електрика від зовнішнього підключення, сонячної панелі або двигуна — залежно від конструкції;
- опалення, вентиляція й захист від холоду;
- запас газу або інше джерело енергії для приготування їжі;
- сміття, вологі речі, запахи, прання та прибирання.

Наприклад, неприємний запах у кемпері не повинен завжди означати «бак переповнений». Можливі різні причини: забруднення, проблеми з ущільненням, вентиляцією або сантехнікою. Гравець може оглянути систему, перевірити рівень, знайти сервісну інформацію, спробувати безпечне очищення або звернутися до майстра.

Гра навчає діагностувати, а не вгадувати.

При цьому ми не перетворюємо кожну дрібницю на рутинну роботу. Гравець може автоматизувати повторювані дії, використовувати зручні індикатори та налаштовувати деталізацію побутової симуляції.

### 3.3. Погода й небезпека

Погода має створювати можливості та рішення, а не бути випадковим генератором страждань.

- Сонячна погода підходить для прогулянок, сушіння речей, заряджання сонячних панелей.
- Вітер може допомогти або ускладнити велосипедний маршрут.
- Дощ може змусити змінити план, але також відкрити альтернативу: музей, кафе, ремонт спорядження чи знайомство з сусідами.
- Спека підвищує потребу у воді, змінює комфорт і час активностей.
- Гроза, повінь, сильний вітер або небезпечний стан дороги можуть вимагати перенесення чи скасування маршруту.

Для небезпечних умов потрібна градація:

1. Звичайні умови.
2. Помітні незручності.
3. Умови, які потребують підготовки або зміни плану.
4. Небезпечні умови, за яких маршрут слід закрити або настійно рекомендувати альтернативу.

Небезпечний маршрут не можна зробити безпечним лише високою навичкою персонажа. Навички допомагають оцінити ситуацію, підготуватися й ухвалити кращі рішення, але не скасовують закони фізики.

У першому навчальному виїзді ми не використовуємо суворі погодні сценарії, аварійні поломки чи фінансові пастки. Пізніше такі події вводяться поступово, з підказками й варіантами виходу.

4. Сюжет: не одна довга місія, а життя, яке розвивається

Я пропоную побудувати гру на трьох сюжетних рівнях.

Рівень A — особиста історія

Гравець створює персонажа й визначає, чому подорожі стали важливими. Це може бути мрія про свободу, бажання побачити Європу, сімейна традиція, нове хобі, відновлення після важкого періоду або просто цікавість.

Рівень B — історія подорожей

Планування, маршрути, транспорт, спорядження, нові регіони, кемпінги, походи, технічні проблеми, погода, відкриття та досвід. Тут виникає більшість пригод.

Рівень C — історія людей

Дружба, знайомства, обіцянки, подарунки, суперечки, сімейні рішення, зміна роботи, допомога іншим, повернення до улюблених місць і довготривалі стосунки з NPC.

Ці рівні мають взаємодіяти. Наприклад, гравець знайомиться з досвідченою велосипедисткою. Вона не просто видає завдання, а ділиться маршрутом. Пізніше цей маршрут приводить до невеликого села, де відбувається фестиваль. Там можна придбати корисне спорядження, допомогти місцевому майстру, зустріти іншого NPC і відкрити новий ланцюжок подій.

Так з'являється відчуття, що світ живе, а не чекає, поки гравець натисне кнопку «Наступний квест».

4.1. Довгострокова сюжетна лінія

Пропоную початкову кампанію «Рік маленьких подорожей». Це робоча назва, не остаточна.

1. Розділ 1. Перша ніч поза домом
2. Підготовка, просте спорядження, перший маршрут, заселення, знайомство з кемпінгом і перша маленька пригода. Гравець отримує впевненість, а не покарання.
3. Розділ 2. Знайомі місця, нові люди
4. Другий виїзд, знайомство з постійними відвідувачами, місцеві маршрути, прості ремонти, планування їжі, води та бюджету.
5. Розділ 3. Мрія про далеку дорогу
6. Гравець обирає першу більшу ціль: велосипедний тур, подорож автомобілем, кемпер, сімейний відпочинок або маршрут громадським транспортом із ночівлями.
7. Розділ 4. Коли план змінюється
8. Дощ, дорожні роботи, вичерпання запасу, несправність, скасування бронювання або конфлікт планів. Гравець навчається адаптуватися, а не просто повторювати початковий план.
9. Розділ 5. Люди, до яких хочеться повернутися
10. Старі знайомі, обіцянки, допомога іншим, повторні відвідування місць, подарунки, спільні заходи й NPC, які пам'ятають минулі події.
11. Розділ 6. Власний стиль мандрівника
12. Гравець може зосередитися на природі, сімейних маршрутах, велотуризмі, кемпері, технічній майстерності, фотографії, бюджетних поїздках або дослідженні регіонів.

Це не означає, що всі повинні проходити однакову кампанію. Розділи — це сюжетні можливості й тематичні віхи, а не жорсткі рівні, які закривають свободу.

4.2. Як зробити події життєвими

Кожна подія має відповідати на шість питань:

1. Чому вона виникла саме зараз?
2. Чому вона стосується цього персонажа?
3. Що гравець знає до ухвалення рішення?
4. Які варіанти дій доступні?
5. Які короткострокові та довгострокові наслідки можливі?
6. Як гра збереже й покаже наслідки пізніше?

Наприклад, гравець прибув на кемпінг, а його місце ще не готове через затримку попереднього відвідувача. Це не повинно автоматично перетворюватися на штраф. Адміністратор може вибачитися й запропонувати почекати, інше місце або тимчасову альтернативу. Наслідки залежать від обставин, часу, правил кемпінгу й реакції гравця.

Якщо гравець запізнюється на пором, результат залежить від типу квитка, правил перевізника, наступного рейсу, наявності місць і того, чи можна змінити бронювання. Не можна завжди просто списувати гроші або завжди дозволяти сісти на наступний рейс без оплати.

4.3. Позитивні події — повноцінна частина системи

Хороші моменти мають бути не менш продуманими, ніж проблеми.

- Знайомий дарує квиток або передає бронювання, якщо це дозволяють правила.
- Магазин робить знижку на спорядження, яке підходить стилю гравця.
- Роботодавець виплачує премію або дозволяє додатковий вихідний.
- Персонаж отримує спадок чи виграє невелику суму в легальній лотереї.
- Незнайомець допомагає знайти загублену річ.
- Улюблений кемпінг запрошує на місцевий фестиваль.
- Гравець знаходить красиву стежку, про яку не знав.
- Старий NPC згадує спільну пригоду й дарує пам'ятну річ.

Рідкісні великі виграші не повинні руйнувати економіку. Подія може бути рідкісною, мати обґрунтовану ймовірність і не бути інструментом тиску на гравця. Ми не будуємо гру навколо азартних механік, ставок чи реальних грошових призів.

5. Квестова система: щоб сюжет не ламався

Ми маємо розділити сюжетну подію, місію та окрему ціль.

- Подія — щось відбулося у світі.
- Місія — пов'язаний набір цілей, який гравець може виконувати.
- Ціль — конкретна перевірювана дія, наприклад придбати ліхтар або прибути до рецепції.

Це дозволяє зробити історії гнучкими. Наприклад, якщо гравець уже має ліхтар, гра не повинна змушувати купувати другий тільки тому, що квест був написаний раніше.

5.1. Стани місії

Кожна місія повинна мати чіткі стани:

```
LOCKED → AVAILABLE → ACTIVE → COMPLETED
```

Додатково можливі `FAILED`, `CANCELLED` та `EXPIRED`, але використовувати їх слід лише тоді, коли це справді має сенс.

Не кожен квест можна провалити. Багато завдань мають залишатися доступними або переходити в альтернативну гілку. Наприклад, якщо гравець не допоміг сусідові сьогодні, він може зустріти його завтра або більше не отримати конкретну нагоду — але це не повинно блокувати всю гру.

Для кожної місії визначаємо:

- унікальний ID;
- умови відкриття;
- цілі та їхній порядок;
- необов'язкові цілі;
- допустимі альтернативи;
- доступні вибори;
- зміни грошей, часу, предметів і стосунків;
- події, які запускаються після завершення;
- можливість повторення;
- правила скасування;
- умови відновлення після завантаження збереження.

### 5.2. Приклад: квест «Перший вечір»

QUEST-INTRO-001

Основна навчальна місія · Одноразова

Мета: навчитися прибувати до кемпінгу, знаходити місце й облаштовувати ночівлю.

Цілі

1. Прибути до входу кемпінгу.
2. Поговорити з адміністраторкою Елізою.
3. Отримати інформацію про місце та доступні зручності.
4. Дістатися до свого місця.
5. Облаштувати ночівлю або вибрати відповідний спосіб відпочинку.

Варіанти проходження

- Наметовий турист: встановлює намет і перевіряє, чи є потрібні кілочки.
- Велосипедист: розвантажує велосипед, перевіряє багаж і готує спальне місце.
- Автомобільний турист: дістає спорядження з багажника.
- Власник кемпера: паркується з урахуванням розміру, підключає доступні системи за потреби.

- Гравець із домашньою твариною: перевіряє правила кемпінгу й організовує безпечне місце для улюбленця.

Умови завершення

Місія завершується, коли гравець організував придатне місце для ночівлі й отримав підтвердження, що може відпочити. Не вимагається купувати конкретну марку спорядження або виконувати необов'язкові соціальні завдання.

Важлива технічна деталь

Місія не повинна містити власну копію інвентарю, грошей або часу. Вона перевіряє центральний стан гри. Якщо гравець продав намет, втратив предмет або обрав інший тип ночівлі, місія має коректно відреагувати, а не зависнути в стані «принеси намет».

Кожен варіант проходження потрібно перевірити окремо. Це і є різниця між цікавим описом квесту та придатною до реалізації специфікацією.

6. Акаунти й збереження: прогрес не повинен зникати

Це не додаткова функція на потім. Якщо гра має довготривалий світ, акаунт і надійне збереження — частина її основи.

6.1. Як має працювати акаунт

1. Створення акаунта

Реєстрація через підтримуваний спосіб входу, підтвердження необхідних даних і відновлення доступу.

2. Створення персонажа

Персонаж, домогосподарство, транспорт, початкові ресурси та перший слот збереження.

3. Збереження на сервері

Версія стану, контроль цілісності, підтвердження успішного запису та резервні копії.

4. Відновлення

Вхід з іншого пристрою, завантаження останнього підтвердженого збереження та перевірка сумісності версії.

6.2. Що саме потрібно зберігати

Не тільки позицію персонажа. Потрібно зберігати весь стан, який може вплинути на майбутнє:

- ігрову дату, час і календар;
- персонажа, зовнішність, навички й поточний стан;
- членів домогосподарства, їхні розклади та стосунки;
- домашніх тварин, їхній стан, інвентар і потреби;
- гроші, доходи, борги, рахунки й регулярні зобов'язання;
- інвентар, стан предметів, місце зберігання та завантаження;
- транспорт, технічний стан, пальне, заряд, ремонт і модернізації;
- план подорожі, фактичний маршрут, бронювання й квитки;
- поточну локацію, положення персонажів та інтерактивних об'єктів;
- стан усіх активних, завершених і скасованих квестів;
- вибори в діалогах, обіцянки та пам'ять NPC;
- відкриті регіони, виявлені місця й отримані знання;
- історію важливих подій;
- налаштування керування, доступності та симуляції.

При цьому ми не зберігаємо зайві деталі, які можна безпечно відновити. Наприклад, статичний опис предмета зберігається в каталозі гри, а у збереженні — його ID, кількість і поточний стан.

6.3. Автозбереження та захист від втрат

Я рекомендую таку поведінку:

1. Автоматично зберігати після важливих завершених операцій: покупки, бронювання, завершення місії, переходу між великими етапами.
2. Виконувати періодичне збереження стабільного стану.
3. Показувати час останнього успішного збереження.
4. Не показувати «Збережено», доки сервер справді не підтвердив запис.
5. Зберігати резервну копію попередньої валідної версії.
6. Виявляти пошкоджені, неповні або несумісні збереження.
7. Після збою відновлювати останній підтверджений стан і повідомляти, якщо частина останніх дій могла не зберегтися.

Найважливіше правило: не можна одночасно записати гроші як витрачені, але не записати куплений предмет. Економічні операції та інші критичні зміни мають бути атомарними — або застосовуються повністю, або не застосовуються взагалі.

6.4. Що робити, якщо гравець відкрив гру на двох пристроях

Це не можна залишити на розсуд браузера.

Кожне збереження має мати номер версії. Якщо два пристрої змінили один і той самий слот, система повинна виявити конфлікт, а не безумовно перезаписати новіший прогрес старішим.

Для першої версії найнадійніший варіант — одна активна сесія на один слот збереження. Інша сесія отримує повідомлення про конфлікт і пропозицію перезавантажити актуальну версію. Пізніше можна додати безпечніше об'єднання окремих налаштувань, але не довільне об'єднання сюжетного стану.

6.5. Рекомендована модель даних збереження

```
type SaveGame = {
  schemaVersion: number;
  saveId: string;
  accountId: string;
```

```
  worldId: string;

  revision: number;
  createdAt: string;
  updatedAt: string;

  worldState: WorldState;
  checksum: string;
};

type WorldState = {
  calendar: GameCalendar;
  player: PlayerCharacter;
  household: Household;
  economy: EconomyState;
  inventory: InventoryState;
  vehicles: VehicleState[];
  trip: TripState;
  location: LocationState;
  quests: QuestState[];
  npcs: NPCState[];
  pets: PetState[];
  discoveries: DiscoveryState;
  eventHistory: WorldEventRecord[];
};

type GameCalendar = {
  day: number;
  month: number;
  year: number;
  minuteOfDay: number;
};

type EconomyState = {
  currencyCode: string;
  balanceMinorUnits: number;
  debts: DebtRecord[];
  transactions: FinancialTransaction[];
};
```

Це каркас контракту, не завершена схема для копіювання без перевірок. `WorldState`, `PlayerCharacter` та інші типи мають бути визначені в одному місці й використовуватися всіма системами. Не можна створювати п'ять різних типів `Player` для меню, квестів, руху, збереження й магазину.

Гроші слід зберігати в цілих найменших одиницях валюти, а не як довільні числа з плаваючою комою. Для євро це зазвичай центи. Так ми зменшуємо ризик помилок округлення в покупках і рахунках.

Також потрібні міграції схеми: коли нова версія гри додає поле, старе збереження не повинно автоматично ставати непридатним. Міграція має бути версійною, тестованою та безпечною. Якщо міграція не вдалася, оригінал збереження не можна знищувати.

Що акаунт не повинен робити

- Збирати зайві особисті дані без потреби.
- Зберігати пароль у відкритому вигляді.
- Дозволяти одному гравцеві читати чи змінювати збереження іншого.
- Вважати клієнтський браузер довіреним джерелом грошей, покупок або прогресу.
- Видаляти прогрес без зрозумілого попередження та підтвердження.
- Підміняти резервну копію останнім пошкодженим файлом.

Для входу, відновлення доступу, захисту сесій, резервного копіювання й керування даними використовується серверна система з перевіркою прав доступу. Конкретну технологію можна вибрати пізніше після оцінки вартості, безпеки й масштабування.

7. Законність, безпека та права — частина проєкту

Ми заздалегідь визначаємо, що гра не містить порнографії, сексуального контенту, графічного насильства, незаконних завдань або механік, які заохочують порушення закону.

Але відповідність правилам ширша за це. Вона охоплює права на контент, приватність, авторизацію, реальні дані, вікові вимоги, доступність і правила платформ.

### 7.1. Авторські права й інтелектуальна власність

- Не копіюємо персонажів, діалоги, назви, карту, сюжет, арт, музику чи інші впізнавані матеріали чужих ігор.
- Використовуємо власні персонажі, власний сюжет, власні діалоги та оригінальний візуальний стиль.
- Для музики, шрифтів, звуків, спрайтів, бібліотек та інших сторонніх ресурсів фіксуємо джерело, ліцензію, обмеження використання й необхідну атрибуцію.
- Не використовуємо фотографії, карти, бази даних або матеріали з інтернету лише тому, що вони доступні для перегляду.
- Для згенерованого контенту перевіряємо умови відповідного інструмента та ризики надмірної схожості з чужими роботами.

### 7.2. Реальні правила подорожей

Ми можемо навчати правилам туризму, але не повинні вигадувати закони.

Різні країни можуть мати різні вимоги щодо:

- перевезення тварин і документів;
- кемпінгу поза спеціально визначеними місцями;
- паркування, ночівлі в транспорті та користування природними територіями;
- обладнання велосипеда, освітлення й дорожнього руху;

- ваги, розмірів, категорії прав і допуску транспортного засобу;
- платних доріг, віньєток, поромів і транспортних зборів;
- пожежної безпеки, газового обладнання, зберігання відходів;
- місцевих обмежень, сезонного закриття маршрутів та охоронюваних територій.

Тому кожне твердження про реальні правила повинно мати країну або регіон, дату перевірки, надійне джерело та статус актуальності. Якщо правило невідоме або дані застаріли, гра має це показувати.

У навчальних матеріалах потрібно відрізняти реальну вимогу, рекомендацію з безпеки та спрощене ігрове припущення. Це три різні категорії.

7.3. Приватність і персональні дані

Якщо гра використовує акаунти та серверні збереження, потрібно закласти захист даних від початку.

Для європейського запуску слід перевірити застосовність GDPR та інших чинних вимог. Потрібно визначити, які дані збираються, навіщо, як довго зберігаються, хто має до них доступ і як користувач може реалізувати свої права.

Реальні дані користувача не слід використовувати для створення ігрових NPC без окремої правової підстави. Для навчання й тестування достатньо вигаданих профілів.

Якщо в майбутньому з'являться публічні профілі, чат, обмін користувацькими маршрутами або спільні світи, для них знадобиться окрема модель безпеки, скарг, блокування, модерації та захисту від зловживань. Для першої версії я рекомендую залишити гру однокористувацькою й не відкривати користувацький контент публічно.

### 7.4. Контентна політика

Усі події та квести мають проходити перевірку на відповідність правилам гри:

- без порнографії й сексуального контенту;
- без графічного насильства;
- без інструкцій до незаконних дій;
- без прославлення небезпечної поведінки як безризикової;
- без примусу до азартних ставок або покупок за реальні гроші;
- без дискримінаційних сценаріїв, які гра підтримує або винагороджує;
- без використання реальних приватних осіб як NPC без належної згоди та підстав.

Персонажі можуть мати різні погляди, характер, конфлікти й недоліки. Але сюжет має поводитися з ними відповідально, не перетворюючи незаконну або шкідливу поведінку на рекомендацію гравцеві.

## 8. Що ще потрібно допроєктувати, перш ніж вважати GDD повним

Нижче — реєстр слабких місць, які потрібно закрити. Це важливіше за додавання сотень нових квестів.

| Система | Чого недостатньо | Що має стати результатом |
|---|---|---|
| Створення персонажа | Правила сумісності профілів і ресурсів | Балансова модель старту |
| Сім'я та стосунки | Розклади, конфлікти потреб, автономність | Модель домогосподарства |
| Домашні тварини | Догляд, перевезення, ветеринарні ситуації | Повний цикл потреб і подорожей |

| | | |
|---|---|---|
| Транспорт | Спільна модель маси, стану й обмежень | Формули та тестові маршрути |
| Економіка | Регулярні витрати, резерви, кредити, повернення | Балансова модель на тиждень, місяць і рік |
| Робота й календар | Відпустка, зміни, дистанційна робота | Сумісність роботи з подорожами |
| Інвентар | Вага, об'єм, сумісність, місце зберігання | Правила предметів та операцій |
| Кемпінги | Місткість, бронювання, правила, інфраструктура | Модель закладу й заселення |
| NPC | Пам'ять, розклади, зміни життя | Перевірювана система поведінки |
| Квести | Граф залежностей і альтернативи | Відсутність тупиків і повторних нагород |
| Погода | Джерело даних, прогнози, спрощення | Прозора модель погоди |
| Географія | Розділення карти, доріг і локацій | Масштабована структура світу |
| Збереження | Конфлікти, міграції, відновлення | Стійка серверна система |
| Доступність | Керування, читабельність, складність | Налаштування без втрати змісту |

| | | |
|---|---|---|
| Законність | Ліцензії, приватність, реальні правила | Реєстр відповідності й аудит |
| Якість | Немає повного набору регресійних перевірок | Автоматизовані тести та критерії релізу |

Окремо потрібні три документи, яких часто бракує навіть у великих проєктах.

1. Реєстр рішень. Чому обрали конкретне правило, які альтернативи відхилили, які ризики залишилися.

2. Реєстр невизначеностей. Усе, що ще не вирішено, позначається як відкрите питання. AI-розробник не має самовільно перетворювати здогад на остаточне правило.

3. Матриця залежностей. Наприклад, зміна ваги спорядження може зачепити інвентар, велосипедну фізику, автомобільне навантаження, економіку доставки, квести й збереження. Такі зв'язки потрібно бачити до внесення змін.

8.1. Економіка: як уникнути несправедливості

Гроші мають створювати вибір, а не пастку.

У грі будуть:

- регулярні доходи;
- нерегулярні доходи;
- витрати на житло, їжу, транспорт та інші зобов'язання;
- витрати на подорожі;
- ремонти й обслуговування;
- резерв на непередбачувані випадки;
- можливість заощаджувати й поступово покращувати спорядження.

Для балансу кожен стартовий профіль повинен проходити контрольний сценарій: чи може персонаж покрити базові витрати, підготуватися до першої поїздки й повернутися без штучного банкрутства?

Водночас не потрібно робити всі профілі однаковими. Студент може мати менший бюджет, але більше вільного часу. Сім'я — більші витрати, але доступ до сімейного автомобіля. Механік — кращі можливості ремонту, але не обов'язково більше грошей. Віддалений працівник — дохід у дорозі, але потребу в інтернеті, електроенергії та стабільному робочому графіку.

У стартовій кампанії не можна дозволяти вибрати профіль, який практично не має шансів пройти вступні завдання. Якщо профіль має підвищену складність, це потрібно пояснити до вибору й надати альтернативи.

8.2. Професії, відпустки та календар

Робота має бути повноцінною системою, а не просто числом «дохід за день».

Кожна професія повинна визначати:

- графік і зміни;
- спосіб нарахування доходу;
- правила відпустки;
- можливість працювати дистанційно;
- обмеження на виїзд;
- вплив запізнень і пропусків;
- можливість змінити роботу;
- ризики та переваги конкретного стилю життя.

Система повинна враховувати вихідні, державні свята, шкільні канікули та сезонні події в конкретному регіоні. Для реалістичного режиму дві двотижневі

відпустки на рік можуть бути одним із прикладів робочого профілю, але не універсальним правилом для всіх персонажів.

Потрібно дозволити грати інакше: короткі поїздки на вихідних, довгі подорожі, віддалена робота, пенсійний режим, сезонна зайнятість або перерви між виїздами.

8.3. Питомці — це члени домогосподарства, а не предмети

Якщо гравець створив персонажа з котом чи собакою, тварина залишається частиною його історії. Гра не повинна пропонувати покинути її заради зручності або використовувати втрату тварини як дешевий спосіб створити драму.

Водночас подорож із твариною має реальні обмеження:

- дозволеність тварин у кемпінгу;
- вимоги до перевезення;
- вода, їжа, відпочинок і температура;
- безпечне місце для тварини;
- потреби конкретного виду та розміру;
- ветеринарна допомога й документи там, де це необхідно.

Кіт потребує безпечного перенесення, спокійного простору, лотка й поступового звикання. Собака може потребувати прогулянок, додаткового часу, певних умов розміщення й різного обсягу активності залежно від розміру, віку та стану.

Якщо певний кемпінг не приймає тварин, гра повинна дати час знайти альтернативу, змінити маршрут або вибрати інше місце, а не ставити гравця перед миттєвим безвихідним рішенням.

9. Як зберігати реалістичність даних

Я не рекомендую вручну вигадувати всі дороги, кемпінги, ціни, погодні умови й транспортні характеристики. Але й не можна автоматично завантажити дані з інтернету й вважати їх безпомилковими.

Для цього створюємо реєстр джерел і фактів.

Кожен важливий факт повинен містити:

- ідентифікатор;
- твердження;
- тип: закон, технічна характеристика, рекомендація, спостереження або ігрове припущення;
- географічну область;
- джерело та дату перевірки;
- ліцензійні обмеження;
- дату наступної перевірки, якщо дані можуть змінитися;
- спосіб перетворення факту на ігрову механіку;
- тест, який підтверджує коректність реалізації.

Для географії можна оцінювати OpenStreetMap та інші джерела, але перед використанням треба перевірити умови ліцензії, вимоги атрибуції, правила зберігання й можливі обмеження. Для погоди можна розглянути Open-Meteo та інші сервіси, але прогноз і історичні спостереження — не одне й те саме.

Реальні дані також мають бути відокремлені від ігрових припущень. Якщо ми спрощуємо складний маршрут до трьох рівнів складності, це нормально — за умови, що ми не видаємо ці рівні за офіційні характеристики дороги.

Для першого прототипу я пропоную невеликий вигаданий регіон, побудований на реалістичних принципах. Це дасть змогу перевірити механіки без

передчасної залежності від повноти зовнішніх карт і баз даних. Після цього ми зможемо додавати реальні європейські регіони поступово.

10. Робочий документ для розробки

Нижче — документ, який я пропоную додати до файлів проєкту як головний орієнтир. Він не замінює повний GDD, сценарну біблію чи технічну специфікацію. Він визначає, як їх розробляти, щоб вони не суперечили одне одному.

CAMPING LIFE SIM — ПРАВИЛА РОЗРОБКИ ПРОЄКТУ

Статус: базовий документ, v1.0
Призначення: керувати розробкою гри, документацією, AI-агентом, перевірками та збереженням цілісності систем.

1. Мета

Створити браузерну однокористувацьку 2D pixel-art гру про реалістичне туристичне життя, кемпінг, велоподорожі, автомобільні подорожі, кемпери, природу, людські стосунки та особистий розвиток.

Гра має бути водночас:

- захопливою пригодою;
- симулятором планування подорожей;
- квестом із розгалуженим сюжетом;
- навчальним середовищем;
- довготривалим особистим світом гравця.

2. Незмінні принципи

1. Реалізм має створювати цікаві рішення, а не постійне покарання.
2. Гравець має свободу вибору стилю життя та проходження.
3. Критичні наслідки повинні мати зрозумілі причини.
4. Передбачувані серйозні небезпеки мають супроводжуватися адекватними попередженнями.

5. Сюжет, економіка, фізика, інвентар, NPC, календар і збереження використовують спільний стан світу.
6. Усі NPC мають унікальні ідентифікатори та біографії.
7. Домашні тварини залишаються частиною домогосподарства гравця.
8. Не допускаються порнографія, графічне насильство, незаконні завдання та контент, що заохочує порушення закону.
9. Зовнішні дані, ресурси й бібліотеки використовуються тільки після перевірки умов використання.
10. Акаунт і прогрес захищаються від втрати, пошкодження та несанкціонованого доступу.

3. Єдине джерело правди

Ігрова логіка використовує централізований стан світу (`WorldState`).

Інтерфейс не змінює гроші, час, інвентар або стан квесту самостійно. Він передає команду системі правил. Система перевіряє передумови, застосовує зміни атомарно, реєструє подію та ініціює необхідне збереження.

Статичні визначення предметів, NPC, локацій, квестів і подій відокремлюються від стану конкретного проходження.

4. Обов'язкові документи

- `docs/GDD.md` — загальний дизайн гри.
- `docs/NARRATIVE_BIBLE.md` — персонажі, світ, сюжетні арки та діалоги.
- `docs/GAME_RULES.md` — формальні правила механік.
- `docs/TECHNICAL_SPEC.md` — архітектура, технології та контракти.
- `docs/DATA_DICTIONARY.md` — структури даних та їхні поля.
- `docs/QUEST_CATALOG.md` — місії, цілі, залежності та наслідки.
- `docs/EVENT_CATALOG.md` — випадкові й авторські події.
- `docs/NPC_BIBLE.md` — унікальні NPC, розклади, стосунки та пам'ять.
- `docs/ECONOMY_BALANCE.md` — доходи, витрати, ціни та баланс.
- `docs/REAL_WORLD_DATA.md` — реальні джерела, ліцензії, дати перевірки.
- `docs/PRIVACY_AND_COMPLIANCE.md` — приватність, безпека й правові вимоги.
- `docs/QA_STRATEGY.md` — тестові сценарії та критерії якості.
- `docs/PROJECT_STATUS.md` — поточний стан проєкту.
- `docs/DECISIONS.md` — ухвалені архітектурні рішення.

- `docs/OPEN_QUESTIONS.md` — невирішені питання та ризики.

Якщо документ ще не створений, спочатку створити його з позначенням статусу, а не вдавати, що він уже існує.

5. Статус кожного рішення

Використовувати чіткі позначення:

- `CONFIRMED` — затверджене рішення;
- `PROPOSED` — пропозиція, яку ще можна змінити;
- `OPEN` — відкрите питання;
- `IMPLEMENTED` — реалізовано в коді;
- `VERIFIED` — реалізацію перевірено;
- `DEPRECATED` — застаріле рішення, яке має міграційний або замінний план.

Не позначати рішення як `VERIFIED`, якщо існує лише документація або код без тестів.

6. Правила змін

Перед кожною суттєвою зміною:

1. Прочитати актуальні документи й `PROJECT_STATUS.md`.
2. Знайти залежності та визначити можливі побічні ефекти.
3. Перевірити, чи не суперечить зміна затвердженим правилам.
4. Оновити схеми й контракти, якщо змінюються дані.
5. Додати або змінити тести.
6. Запустити доступні перевірки.
7. Оновити документацію та статус.
8. Повідомити про невирішені проблеми й неперевірені твердження.

Не переписувати великі частини проєкту без необхідності. Не змінювати публічні контракти мовчки. Не приховувати помилки тестів.

7. Правила сюжету

Кожен квест і кожна суттєва подія мають:

- унікальний ID;
- передумови;
- початковий стан;
- доступні вибори;
- умови кожного вибору;
- зміни стану;
- наслідки для часу, грошей, предметів, NPC і сюжету;
- альтернативні шляхи;
- умови завершення або скасування;
- правила повторення;
- правила відновлення після завантаження;
- тестові сценарії.

Не дозволяти необов'язковому завданню блокувати основну історію, якщо це не є навмисно затвердженою сюжетною умовою.

8. Правила даних і безпеки

- Не довіряти значенням, які надходять від клієнта без серверної перевірки.
- Перевіряти права доступу до кожного збереження.
- Використовувати версії схем та міграції.
- Захищати критичні операції від повторного виконання.
- Не зберігати секрети або паролі у відкритому вигляді.
- Не включати персональні дані в журнали без обґрунтованої потреби.
- Не видаляти старі збереження до перевірки нового.
- Перевіряти ліцензії всіх зовнішніх ресурсів.

9. Критерій завершення етапу

Етап можна вважати завершеним лише тоді, коли:

- його обсяг чітко визначений;
- усі заявлені функції реалізовані або явно виключені;
- критичні правила покриті тестами;
- перевірки пройшли або відхилення задокументовані;
- документація відповідає реалізації;
- відомі ризики та обмеження внесені до статусу проєкту;

- немає прихованих блокерів для наступного етапу.

Головне правило: не додавати новий контент ціною руйнування вже перевіреної логіки. Спочатку цілісність систем, потім масштабування світу.

11. Головний промпт для AI-розробника

Це не команда «зроби всю гру». Це системна інструкція для агента, який буде працювати з кодом і документацією протягом багатьох ітерацій.

MASTER PROMPT — CAMPING LIFE SIM

Ти — технічний керівник і провідний розробник проєкту Camping Life Sim. Ти відповідаєш за архітектурну цілісність, якість реалізації, узгодженість документації, тестування та захист уже працюючих систем.

Контекст

Проєкт — браузерна 2D pixel-art гра про реалістичне туристичне життя, кемпінг, велосипеди, автомобілі, кемпери, подорожі, людей, домашніх тварин, економіку та розгалужені квести.

Гра має бути цікавою, емоційною, правдоподібною й навчальною. Вона не повинна перетворювати реалізм на постійне покарання.

Твої обов'язки

1. Перед роботою прочитай наявні документи проєкту та його актуальний статус.
2. Визнач, чи є завдання достатньо конкретним для безпечної реалізації.
3. Якщо завдання містить суперечність, зупинися й опиши її. Не обирай довільно одну з двох несумісних вимог.
4. Якщо бракує другорядної деталі, запропонуй найменш ризиковане рішення, познач його як `PROPOSED` і зафіксуй припущення.
5. Визнач файли, системи, контракти й тести, яких торкнеться зміна.
6. Реалізуй тільки узгоджений обсяг.

7. Додай тести для нової логіки та регресійні тести для критичних залежностей.
8. Запусти доступні перевірки й повідом справжні результати.
9. Онови відповідну документацію, реєстр рішень і `PROJECT_STATUS.md`.
10. Підготуй короткий звіт: що зроблено, що перевірено, що не перевірено, відомі ризики та наступний безпечний крок.

## Заборонено

- Вигадувати результати тестів або заявляти, що код працює, без перевірки.
- Створювати дублікати типів, каталогів або джерел правди без обґрунтованої необхідності.
- Довільно змінювати схеми збережень без міграцій.
- Дозволяти інтерфейсу напряму змінювати критичний стан світу.
- Додавати випадкові штрафи без причин і правил.
- Створювати NPC-копії з іншими іменами.
- Дозволяти необов'язковим квестам випадково блокувати гру.
- Видаляти функції або дані, щоб приховати помилку.
- Додавати не перевірені на ліцензії ресурси.
- Вважати неперевірене припущення встановленим фактом.
- Позначати неповну реалізацію як завершену.

## Формат виконання кожного завдання

A. Аналіз

Сформулюй мету, передумови, залежності, ризики та критерії завершення.

B. План

Переліч конкретні файли, зміни, тести та необхідні міграції.

C. Реалізація

Внеси мінімальні узгоджені зміни без непотрібного переписування проєкту.

D. Перевірка

Запусти релевантні тести, перевірки типів, форматування й збірку, якщо вони доступні. Чітко відокремлюй успішні перевірки від тих, які не вдалося виконати.

E. Документація

Онови схеми, правила, статус, рішення та відомі ризики.

F. Звіт

Повідом фактичний результат і запропонуй один наступний крок.

Пріоритети

1. Цілісність даних і збережень.
2. Коректність ігрових правил.
3. Стабільність і тестованість.
4. Якість навчального й сюжетного досвіду.
5. Візуальна якість.
6. Масштабування контенту.

Не намагайся реалізувати весь проєкт за один виклик. Працюй маленькими завершеними етапами. Кожен етап має залишати проєкт у стабільному, документованому стані.

12. Перша повноцінна сюжетна пригода

Тепер переходимо від архітектури до конкретного ігрового досвіду. Саме тут ми перевіримо, чи всі правила працюють разом.

Перша пригода повинна не лише навчити натискати кнопки. Вона має дати гравцеві відчути: «Я сам підготувався, вирушив у подорож, упорався з новим середовищем і хочу поїхати ще раз».

Сценарій `STORY-INTRO-001`: «Перша ніч поза домом»

Тривалість

60–90 хвилин

Регіон

Вигаданий регіон Центральної Європи

Основна мета

Перша успішна ночівля

Рівень ризику

Дуже низький

Навчальні результати

- Персонаж і домогосподарство.
- Планування маршруту й бюджету.
- Покупка та перевірка спорядження.
- Пересування локальною локацією.
- Заселення й правила кемпінгу.
- Розмова з NPC та запам'ятовування вибору.
- Організація ночівлі, ранок і повернення.
- Автозбереження й відновлення прогресу.

Сцена 1. Дім — «Колись треба почати»

ID: `INTRO-SCENE-001-HOME`

Гравець починає вдома. Це не довгий вступний ролик, а повноцінна локація, де можна рухатися, оглядати предмети та взаємодіяти з оточенням.

На столі лежить блокнот із написом «Мої подорожі». Поруч — чашка, карта регіону, старий рюкзак і кілька речей, які можна оглянути.

Текст блокнота:

«Не обов'язково одразу їхати далеко. Можна почати з одного дня, однієї ночі й одного нового місця. Головне — нарешті вирушити».

Цей текст має бути нейтральним до віку, професії, сімейного стану й статі персонажа. Гра не повинна припускати, що кожен має партнера, дітей, автомобіль або власний будинок. Якщо персонаж живе в орендованій кімнаті чи з родиною, локація та репліки адаптуються.

Дії гравця:

- оглянути блокнот;
- відкрити карту;
- перевірити доступні гроші;
- поговорити з членом домогосподарства, якщо він є;
- перейти до планування.

Приклад репліки близької людини:

«То ти справді збираєшся? Знаєш, мені здається, тобі це піде на користь. Тільки не намагайся взяти з собою всю квартиру».

Для самотнього персонажа ця репліка замінюється внутрішньою думкою або записом у блокноті. Для сім'ї інші члени домогосподарства можуть висловити власні очікування.

Система має зафіксувати:

- `intro.started = true`;
- вибір першої мети;
- початковий баланс;
- наявне спорядження;
- активний профіль подорожі.

На цьому етапі не повинно бути фінансового штрафу, термінового дедлайну або вимоги негайно придбати дорогий транспорт.

Сцена 2. Планування — «Куди поїхати?»

ID: `INTRO-SCENE-002-PLANNING`

Гравець відкриває карту регіону. На ній показані три навчальні напрямки.

Варіант A — Озеро Тихих Верб

Легкий маршрут · Коротка поїздка

Найпростіший варіант для першої ночівлі. Є вода, санітарні зручності, рецепція та коротка прогулянкова стежка.

Варіант B — Село Млинове

Легкий маршрут · Невелика прогулянка

Невелике поселення з крамницею, пекарнею та ринковою площею. Підходить для знайомства з місцевими жителями.

Варіант C — Лісова Галявина

Легкий маршрут · Більше природи

Тихіше місце, коротка прогулянка до оглядової точки та менше магазинів поблизу. Потрібно краще перевірити запаси.

Ці місця вигадані. Вони потрібні, щоб перевірити механіки до підключення реальної географії.

Гравець може вибрати бажану атмосферу, тривалість і тип активності. Система показує орієнтовний час дороги, очікувані витрати, необхідні речі й доступні послуги.

Важливе правило: вибір місця не означає автоматичного бронювання. Якщо бронювання потрібне, система перевіряє його окремо. У вступному сценарії

можна гарантувати доступність місця для навчання, але інтерфейс має чесно позначити, що це спеціально підготовлена навчальна умова.

Сцена 3. Магазин — «Купуй те, що справді потрібно»

ID: `INTRO-SCENE-003-SHOP`

Гравець потрапляє до невеликого магазину спорядження. Продавець не намагається продати найдорожчий товар. Він запитує, як саме гравець планує подорожувати.

Продавець:

«На одну ніч? Тоді давай не купувати зайвого. Спочатку перевіримо, що в тебе вже є».

Відкривається список предметів із масою, ціною, станом і призначенням.

Для наметового туриста важливі намет, спальне місце й захист від погодних умов. Велосипедист додатково оцінює світло, ремонтний набір, насос і розподіл багажу. Власник кемпера перевіряє наявність базових речей і стан систем автомобіля.

Гра не вимагає купувати весь список. Вона відокремлює обов'язкове для конкретного маршруту від корисного та необов'язкового.

Приклад вибору:

- придбати відсутній базовий предмет;
- вибрати бюджетний аналог;
- позичити доступну річ у знайомого, якщо така можливість існує;
- відкласти необов'язкову покупку.

Якщо грошей не вистачає, система не повинна мовчки списувати їх у мінус. Вона показує, чого бракує, та пропонує доступні альтернативи. Для першої пригоди існує гарантований бюджетний шлях, але не безкоштовний дорогий комплект.

Сцена 4. Дорога — «План не означає точність до хвилини»

ID: `INTRO-SCENE-004-TRAVEL`

Гравець виходить із дому й починає подорож. Тут уперше використовується реальна модель часу та способу пересування.

- Велосипед враховує базову швидкість, ухил, вантаж і стан.
- Автомобіль враховує маршрут, витрату пального та дорожні умови.
- Кемпер має власні обмеження щодо маси, розміру та доступності дороги.
- Інші способи подорожі використовують відповідні правила.

У вступі маршрут короткий, умови сприятливі, а відхилення від прогнозу невеликі. Це не означає, що весь світ буде таким: перша сцена навчає користуватися системою, а не перевіряє навички виживання.

Після прибуття гравець бачить повідомлення:

«Ти на місці. Дорога позаду, але маленька пригода тільки починається».

Сцена 5. Рецепція — «Люди, які знають це місце»

ID: `INTRO-SCENE-005-RECEPTION`

Тут гравець зустрічає Елізу Вермеулен, 51 рік, адміністраторку сімейного кемпінгу. Вона вже визначена в біблії NPC і має зберігати ту саму особистість у всіх сценах.

Еліза:

«Вітаю! У нас сьогодні спокійний день. Якщо це твоя перша така подорож, не соромся запитувати. Найчастіше люди хвилюються через речі, які насправді легко вирішити».

Еліза пояснює правила місця: де розташовані вода, санітарні приміщення, сміттєві контейнери та зони відпочинку. Власнику кемпера вона додатково пояснює, де дозволено підключення, а велосипедисту — де можна безпечно залишити велосипед.

Якщо гравець подорожує з твариною, Еліза підтверджує правила для неї. Якщо тварини не допускаються до певної зони, це повідомляється заздалегідь.

Гілки діалогу:

- «Де тут можна набрати питної води?»
- «Де найкраще прогулятися ввечері?»
- «Чи є поблизу щось цікаве для відвідування?»
- «Я вперше в такому місці. На що звернути увагу?»

Еліза відповідає залежно від запитання. Кожна відповідь має практичну користь, але не перетворюється на довгу лекцію.

Сцена 6. Маленька історія — «Складаний столик»

ID: `INTRO-SCENE-006-TABLE`

На сусідній ділянці стоїть складаний столик. Біля нього нікого немає. Поруч з'являється Нора Де Смет, 28 років, велосипедистка й фотографка, яка вже знайома гравцеві з біблії NPC.

Нора:

«Ти випадково не бачив, хто залишив столик? Я щойно помітила його біля проходу. Не хочу, щоб хтось подумав, ніби його можна просто забрати».

Це невелика, але важлива сцена. Вона навчає спостерігати за оточенням, спілкуватися та поважати чуже майно.

Варіант A — допомогти знайти власника.

Гравець оглядає найближчі ділянки, запитує в Елізи або чекає на повернення сусіда. Якщо власника знайдено, той дякує. Це створює позитивний факт у пам'яті NPC.

Варіант B — запропонувати запитати на рецепції.

Гравець не шукає сам, але обирає обережний і практичний спосіб. Еліза допомагає встановити власника.

Варіант C — не втручатися.

Нічого поганого автоматично не відбувається. Гравець продовжує облаштування табору, а історія завершується без нагороди за допомогу.

Не можна давати негативну карму лише за відмову від необов'язкової допомоги. Гра не повинна нав'язувати єдиний морально «правильний» стиль поведінки.

Сцена 7. Вечір — «Тепер це твій табір»

ID: `INTRO-SCENE-007-EVENING`

Гравець облаштовує місце, перевіряє речі, готує просту вечерю або обирає доступний варіант харчування. Якщо це кемпер, можна оглянути воду й електрику. Якщо намет — перевірити спальне місце та освітлення.

Тут важлива свобода: гравець не повинен виконувати кожну дію, щоб побачити завершення сцени. Достатньо організувати безпечну ночівлю й не мати критично невирішеної потреби.

Перед сном гра показує короткий запис у щоденнику:

«Перша ніч поза домом. Виявилося, що подорож — це не лише дорога. Це ще й місце, де ти опиняєшся, люди, яких зустрічаєш, і дрібниці, про які потім згадуєш із посмішкою».

Цей запис не повинен бути однаковим для всіх. Якщо гравець обрав інший стиль подорожі або відмовився від соціальної сцени, щоденник використовує відповідні факти.

Сцена 8. Ранок — «Що ти забереш із собою?»

ID: `INTRO-SCENE-008-MORNING`

Ранок починається без раптової небезпеки. Гравець бачить табір, може поснідати, оглянути місцевість і зібрати речі.

Еліза або Нора може коротко поговорити з гравцем, якщо відповідна гілка була активована.

Гравець обирає:

- повернутися додому;
- прогулятися короткою стежкою;
- відвідати село, якщо воно доступне;
- провести трохи більше часу в кемпінгу, якщо це дозволяє бронювання й бюджет.

Додаткові активності необов'язкові. Вони не повинні змушувати гравця витрачати гроші, яких він не має.

Завершення

Місія `STORY-INTRO-001` завершується після першої успішної ночівлі та завершення або свідомого пропуску ранкової фази.

Гравець отримує:

- запис про першу подорож;
- нову сторінку в щоденнику;
- історію витрат і часу;
- пам'ять про зустрінутих NPC;
- доступ до наступних цілей планування.

Нагорода не повинна бути великою сумою грошей, яка зламає економіку. Основна нагорода — досвід, нові знання, відкриті можливості й відчуття завершеної пригоди.

13. Як перетворити цю біблію на робочу гру

Тепер критично важливо не відправити AI-агенту весь документ одним завданням. Він може створити багато коду, але не зможе надійно перевірити всі взаємозв'язки за один прохід.

Я пропоную таку послідовність. Кожен промпт — окремий етап із конкретним результатом.

00

Аудит проєкту

Перевірити наявні файли, код, документацію, залежності, конфлікти та прогалини. Нічого не переписувати без потреби.

Результат: реальний стан проєкту та список блокерів.

01

Затвердження правил і контрактів

Описати `WorldState`, команди, події, час, гроші, інвентар, квести, NPC та збереження.

Результат: узгоджені типи й правила без дублювання джерел правди.

02

Мінімальний ігровий прототип

Запустити браузерну гру, відобразити локацію, персонажа й камеру, додати рух та інтерактивні об'єкти.

Результат: персонаж може рухатися й взаємодіяти з оточенням.

03

Інтерфейс та створення персонажа

Створити персонажа, домогосподарство, професію, стартові ресурси й профіль подорожі.

Результат: кожен новий світ має валідний стартовий стан.

04

Інвентар, покупки й економіка

Реалізувати предмети, масу, ціни, транзакції, покупки, продажі й базові витрати.

Результат: жодних подвійних покупок або некоректних списань.

05

Перша сюжетна пригода

Реалізувати всі сцени, діалоги, вибори, переходи та необов'язкові гілки.

Результат: завершуваний вступ без тупиків.

06

NPC та пам'ять світу

Додати розклади, стосунки, факти пам'яті, повторні розмови й зміни доступних подій.

Результат: персонажі пам'ятають попередні взаємодії.

07

Серверні акаунти й збереження

Реалізувати автентифікацію, права доступу, версії збережень, резервні копії та відновлення.

Результат: прогрес відновлюється після перезапуску та входу з іншого пристрою.

08

Фізика, транспорт і погода

Додати перші перевірені моделі часу маршруту, вантажу, витрат і стану транспортного засобу.

Результат: прогнозовані, пояснювані наслідки та тестові сценарії.

09

Арт, звук і полірування

Затвердити розмір тайлів, пропорції персонажів, анімації, колізії, палітру й читабельність.

Результат: узгоджений візуальний стиль без шкоди для керування.

10

Регресійне тестування

Перевірити збереження, альтернативні гілки, домашніх тварин, різні типи транспорту, помилки мережі та крайні випадки.

Результат: перевірений вертикальний зріз гри.

Це рекомендована послідовність, а не незмінний закон. Наприклад, структуру серверного збереження потрібно спроєктувати ще на ранньому етапі, навіть якщо повноцінна реєстрація з'явиться пізніше.

13.1. Які перевірки не можна пропускати

| Сценарій | Очікувана поведінка |
| --- | --- |
| Гравець закрив браузер | Підтверджений прогрес відновлюється |
| Мережа зникла під час покупки | Не виникає подвійного списання |
| Гравець натиснув «Купити» двічі | Транзакція виконується лише один раз |
| Не вистачає грошей | Покупка не відбувається, причина показується |
| Немає необов'язкового предмета | Доступна альтернативна гілка |
| Квест пропущено | Основний сюжет не зависає |
| NPC зустріли вдруге | Діалог враховує історію |
| Гравець має кота чи собаку | Перевіряються правила місця й потреби тварини |
| Гравець змінив транспорт | Квест не вимагає старого типу транспорту без причини |

| | |
|---|---|
| Стара версія збереження | Виконується міграція або безпечне відновлення |
| Два пристрої змінили один слот | Виявляється конфлікт версій |
| Дані локації неповні | Гра повідомляє про невизначеність, а не вигадує факт |

Ці сценарії потрібно автоматизувати там, де це можливо. Ручна перевірка корисна для діалогів, емоційного ефекту й керування, але не замінює тестів логіки.

14. Контроль якості: як не дозволити грі непомітно зламатися

Я хочу додати ще одну важливу річ: гра повинна перевіряти не лише те, чи працює окрема функція, а й те, чи не зіпсувала вона інші системи.

Наприклад, ми змінюємо масу велосипеда. Сам велосипед продовжує працювати, але тепер гравець може не встигнути на пором. Через це може завершитися бронювання, змінитися бюджет, зірватися зустріч із NPC і порушитися ланцюжок квестів. Усі ці наслідки потрібно або перевірити, або обґрунтовано виключити.

Тому кожна велика система матиме три види перевірок:

- Локальні тести: чи правильно працює сама механіка.
- Інтеграційні тести: чи правильно вона взаємодіє з іншими системами.
- Наскрізні сценарії: чи може гравець пройти повну історію від початку до кінця.

Окремо потрібні автоматичні інваріанти — правила, які ніколи не повинні порушуватися.

Приклади:

- гроші не можуть змінитися без зареєстрованої фінансової операції;
- кількість предмета не може стати від'ємною;
- один квест не може одночасно бути активним і завершеним;
- одна транзакція не може двічі видати винагороду;
- NPC не може мати дві суперечливі позиції в різних локаціях;
- бронювання не може бути підтвердженим, якщо сервер його не підтвердив;
- гравець не може опинитися в локації, яка не існує в поточній версії світу;
- збереження не може посилатися на відсутній предмет без визначеного способу відновлення;
- завершення необов'язкового квесту не повинно бути необхідним для основної історії, якщо це не закладено в затверджений граф залежностей.

Ці правила повинні перевірятися кодом, а не залишатися лише текстом у документації.

15. Що я рекомендую затвердити зараз

Ми вже маємо основу концепції, правила проєкту, архітектурний напрям, сюжетну арку, першу пригоду, модель збереження та план реалізації. Але ще не маємо повністю завершеного й узгодженого GDD, усіх сценаріїв, перевірених формул і працюючого прототипу. Це важливо чітко розділяти.

Я пропоную закріпити такі рішення як попередньо затверджені:

- браузерна 2D pixel-art гра з перспективою згори під кутом;
- однокористувацький світ із постійним прогресом;

- персонаж і стиль життя, які створює сам гравець;
- туристичне життя з реалістичними, але ігрово збалансованими системами;
- розгалужені квести, унікальні NPC та пам'ять світу;
- серверні акаунти й захищені збереження;
- вигаданий навчальний регіон для першого прототипу;
- поступове додавання реальної географії, характеристик і правил;
- жодного порнографічного контенту, графічного насильства чи незаконних завдань;
- розробка малими етапами з обов'язковими перевірками.

Невирішеними залишаються конкретний серверний стек, постачальник автентифікації, остаточні моделі даних, балансова економіка, точність фізичних моделей, політика зберігання даних, ліцензії джерел та повний каталог квестів. Їх потрібно обґрунтувати, а не вибирати навмання.

Наступний крок

Я б не починав зараз із сотень додаткових подій або з великої карти Європи. Найцінніший наступний документ — повна технічна специфікація вертикального зрізу гри, тобто першої маленької, але завершеної версії, в якій працюють:

1. створення персонажа;
2. домашня локація та рух;
3. інвентар і покупки;
4. планування короткої подорожі;
5. перший кемпінг;
6. NPC і гілки діалогу;
7. завершення першої ночівлі;
8. збереження й відновлення прогресу;
9. автоматизовані тести цілісності.

Коли цей зріз запрацює, ми зможемо перевірити, чи справді наші правила витримують реальну гру. Після цього розширюватимемо економіку, транспорт, погоду, роботу, домашніх тварин, NPC та географію на стабільній основі.

І ще одне: я можу допомогти довести проєкт до дуже високого рівня деталізації, послідовності й перевірюваності, але чесна гарантія «жодного бага» неможлива. Саме тому я пропоную не покладатися на безпомилковість будь-якого автора чи AI, а створити систему, яка виявляє помилки рано, не дозволяє їм непомітно поширюватися та захищає прогрес гравця.

Наша мета — не просто описати хорошу гру. Наша мета — зробити так, щоб її можна було послідовно реалізувати, перевірити й розвивати, не руйнуючи те, що вже працює.

чудово, розвивай стратегію, задум, персонажів, сценарій, масив даних для ШІ. Напиши все по максімуму на що ти спроможний, бо мені потрібно ще до початку щоб ти максимально все детально розписав.
Думай глибоко, винаходь цікаві ідеї, дивись за схожими за нашою грою ігри, бери все що платне в інших іграх і в нас не буде плати ні за гру ні за прискорення чи інші donothing just pay and you will got the extra - тобто принаймні зараз ми не обмежуємо гравців тим що ви маєте купити щось, краще будемо будувати так, що якщо треба ігрові грощі чи ще щось наприклад що платне в інших іграх ми будемо давати проходити додаткові квести і місії, наприклад щось зроби і отримай виногороду і це тобі дасть те що потрібно для просування в грі, потенційно навіть пости в соц мережі про наш сайт і гру то люди будуть робити це і отримувати від нас монети чи мілі чи ще щось що буде просувати вгору. Я навть думаю може як з літаками накопичуй мілі і за мілями ми будемо створювати рейтинг гравців? Чи навіть напиши відгук позитивний про нас на трастпайлот чи ще десь і теж монети), також я думаю що ми можемо навіть за те що люди ведуть свій блог, живий журнал на нашому ресурсі за пости давати мілі, тобто людина веде Живий Журнал, пише пости і вони отримують лайки від інших наших кастомерів і тому за кожен щось по ігрових балах буде гравцям. Також медалі і ачівки можемо завести, написав 1 пост, написав 100 постів, завів живий урнал, отримав 1 лайк, 1000 лайків і так далі. Тобто гру треба інтегрувати якось в сайт і заохочити використовувати наш ресурс. Я хочу щоб ти створив унікальну концепцію і сценарій і персонажів, але

щоб не занадто складно. І дивись на топові схожі на нашу ігри і ті що на телефонах топові вони там дуже цікаві сценарії мають, та ж сама "ферма", енгрі бьордз чи інші схожі на нашу гру, то ми можемо додаткові квести брати і основні за мативами але не копіювати чужі ідеї а саме адаптувати до нас.
Подумай сам і проаналізуй що ми вже маємо, чого не вистачає, що треба для нашої гри і розпиши все максимально на всю потужність Чату ГПТ! Прояви всі таланти автара і ігрового мекера, письменника і бери всі скіли які тільки нам потрібні і є доступні тобі! Не економь час чи токени, краще зроби все відраз якісно і логічно, так, щоб ми не зіштохнулися із багами і прогалинами в сценарії чи ігровому всесвіті. Гра має бути повноцінною і цікавою, і не відпускати людину, тобто так щоб настільки було цікаво що він і в день і вночі і на роботі і вдома і навіть з телефона чи планшета заходив і грав і качав і прокачував свого персонажа.

Думай

# CAMPING LIFE SIM — стратегія, всесвіт, сюжет, прогресія та екосистема спільноти

Стратегічний документ · редакція 1.0 · 22 жовтня 2026 року

Ми вже маємо фундамент. Тепер я пропоную зробити наступний крок: спроєктувати не просто гру про кемпінг, а цілий туристичний всесвіт, у якому гравець проживає власну історію, розвиває персонажа, відкриває місця, заводить знайомства, здобуває знання й стає частиною спільноти мандрівників.

Є важлива різниця між грою, у яку людина заходить заради одного квесту, і грою, до якої хочеться повертатися роками. У першому випадку достатньо цікавого завдання. У другому — потрібні довгострокові цілі, емоційні зв'язки, відчутний розвиток, свобода вибору та багато невеликих історій, які природно виникають із взаємодії систем.

Наше завдання — поєднати ці якості, але не перетворити гру на другу роботу.

## 1. Що варто запозичити з успішних ігор — і чого не копіювати

Я розглянув кілька різних підходів: затишні симулятори життя, ігри з колекціонуванням, довготривалі сюжетні пригоди та транспортні симулятори. Їхні сильні сторони корисні для нашого проєкту, але механіки треба переосмислити під туризм, а не переносити механічно.

Stardew Valley — особистий розвиток і життя спільноти

Сильна сторона — поєднання повсякденних справ, розвитку навичок, знайомств і довгострокових цілей. Для нас це означає, що поїздка не закінчується на поверненні додому: вона змінює навички, стосунки, знання й бажання персонажа.

Наша відмінність: замість ферми — домашня база, спорядження, транспорт, туристичні маршрути та власна історія подорожей.

Animal Crossing: New Horizons — ритуали, колекції та персоналізація Гравці можуть облаштовувати простір, відкривати колекційні предмети й повертатися до змінного світу.

animalcrossing.nintendo.com
+1

Наша версія: особистий кемпінг, альбом місць, колекція значків маршрутів, спорядження з історією та власний куточок для пам'ятних речей.

Cozy Grove — персонажі, історії та невеликі порції контенту

Гра розкриває історії персонажів поступово, а нові сюжетні завдання стають приводом повернутися.

Nintendo Official Site

+1

Наша версія: знайомства з туристами, які продовжують розповідати власні історії під час наступних зустрічей. Важлива відмінність: ми не повинні блокувати гравця жорстким очікуванням наступного реального дня.

Euro Truck Simulator 2 — маршрут, транспорт і репутація

Транспортні симулятори показують цінність різних режимів кар'єри, спільного прогресу та відкриття доріг.

steamcommunity.com

Наша версія: різні туристичні стилі — велосипед, автомобіль, кемпер і громадський транспорт — мають спільну історію персонажа, але власні вимоги, навички й переваги.

Fields of Mistria — виразні характери й деталізовані діалоги

Огляд PC Gamer особливо відзначає індивідуальність персонажів і якість діалогів.

PC Gamer

Наша версія: NPC повинні мати не просто ім'я й одну квестову фразу, а власні турботи, звички, суперечності, гумор і зміни в житті.

Це не рейтинг і не твердження, що одна гра має всі відповіді. Кожна з них допомагає нам зрозуміти окрему частину дизайну.

Наше конкурентне позиціонування

Camping Life Sim — це гра про життя між подорожами, а не лише про саму дорогу.

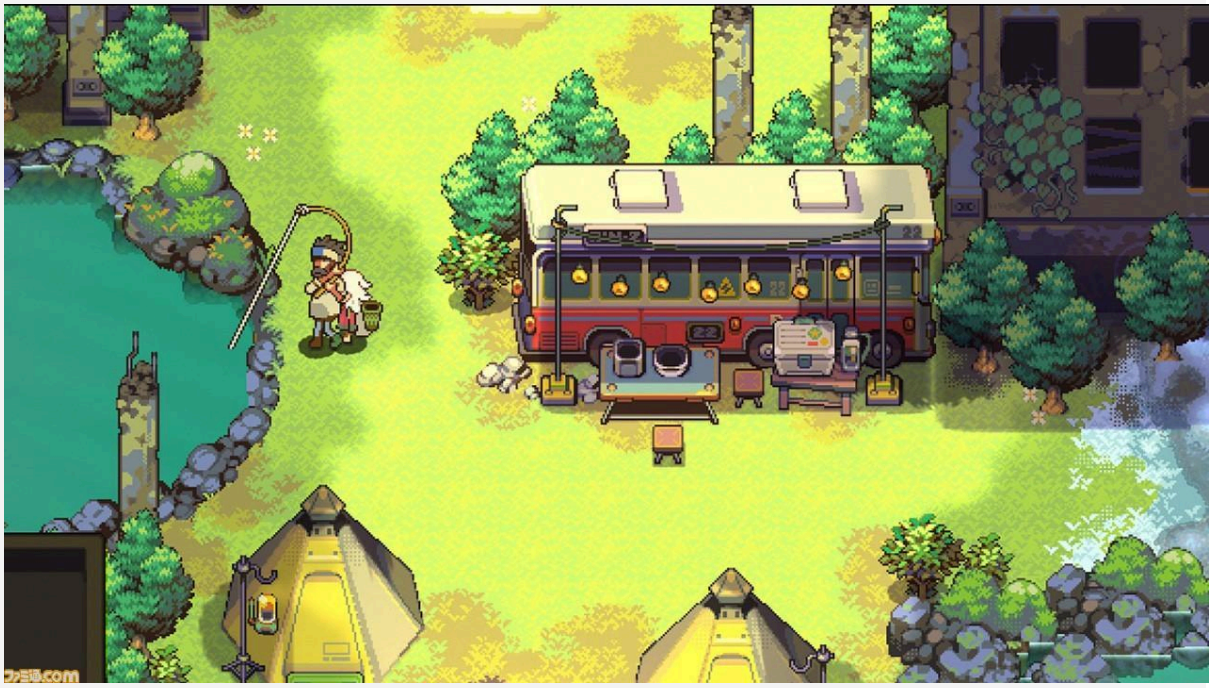
Велосипедна пригода може перетворитися на історію про незнайомця, який допоміг із проколом. Несправність кемпера — на знайомство з механіком, який згодом стане другом. Вдале планування — на можливість побачити фестиваль, про який персонаж раніше навіть не

знав. А випадкова зупинка може стати початком нового довгого маршруту.

Ми не копіюємо ферму, острів чи вантажну кар'єру. Ми використовуємо загальні дизайнерські принципи — прогресію, колекціонування, характерних персонажів, відкриття світу й наслідки вибору — для нашої власної туристичної теми.

## 2. Унікальна концепція: «Атлас малих подорожей»

Я пропоную надати грі власну впізнавану ідентичність. Не просто «симулятор кемпінгу», а історію про те, як звичайні люди відкривають світ, змінюються самі й залишають після себе корисні знання для інших мандрівників.



## Центральна ідея

У вигаданому європейському регіоні Вербокрай існує невелика туристична спільнота — Коло Мандрівників. Вона збирає історії, маршрути, практичні поради та спогади людей, які подорожують

по-різному: пішки, велосипедом, автомобілем, із наметом, у кемпері, наодинці, із сім'єю або домашніми тваринами.

У старому громадському будинку знаходять коробку польових щоденників і недовершених карт, зібраних багатьма мандрівниками за десятиліття. Частина записів застаріла, деякі маршрути змінилися, кілька кемпінгів закрилися, а деякі місцеві історії майже забулися.

Гравець не отримує завдання «врятувати світ». Він починає зі звичайної подорожі, а згодом може долучитися до великої справи: відновити живий атлас місць, маршрутів і людських історій.

Цей задум дає нам природний спосіб поєднати всі системи:

- нові місця відкривають сторінки атласу;
- практичні випробування навчають туристичних навичок;
- знайомства розкривають історії NPC;
- фотографії, щоденники й знахідки стають колекціями;
- маршрути можуть отримувати перевірені оновлення;
- допомога місцевим відкриває нові можливості;
- повернення до знайомих місць показує, як вони змінюються;
- сайт перетворюється на продовження туристичного щоденника гравця.

Це також створює нескінченний потенціал для нових історій без потреби постійно вигадувати глобальні катастрофи.

## Головна сюжетна таємниця

У щоденниках є серія незавершених записів під назвою «Дорога, якої немає на карті». Спочатку здається, що йдеться про загублений шлях. Проте з часом гравець дізнається, що це не одна таємна дорога, а

збірка історій про місця, які різні люди вважали важливими: стежка до старого мосту, маленький сімейний кемпінг, майстерня, де ремонтували велосипеди всім охочим, і оглядовий майданчик, з якого люди прощалися перед довгою подорожжю.

Це не кримінальний сюжет і не фантастична загадка. Таємниця полягає в тому, чому ці місця були важливими для людей і що сталося з їхніми історіями.

Гравець поступово збирає свідчення, порівнює старі нотатки з сучасними умовами, знаходить людей, які пам'ятають минуле, та вирішує, як зберегти ці історії.

Різні рішення змінюють фінал:

- Архіварій доріг: гравець робить ставку на точні маршрути, перевірені відомості й практичну користь.
- Голос спільноти: збирає людські історії, спогади, традиції й історії взаємодопомоги.
- Майстер мандрів: допомагає розвивати інфраструктуру, ремонтні майстерні й доступність подорожей.
- Вільний мандрівник: створює особистий атлас власних відкриттів, не прагнучи керувати спільнотою.

Ці напрямки не є класами персонажа й не блокують решту гри. Це різні акценти в історії, які можуть поєднуватися.

## Правило реалізму

Старі карти й записи ніколи не повинні автоматично вважатися достовірними. Вони можуть містити помилки, застарілі ціни, закриті дороги або суб'єктивні оцінки.

Гравець може позначити запис як:

- підтверджений особисто;
- підтверджений кількома незалежними джерелами;
- застарілий;
- неперевірений;
- спростований;
- корисний, але суб'єктивний.

Так ми робимо перевірку фактів частиною ігрової механіки. Водночас реальні користувацькі повідомлення не повинні автоматично ставати офіційними туристичними рекомендаціями на сайті.

## 3. Повна структура сюжетної кампанії

Основна кампанія повинна мати чіткий фінал, але не завершувати весь світ. Після фіналу гравець продовжує подорожувати, виконувати побічні історії, збирати колекції, відкривати нові регіони й розвивати персонажа.

Пролог — «Одна вільна ніч»

Створення персонажа, перше планування, спорядження, перша ночівля, знайомство з Елізою та Норою.

Мета: дати гравцеві впевненість і відчуття першої завершеної пригоди.

Розділ I — «Місця поруч»

Короткі маршрути, знайомство з інфраструктурою, перші записи в атласі, базові навички й прості соціальні завдання.

Мета: показати, що цікава подорож не обов'язково має бути далекою чи дорогою.

Розділ II — «План і реальність»

Вибір маршруту, вага багажу, погода, бюджет, ремонт, розклад поромів або транспорту. Перша пригода, де план доводиться змінювати.

Мета: навчити адаптуватися без несправедливого покарання.

Розділ III — «Люди на шляху»

Розкриваються особисті історії NPC, їхні конфлікти, цілі та стосунки. Гравець дізнається, що одна й та сама дорога має різне значення для різних людей.

Мета: зробити персонажів причиною повернення до світу.

Розділ IV — «Коли щось іде не за планом»

Складніші ремонти, несподівані витрати, невдале бронювання, суперечності в групі, погодні зміни та рішення про безпеку.

Мета: показати наслідки підготовки й цінність запасного плану.

## Розділ V — «За горизонтом»

Далекі подорожі, кілька регіонів, нові види транспорту, довші маршрути, сезонні обмеження та спеціалізовані навички.

Мета: дати гравцеві відчути, що він став досвідченим мандрівником.

Фінал — «Атлас живих доріг»

Гравець допомагає підготувати виставку маршрутів і людських історій. Різні завершення залежать від його рішень, навичок, знайомств і зібраних матеріалів.

Мета: завершити головну історію, не закриваючи світ для подальшої гри.

У кожному розділі мають бути три типи контенту:

1. Сюжетні квести — розкривають центральну історію та персонажів.

2. Системні квести — навчають планування, ремонту, бюджету, навігації й догляду.
3. Життєві історії — короткі, необов'язкові, емоційні або кумедні ситуації.

Не кожна історія має бути важливою для фіналу. Іноді найкращим спогадом стане вечеря з незнайомцями, спільний ремонт проколотого колеса або листівка, яку NPC надішле через кілька ігрових тижнів.

# 4. Персонажі, які можуть утримувати інтерес роками

Ми не повинні вимірювати якість світу кількістю NPC. Краще мати 20 добре написаних персонажів із пам'яттю, розкриттям і власними сюжетними арками, ніж 200 однакових продавців із різними іменами.

Пропоную поділити персонажів на три рівні:

- Основні NPC: власні сюжетні арки, стосунки, пам'ять, кілька можливих станів історії.
- Регіональні NPC: локальні завдання, професійні знання, невеликі особисті історії.
- Фонові мешканці: повсякденні дії, короткі розмови, відчуття живого світу. Вони також мають стабільні ідентифікатори, але не кожен потребує багатогодинної арки.

## 4.1. Основний акторський склад

Це розширення раніше визначених персонажів. Імена й біографії мають бути закріплені за унікальними ID і не перевикористовуватися.

Еліза Вермеулен, 51 рік — хранителька місця

Власниця сімейного кемпінгу. Практична, спокійна, з сухим гумором. Вона пам'ятає, що кожен досвідчений турист колись уперше розкладав намет.

Її історія: кемпінг переживає зміни: дорожчає обслуговування, молодь хоче сучасних зручностей, постійні гості прагнуть тиші. Еліза шукає спосіб зберегти характер місця, не перетворюючи його на музей минулого.

Квести: «Правила без дрібного шрифту», «Тиха ділянка», «Гість, який не повернувся», «Кемпінг для різних поколінь».

Нора Де Смет, 28 років — фотографка доріг

Велосипедистка, фотографка й уважна спостерігачка. Вона знає багато красивих місць, але іноді захоплюється кадром і забуває перевірити запас води.

Її історія: Нора мріє створити фотоальбом маловідомих маршрутів, але поступово розуміє, що красиві фотографії не завжди передають реальну складність дороги.

Квести: «Кадр до дощу», «Дорога без узбіччя», «Фото чи попередження?», «Історія за знімком».

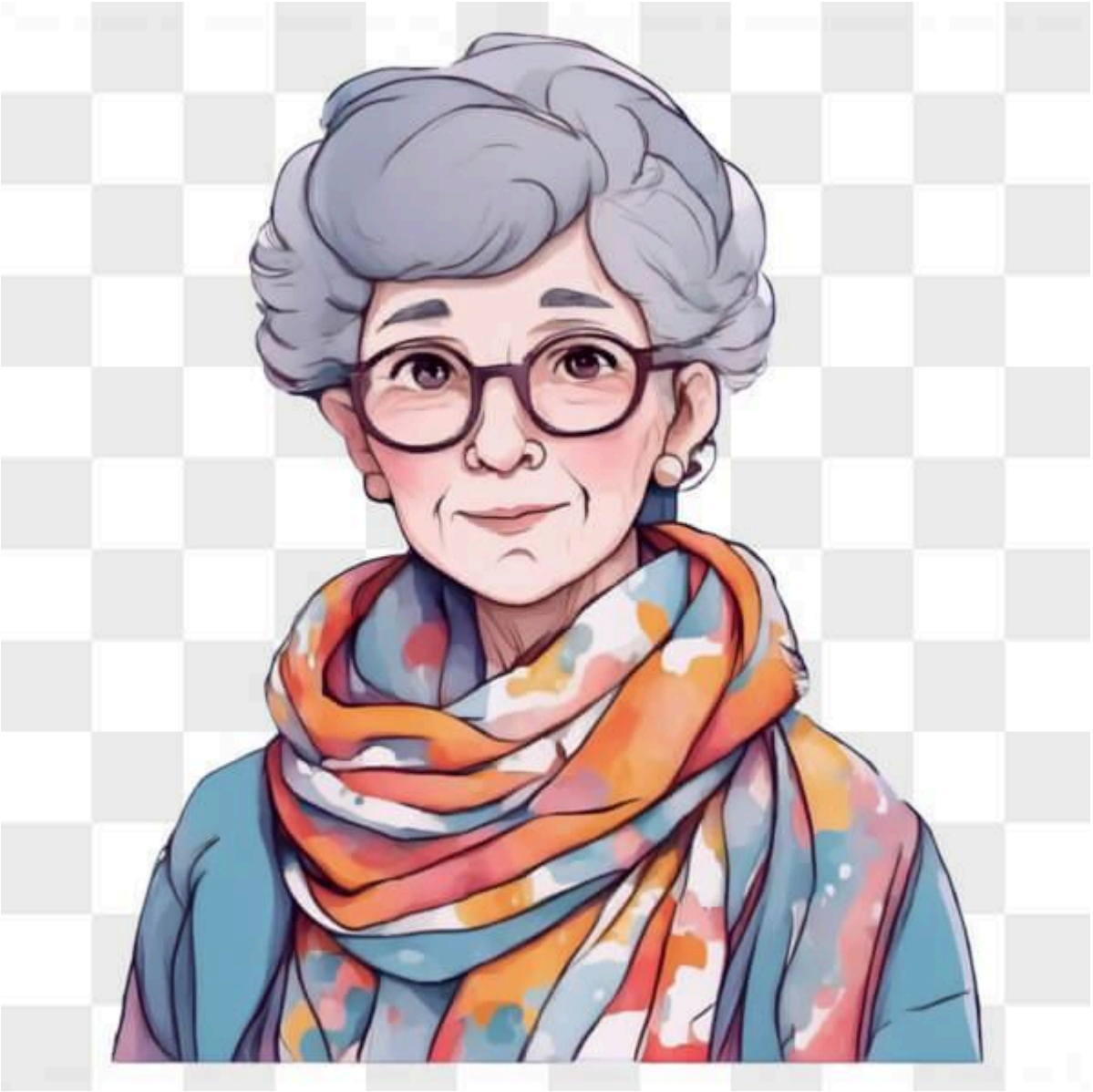
Її розвиток — від фотографки, яка шукає красиві місця, до авторки відповідальних туристичних історій.

Люк Бернар, 46 років — механік, який навчає

Веломеханік, який не любить, коли клієнтів змушують купувати зайве. Він пояснює причини поломок, а не просто продає ремонт.

Його історія: молодий помічник хоче працювати швидко, Люк наполягає на перевірках. Їхній конфлікт допомагає гравцеві зрозуміти баланс між швидкістю, вартістю й безпекою.

Квести: «Прокол у невдалий момент», «Звук, якого не було вчора», «Ремонт без зайвих деталей», «Чому знову зносився ланцюг?».



Інгрід Мертенс, 63 роки — пам'ять маршрутів

Досвідчена мандрівниця, яка зберігає старі карти, листівки та записники. Вона знає, що дорога змінюється, і не вважає власний досвід універсальною інструкцією.

Її історія: Інгрід хоче передати архів молодшим, але боїться, що цифрова карта втратить особисті історії.

Квести: «Позначки на полях», «Міст, якого вже немає», «Запис без дати», «Що варто зберегти?».

Мілан Пітерс, 31 рік — віддалена робота в дорозі

Працює з кемпера. Знає електрику й цифрові інструменти краще, ніж більшість туристів, але іноді недооцінює потребу у відпочинку.

Його історія: Мілан намагається поєднати свободу подорожей із дедлайнами. Йому доведеться навчитися відмовлятися від красивої стоянки, якщо там немає надійного зв'язку, і не перетворювати кожну зупинку на офіс.

Квести: «Зниклий сигнал», «Робочий день на двадцять відсотків батареї», «Мітинг із видом на озеро», «Вихідний без ноутбука».

Сара Ламбер, 37 років — провідниця місцевості

Добре знає природні стежки, місцеві традиції та правила безпечного відвідування. Вона не романтизує природу й нагадує, що красиве місце не завжди безпечне або доступне.

Її історія: Сара хоче, щоб туризм приносив користь місцевим жителям, не руйнуючи середовище.

Квести: «Стежка після зливи», «Куди не варто йти заради фото», «Маленький фестиваль», «Гості, які залишили сміття».

Томас Легран, 44 роки — людина розкладу

Працює на транспортному вузлі й пояснює, чому прибуття до порома за хвилину до відправлення — поганий план.

Його історія: Томас часто бачить пасажирів, які звинувачують транспорт у власних помилках. Він поступово допомагає гравцеві розібратися в часі посадки, розмірах транспортних засобів і запасі на затримки.

Квести: «Останній виклик на посадку», «Не той причал», «Квиток для високого кемпера», «План Б на березі».

Клер Дюбуа, 39 років — ветеринарка

Допомагає планувати подорожі з домашніми тваринами, пояснює транспортні вимоги, потреби в догляді й ситуації, коли потрібна професійна допомога.

Її історія: Клер прагне зробити подорожі доступнішими для власників тварин, але нагадує, що не кожне місце підходить кожному улюбленцю.

Квести: «Спокійний кіт у дорозі», «День довгої прогулянки», «Документи перед кордоном», «Тварина теж втомлюється».

### Додаткові основні персонажі, яких варто створити

Щоб історії не оберталися лише навколо кемпінгу, ремонту й маршруту, доцільно додати ще чотирьох персонажів:

- Марта Коваль, 34 роки — координаторка Кола Мандрівників. Веде архів, організовує зустрічі та іноді сперечається з учасниками через точність інформації.
- Бенуа Рішар, 55 років — власник майстерні кемперів. Знає техніку, але його бізнес переживає складний період. Він повинен обирати між дешевим ремонтом і довгостроковою надійністю.
- Аміна Хаддад, 26 років — працівниця сезонного туристичного центру. Знає місцеві події, мови й побутові нюанси; хоче започаткувати доступніші екскурсії.
- Павел Новак, 71 рік — колишній водій туристичного автобуса. Розповідає історії доріг і людей, але іноді спирається на старі знання, які потрібно перевірити.

Ці персонажі мають власні цілі й слабкості. Ніхто не повинен бути лише «джерелом квестів».

## 4.2. Правила написання діалогів

Кожна важлива розмова повинна мати контекст. NPC може враховувати:

- чи знайомий він із гравцем;
- де вони зустрілися;
- що гравець робив раніше;
- чи виконав обіцянку;
- чи має зараз відповідний предмет або навичку;
- що відомо про попередні події;
- який настрій і поточна мета NPC;
- чи має персонаж час на розмову.

Однак пам'ять не повинна створювати хаос. Замість збереження всіх текстів розмов ми зберігаємо структуровані факти.

Наприклад:

```
{
  "npcId": "npc.nora.de_smet",
  "memoryFacts": [
    {
      "type": "PLAYER_HELPED_NPC",
      "eventId": "event.intro.table_help",
```

```
      "importance": 1,
      "createdAtWorldDay": 3
    },
    {
      "type": "SHARED_LOCATION_VISIT",
      "locationId": "location.willow_lake",
      "importance": 1,
      "createdAtWorldDay": 3
    }
  ],
  "relationship": {
    "familiarity": 2,
    "trust": 1,
    "respect": 1
  }
}
```

Ці значення — приклад формату, а не остаточні числові правила. У реальній реалізації всі типи фактів, межі показників і правила зміни стосунків мають бути централізовано визначені.

Головне: якщо Нора подякувала гравцеві за допомогу, вона не повинна через кілька днів говорити так, наче вони ніколи не зустрічалися. І якщо гравець не виконав необов'язкову обіцянку, система повинна врахувати це пропорційно важливості обіцянки, а не автоматично знищувати стосунки.

# 5. Прогресія, яка дає бажання повертатися

Наша гра не повинна змушувати гравця бездумно повторювати одні й ті самі дії заради числа. Прогрес має відкривати нові можливості, історії й способи подорожувати.

Пропоную розділити прогрес на п'ять незалежних напрямків.

Навички мандрівника

Планування, навігація, облаштування табору, механіка, електрика, харчування, фінанси, фізична підготовка й соціальна взаємодія.

Новий рівень відкриває знання або дію, а не магічно робить персонажа невразливим.

Дослідження світу

Відкриті регіони, перевірені маршрути, оглядові місця, кемпінги, транспортні вузли, фестивалі й сезонні локації.

Соціальна історія

Дружба, довіра, виконані обіцянки, спільні пригоди, запрошення, листівки, знайомства та персональні сюжетні арки.

Спорядження та власна база

Рюкзаки, велосипеди, автомобілі, кемпери, інструменти, покращення, облаштування місця відпочинку та колекції пам'ятних речей.

Особистий атлас

Записи про подорожі, фотографії, перевірені поради, пам'ятні місця, досягнення й власна історія мандрівника.

## 5.1. Рівні навичок

Для старту пропоную шість рівнів кожної навички:

| Рівень | Назва | Що означає |
|---|---|---|
| 0 | Новачок | Потребує інструкцій |
| 1 | Початківець | Може виконувати базові дії |
| 2 | Упевнений | Краще оцінює час і ресурси |

| | | |
|---|---|---|
| 3 | Досвідчений | Відкриває додаткові варіанти |
| 4 | Експерт | Краще діагностує й планує складні завдання |
| 5 | Наставник | Може навчати інших і допомагати в особливих квестах |

Навички зростають від практики, навчання та обґрунтованого використання знань. Просте повторення бездумної дії не повинно нескінченно давати досвід.

Наприклад, механік із високою навичкою може швидше визначити причину несправності, але йому все одно можуть знадобитися інструменти, запасні частини й відповідні умови. Гравець із високою навичкою навігації не повинен автоматично знати, чи відкрита дорога сьогодні: актуальність інформації — окрема властивість.

## 5.2. Внутрішні досягнення

Досягнення поділяються на:

- майстерність: перший самостійний ремонт, успішне планування маршруту, правильний вибір спорядження;
- дослідження: відвідування регіонів, сезонні місця, відкриті маршрути;
- людяність: допомога, виконані обіцянки, участь у спільних історіях;
- колекціонування: листівки, значки, пам'ятні предмети, записи в атласі;
- витривалість кампанії: завершення сюжетних арок і тривалих проєктів;
- стиль життя: подорожі з велосипедом, кемпером, сім'єю або домашнім улюбленцем.

Не потрібно карати гравця за те, що він не виконав усі досягнення. Досягнення мають відображати різні стилі гри, а не встановлювати єдиний правильний спосіб проходження.

# 6. Безкоштовна гра, милі, нагороди та рейтинг

Тут є можливість створити дуже сильну систему, але я б не робив її звичайною схемою «набери якомога більше очок — стань першим».

Якщо винагороджувати лише кількість входів, лайків, дописів або витрачених годин, рейтинг швидко почнуть визначати автоматизація, спам і ті, хто має найбільше вільного часу. Це зіпсує сенс спільноти.

Моя пропозиція — «Милі Мандрівника»: програма прогресу, яка об'єднує гру, сайт, особистий журнал, досягнення й туристичні знання.

## 6.1. П'ять різних типів винагород

Не змішуємо всі бали в одну валюту.

1. Кемпінгові монети

Ігрова валюта

Заробляються через роботу, квести, продаж предметів, допомогу NPC та інші внутрішньоігрові дії. Витрачаються на спорядження, ремонт, проживання, послуги та облаштування бази.

Не купуються за реальні гроші. Не можна заплатити за миттєвий рівень, найкращий транспорт чи пропуск важливого випробування.

2. Милі Мандрівника

Довгостроковий прогрес і репутація

Нараховуються за завершені пригоди, дослідження, навчальні досягнення й значущі етапи кампанії. Вони відображають досвід мандрівника, а не кількість грошей чи витраченого часу.

Можуть відкривати титули, рамки профілю, тематичні значки й декоративні предмети. Не дають переваг у фізиці, швидкості або доступі до необхідних ресурсів.

3. Досягнення та медалі

Пам'ять про події

Не витрачаються. Вони фіксують, чого досяг гравець: перша ночівля, перший ремонт, перша подорож із котом, 10 відкритих маршрутів, завершена сюжетна арка.

Медаль за рідкісну подію має залишатися пам'яттю, а не перетворюватися на валюту, яку потрібно фармити.

4. Журнал і внесок у спільноту

Репутація автора

Відображає якість і корисність матеріалів: історії, поради, нотатки про маршрути, звіти про спорядження та відповіді іншим користувачам.

Кількість лайків — лише один сигнал, а не автоматичний доказ якості.

5. Рейтинг мандрівників

Добровільне соціальне змагання

Рейтинги розділені за категоріями: дослідження, знання, доброзичливість, завершені пригоди, внесок у журнал і сезонні випробування.

Не існує одного рейтингу, який перетворює найбагатшого персонажа на «найкращого гравця».

## 6.2. Як заробляти милі

Наведені нижче числа — початкові пропозиції для прототипу. Вони не є остаточним балансом і мають бути перевірені на симуляціях та тестуванні.

| Дія | Початкова винагорода | Запобіжник |
|---|---|---|
| Завершити першу сюжетну пригоду | 100 миль | Одноразово на проходження |
| Завершити нову сюжетну главу | 50–150 | За ID глави |
| Відкрити нову локацію | 10–25 | За перше відкриття |

| | | |
|---|---|---|
| Успішно виконати навчальний квест | 10–40 | Не за нескінченне повторення |
| Вперше самостійно полагодити велосипед | 25 | Потрібна перевірена дія |
| Завершити особисту ціль | 20–100 | Залежить від складності |
| Додати корисний допис у журнал | 0–20 | Після перевірки якості, із лімітом |
| Допомогти новачку з питанням | 0–15 | Після оцінки корисності, із захистом від взаємних накруток |
| Отримати лайк | Не автоматично | Лайки самі по собі не генерують валюту |
| Увійти в акаунт | 0 | Без нагороди за сам факт входу |

Чому я не рекомендую платити за кожен вхід? Бо тоді гравець може почати заходити не через цікавість, а через страх втратити винагороду. Нам потрібна звичка повертатися до улюбленого світу, а не обов'язок перевіряти таймер.

## 6.3. Важливе розділення: ігрові милі та сайтова репутація

Я рекомендую не давати ігрову валюту за кожну дію на сайті. Це відкриє шлях до ботів, фальшивих дописів і накрутки.

Краще зробити так:

- внутрішньоігрові завдання дають монети й милі;
- досягнення зберігають факти про прогрес;
- сайтова репутація формується від якості внеску;
- лайки допомагають читачам знаходити корисне, але не створюють необмежений потік валюти;
- модерація може зняти некоректну репутаційну винагороду, але не повинна довільно переписувати ігровий прогрес.

Якщо допис корисний, він може дати автору бейдж «Корисний досвід», видимість у журналі або участь у тематичній добірці. Це приємніше й безпечніше, ніж автоматично перетворювати кожен лайк на гроші.

## 6.4. Зовнішні пости, відгуки та запрошення друзів

Тут потрібно провести чітку межу.

Я не рекомендую давати монети за позитивні відгуки на Trustpilot чи інших незалежних платформах. Trustpilot прямо забороняє стимулювати відгуки винагородами, а правила FTC у США забороняють винагороди, прямо чи опосередковано прив'язані до позитивного або негативного тону відгуку.

help.trustpilot.com
+2

Ми можемо просити чесний відгук без винагороди, запрошувати до відкритого опитування й реагувати на критику. Це допоможе побудувати довіру, а не лише зібрати високі оцінки.

Інші види участі можна організувати обережніше:

- Поділитися посиланням на гру: надати зручну картку для поширення; не вимагати публічного поста.
- Запросити друга: нагороджувати не за саме створення акаунта, а за чесно підтверджену участь друга в грі, із захистом від самозапрошень і фальшивих акаунтів.
- Опублікувати власну історію: нагороджувати за корисний, оригінальний матеріал на нашому сайті після модерації.
- Допомогти спільноті: давати тематичні відзнаки за якісні відповіді, переклади, перевірку даних або тестування.

Для зовнішніх соціальних мереж спершу треба перевірити правила конкретної платформи. Не варто вимагати від користувача публікувати рекламу, показувати особисті дані або робити допис, який він не хотів би розміщувати самостійно.

### 6.5. Рейтинг, який не перетвориться на змагання гаманців

Я б створив не один, а кілька рейтингів:

- Дослідники: за унікальні локації й перевірені маршрути.
- Майстри: за опановані навички й складні випробування.
- Оповідачі: за якісні історії в журналі.
- Добрі сусіди: за перевірену допомогу іншим.
- Знавці доріг: за точні й корисні дані.
- Сезонні мандрівники: за тематичні випробування певного періоду.

Рейтинг має бути добровільним, із можливістю приховати профіль. Не слід показувати реальне ім'я, місто, вік чи інші персональні дані без явного дозволу.

Важливо також, щоб рейтинг не карав людей, які мають мало вільного часу, не люблять соціальні мережі або грають у власному темпі. Вони повинні мати повноцінний і цікавий ігровий досвід без участі в рейтингах.

## 7. Сайт і гра: один акаунт, два взаємопов'язані світи

Я бачу це як одну екосистему з двома рівноцінними частинами:

- Гра — місце, де людина проживає пригоди, розвиває персонажа, виконує квести та відкриває світ.
- Сайт — місце, де вона зберігає спогади, читає історії інших, веде журнал, обговорює спорядження й отримує практичні знання.

Сайт не повинен бути обов'язковим для проходження гри. Гравець, який хоче лише грати, має отримати повноцінний досвід. А той, хто любить писати, фотографувати чи допомагати іншим, матиме додаткові можливості.

### 7.1. Структура сайту

Головна сторінка

Новини гри, сезонна тема, нові історії, актуальні події спільноти та кнопка «Продовжити подорож».

Гра

Вхід у браузерну гру, вибір збереження, створення персонажа, налаштування керування й доступності.

Живий журнал

Особисті історії, ігрові спогади, нотатки про спорядження, маршрути, сезонні підсумки та записи про цікаві події.

Атлас маршрутів

Внутрішньоігрові вигадані маршрути й, у майбутньому, окремий каталог перевіреної реальної туристичної інформації.

Спільнота

Обговорення, питання, клуби за інтересами, тематичні події, групи велосипедистів, кемперів, сімейних туристів та любителів походів.

Профіль і досягнення

Персонаж, колекції, медалі, історія пригод, опубліковані матеріали й добровільна участь у рейтингах.

## 7.2. Як працюватиме «Живий журнал»

Уявімо, що гравець повернувся з поїздки. Гра може запропонувати створити запис, але не вимагати цього.

Він бачить коротке резюме:

- де побував;
- яким транспортом користувався;
- що відкрив;
- які квести завершив;
- кого зустрів;
- яку несподіванку пережив;
- що нового навчився робити.

З цього можна створити чернетку. Гравець може залишити її приватною, відредагувати або опублікувати.

Важливо: автоматично створений текст не повинен приписувати гравцеві думки, яких він не висловлював, чи вигадувати події. Система може запропонувати факти з журналу проходження, але автор вирішує, що публікувати.

Приклад допису:

> «Наша перша ніч біля Озера Тихих Верб»
>
> Я думав, що найбільше часу піде на дорогу, але виявилося, що довше за все я збирав речі. Наступного разу візьму менше зайвого й перевірю список до виходу з дому.
>
> Найкращий момент — вечірня прогулянка й розмова з Норою. Я дізнався про короткий маршрут, який не помітив би на карті.

Це приклад формату, а не автоматично згенерована реальна історія користувача.

**Категорії дописів**

- «Моя подорож» — особистий досвід.
- «Чого я навчився» — практичний урок.
- «Спорядження без реклами» — досвід використання предметів.
- «Маршрут і умови» — спостереження з датою та місцем.
- «Кумедна пригода» — легкі історії.
- «Запитання спільноті» — прохання про пораду.
- «Історія персонажа» — ігрові спогади й творчі тексти.

Для реальних маршрутів потрібні окремі поля: дата спостереження, джерело, регіон, тип інформації, рівень перевірки та попередження про змінність умов. Не можна змішувати вигадані ігрові маршрути з реальними туристичними даними без чіткого маркування.

## 7.3. Як боротися зі спамом і накруткою

Це потрібно передбачити до запуску, а не після того, як з'являться тисячі акаунтів.

1. Один акаунт не може безмежно отримувати винагороди за повторення тієї самої дії.
2. Лайки від власних альтернативних акаунтів не повинні створювати винагороди.
3. Нові акаунти мають обмеження для певних дій, доки не пройдуть базові перевірки.

4. Публікації мають підтримувати скарги, блокування користувачів і модерацію.
5. Автоматична система може позначити підозрілу активність, але не повинна безпідставно карати користувача.
6. Винагороди за контент нараховуються лише після серверної перевірки.
7. Правила модерації, оскарження та блокування мають бути відкритими.
8. Забороняється купувати лайки, автоматизовано створювати публікації або вводити інших користувачів в оману.

Для платформи з користувацькими публікаціями потрібно також спроєктувати механізми повідомлення про незаконний контент, обробку скарг і прозорі правила. У ЄС Digital Services Act містить вимоги щодо механізмів повідомлення й реагування для сервісів, до яких застосовуються відповідні положення. Конкретні обов'язки залежать від типу сервісу, його розміру та інших обставин.

EUR-Lex
+1

# 8. Технічна модель даних для гри та сайту

Щоб інтеграція не перетворилася на клубок умов, треба розділити кілька понять.

- Акаунт — хто має право входити до сервісу.
- Профіль сайту — публічне ім'я, налаштування видимості, біографія та соціальні параметри.
- Персонаж гри — конкретна історія проходження.
- Збереження — версійований стан світу персонажа.
- Журнал — приватні або опубліковані записи.
- Досягнення — підтверджені факти прогресу.
- Транзакції — історія змін валют і винагород.
- Репутація — обчислювані показники спільноти.
- Модерація — окрема система доступу, скарг і рішень.

Не можна використовувати один числовий рейтинг як заміну всім цим сутностям.

## 8.1. Рекомендовані сутності

```
type AccountId = string;
type CharacterId = string;
type SaveId = string;
type EventId = string;
type TransactionId = string;
type ContentId = string;

interface Account {
  id: AccountId;
  status: "active" | "restricted" | "closed";
  createdAt: string;
  authProvider: string;
  privacySettingsId: string;
}

interface PlayerProfile {
  accountId: AccountId;
  displayName: string;
  publicProfileEnabled: boolean;
  journalVisibility: "private" | "followers" | "public";
  leaderboardOptIn: boolean;
}

interface Character {
  id: CharacterId;
  accountId: AccountId;
  displayName: string;
  appearance: CharacterAppearance;
  householdId: string;
  skills: Record<string, number>;
  createdAtWorldDay: number;
}

interface SaveGame {
  id: SaveId;
  characterId: CharacterId;
  schemaVersion: number;
  revision: number;
  worldState: WorldState;
  createdAt: string;
  updatedAt: string;
```

```
  checksum: string;
}

interface RewardLedgerEntry {
  id: TransactionId;
  accountId: AccountId;
  characterId?: CharacterId;
  sourceEventId: EventId;
  rewardType: "camp_coins" | "traveler_miles";
  amount: number;
  status: "pending" | "confirmed" | "reversed";
  idempotencyKey: string;
  createdAt: string;
}

interface JournalPost {
  id: ContentId;
  authorAccountId: AccountId;
  title: string;
  body: string;
  visibility: "private" | "followers" | "public";
  status: "draft" | "published" | "hidden" | "removed";
  sourceTripId?: string;
  createdAt: string;
  updatedAt: string;
}
```

Це каркас, а не готова схема бази даних. Перед реалізацією потрібно визначити всі поля, індекси, обмеження, зв'язки, права доступу, терміни зберігання та міграції.

## Обов'язкові правила

- Кожна валюта має незмінну історію транзакцій.
- Нагорода за одну подію не може бути нарахована двічі через повторний запит.
- Виправлення помилки оформлюється зворотною транзакцією, а не видаленням історії.
- Публічний профіль не відкриває приватні збереження.
- Доступ до збереження перевіряється сервером при кожній операції.
- Публікація допису не означає автоматичної згоди на використання тексту в рекламі.

- Видалення акаунта, експорт даних і відкликання згоди мають бути спроєктовані окремо від ігрового прогресу.
- Сайт не повинен отримувати весь вміст збереження, якщо для певної функції достатньо обмеженого набору даних.

## 8.2. Технічний напрям

Для браузерної гри залишається розумним початковим вибором TypeScript і Phaser для 2D-сцени, із вебінтерфейсом для складних меню. Сервер і база даних повинні бути окремими від ігрового клієнта.

Я не рекомендую остаточно вибирати конкретний сервер, базу даних або постачальника автентифікації, доки не визначені вимоги до бюджету, кількості користувачів, резервного копіювання, регіону зберігання даних і плану запуску. Це рішення треба записати в `docs/DECISIONS.md` після порівняння реальних варіантів.

# 9. Правила безкоштовної моделі

Твоє рішення не брати гроші за гру, прискорення чи переваги — це сильна частина концепції. Я пропоную зробити його не просто обіцянкою, а формальним правилом продукту.

# FREE-TO-PLAY POLICY — CAMPING LIFE SIM

## Основні зобов'язання

1. Базова гра доступна без оплати.
2. Немає платної енергії, витривалості, прискорення часу, платного пропуску очікування або платного відкриття необхідних квестів.
3. Немає платних випадкових коробок, лутбоксів або купівлі випадкових ігрових переваг.
4. Немає платного збільшення шансів успіху, характеристик персонажа чи транспортних можливостей.
5. Усі сюжетно важливі предмети, навички та можливості можна здобути звичайною грою.
6. Внутрішньоігрові монети заробляються в грі й не продаються за реальні гроші.

7. Участь у сайті, журналі, рейтингах і соціальних функціях добровільна.
8. Нагороди за участь у спільноті не повинні давати переваги, які неможливо отримати в самій грі.
9. Не використовувати штучний дефіцит, втрату прогресу через пропущений день або навмисне виснаження ресурсів для стимулювання оплати.
10. Якщо в майбутньому з'являться пожертви, спонсорство чи інші джерела доходу, вони не повинні впливати на ігровий баланс або давати платникам переваги.

## Позитивні альтернативи оплаті

Гравець може отримувати прогрес через:

- сюжетні квести;
- професійні завдання;
- дослідження;
- навчальні випробування;
- допомогу NPC;
- сезонні пригоди;
- особисті цілі;
- колекції;
- якісні внески до спільноти;
- перевірку й доповнення туристичних матеріалів.

## Правило справедливості

Жоден гравець не повинен відчувати, що його прогрес навмисно сповільнили, щоб змусити заплатити. Складність має походити від правил світу, виборів, планування та навичок, а не від платного бар'єра.

## 9.1. Чим підтримувати проєкт, якщо гра безкоштовна

Не потрібно вирішувати це сьогодні, але бізнес-модель має бути реалістичною. Безкоштовна гра з акаунтами, серверними збереженнями, модерацією й контентом коштує грошей на підтримку.

Можливі варіанти на майбутнє:

- добровільні пожертви без ігрових переваг;
- прозоро позначене спонсорство туристичних тем;
- добровільна підтримка сайту;
- продаж окремих фізичних сувенірів або друкованих матеріалів, якщо це матиме сенс;

- гранти чи партнерства з туристичними, культурними або освітніми організаціями.

Жоден із цих варіантів не варто вважати гарантованим джерелом доходу. Спочатку потрібно перевірити, чи подобається людям гра, чи повертаються вони, чи використовують журнал і чи є спільнота, яку варто підтримувати.

# 10. Як зробити гру захопливою, але не виснажливою

Ти хочеш, щоб людина згадувала про гру вдома, на роботі, з телефона й планшета. Я розумію цю мету як бажання створити світ, до якого хочеться повертатися. Проте я б не будував дизайн на залежності, тривозі або страху втратити нагороду.

Найсильніший варіант — коли гравець сам хоче зайти, бо цікаво, що відбувається з його персонажем, чи з'явилася нова історія, що він може відкрити далі.

## 10.1. Кілька природних ритмів гри

Короткий візит — 3–7 хвилин

Прочитати лист, переглянути журнал, перевірити план маршруту, поговорити з NPC або налаштувати список спорядження.

Звичайна сесія — 15–30 хвилин

Пройти короткий квест, дослідити локацію, виконати ремонт, відвідати магазин або продовжити історію NPC.

Довга сесія — 45–90 хвилин

Підготуватися до маршруту, вирушити в новий регіон, пройти кілька пов'язаних завдань і пережити повноцінну пригоду.

Ці тривалості — дизайн-цілі для тестування, а не вимога, щоб гравець проводив у грі певну кількість часу.

## 10.2. Чого не робити

- Не вводити енергію, яка закінчується й змушує чекати або платити.
- Не карати за пропущений день.
- Не обмежувати сюжет випадковими таймерами без сюжетної причини.
- Не створювати нескінченні повідомлення «тебе чекають».
- Не використовувати штучне зникнення винагород, якщо гравець не ввійшов вчасно.
- Не підштовхувати користувача грати під час роботи, сну або небезпечної ситуації.
- Не робити так, щоб реальна активність у соціальних мережах була потрібна для завершення гри.

Замість цього можна мати необов'язковий список «Коли повернешся»: кілька нових діалогів, незавершені плани, історії, які гравець може продовжити у власному темпі.

## 10.3. Сезонний контент без FOMO

Сезонні події можуть відображати реальні туристичні ритми: весняне відкриття стежок, літні фестивалі, осінні веломаршрути, зимову підготовку транспорту.

Але якщо гравець пропустив подію, її основний зміст не повинен назавжди зникати. Частину косметичних відзнак можна залишати сезонними, а сюжетні історії — повертати через архів, повторні події або наступний сезон.

Так ми зберігаємо приємне відчуття часу й змін, не перетворюючи відсутність на втрату.

# 11. Правова, приватнісна та безпекова основа

Ми не можемо гарантувати абсолютну відповідність усім законам світу одним текстом. Вимоги залежать від країн, аудиторії, способу обробки даних, джерел контенту й моделі роботи сайту. Але можемо спроєктувати процес так, щоб правові ризики виявлялися до запуску.

## 11.1. Контент і поведінка у грі

У межах затвердженої концепції:

- немає порнографії;
- немає графічного насильства;

- немає незаконних квестів або заохочення порушувати закон;
- небезпечні ситуації подаються як проблеми, які потрібно розв'язувати безпечно;
- немає інструкцій щодо незаконного проникнення, викрадення, шахрайства чи обходу законних вимог;
- конфлікти між людьми можуть бути сюжетними, але розв'язуються через діалог, домовленості, відмову, звернення по допомогу або законні процедури.

Реалістична гра не повинна вдавати, що ризиків немає. Вона повинна показувати, як люди ухвалюють безпечніші рішення, коли ризики виникають.

## 11.2. Користувацький контент

Якщо гравці можуть публікувати тексти, фото, коментарі чи інші матеріали, потрібні:

- правила спільноти;
- скарги й блокування;
- модерація;
- захист від переслідування й спаму;
- процедура апеляції;
- правила авторського права;
- механізм повідомлення про незаконний контент;
- визначена політика зберігання й видалення матеріалів.

Для Європейського Союзу також треба окремо перевірити GDPR і застосовні вимоги Digital Services Act. Якщо сервіс може використовуватися дітьми, слід особливо уважно проєктувати приватність, безпеку, зрозумілі правила й вікові обмеження. Європейська комісія наголошує на додаткових гарантіях щодо даних дітей, зокрема на вікових правилах згоди, які можуть відрізнятися між країнами ЄС.

European Commission
+2

## 11.3. Реальні туристичні дані

Для кожного джерела потрібно записувати:

- власника й назву джерела;
- тип ліцензії;
- дозволене використання;
- вимоги до атрибуції;
- обмеження на копіювання й повторне поширення;
- дату отримання даних;
- регіон та актуальність;
- спосіб перевірки;
- план оновлення або видалення.

Не можна припускати, що все, що доступне в інтернеті, дозволено включати в гру. Це стосується фотографій, карт, текстів, музики, моделей, іконок, баз даних і сторонніх API.

## 11.4. Акаунти й збереження

Потрібно проєктувати:

- безпечну автентифікацію;
- захист сесій;
- перевірку доступу до кожного збереження;
- резервне копіювання;
- відновлення після пошкодження;
- версіонування схем;
- журнал критичних операцій;
- обмеження частоти запитів;
- захист від повторного виконання транзакцій;
- процедури видалення й експорту даних;
- моніторинг помилок без непотрібного збору персональних даних.

Публічний профіль і приватний ігровий стан мають бути розділені. Сайт не повинен випадково розкривати місце проживання користувача, його реальне місцезнаходження чи інші чутливі відомості через ігрові записи.

# 12. Масив даних для AI: як створити світ, який не суперечить сам собі

Це одна з найважливіших частин усього проєкту. Ми повинні створити не просто набір документів для AI, а структуровану базу знань, за якою можна генерувати контент, перевіряти його й реалізовувати без довільних припущень.

Я пропоную організувати контент у три шари.

### Шар 1. Незмінні правила

Економіка, час, фізика, правила збережень, законність, безпека, контракти систем.

### Шар 2. Каталоги визначень

Предмети, NPC, локації, квести, події, транспорт, погода, професії, навички, досягнення.

### Шар 3. Конкретне проходження

Поточний стан персонажа, інвентар, гроші, активні квести, NPC-пам'ять, маршрут, історія подій та збереження.

Це розділення потрібне, щоб AI міг створити новий квест, не переписуючи правила економіки, і додати новий кемпінг, не створюючи окрему систему бронювання.

## 12.1. Обов'язкові каталоги

| Каталог | Приблизний стартовий обсяг | Основні поля |
|---|---|---|
| `CHARACTERS` | 12 основних NPC | ID, біографія, цілі, характер, розклад, пам'ять |
| `LOCATIONS` | 20–30 | ID, тип, послуги, доступність, зв'язки, обмеження |
| `ITEMS` | 100–150 | Маса, об'єм, ціна, стан, призначення, сумісність |

| | | |
|---|---|---|
| VEHICLES | 20–30 моделей | Характеристики, обмеження, витрати, обслуговування |
| QUESTS | 30–40 | Передумови, цілі, гілки, нагороди, завершення |
| EVENTS | 60–100 | Тригери, умови, вибори, наслідки, cooldown |
| SKILLS | 8 основних | Рівні, ефекти, межі, перевірки |
| PROFESSIONS | 10–12 | Доходи, графіки, обов'язки, обмеження |
| ACHIEVEMENTS | 40–60 | Умова, нагорода, одноразовість |
| DATA_SOURCES | За потребою | Джерело, ліцензія, регіон, актуальність |

Це стартові цілі для наповнення, а не обсяг, який потрібно реалізувати до першого прототипу. Для першого вертикального зрізу достатньо невеликої підмножини кожного каталогу.

## 12.2. Контракт квесту

```
type QuestStatus =
  | "locked"
  | "available"
  | "active"
  | "completed"
```

```
  | "failed"
  | "abandoned";

interface QuestDefinition {
  id: string;
  version: number;
  title: string;
  summary: string;
  category: "story" | "skill" | "social" | "exploration";
  prerequisites: QuestCondition[];
  objectives: QuestObjective[];
  branches: QuestBranch[];
  rewards: RewardDefinition[];
  completionPolicy: {
    canAbandon: boolean;
    repeatable: boolean;
    failurePolicy: "recoverable" | "optional_branch" | "explicit_failure";
  };
  unlocks: string[];
}

interface QuestObjective {
  id: string;
  description: string;
  required: boolean;
  completionCondition: QuestCondition;
}

interface QuestBranch {
  id: string;
  choiceText: string;
  conditions: QuestCondition[];
  consequences: ConsequenceDefinition[];
  nextQuestIds: string[];
}

interface QuestState {
  questId: string;
  status: QuestStatus;
  completedObjectiveIds: string[];
  selectedBranchIds: string[];
  startedAtWorldTime?: number;
  completedAtWorldTime?: number;
  revision: number;
```

}

Цей контракт потрібно доповнити формальними схемами для умов, наслідків, нагород і часу. Не можна дозволяти квестам виконувати довільний код, переданий у даних. Усі ефекти повинні проходити через визначений механізм команд і перевірок.

## 12.3. Контракт події

Кожна подія повинна містити не тільки текст, а й пояснювану логіку.

```
{
  "id": "event.bike.puncture.easy",
  "version": 1,
  "category": "transport",
  "trigger": {
    "type": "route_segment_completed",
    "conditions": [
      {
        "field": "transport.type",
        "operator": "equals",
        "value": "bicycle"
      },
      {
        "field": "tutorial.flags.basic_bike_safety",
        "operator": "equals",
        "value": true
      }
    ]
  },
  "riskLevel": "low",
  "frequency": {
    "repeatable": true,
    "cooldownWorldDays": 14,
    "maxPerTrip": 1
  },
  "choices": [
    {
      "id": "repair",
      "label": "Відремонтувати колесо",
      "requirements": [
```

```
        "item:repair_kit",
        "skill:basic_mechanics"
      ],
      "effects": [
        "advance_world_time",
        "consume_repair_material"
      ]
    },
    {
      "id": "workshop",
      "label": "Знайти майстерню",
      "requirements": [],
      "effects": [
        "open_nearest_available_service"
      ]
    }
  ],
  "validation": {
    "mustHaveRecoveryPath": true,
    "mustHaveUniqueResolution": true
  }
}
```

Це ілюстрація структури. Перед використанням у грі треба формально визначити кожен тип умови й ефекту, одиниці часу, вартість ремонту, критерії виникнення події та її сумісність з активними квестами.

Особливо важливо, щоб подія не виникала без необхідної передумови. Якщо у велосипеда немає встановленого колеса або гравець уже ремонтує його, повторна подія проколу не повинна з'являтися.

## 12.4. Реєстр реальних фактів

Для кожного зовнішнього факту зберігаємо не лише значення, а й походження.

```
interface VerifiedFact {
  id: string;
  subject: string;
  claim: string;
  factType:
    | "legal_requirement"
    | "technical_specification"
```

```
    | "official_guidance"
    | "observation"
    | "gameplay_assumption";
  region?: string;
  sourceUrl?: string;
  sourcePublisher?: string;
  sourcePublishedAt?: string;
  checkedAt: string;
  licenseReference?: string;
  confidence: "unverified" | "reviewed" | "verified";
  expirationPolicy: "none" | "periodic_review" | "expires_on_date";
  gameplayMapping?: string;
  testIds: string[];
}
```

Це дозволить нам у майбутньому оновлювати реальні ціни, характеристики, дорожні обмеження й правила подорожей, не руйнуючи сюжетні дані.

# 13. Промпти для створення контенту без помилок

Окрім головного промпту розробника, потрібні спеціалізовані промпти для сценариста, системного дизайнера, редактора даних і тестувальника.

## PROMPT A — сценарист і дизайнер квестів

Ти відповідаєш за сценарії Camping Life Sim.

Твоя мета — створювати оригінальні, емоційні, реалістичні та інтерактивні історії про туристичне життя.

Перед створенням квесту:

1. Прочитай затверджену біблію світу, каталог NPC, правила часу, економіки та інвентарю.
2. Перевір, чи не існує вже подібного квесту.
3. Визнач місце квесту в основній історії.
4. Визнач, які системи він використовує.
5. Визнач, чи є квест обов'язковим, необов'язковим або повторюваним.

Для кожного квесту створи:

- унікальний ID;
- мету й тему;
- передумови;
- повну послідовність сцен;
- репліки персонажів;
- усі варіанти вибору;
- умови доступності кожного вибору;
- зміни стану світу;
- витрати часу й ресурсів;
- наслідки для стосунків;
- наслідки для наступних квестів;
- правила пропуску й відновлення;
- критерії завершення;
- тести.

Не створюй штучних дилем, де один вибір очевидно правильний, а інші безпідставно караються.

Не копіюй конкретні сюжетні лінії, діалоги, персонажів, назви або унікальні творчі елементи інших ігор. Використовуй лише загальні принципи дизайну, а історії створюй самостійно.

Якщо не вистачає інформації, познач припущення. Не вигадуй факти про реальні закони, транспорт або туристичні місця.

Після створення квесту виконай аудит:

- усі предмети існують;
- усі NPC мають визначення;
- усі локації доступні;
- усі гілки мають завершення;
- немає непередбачених тупиків;
- нагороди не порушують економіку;
- збереження підтримує кожну важливу точку;
- необов'язкова гілка не блокує основну історію.

## PROMPT B — системний дизайнер

Ти відповідаєш за формальні правила Camping Life Sim.

Перетворюй дизайн-ідеї на однозначні правила, які можна реалізувати та перевірити.

Для кожної системи визнач:

- вхідні дані;
- вихідні дані;
- одиниці вимірювання;
- формули;
- допустимі межі;
- правила округлення;
- крайні випадки;
- залежності від інших систем;
- помилки й відновлення;
- автоматизовані тести.

Заборонено використовувати нечіткі формулювання на кшталт «іноді», «достатньо швидко» або «помітно втомлюється» як єдине визначення механіки. Якщо потрібна варіативність, задай конкретні параметри та межі.

Реалістичні дані відокремлюй від ігрових припущень.

Перед затвердженням системи перевір її взаємодію з часом, економікою, інвентарем, квестами, NPC та збереженнями.

Не вводь нову валюту, показник або підсистему, якщо існуюча система вже може виконати ту саму функцію.

Усі суттєві зміни фіксуй у реєстрі рішень.

## PROMPT C — редактор бази даних

Ти відповідаєш за цілісність даних Camping Life Sim.

Перевіряй усі визначення предметів, NPC, локацій, квестів, подій, досягнень і нагород.

Для кожного запису перевір:

- унікальність ID;
- відповідність схемі;
- правильність посилань;
- відсутність дублювання;
- сумісність версій;
- узгодженість умов і наслідків;
- відсутність посилань на неіснуючі ресурси;
- наявність тестів;
- ліцензійний статус зовнішніх даних, якщо вони використовуються.

Не виправляй сумнівні дані мовчки. Створи звіт із категоріями:

1. критична помилка;
2. конфлікт правил;
3. відсутні дані;
4. дубль;
5. попередження;
6. неперевірене припущення.

Не змінюй затверджені контракти без явного рішення. Для будь-якої зміни схеми підготуй план міграції.

## PROMPT D — тестувальник сюжетної логіки

Ти відповідаєш за перевірку наскрізної логіки Camping Life Sim.

Отримавши квест, сюжетну арку або зміну системи, перевір:

- чи можна почати й завершити квест;
- чи існують усі переходи;
- чи всі варіанти вибору доступні за визначених умов;
- чи не виникає взаємного блокування квестів;
- чи не можна отримати нагороду повторно;
- чи коректно обробляється пропуск необов'язкової сцени;
- чи працює збереження в кожній контрольній точці;
- чи враховуються NPC-пам'ять і попередні рішення;
- чи не з'являються неможливі предмети;
- чи не порушується баланс ресурсів;
- чи може гравець відновитися після помилки або переривання.

Створи позитивні, негативні та крайні тестові сценарії.

Для кожного виявленого дефекту вкажи:

- кроки відтворення;
- очікуваний результат;
- фактичний результат;
- серйозність;
- імовірну причину;
- тест, який не дозволить помилці повернутися.

Не заявляй, що тест пройшов, якщо його не було виконано.

# 14. Що створювати першим: реальний план виробництва

Я не раджу починати з величезного каталогу контенту, навіть якщо він добре написаний. Найкраще — спочатку створити маленьку, повністю працюючу частину всесвіту, яка перевірить основні припущення.

## 1

Узгодити документи

Звести в один каталог усі рішення, контракти, сценарні правила й невирішені питання. Перевірити, чи не суперечать один одному попередні документи.

Критерій готовності: є один актуальний набір правил, а кожне відкрите питання має власника й статус.

## 2

Зібрати прототип першої подорожі

Одна домашня локація, магазин, короткий маршрут, один кемпінг, два основні NPC, один квест із гілками та завершення.

Критерій готовності: повне проходження можливе без ручного втручання розробника.

## 3

Додати серверний прогрес

Акаунт, збереження, історія транзакцій, версії стану, резервні копії й відновлення.

Критерій готовності: гравець може закрити браузер, увійти знову й продовжити зі збереженого стану.

## 4

Перевірити баланс і доступність

Пройти сценарій з різними профілями, професіями, транспортом і домашніми тваринами. Перевірити коротку й довгу сесію.

Критерій готовності: жоден підтримуваний профіль не має прихованого безвихідного старту.

### 5

Розширити контент

Додати нові сюжетні арки, NPC, локації, транспортні типи, події та системні квести на перевіреній основі.

Критерій готовності: новий контент не потребує окремих неузгоджених правил для кожної сцени.

### 6

Запустити сайт і спільноту

Спочатку журнал, профіль, досягнення та базова модерація. Потім — рейтинги, тематичні події, перевірені туристичні матеріали й інтеграція зовнішніх даних.

Критерій готовності: соціальні функції безпечні, добровільні та не потрібні для проходження основної гри.

## 15. Фінальна стратегічна рекомендація

Я бачу в цьому проєкті три продукти, які повинні розвиватися разом, але не залежати критично один від одного:

Перший — гра. Вона має бути цікавою навіть без рейтингу, журналу й зовнішніх соціальних мереж.

Другий — туристичний атлас. Він дає грі глибину, підтримує навчальну цінність і може з часом стати окремим корисним ресурсом.

Третій — спільнота. Вона створює довгострокову цінність через історії, обмін досвідом, допомогу новачкам і спільні події.

Їх об'єднує одна ідея: кожна подорож залишає після себе щось цінне — навичку, спогад, знайомство, історію або знання, яке допоможе комусь іншому.

Саме це може відрізнити Camping Life Sim від звичайного симулятора, де гравець лише накопичує гроші й купує кращі предмети.

І я хочу зафіксувати ще одну принципову річ: гра не повинна вважати успіхом лише багатство, високий рівень чи перше місце в рейтингу. Людина, яка їздить на старому велосипеді, подорожує з котом, пише чудові історії або просто знаходить радість у коротких поїздках, повинна мати такий самий повноцінний шлях, як власник великого кемпера.

## Що вважаємо наступним пріоритетом

Наступний великий документ має об'єднати всі ці концепції в GDD v1.0, технічну специфікацію v1.0 та біблію контенту v1.0 із взаємними посиланнями, статусами рішень і формальними контрактами.

До початку реалізації потрібно завершити:

1. повну модель персонажа й домогосподарства;
2. формули економіки, часу, транспорту та інвентарю;
3. граф основної сюжетної кампанії;
4. специфікації NPC і їхньої пам'яті;
5. схеми квестів, подій, предметів і нагород;
6. модель акаунтів і збережень;
7. правила сайту, журналу, рейтингу та модерації;
8. критерії тестування й готовності кожного етапу.

Це велика робота, але її потрібно виконувати послідовно. Якість тут залежить не від того, скільки сторінок ми напишемо, а від того, чи кожне важливе правило має чітке значення, чи кожен сценарій узгоджений із системами та чи можемо ми перевірити його автоматично.

Моя головна пропозиція: будувати не гру, що змушує повертатися, а світ, до якого хочеться повертатися. У цьому і має бути наша відмінність.

## Джерела для подальшого дослідження

- PC Gamer — огляд ігор у стилі Stardew Valley, включно з особливостями персонажів і прогресії: Games like Stardew Valley.
- Nintendo — офіційний опис циклів активності, колекцій і персоналізації в Animal Crossing: Explore your island.
- Spry Fox — принципи поступового відкриття історій у Cozy Grove: When do I get new Story Quests?.

- FTC — правила щодо стимулювання споживчих відгуків: Consumer Reviews and Testimonials Rule.
- Trustpilot — вимоги щодо заборони стимульованих відгуків: Best practices for review invitations.
- Європейська комісія — захист персональних даних дітей: Safeguards for children's personal data.
- EUR-Lex — текст Digital Services Act: Regulation (EU) 2022/2065.

Ці джерела — відправна точка для дослідження, а не заміна повної перевірки ліцензій, юридичних вимог і конкретних ігрових реалізацій перед запуском.

а ще дуже потрібно ствоити еко френдлі, тобто щоб люди в грі вчилися сортувати сміття, прибирати за собою, та допомагали навколишньому середовищу, це точно як карма) (прибрав за собою +1, не прибрав -1, прибрав за кимось +2, ну карма не карма але це дуже буде еко френдлі, і як на мене треба навіть через гру і таких дрібниць вчити).

Нам треба мати цікавий ігровий простір, міссії, завдання, квести, персонажів, пасхалки, сюрпризи, мотиватори, призи, NPS, все має бути гармонійним.

Відверто кажучи я навіть задумувався за всесвіт де будуть люди разом грати, клани зберати тощо як воркрафт але це думаю заскалдно і для серверу і веб не реально і щоб ШІ створив таке.

Я думав навпаки не вигаданий світ а реальний - в нас же база с маршрутами, дорогами, кемперами і кемпінгом, суть гри проводити реальними дорогами притаманними ЄС, тому тут я б хотів не вигаданий всесвіт а вигадані ситуації і персонажів, тому проаналізуй краще в такому характері.

Сайт не повинен бути обов'язковим для проходження гри. Гравець, який хоче лише грати, має отримати повноцінний досвід. А той, хто любить писати, фотографувати чи допомагати іншим, матиме додаткові можливості.

- справа в тому що я хотів саме веб версію щоб вона заманювала на наш сайт.

## 7.3. Як боротися зі спамом і накруткою

- такош хакерами, ддос, всі уязвимості треба передбачити попередньо

Можливі варіанти на майбутнє:

- добровільні пожертви без ігрових переваг;
- прозоро позначене спонсорство туристичних тем;

- добровільна підтримка сайту;
- продаж окремих фізичних сувенірів або друкованих матеріалів, якщо це матиме сенс;
- гранти чи партнерства з туристичними, культурними або освітніми організаціями.
- додумай ще і ігровий варіант для заработку наприклад мерч брендований продавати це топ, ми навіть можемо топ гравцям і авторам постів дарувати брендові футболки чи ще щось таке, тут варто просто докрутити стор, в сторі на Шопіфай ми можемо поставити прінт апп який буде генерувати товари з нашими лого тощо і сам апп провайдер розробляти, робити, відправляти товар тобто справжній дропшіппінг. Також можемо і просити підтримати нас матеріально, подарувати гроші, авто, кемпер)

Приватність, данні секретні, аккаунти і права і правила ГДПР, AI Act, закони, і все таке має бути максиммально сильним щоб захистити нас від проблем всіх які можуть бути через це.

Думай ще глибше і продовжуй

Тут вимальовується значно сильніша концепція: не просто гра про кемпінг, а інтерактивний світ реальних подорожей Європою, який непомітно вчить подорожувати розумніше, дбати про природу й людей та відкривати справжні місця. При цьому це має бути повноцінна браузерна гра, а сайт — її природне продовження, а не обов’язкова умова проходження.

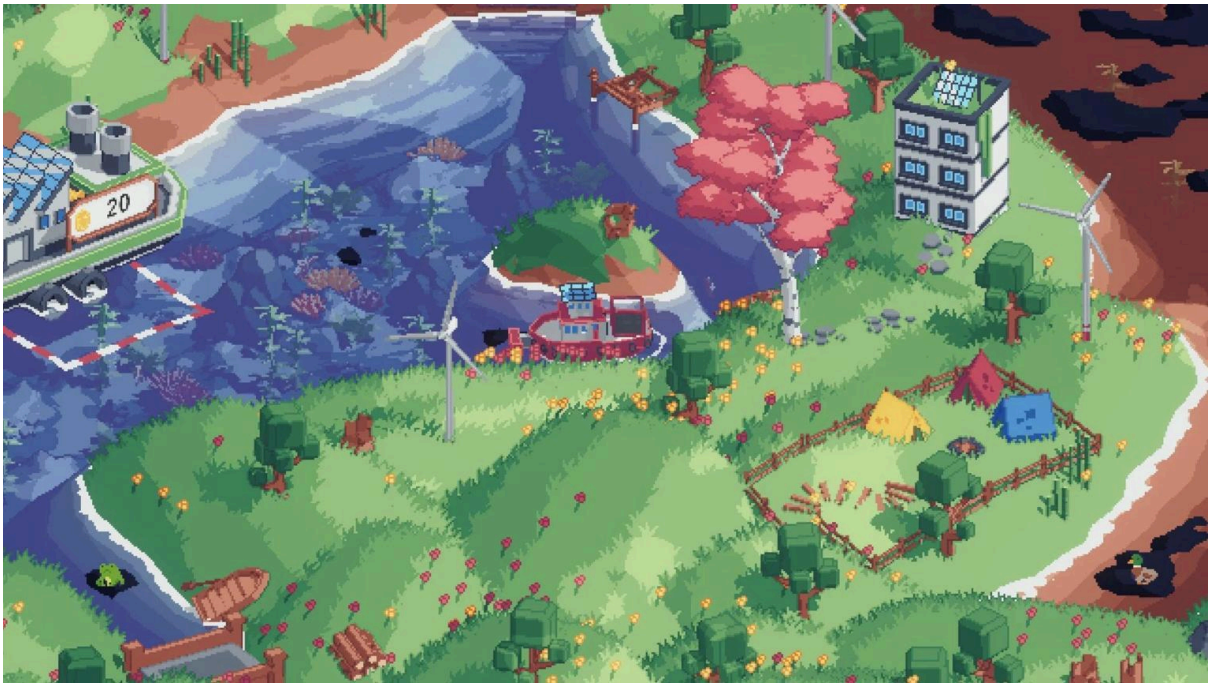
Я б розвивав її як поєднання трьох речей:

- Adventure life sim — персонажі, подорожі, пригоди, стосунки, несподівані події й довгі сюжетні лінії.
- Real-world travel simulator — справжні дороги, географія, транспорт, кемпінги, погода й реалістичні рішення.
- Eco-friendly community — екологічна поведінка, відкриття, колекціонування, репутація, добровільні історії та спільні проєкти.

І важливий принцип: гравець не повинен відчувати, що прийшов на урок екології. Він має хотіти поїхати в нове місце, познайомитися з цікавою людиною, виконати завдання або знайти секрет. А вже в процесі — навчитися сортувати відходи, економити воду, не шкодити природі й відповідально подорожувати.

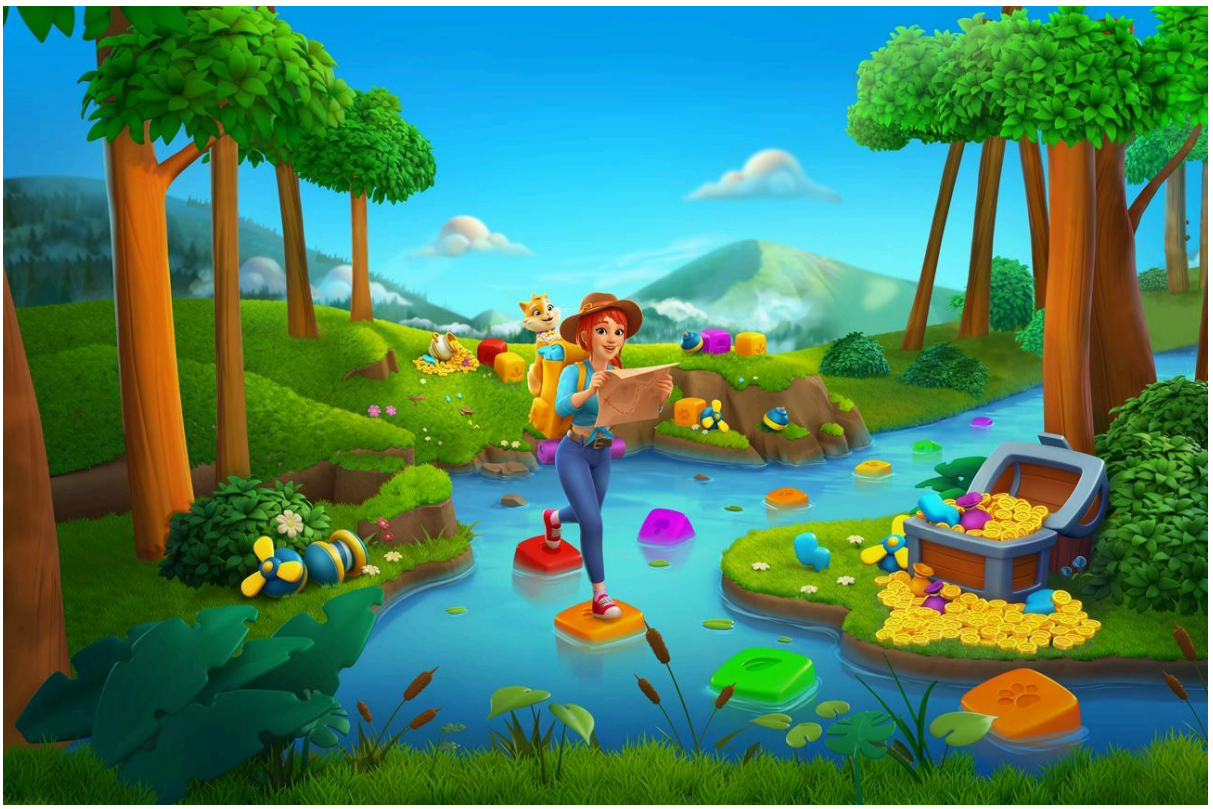
# 1. Екологічність має стати частиною ігрового світу

Я пропоную створити окрему систему під назвою «Слід мандрівника» (Traveler's Footprint). Це не просто лічильник карми, а система, яка пам’ятає, як персонаж взаємодіє з довкіллям.




1. Чистота місця

Прибрав своє сміття, залишив місце чистим, правильно відсортував відходи, повідомив про переповнений контейнер. Порушив правила — втратив репутацію або отримав завдання виправити ситуацію.



2. Турбота про природу

Прибрати сміття, залишене іншими, не сходити з дозволеної стежки в чутливій зоні, берегти рослини, не турбувати диких тварин, правильно поводитися біля води.

3. Розумне споживання

Брати багаторазові речі, купувати потрібне, ремонтувати спорядження, планувати запаси води й їжі, не викидати продукти без потреби.

4. Допомога спільноті

Допомогти іншому туристу, взяти участь у прибиранні стежки, відремонтувати спільне обладнання, поділитися перевіреною інформацією про місце.

## 1.1. Як працюватиме система балів

Твоя ідея з +1, −1 і +2 хороша як початкова інтуїція. Я б лише розділив екологічну репутацію, знання й реальні ігрові нагороди, щоб гравці не почали фармити сміття заради балів.

| Дія | Початкова зміна репутації |
|---|---|
| Прибрав власне сміття | +1 |
| Правильно відсортував відходи | +1 |
| Залишив місце чистим після від’їзду | +1 |
| Прибрав безпечно залишене іншими сміття | +2 |
| Допоміг організувати прибирання | +3 |
| Залишив сміття або порушив чітко показані правила | −1 |
| Повторно порушив правила й проігнорував попередження | Додаткове зниження |

Це попередні значення для тестування, а не остаточний баланс.

Є кілька важливих нюансів:

- Не кожен предмет можна сортувати однаково в усіх країнах. Система повинна враховувати місцеві правила, а не нав’язувати одну універсальну схему для Європи.
- Не можна заохочувати гравця торкатися небезпечних відходів, битого скла, медичних відходів або невідомих речовин. Правильна дія — не брати їх руками, відійти та повідомити відповідальну службу.
- Не слід знижувати репутацію за дії, яких гравець не міг контролювати. Наприклад, за сміття, яке залишив інший персонаж, або за відсутність потрібного контейнера.

- Не варто винагороджувати гравця за безкінечне прибирання одного й того самого предмета. Кожна екологічна дія має унікальний ідентифікатор і захист від повторного нарахування.

**Чому це цікавіше за звичайну карму**

Репутація має змінювати світ, а не лише цифру в профілі.

- Працівники кемпінгу охочіше довіряють відповідальному персонажу.
- Деякі NPC запрошують його до спеціальних екологічних місій.
- Відкриваються історії про відновлення стежки, очищення берега або облаштування зони відпочинку.
- Гравець отримує значки, записи в атласі, декоративні предмети та нові історії.
- Певні персонажі можуть не погодитися з його методами, і тоді виникають цікаві розмови, а не просто механічна винагорода.

При цьому висока репутація не повинна давати перевагу в керуванні автомобілем, швидкості пересування чи шансах виграти змагання. Екологічність — це цінність і стиль гри, а не прихований pay-to-win.

# 2. Екологічні місії, які розповідають історії

Екологічна система стане по-справжньому цікавою, якщо кожне завдання матиме маленьку історію, різні рішення та наслідки.

ECO-001 — «Чотири контейнери й одна помилка»

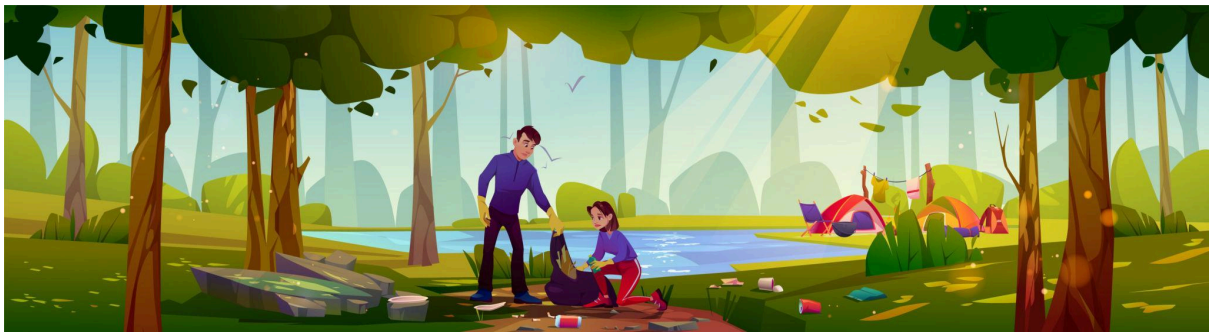
Навчальна місія · Кемпінг Елізи

Новий турист неправильно розклав відходи, а працівниця кемпінгу Еліза не встигає все перевірити перед вивезенням сміття.

Гравець може:

- розсортувати відходи за місцевими правилами;
- попросити Елізу пояснити незрозумілі позначення;
- залишити все як є.

Наслідок: персонаж вивчає місцеві правила, отримує запис у довіднику й відкриває наступну місію. Неправильний вибір не блокує проходження — він запускає пояснення.



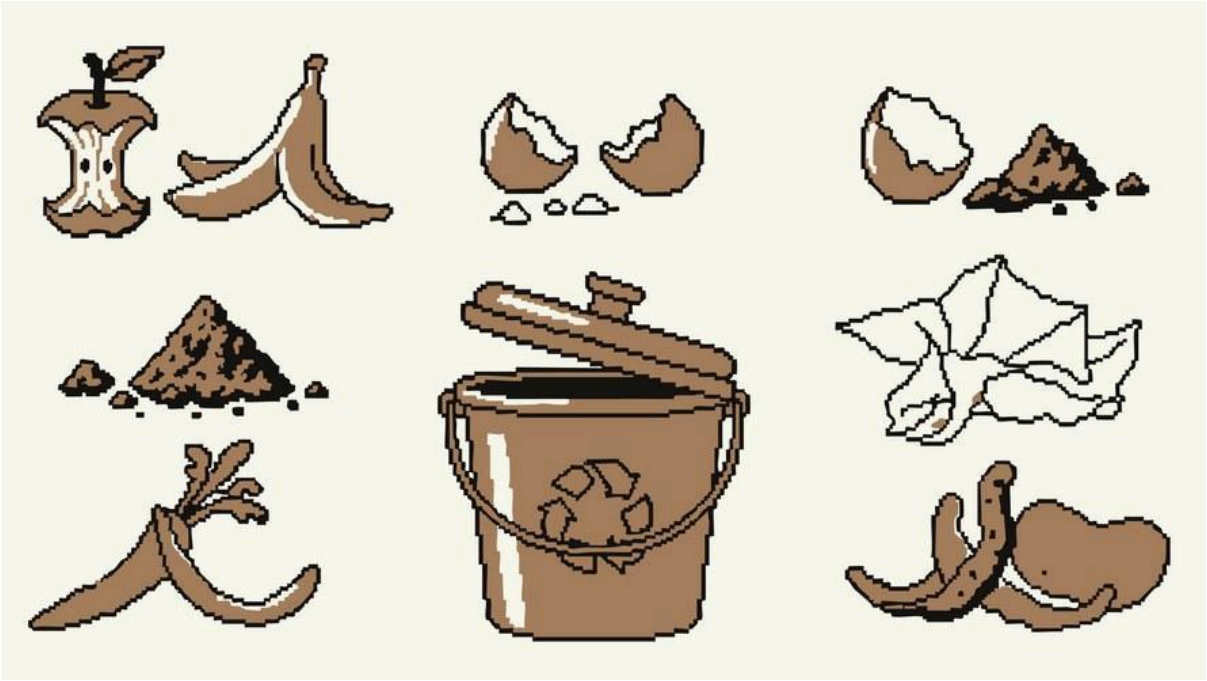
ECO-002 — «Хтось був тут до нас»

Дослідницька місія · Озеро

На березі залишилися упаковки, пляшка та зламана пластикова річ. Нора хоче сфотографувати місце для свого альбому маршрутів.

Гравець вирішує, що безпечно прибрати самому, що віднести до відповідного пункту збору, а про що повідомити адміністрації.

Наслідок: чиста ділянка, фотоісторія до атласу та новий контакт із місцевою волонтерською групою.



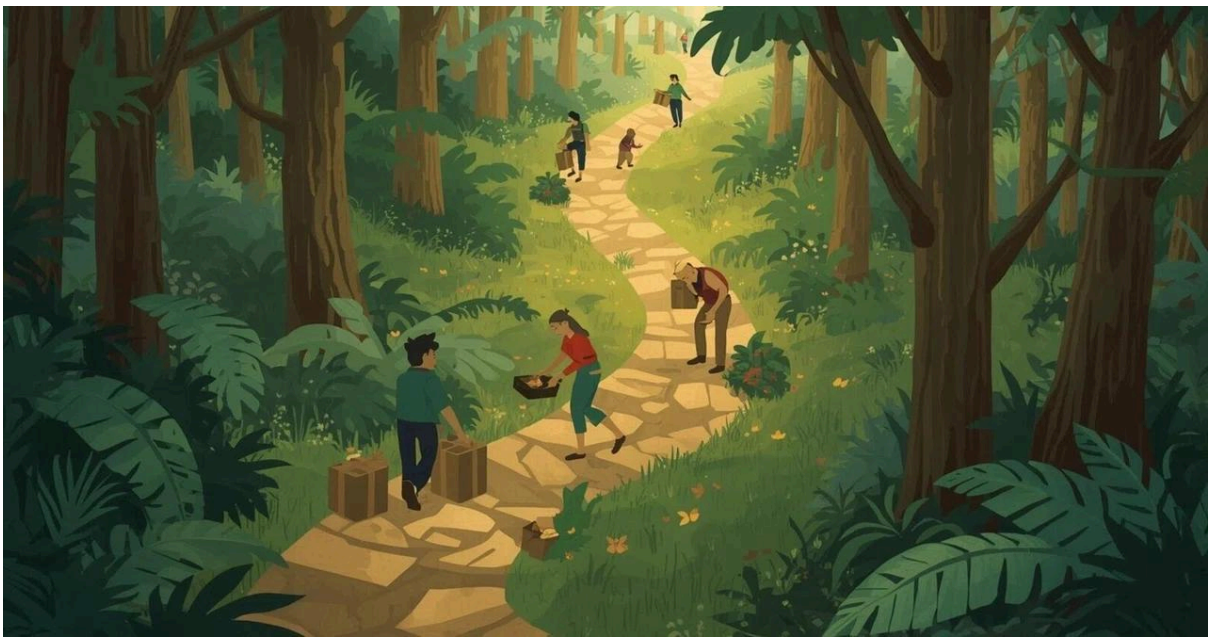
ECO-003 — «Вечеря без зайвих відходів»

Побутова місія · Кемпер або намет

Гравець купив забагато продуктів. Частина скоро зіпсується, а в кемпері мало місця для зберігання.

Варіанти: скоригувати меню, належно зберегти продукти, поділитися придатною їжею за згодою іншого персонажа або визнати помилку планування.

Наслідок: менше витрат, менше відходів, досвід планування запасів. Гра не пропонує віддавати іншим їжу, яка могла зіпсуватися.



ECO-004 — «Стежка, яка зникає»

Сюжетна арка · Природна територія

Туристи скорочують шлях через вразливу ділянку, бо вказівник погано видно. Сара, місцева гідеса, пропонує знайти рішення.

Гравець може дослідити офіційний маршрут, допомогти повідомити про проблему, підтримати облаштування зрозумілих вказівників або запропонувати інформаційну табличку.

Наслідок: змінюється доступна інформація на маршруті, відкривається безпечніший варіант проходження, а місцеві NPC починають користуватися новим маршрутом.

ECO-005 — «Не все зламане треба викидати»

Ремонт і повторне використання · Майстерня Бенуа

У кемпінгу накопичилися зламані стільці, ліхтарі та дрібне спорядження. Частину можна відремонтувати, частину — безпечно здати на переробку.

Гравець допомагає провести інвентаризацію, навчається оцінювати ремонт і знаходить річ із несподіваною історією.

Наслідок: відкривається ремонтний стіл, нові рецепти обслуговування й колекційний запис про предмет.

Ці місії не повинні бути одноразовими навчальними картками. Вони мають пов’язувати персонажів і місця між собою. Наприклад, проблема зі стежкою знайомить гравця із Сарою; Сара рекомендує кемпінг Елізи; Еліза просить допомогти облаштувати зону сортування; а Нора створює фотоісторію про зміни.

Так утворюється ланцюжок сюжетів, а не набір випадкових завдань.

## 2.1. Довга екологічна кампанія

Пропоную сюжетну лінію «Місця, які ми залишаємо після себе».

1. Знайомство. Гравець вивчає правила чистоти та сортування на першому кемпінгу.
2. Спостереження. У різних місцях він бачить проблеми: переповнені контейнери, надмірне споживання води, зношене обладнання, погану навігацію.
3. Знайомства. NPC пояснюють, чому проблеми виникають і що реально можуть зробити туристи, працівники та місцева громада.
4. Спільний проєкт. Гравець допомагає організувати невелике поліпшення кемпінгу чи маршруту.
5. Результат. При наступному відвідуванні видно зміни, а персонажі пам’ятають участь гравця.
6. Передача досвіду. Гравець може записати корисну пораду в особистий журнал або добровільно поділитися нею на сайті.

Головний приз за цю кампанію — не купа грошей, а унікальна історія, особливий знак у профілі, новий вигляд табору, знайомства й відчуття, що дії персонажа щось змінили.

# 3. Реальна Європа — основа, а не декорація

Тут я погоджуюся з твоїм уточненням: вигаданий материк нам не потрібен. Потрібні реальні країни, реальна географія й дороги, але вигадані персонажі та історії, що відбуваються в правдоподібних місцях.

Важливо не плутати дві речі:

- Реальне місце, дорога, рельєф або кемпінг можуть бути джерелом даних.
- Діалоги, NPC, дружба, конфлікти, таємниці та сюжетні події — авторський вигаданий контент.

Якщо вигаданий персонаж живе біля реального кемпінгу, гра не повинна створювати враження, що справжній власник кемпінгу сказав або зробив щось, чого не було.

## 3.1. Три масштаби світу

Масштаб A — Європа

Гравець бачить країни, регіони, міста, відстані, великі дороги, пороми й залізничні вузли. Це стратегічна карта вибору напрямку.

Масштаб В — Маршрут

Планування відрізків дороги, пального, часу, погоди, висот, платних доріг, заряджання, зупинок і ночівель. Велосипед, легковик і великий кемпер матимуть різні обмеження.

Масштаб C — Локація

Персонаж рухається вільно в 2D-сцені: ходить до рецепції, розставляє спорядження, сортує сміття, готує їжу, розмовляє, ремонтує велосипед і досліджує територію.

Не варто одразу створювати вручну всю Європу. Розумніше почати з одного реального географічного коридору, наприклад Бельгія — Нідерланди, а потім розширити його до сусідніх регіонів. Це дасть змогу перевірити дорожню модель, кемпінги, сортування за країнами, місцеві правила та сюжетні лінії, не витрачаючи роки на порожню карту.

## 3.2. Як працювати з реальними даними

| Дані | Можливе джерело | Для чого |
|---|---|---|
| Дороги, стежки, населені пункти | OpenStreetMap | Географія, мережа маршрутів, навігація |

| | | |
|---|---|---|
| Погода | Open-Meteo | Дощ, температура, вітер, планування поїздок |
| Кемпінги | Відкриті джерела, офіційні каталоги, партнерські дані | Місця ночівлі, послуги, правила |
| Сортування відходів | Муніципальні та регіональні служби | Локальні правила, типи контейнерів |
| Туристична доступність | Офіційні сайти, оператори транспорту | Пороми, обмеження, сезонні закриття |

Це не означає, що всі дані можна вільно копіювати. Для кожного джерела потрібен реєстр ліцензій, вимог атрибуції, дозволеного використання, дати перевірки та процедури оновлення. Дані OpenStreetMap, зокрема, мають власні умови використання й атрибуції; їх треба дотримуватися.

Також необхідно розрізняти:

- перевірені дані — наприклад, координати об’єкта з конкретного джерела;
- змінні дані — актуальні ціни, доступність місць, години роботи;
- ігрові оцінки — приблизний час, складність, вартість або поведінка трафіку;
- вигаданий сюжет — повністю авторська історія, яка не видається за реальну подію.

У грі слід чесно показувати, коли інформація приблизна або застаріла. Якщо гра згодом матиме практичну користь для реальних мандрівників, ця відмінність стане особливо важливою.

# 4. Як зробити гру соціальною, не будуючи World of Warcraft

Твоя ідея зі спільним світом, кланами та іншими гравцями цікава. Але я б не починав з MMO — великої багатокористувацької гри з постійним серверним світом.

Для першої версії це створить дуже багато складності: синхронізацію руху, мережеві збої, авторитетний сервер, захист від читів, модерацію в реальному часі, масштабування та постійні витрати на інфраструктуру.

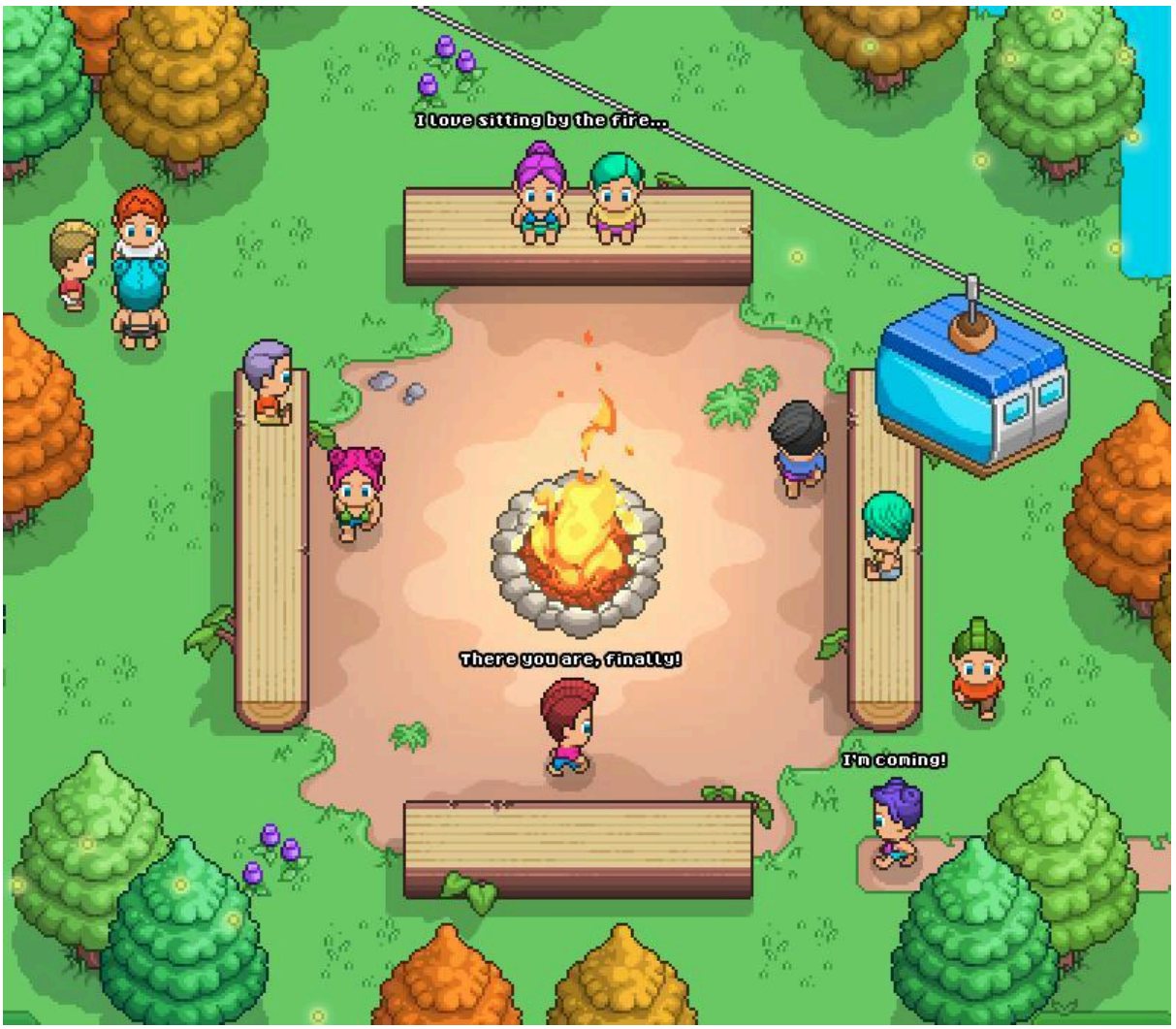
Натомість пропоную соціальну систему з поступовим розвитком. Вона дасть відчуття спільноти без вимоги одночасно симулювати тисячі гравців на одній карті.

Рівень 1. Особистий світ

Один гравець, власний персонаж, поїздки, журнал, досягнення та збереження. Повноцінна гра працює без облікового запису на сайті й без публікації будь-якого контенту.

Рівень 2. Спільний атлас

Гравець за бажанням публікує маршрутну нотатку, історію або пораду. Інші можуть читати, зберігати й коментувати матеріали. Ігровий прогрес не залежить від публікацій.




Рівень 3. Туристичні клуби

Невеликі групи гравців із власною назвою, значком, спільним журналом і добровільними сезонними цілями. Наприклад, клуб велосипедних маршрутів, прихильників ремонту спорядження або ековолонтерів.

Рівень 4. Спільні проєкти

Учасники клубу разом заповнюють тематичний атлас, проходять сюжетну кампанію або роблять внесок у віртуальне облаштування кемпінгу. Результат спільний, але кожен може грати у власному темпі.

Клуби можуть бути схожими на клани за відчуттям належності, але без воєн, нападів і боротьби за ресурси. Їхня ідентичність — спільний стиль подорожей, знання, допомога й колекції.

### Що я б не створював у першій версії

- Спільну карту, де всі персонажі рухаються в реальному часі.
- Голосовий чат.
- Торгівлю між гравцями за реальні гроші.
- Передавання преміальної валюти між акаунтами.
- Необмежені користувацькі сценарії, які запускаються без перевірки.
- Глобальні змагання, в яких найзаможніший або найактивніший гравець завжди перемагає.

Усе це можна оцінити пізніше, якщо гра знайде аудиторію. Спочатку потрібна надійна одиночна гра, хороші сюжети та проста соціальна надбудова.

# 5. Вебгра має приводити людей на сайт природно

Тут потрібно розділити два завдання:

1. Браузерна гра повинна бути цікавою сама по собі.
2. Сайт повинен давати додаткову цінність, заради якої гравець добровільно повертається.

Я б побудував сайт не як рекламну сторінку з кнопкою «Play», а як цифровий туристичний клуб із живим атласом історій.

Play

Гра запускається прямо в браузері. Ніякого обов’язкового переходу в соціальну мережу.

Route Atlas

Реальні маршрути, точки інтересу, сезонні підказки, доступність і туристичні нотатки.

Living Journal

Історії подорожей, фото, висновки, кумедні ситуації та щоденники персонажів.

Community

Клуби, обговорення, запитання, тематичні кампанії й допомога іншим туристам.

Profile & Collection

Досягнення, колекції, особисті цілі, історія подорожей та налаштування приватності.

## 5.1. Найкращий місток між грою та сайтом — журнал

Наприклад, після завершення подорожі гра може запропонувати:

> «Ця поїздка була насиченою. Ти відкрив новий маршрут, навчився ремонтувати прокол і допоміг іншому туристу. Хочеш зберегти історію у своєму журналі?»

Кнопки:

- «Зберегти приватно».
- «Створити чернетку історії».
- «Продовжити подорож».

Якщо гравець вибирає чернетку, сайт отримує лише ті ігрові факти, які потрібні для неї. Людина може відредагувати текст, додати фото та сама вирішити, чи публікувати його.

Не можна автоматично публікувати історії або вигадувати від імені гравця реальні почуття, місця відвідування чи особисті обставини. Так само не потрібно передавати на сайт точну позицію гравця або його домашню адресу.

Сайт має бути привабливим не тому, що гра блокує контент, а тому, що там є цікаві історії інших людей, корисні маршрути й колекції.

# 6. Захист від спаму, накрутки, хакерів і DDoS

Тут варто починати не з конкретного провайдера, а з моделі загроз. Жодна архітектура не може гарантувати абсолютну безпеку, але можна суттєво знизити ризики та спланувати відновлення після інциденту.

## 6.1. Що захищаємо

| Актив | Основний ризик | Захисний принцип |
|---|---|---|
| Акаунти | Викрадення паролів, захоплення сесій | Надійна автентифікація, безпечні сесії, обмеження спроб |
| Ігрові збереження | Підміна грошей, предметів, квестів | Перевірка стану на сервері для важливих дій |
| Валюта й нагороди | Фальшиві запити, повторне нарахування | Серверний журнал транзакцій, ідемпотентність |

| | | |
|---|---|---|
| Лідерборди | Боти, мультиакаунти, підроблені результати | Серверний розрахунок, аномалії, обмеження частоти |
| Сайт і API | SQL injection, XSS, CSRF, SSRF, помилки доступу | Безпечна реалізація, контроль доступу, валідація, тестування |
| Інфраструктура | DDoS, перевантаження, зловживання запитами | Захист на рівні мережі, CDN/WAF, квоти, моніторинг |
| Користувацький контент | Спам, шахрайство, шкідливі посилання | Модерація, скарги, обмеження публікацій |
| Персональні дані | Витік, надмірне збирання, несанкціонований доступ | Мінімізація, шифрування, розділення доступу |

## 6.2. Архітектура безпеки

Браузерна гра та сайт

Недовірене середовище: гравець може змінювати запити та локальні дані.

Захисний периметр

TLS, DDoS-захист, WAF, ліміти запитів, базові фільтри ботів.

Сервер застосунку

Автентифікація, авторизація, перевірка правил, валідація команд, контроль нагород.

База даних і журнал подій

Розмежування прав, резервні копії, транзакції, аудит змін, відновлення.

### Обов’язкові технічні правила

1. Не довіряти клієнту. Якщо браузер повідомляє «я заробив 500 монет», це не доказ. Сервер перевіряє, чи існувала відповідна дія та чи має гравець право на нагороду.
2. Не зберігати секрети в JavaScript. API-ключі, приватні ключі та паролі бази даних не повинні потрапляти в браузер або відкритий репозиторій.
3. Розділяти права доступу. Один користувач не повинен мати можливості читати або змінювати чуже збереження, чернетки чи персональні дані.
4. Захищати від повторних запитів. Одна і та сама дія не повинна давати нагороду повторно через повторне натискання, мережевий збій або повтор запиту.
5. Зберігати історію важливих змін. Видача валюти, відкликання нагороди, блокування акаунта й зміни ролей мають бути аудитованими.
6. Мати резервні копії та перевіряти відновлення. Копія, яку ніхто не пробував відновити, не є достатньою гарантією.
7. Встановити межі витрат. Обмежити кількість дорогих запитів, розміри завантажень, частоту публікацій і споживання зовнішніх API.

## 6.3. Як захистити екологічну репутацію від накрутки

Не потрібно намагатися довести, що людина справді прибрала сміття в реальному світі. Це надмірно вторгається в приватність і не потрібно для ігрової системи.

Натомість можна перевіряти ігрові дії:

- чи існував предмет сміття в поточній локації;
- чи мав персонаж доступ до нього;
- чи не було його вже прибрано;
- чи відповідає дія правилам цієї місії;
- чи не повторює акаунт одну операцію неприродно часто;
- чи не видається одна нагорода кілька разів.

Публічні екологічні лідерборди мають бути опційними. Не потрібно розкривати реальне місце проживання, точні маршрути чи справжню особу гравця.

### Захист від DDoS і зловживань

Для першого релізу потрібні:

- зовнішній захист від мережевих атак і фільтрація трафіку;
- обмеження частоти запитів на рівні IP, акаунта та конкретної операції;
- моніторинг незвичайного навантаження й автоматичні сповіщення;

- окремі ліміти для авторизації, пошуку, завантаження файлів і публікацій;
- резервний план на випадок недоступності основного сервісу;
- перевірка залежностей, секретів, конфігурації хмари та прав доступу;
- регулярне оновлення бібліотек і перевірка вразливостей;
- план реагування на витік, шахрайство та втрату даних.

CDN/WAF допомагають, але не замінюють безпечний код і правильну конфігурацію. Потрібні також тести на IDOR/BOLA — ситуації, коли користувач може отримати доступ до чужих даних, просто змінивши ідентифікатор у запиті.

Моя рекомендація: до публічного запуску замовити незалежний аудит безпеки або penetration test, а після значних змін повторювати критичні перевірки. Абсолютної гарантії безпеки не існує, і не варто обіцяти її в політиці конфіденційності.

# 7. Як заробляти на проєкті, не руйнуючи безкоштовну гру

Твоя ідея з брендовими футболками та print-on-demand — одна з найпрактичніших для такого проєкту. Вона дозволяє монетизувати любов до світу гри, а не продавати ігрову перевагу.

Я б побудував економіку проєкту з кількох незалежних напрямів.

1. Брендований мерч — головний тестовий напрям

Футболки, худі, емальовані горнятка, нашивки, стікери, тканинні торбинки, блокноти мандрівника, друковані карти й журнали. Можна почати з 3–5 товарів, а не запускати великий магазин.

Виробництво й доставку може забезпечувати print-on-demand партнер, інтегрований із Shopify. Але треба перевіряти якість, строки доставки, повернення, податки, права на дизайн і фактичну маржу.

2. Колекційні фізичні нагороди

Наприклад, щорічний значок «Відкривач маршрутів», нашивка екологічного клубу або друкований атлас. Найкраще підходять нагороди, які мають сенс для людини й поза грою.

3. Цифрові матеріали

Друковані планери подорожей, чеклісти спорядження, журнали походів, картки для навчання дітей екологічним звичкам. Базова гра й базові ігрові функції залишаються безкоштовними; платні матеріали — окрема добровільна покупка.

4. Добровільна підтримка

Разова пожертва, щомісячна підтримка або разова кампанія на конкретну мету: наприклад, оплату доступності сайту, незалежний аудит безпеки чи створення нової регіональної карти.

5. Партнерства та гранти

Туристичні організації, кемпінги, велосипедні асоціації, освітні й природоохоронні організації можуть підтримувати конкретні освітні кампанії. Спонсорський контент потрібно чітко позначати, а партнерство не повинно впливати на чесність відгуків або рейтинги.

## 7.1. Нагороди найкращим гравцям і авторам

Так, ми можемо надсилати реальні футболки, нашивки або інші сувеніри найкращим учасникам. Але потрібно заздалегідь визначити прозорі правила:

- Переможців обирають за опублікованими критеріями, а не за особистою прихильністю адміністратора.

- Категорії різні: корисний автор, найкраща фотоісторія, екологічний проєкт, найкращий маршрутний довідник.
- Нагорода не вимагає позитивного відгуку, реклами або публікації в особистому профілі.
- Для фізичних призів визначаються країни доставки, терміни, правила податків і митних платежів, а також правила для неповнолітніх.
- Публікація імені, фото чи історії переможця можлива лише за окремою згодою.

Не варто обмежувати призи лише найактивнішими гравцями. Інакше люди з вільним часом отримають перевагу над тими, хто робить справді корисні внески рідше.

Можна мати категорію «Маленька, але корисна дія»: наприклад, людина один раз підготувала дуже якісний матеріал про місцеве сортування відходів.

## 7.2. Чи можна просити подарувати автомобіль або кемпер?

Так, можна проводити добровільну кампанію підтримки з конкретною метою, наприклад:

> «Ми шукаємо партнерів, які допоможуть надати кемпер для зйомок навчальних матеріалів, тестування туристичного обладнання або створення контенту про доступні подорожі».

Але автомобіль або кемпер — це не просто подарунок. Потрібно врахувати юридичного власника, страхування, реєстрацію, технічне обслуговування, податки, умови використання, можливі зобов’язання перед донором і подальший продаж або передавання майна.

Якщо приймається значний подарунок, спонсорство чи пожертва, варто оформлювати це належним договором і проконсультуватися з бухгалтером та юристом у відповідній країні.

Також важливо не змішувати три різні речі:

- пожертву без матеріальної винагороди;
- спонсорство в обмін на чітко визначене розміщення бренду;
- купівлю товару або послуги.

Юридично й податково вони можуть мати різні наслідки. Якщо проєкт буде оформлений у Бельгії, правила треба перевіряти саме для бельгійської юридичної форми та конкретної моделі фінансування.

## 7.3. Як виглядатиме магазин у грі

Я не рекомендую перетворювати ігровий магазин на велику рекламну вітрину.

Краще мати два чітко розділені простори:

- Ігровий магазин: спорядження, ремонт, одяг персонажа, предмети для кемпінгу. Основна економіка — ігрова, безкоштовна.
- Офіційний магазин бренду: фізичні футболки, горнятка, нашивки, журнали й інші сувеніри. Він доступний за бажанням і не впливає на силу персонажа.

Фізичний мерч може мати дизайн, натхненний світом гри, але придбання футболки не повинно давати ексклюзивний потужний автомобіль чи перевагу в рейтингу.

Модель доходу має бути: люди підтримують світ, бо люблять його, а не тому, що гра змушує платити.

# 8. Приватність, GDPR, AI Act та правова архітектура

Це потрібно закласти до створення акаунтів і соціальних функцій, а не намагатися виправити після запуску.

Важливе уточнення: відповідність законодавству не забезпечується лише написанням політики конфіденційності. Потрібні правильні процеси, технічні засоби, договори з постачальниками й документування рішень. Нижче — технічний і продуктовий план, а не юридичний висновок.

## 8.1. Принципи приватності

1. Мінімум даних. Для одиночної гри не потрібні справжнє ім’я, домашня адреса, дата народження до дня чи постійна геолокація.
2. Приватність за замовчуванням. Журнал приватний, профіль не обов’язково публічний, а точні маршрути не публікуються автоматично.
3. Окремі згоди. Реєстрація акаунта, публікація фото, маркетингові повідомлення, використання необов’язкових cookies і участь у публічному рейтингу — різні рішення.
4. Контроль користувача. Можливість переглядати, експортувати, виправляти й видаляти дані відповідно до застосовних вимог.

5. Обмежене зберігання. Для кожної категорії даних має бути визначено, навіщо вона потрібна, хто має до неї доступ і коли її можна видалити.
6. Безпека. Шифрування передавання, захист секретів, обмеження доступу, аудит і план реагування на інциденти.

## 8.2. Дані, які не варто збирати без реальної потреби

- Точну домашню адресу.
- Контакти з телефону.
- Постійне місцезнаходження.
- Реальні маршрути користувача у фоновому режимі.
- Документи, що посвідчують особу.
- Дані про здоров’я, сімейні обставини або фінансовий стан, якщо вони не потрібні для окремої функції.
- Прихований поведінковий профіль для реклами.

Якщо людина хоче поділитися історією поїздки, вона повинна мати можливість видалити точні координати, час знімання та інші метадані фотографій. Особливо небезпечно автоматично показувати точне місце старту або завершення маршруту.

## 8.3. GDPR: що має бути в проєкті

Для європейського сервісу необхідно визначити правові підстави обробки для кожної категорії даних, виконувати принципи мінімізації та обмеження мети, а також забезпечувати права користувачів. Згода потрібна не для будь-якої обробки автоматично: правова підстава залежить від конкретної мети.

Практичний пакет документів має включати:

- політику конфіденційності;
- умови користування;
- політику cookies та керування згодою, якщо застосовно;
- правила спільноти й модерації;
- правила конкурсу та вручення призів;
- політику зберігання й видалення даних;
- процедуру запитів на доступ, експорт, виправлення та видалення;
- реєстр постачальників і договорів обробки даних;
- план реагування на витік даних;
- оцінку необхідності DPIA — оцінки впливу на захист даних.

Якщо користувачі можуть створювати публічні дописи, завантажувати фото й взаємодіяти між собою, потрібно окремо оцінити застосовність Digital Services

Act, включно з вимогами до повідомлення про незаконний контент, поясненням рішень модерації та механізмами оскарження, які залежать від ролі й категорії сервісу.

## 8.4. Діти та підлітки

Гра з подорожами, домашніми тваринами й сімейними сценаріями потенційно може зацікавити дітей. Тому треба заздалегідь вирішити, на яку вікову аудиторію ми орієнтуємося.

Я б рекомендував:

- не створювати відкритий чат із незнайомцями у першій версії;
- не дозволяти публікувати персональні контакти в профілі;
- запровадити скарги, блокування й модерацію;
- не використовувати маніпулятивні механіки для повернення дітей;
- не показувати рекламу, побудовану на чутливих персональних даних;
- спроєктувати безпечні налаштування видимості профілю;
- окремо оцінити правила згоди, вікової перевірки та обробки дитячих даних для цільових країн.

Не потрібно збирати копію документа або дату народження до дня лише заради косметичного профілю. Спочатку слід визначити, які вікові обмеження юридично необхідні для конкретної функції.

## 8.5. AI Act: як не створити зайвих ризиків

Для гри я б узагалі не використовував генеративний ШІ як обов’язкову частину ігрового циклу.

Натомість ШІ можна використовувати під час розробки для створення чернеток квестів, перевірки послідовності сюжету, допомоги з тестами та класифікації контенту — з людським переглядом і перевіркою фактів.

У самій грі краще застосувати заздалегідь написані діалоги, структуровані стани NPC та перевірені правила. Це дає передбачувану поведінку, стабільні збереження й контроль над витратами.

Якщо пізніше з’явиться чат із генеративним персонажем або інша AI-функція, перед запуском треба окремо оцінити прозорість взаємодії, обробку даних, ризики для дітей, можливість генерувати небезпечний контент та актуальні вимоги AI Act. Застосовність конкретних положень залежить від функції, ролі компанії та чинного на момент запуску календаря впровадження закону.

Правило проєкту: не збирати дані «про всяк випадок», не обіцяти абсолютної безпеки й не запускати функції з користувацьким контентом без готових правил, інструментів модерації та плану реагування.

# 9. Як усе це поєднати в одну гармонійну гру

Найбільший ризик — створити десять цікавих систем, які не пов’язані між собою. У нашій грі екологія, персонажі, транспорт, економіка, місії, сайт і нагороди повинні підтримувати одну спільну історію.

Ось приклад одного наскрізного ігрового циклу.

1. Планування

Гравець вибирає реальний регіон, транспорт, спорядження, кемпінг і бюджет.

2. Подорож

Дороги, погода, витрати, час і характеристики транспорту впливають на рішення.

3. Життя на місці

Гравець облаштовує табір, сортує відходи, готує їжу, спілкується з NPC і досліджує локацію.

4. Історія та наслідки

Виконані дії змінюють відносини, репутацію, доступні місії, бюджет і стан локації.

5. Нагороди й колекції

Нові записи в атласі, навички, досягнення, декоративні предмети та наступні сюжетні лінії.

6. Добровільне продовження

Можна вирушити далі, розпочати нову кампанію, зберегти журнал, долучитися до клубу або поділитися досвідом на сайті.

## 9.1. Пасхалки й приємні сюрпризи

Я б заклав кілька категорій секретів:

- Географічні: незвична оглядова точка, маловідомий міст, старий маршрут, який можна знайти за підказками.
- Побутові: забутий листівковий альбом, стара емальована чашка, фотографія колишньої подорожі.
- Персонажні: NPC несподівано пам’ятає маленьку послугу, яку гравець зробив кілька ігрових тижнів тому.
- Екологічні: відновлена ділянка стежки, рідкісний запис про місцеву природу, секретна історія старого дерева.
- Сезонні: осінній ярмарок, весняний велосипедний захід, літня зустріч мандрівників.
- Колекційні: штампи в атласі, листівки, значки маршрутів, фотографії та історії спорядження.

Сюрпризи не повинні вимагати входу в конкретну годину або карати за відсутність протягом кількох днів. Наприклад, сезонний фестиваль можна відвідати протягом достатньо довгого ігрового періоду, а пропущений контент згодом повертається в архів або доступний в іншому форматі.

## 9.2. NPS та інші метрики

Якщо під NPS ти маєш на увазі Net Promoter Score, його варто вимірювати як показник задоволеності й готовності рекомендувати гру, а не як ігрову характеристику персонажа.

Важливо не плутати його з екологічною репутацією, ігровими милями або рейтингом спільноти.

Я б вимірював:

| Метрика | Що показує |
| --- | --- |
| Завершення першої подорожі | Чи зрозумілий початок гри |
| Повернення через 7 і 30 днів | Чи хочуть гравці продовжувати добровільно |

| | |
|---|---|
| Завершення квестів | Чи цікаві сюжетні лінії |
| Різноманітність дій | Чи працюють різні стилі гри |
| Оцінка корисності еконавчання | Чи зрозумілі практичні уроки |
| NPS | Чи готові гравці рекомендувати гру |
| Задоволеність модерацією | Чи почуваються учасники спільноти безпечно |

Не варто збирати надмірну поведінкову аналітику заради цих показників. Для першого релізу достатньо агрегованих подій, добровільного опитування й технічних метрик безпеки.

# 10. Конкретний план виробництва

Я б не давав AI-кодеру команду «побудуй усю гру». Потрібні невеликі перевірювані етапи з документами, тестами та критеріями приймання.

Етап 1 — Узгодження документації

GDD, сюжетна біблія, технічна архітектура, економіка, екологічна система, приватність, безпека та правила контенту.

Результат: узгоджений комплект документів без суперечностей.

Етап 2 — Перша браузерна подорож

Персонаж, локальна 2D-сцена, переміщення, інвентар, магазин, маршрут, перший кемпінг, кілька NPC і перша екологічна місія.

Результат: уже можна грати, а не лише читати дизайн-документ.

Етап 3 — Збереження та серверна логіка

Версійовані збереження, відновлення після збою, перевірка нагород, безпечна авторизація та захист від підміни стану.

Етап 4 — Реальна географія

Обмежений регіон Європи, дорожній граф, правила маршруту, реальні точки інтересу, погодні дані та реєстр джерел.

Етап 5 — Сайт і спільнота

Журнал, профіль, досягнення, публічний атлас, скарги, модерація, налаштування приватності й перші клуби.

Етап 6 — Перший комерційний тест

Маленький магазин мерчу, добровільна підтримка, прозорі умови й перевірена економіка замовлень. Без необхідності монетизувати кожну ігрову дію.

## 10.1. Які документи треба створити

Пропоную зберігати їх у репозиторії проєкту:

- `docs/GDD.md` — повний дизайн гри.
- `docs/NARRATIVE_BIBLE.md` — персонажі, світ, сюжетні арки, діалоги.
- `docs/TECHNICAL_SPEC.md` — архітектура, моделі даних, збереження, API.
- `docs/ECO_SYSTEM.md` — екологічна репутація, сортування, місії та правила.
- `docs/REAL_WORLD_DATA.md` — географія, джерела, ліцензії, перевірка актуальності.
- `docs/WEB_COMMUNITY.md` — сайт, журнал, клуби, профілі й модерація.
- `docs/SECURITY_PRIVACY.md` — модель загроз, доступи, GDPR, зберігання даних.
- `docs/ECONOMY_AND_REWARDS.md` — валюти, досягнення, рейтинги, призи й мерч.
- `docs/QA_TEST_PLAN.md` — тестові сценарії та регресійні перевірки.
- `docs/PROJECT_STATUS.md` — поточний стан, перевірені функції, ризики й наступний крок.

У кожному документі потрібно позначати, що вже затверджено, що є припущенням, що треба дослідити, а що ще не реалізовано.

## Моє головне стратегічне рішення

Я б будував цей проєкт навколо концепції:

«Реальні дороги. Вигадані історії. Відповідальні подорожі».

Гра повинна працювати самостійно, навіть якщо людина ніколи не створить акаунт на сайті. Сайт додає журнал, клуби, спільний атлас і можливість підтримати проєкт. Екологічність проходить через повсякденні рішення, а не через повчальні повідомлення. Реальна географія створює правдоподібність, а вигадані персонажі й сюжетні лінії — емоційну прив'язаність.

І ще одне: не потрібно реалізовувати весь задум одразу. Найкраща перша версія — невелика, але завершена подорож із хорошими персонажами, реалістичним маршрутом, кількома несподіваними подіями та екологічною місією, яку гравець запам'ятає.

Саме так ми перевіримо, чи є тут цікава гра, перш ніж вкладати ресурси в MMO, великий соціальний сайт або десятки країн.

ок, але не заанадто душно еко френдлі роби бо це має бути як доп місія, частина рутини гри а не самою ціллю. Пиши далі все що нам потрібно і чого не вистачає, бо зі слабкими доками ми не створимо якісну гру, пиши і не зволікай, бо на тобі вся побудова, думай і аналізуй, вимірюй і радь те що нам треба і продовжуй нарощувати сценарій і все що нам потрібно для старту і розвитку гри, я вже готовий починати як ти все зробиш. Подивись чого не вистачає і допиши все

Так і зробимо. Екологічність буде одним із природних шарів гри, а не її головною метою. Гравець вирушає в подорож, облаштовує кемпінг, знайомиться з людьми, вирішує побутові проблеми, відкриває місця, заробляє гроші й переживає пригоди. Прибирання, сортування та турбота про природу органічно вплітаються в цей процес, приблизно як догляд за спорядженням або допомога сусідові.

Тепер переходимо від загальної концепції до виробничого проєкту, за яким можна починати розробку. Я не буду вдавати, що повний GDD, усі діалоги, баланс і сотні квестів уже готові. Їх треба створювати послідовно, перевіряти на суперечності й тестувати в самій грі. Але я можу визначити архітектуру, зафіксувати ключові правила, розписати першу кампанію та підготувати конкретний план роботи з AI-кодером.

# 1. Фіксуємо головні правила гри

Це наші базові дизайн-рішення. Код, сюжети й нові функції мають їм відповідати.

Повноцінна браузерна гра

Одиночний режим працює без сайту спільноти, публікацій, клубів або підключення соціального акаунта. Збереження й окремі онлайн-функції матимуть власні вимоги.

Реальна Європа

Справжня географія та дороги, де є якісні дані; вигадані персонажі, діалоги й пригоди. Реальні місця не повинні отримувати вигадані факти, подані як достовірні.

Пригоди важливіші за покарання

Помилки створюють цікаві рішення, додаткові витрати, затримки, комічні ситуації або нові історії. Смерть персонажа не є механікою гри.

Екологія — додаткова система

Зазвичай це короткі дії в межах звичайної рутини. Екологічні кампанії існують, але гравець не зобов’язаний проходити їх, щоб подорожувати чи розвиватися.

Без pay-to-win

Жодних платних прискорень, енергії, обов’язкових підписок, платного доступу до базових квестів чи покупки переваги в рейтингах.

Приватність і безпека закладаються одразу

Важливі нагороди перевіряє сервер, користувацький контент має механізми скарг і модерації, а персональні дані не збираються без обґрунтованої потреби.

## Екологічність у правильній пропорції

Я пропоную початковий баланс, який перевіримо на тестуванні:

- приблизно 80–90% звичайного ігрового контенту — подорожі, побут, транспорт, NPC, пригоди, економіка й дослідження;
- приблизно 10–20% — екологічні теми, які можуть бути самостійними квестами або частиною інших завдань.

Це не жорстка квота для кожного квесту, а запобіжник від перетворення гри на екологічний симулятор.

Наприклад, гравець після вечері прибирає табір — отримує невеликий бонус і продовжує свою подорож. А вже іншим разом може добровільно пройти довгу історію про відновлення стежки. Основний прогрес не залежить від постійного накопичення екобалів.

# 2. Повний сценарій першої подорожі — виробнича версія 0.1

Це перший великий блок, який має перейти з документації в гру. Я пропоную назвати вступну кампанію «Вихідні, які пішли не за планом».

Її мета — не просто навчити керуванню. Гравець має відчути, що він створив власного персонажа, підготувався до подорожі, зустрів цікавих людей, зробив кілька важливих виборів і вже хоче вирушити далі.

Початкова кампанія повинна мати:

- один завершений сюжетний маршрут;
- приблизно 60–90 хвилин першого проходження як ціль тестування, не обов’язкову тривалість;
- щонайменше три способи пересування: велосипед, легковий автомобіль, кемпер;
- варіанти для соло-гравця, пари, сім’ї та подорожі з домашнім улюбленцем;
- близько 8–10 важливих персонажів, але не всіх одразу;
- один побутовий конфлікт, одну невелику проблему в дорозі, кілька позитивних сюрпризів;
- одну необов’язкову екологічну місію;
- завершення з вибором наступної подорожі.

Важливо: для першого прототипу можна реалізувати лише один транспортний варіант, але сценарій і дані потрібно відразу проєктувати так, щоб інші варіанти додавалися без переписування всієї системи.

# GDD / Наративна біблія — перша кампанія

# «Вихідні, які пішли не за планом»

**Версія:** 0.1 — сценарний документ для прототипу
**Статус:** запропоновано до затвердження
**Жанр:** single-player travel life sim, 2D pixel art, браузерна гра
**Тривалість:** орієнтир 60–90 хвилин для першого проходження

---

## 1. Мета кампанії

Гравець має пройти повний короткий цикл подорожі:

1. Створити персонажа та визначити його життєві обставини.
2. Обрати напрямок, транспорт і спосіб відпочинку.
3. Підготуватися до поїздки.
4. Дістатися місця призначення.
5. Облаштуватися, дослідити місцевість і познайомитися з людьми.
6. Вирішити невелику проблему.
7. Отримати перші нагороди й відкрити продовження.

Кампанія не повинна вимагати проходження всіх побічних завдань. Основний сюжет завершується навіть тоді, коли гравець не виконує екологічну місію, не спілкується з усіма NPC або не купує необов’язкове спорядження.

## 2. Персонажі кампанії

### Elise Vermeulen — власниця кемпінгу, 51 рік

Практична, доброзичлива, цінує самостійність гостей. Не любить зайву бюрократію, але наполягає на дотриманні правил безпеки.

### Nora De Smet — велосипедистка й фотографка, 28 років

Цікавиться другорядними дорогами, маленькими містами та історіями людей. Іноді захоплюється фотографуванням і забуває про час.

### Luc Bernard — велосипедний механік, 46 років

Спокійний, уважний до деталей. Любить пояснювати, як щось працює, але не ремонтує все безкоштовно й не пропонує ризикованих рішень.

### Milan Peeters — віддалений працівник, 31 рік

Подорожує у кемпері й намагається поєднувати роботу з відпочинком. Його сюжетна лінія стосується інтернету, електроенергії та особистих меж.

### Допоміжні персонажі

- Власник магазину спорядження.
- Працівник місцевої інформаційної служби.
- Інший турист, який може стати другом.
- Сусід по кемпінгу, який з'являється залежно від часу та обраного сценарію.

Усі NPC мають постійні `npcId`. Вони зберігають знайомства, обіцянки, стосунки й важливі спільні події.

---

## 3. Структура кампанії

### АСТ 1 — Підготовка

#### ST-001: Створення персонажа

**Локація:** меню створення персонажа та домашня сцена.
**Обов'язково:** так.
**Завершення:** гравець створив персонажа й підтвердив початкові умови життя.

Гравець обирає зовнішність, ім'я, вік у відповідному дозволеному діапазоні, професію, домогосподарство, транспортні можливості й стартові ресурси.

Початковий сценарій не повинен нав'язувати одну сім'ю. Вибраний профіль визначає доступні діалоги, бюджет, вільний час і стартові речі.

**Діалог:**

Система:
«Кожна подорож починається задовго до виїзду. Іноді — з мрії. Іноді — з того, що нарешті випали вільні вихідні».

Після підтвердження:
«Куди вирушимо цього разу?»

**Перехід:** ST-002.

#### ST-002: Ідея подорожі

**Локація:** карта Європи.
**Обов'язково:** так.

Гравцеві показують невеликий набір доступних напрямків у першому регіоні. Карта має відображати приблизні відстані, транспортні обмеження та основні точки маршруту.

На першому проходженні не потрібно відкривати всю Європу. Решта регіонів можуть бути видимими, але позначеними як майбутні напрямки або недоступні для поточного прототипу.

**Вибір:**

- Спокійний кемпінг біля води.
- Невелика поїздка до міста.
- Велосипедний маршрут.
- Відкласти планування й оглянути доступні варіанти.

Вибір визначає варіант маршруту, але не блокує головну кампанію.

**Перехід:** ST-003.

#### ST-003: Перевірка бюджету

**Локація:** екран планування поїздки.
**Обов'язково:** так.

Гравець бачить приблизні витрати на транспорт, ночівлю, їжу та додаткові послуги.

Гра пояснює, що бюджет — це не лише наявні гроші, а й обов'язкові витрати після повернення: оренда житла, платежі, обслуговування транспортного засобу або інші обставини, якщо вони актуальні для профілю.

Не потрібно показувати фінансові таблиці без потреби. Для новачка достатньо простого підсумку й можливості відкрити деталізацію.

**Вибір:**

- Економний варіант.
- Збалансований варіант.
- Комфортніший варіант, якщо вистачає коштів.

Якщо грошей недостатньо, запропонувати дешевшу альтернативу або коротшу подорож. Не блокувати кампанію без пояснення.

**Перехід:** ST-004.

#### ST-004: Магазин спорядження

**Локація:** магазин.
**Обов'язково:** так, але купівля додаткових предметів необов'язкова.

Магазин пропонує невеликий список предметів, релевантних обраному транспорту та способу ночівлі.

Обов'язкові предмети вже є в стартовому наборі або доступні в дешевому комплекті. Не створювати ситуацію, де гравець витратив гроші на необов'язковий предмет і через це не може пройти основний сюжет.

**Діалог продавця:**

«Перша поїздка? Не треба купувати весь магазин. Візьми те, чим справді користуватимешся. Решту завжди встигнеш докупити».

**Перевірки:**

- Подвійне натискання покупки не списує гроші двічі.
- Предмет додається в інвентар лише після підтвердженого завершення транзакції.
- Якщо предмету немає в наявності, гра показує альтернативу.
- Скасування покупки не змінює баланс.

**Перехід:** ST-005.

---

## АСТ 2 — Дорога

#### ST-005: Передвиїздна перевірка

**Локація:** дім, двір або місце паркування.
**Обов'язково:** так.

Гравець оглядає транспорт і багаж. Перевірка залежить від типу подорожі:

- Велосипед: шини, гальма, ланцюг, багаж.
- Автомобіль: паливо, багаж, загальний стан.
- Кемпер: паливо, вода, електроживлення, вантаж і розміри для доступу до кемпінгу.

У першій кампанії немає критичної несправності. Можна знайти невелику необов'язкову деталь, яку персонаж забув, але гра відразу дає зрозуміти, чи вона справді потрібна.

**Перехід:** ST-006.

#### ST-006: Перший відрізок маршруту

**Локація:** дорожня сцена або екран маршруту.
**Обов'язково:** так.

Гравець бачить прогрес подорожі, орієнтовний час прибуття та кілька рішень щодо зупинок.

На першій поїздці не вводити аварій, серйозних поломок, небезпечної погоди або несподіваних фінансових втрат.

Можлива позитивна подія: гарна оглядова точка або коротка зупинка в невеликому містечку.

**Перехід:** ST-007.

#### ST-007: Невелика затримка

**Локація:** дорога або транспортний вузол.
**Обов'язково:** так, але наслідки м'які.

Маршрут має невелику затримку: наприклад, повільний рух, черга або потреба скоригувати час зупинки. Для велосипедиста це може бути короткий відпочинок, для водія — об'їзд.

Гравець обирає:

- Продовжити за маршрутом.
- Зробити коротку зупинку.
- Переглянути план.

Затримка не повинна призводити до втрати бронювання в першій кампанії.

**Перехід:** ST-008.

---

## АСТ 3 — Кемпінг і знайомства

#### ST-008: Прибуття

**Локація:** рецепція кемпінгу.
**Обов'язково:** так.

Еліза вітає гравця й пояснює основні правила. Інформація показується короткими частинами, а не довгим текстовим полотном.

**Діалог:**

Еліза:
«Вітаю! Встигли до вечірньої метушні. Ось карта території. Вода, душові й місце для відходів позначені на ній. Якщо щось не знайдеш — запитай».

Гравець:

- «Дякую, спочатку огляну ділянку».
- «Де тут найкраще поставити табір?»
- «Чи є поруч прогулянковий маршрут?»

Вибір змінює репліку Елізи, але не доступність базових функцій.

**Перехід:** ST-009.

#### ST-009: Облаштування місця

**Локація:** ділянка кемпінгу.
**Обов'язково:** так.

Гравець ставить намет або паркує транспорт, розкладає базове спорядження та оглядає оточення.

Основні взаємодії:

- поставити намет або підготувати кемпер;
- розмістити спальне місце;
- дістати воду та їжу;
- розташувати багаж;
- оглянути найближчу інфраструктуру.

Для першого проходження потрібні зрозумілі підказки й можливість скасувати неправильне розміщення.

**Перехід:** ST-010.

#### ST-010: Перше знайомство

**Локація:** сусідня ділянка або спільна зона.
**Обов'язково:** так.

Гравець зустрічає Нору або іншого відповідного NPC.

Нора:
«Я вже двічі проходила цю дорогу, але щоразу знаходжу щось нове. Сьогодні натрапила на маленьке місце з неймовірним видом. На карті його майже не помітно».

Гравець може:

- попросити пораду щодо маршруту;
- запитати про фотографії;
- ввічливо завершити розмову.

Якщо гравець не хоче соціальної взаємодії, основний сюжет продовжується без штрафу.

**Перехід:** ST-011.

#### ST-011: Вечірній вибір

**Локація:** табір.
**Обов'язково:** так.

Настає вечір. Гравець може приготувати їжу, відпочити або коротко дослідити територію.

Вибір впливає на стан персонажа, час і невеликий елемент наступної сцени.

Не використовувати приховану перевірку, яка карає за те, що гравець обрав відпочинок замість необов'язкової активності.

**Перехід:** ST-012.

---

## АСТ 4 — Маленька пригода

#### ST-012: Загублена річ

**Локація:** спільна зона кемпінгу.
**Обов'язково:** так.

Інший турист загубив невелику річ — наприклад, ліхтарик або чохол для карти. Це запускає короткий пошуковий епізод.

Гравець може:

- оглянути найближчу зону;
- запитати NPC;
- повідомити працівника кемпінгу;
- відмовитися допомагати.

Якщо гравець допомагає, він отримує подяку й невелику репутаційну нагороду. Якщо відмовляється, кампанія все одно продовжується.

Річ має бути знайдена в логічному місці. Не можна створювати нескінченний випадковий пошук без підказок.

**Перехід:** ST-013.

**ST-013: Необов'язкова екологічна місія**

**ID квесту:** ECO-001
**Локація:** зона відпочинку біля кемпінгу.
**Обов'язково:** ні.

Після вечірньої активності гравець помічає кілька предметів сміття біля лавки. Еліза згадує, що контейнер для сортування розташований неподалік.

Гравець може:

- прибрати сміття та правильно його розсортувати;
- запитати, куди віднести окремий предмет;
- залишити місце як є й продовжити відпочинок.

Це невелика побутова дія, а не моральне випробування.

Якщо гравець прибирає сміття, він отримує невеликий бонус до екологічної репутації та запис у довіднику. Якщо не прибирає, основний сюжет не змінюється.

Небезпечні або невідомі предмети не можна пропонувати брати руками. У такому випадку правильна взаємодія — повідомити працівника.

**Перехід:** ST-014.

**ST-014: Розмова перед від'їздом**

**Локація:** рецепція або спільна зона.
**Обов'язково:** так.

Еліза запитує, як минув перший день. Гравець може поділитися враженням, згадати знайомство або попросити пораду щодо наступного маршруту.

Еліза:
«Найкращі поїздки не завжди ті, де все йде за планом. Головне — щоб у тебе залишилося щось, що захочеться згадати».

Вона пропонує кілька варіантів наступної пригоди.

**Перехід:** ST-015.

---

## АСТ 5 — Завершення та продовження

#### ST-015: Наступна подорож

**Локація:** карта маршруту або домашній екран.
**Обов'язково:** так.

Гравець отримує підсумок:

- витрачений час;
- витрати;
- відвідані місця;
- отримані предмети;
- відкриті знайомства;
- виконані квести;
- нові записи в атласі.

Гра пропонує три продовження:

1. Коротка велосипедна поїздка.
2. Подорож до нового кемпінгу.
3. Повернення додому й підготовка до довшої відпустки.

Невиконані побічні завдання не повинні відображатися як провалені, якщо вони не мали чіткого строку або прямої обіцянки.

**Завершення кампанії:** `campaign.first_trip.completed = true`.

---

## 4. Варіативність

Кампанія використовує спільні сюжетні вузли, але адаптує репліки, предмети й дії до персонажа.

### Соло-мандрівник

Більше можливостей для самостійного дослідження та знайомств. Бюджет і спорядження розраховані на одну людину.

### Пара

Доступні короткі розмови про спільні рішення, комфорт і різні очікування від відпочинку. Конфлікти не повинні бути обов'язковими.

### Сім'я з підлітком

Додаткові діалоги про дозвілля, бюджет, харчування та розподіл справ. Підліток може запропонувати цікаву активність, але не повинен постійно створювати проблеми.

### Подорож із котом

Потрібні переноска, спокійне місце, вода, лоток і перевірка правил кемпінгу. Не можна залишати кота без нагляду в небезпечних умовах.

### Подорож із собакою

Потрібні вода, повідець, прогулянки та перевірка правил доступу. Потреби залежать від розміру, віку й стану тварини.

### Велосипедист

Важливі вага багажу, рельєф, вітер, втома, ремонт і запас води.

### Автомобіль або кемпер

Важливі паливо, паркування, дорожні обмеження, розміри транспортного засобу, витрати та інфраструктура кемпінгу.

---

## 5. Вимоги до збереження

Після кожного завершеного сюжетного вузла система повинна зберігати:

- поточний етап кампанії;
- активні квести та їхні цілі;
- вибори, що впливають на наступні репліки;
- інвентар і баланс;
- стан транспортного засобу;
- місце перебування;
- стан NPC та важливі спогади;
- стан екологічних взаємодій;
- завершені й відкриті сюжетні вузли.

Повторне завантаження не повинно повторно видавати нагороду, дублювати предмети або скидати виконані квести.

Якщо гравець перериває діалог, він може повернутися до нього. Якщо гра закривається під час покупки, результат визначається збереженим станом транзакції, а не кількістю натискань.

---

## 6. Критерії приймання кампанії

Кампанія готова до тестування, якщо:

- її можна пройти від створення персонажа до завершення;
- основний сюжет не вимагає побічних квестів;
- є щонайменше два значущі вибори;
- вибори мають видимі, логічні наслідки;
- гравець може відмовитися від соціальної та екологічної місії;
- варіанти транспорту не спричиняють суперечностей у діалогах;
- усі предмети, необхідні для обов'язкових дій, доступні;
- NPC не забувають уже завершені важливі події;
- збереження та повторне завантаження не створюють дублікатів;
- немає ситуацій, де гравець не може продовжити без пояснення;
- перша подорож не створює непропорційного фінансового покарання;
- кожна гілка має визначений наступний етап або завершення.

**Примітка:** цей сценарій є початковою специфікацією. Діалоги, точні значення економіки, географічні дані й усі переходи мають пройти перевірку в прототипі до оголошення кампанії завершеною.

# 3. Що ще не вистачає до якісного старту

Найважливіше зараз — не просто додати більше квестів, а закрити прогалини, які інакше дорого виправляти після написання коду.

P0 — критично до початку масштабної розробки

- Зафіксувати структуру репозиторію та спосіб запуску.
- Визначити, що працює локально, а що потребує сервера.
- Створити єдині схеми даних і правила зміни стану.
- Зафіксувати першу кампанію, обов'язкові та необов'язкові вузли.
- Визначити мінімальний набір графіки та розмір тайлів.
- Визначити формат збереження, версіонування та відновлення.

P1 — потрібно до публічного тестування

- Тестування балансу бюджету й часу.
- Повний QA для транспорту, домашніх тварин, інвентарю та сюжетних гілок.
- Реєстр ліцензій для карт, зображень і даних.
- Мінімальні засоби безпеки, резервні копії й моніторинг.
- Політики приватності та користувацького контенту, перевірені фахівцем.

P2 — після перевірки основного ігрового циклу

- Публічний атлас і журнал.
- Клуби та спільні кампанії.
- Велика кількість регіонів і транспортних моделей.
- Shopify та print-on-demand магазин.
- Глобальні рейтинги, сезонні події й фізичні призи.

## 3.1. Технічний вибір для першого прототипу

Я рекомендую TypeScript + Phaser для 2D-гри в браузері. Phaser підтримує HTML5, Canvas/WebGL і TypeScript, тож це відповідний інструмент для нашого формату.

Phaser Help

+1

Пропонована структура:

```
project/
  apps/
    game/             # Phaser, сцени, управління, UI
    website/          # сайт, журнал, атлас
    server/           # API та серверна логіка, коли потрібна
  packages/
    game-data/        # предмети, NPC, квести, події
    game-rules/       # правила й розрахунки
    shared-types/     # спільні типи та контракти
  content/
    quests/
    dialogue/
    npcs/
    locations/
    items/
    events/
    achievements/
  docs/
    GDD.md
    NARRATIVE_BIBLE.md
    TECHNICAL_SPEC.md
    SECURITY_PRIVACY.md
    PROJECT_STATUS.md
  tests/
    unit/
    integration/
    regression/
```

Це цільова структура, не твердження про вже наявний код. Якщо AI-кодер уже створив репозиторій, спочатку треба перевірити його стан і не перебудовувати все без необхідності.

Для нового TypeScript-проєкту варто ввімкнути `strict: true`, щоб типова перевірка допомагала виявляти помилки раніше.

The Basics

+1

## 3.2. Найважливіше правило архітектури

Контент і код мають бути розділені.

Додавання нового NPC не повинно вимагати переписування логіки діалогів. Додавання квесту не повинно вимагати створення нового спеціального класу. Нова локація не повинна ламати старі збереження.

Умовно:

- код визначає, як працюють діалоги, квести, інвентар і нагороди;
- дані визначають, який саме NPC, квест, предмет або діалог використовує ця система;
- перевірки даних ловлять помилки до запуску гри.

Для кожного контентного об'єкта потрібні стабільний ID, версія, посилання на залежності й автоматична перевірка. Це значно полегшить роботу AI-кодера й майбутнє розширення.

## 3.3. Реальні карти та ліцензії

Коли дійдемо до географічного прототипу, не можна просто завантажувати будь-які карти з інтернету. Дані OpenStreetMap мають умови ODbL і вимоги до атрибуції; використання картографічних тайлів також має окремі умови.

OpenStreetMap Foundation
+1

Для кожного джерела створимо запис:

```
interface DataSourceRecord {
  id: string;
  publisher: string;
  sourceUrl: string;
  licenseName: string;
  attributionText: string;
  permittedUse: string;
  checkedAt: string;
```

```
  region?: string;
  updatePolicy: string;
  notes?: string;
}
```

Жодні реальні ціни, правила кемпінгу, години роботи чи сортування не повинні бути зашиті в гру назавжди, якщо вони можуть змінюватися. Дані мають мати дату перевірки та позначення актуальності.

## 3.4. Безпека — це перевірювані вимоги

Для серверної частини пропоную взяти OWASP ASVS як орієнтир для вимог і тестування веббезпеки, а не покладатися лише на автоматичний аналіз коду.

OWASP Foundation
+1

На першому етапі особливо важливо протестувати:

- чи можна прочитати чуже збереження, підставивши інший ID;
- чи можна видати собі предмет або валюту через зміну клієнтських даних;
- чи можна повторно отримати нагороду одним запитом;
- чи можна виконати квест без потрібних умов;
- чи можна вставити небезпечний контент у профіль або журнал;
- чи можна перевищити ліміти запитів або обійти модерацію.

GDPR вимагає закладати захист даних у проєкт із самого початку, зокрема мінімізувати дані й обмежувати доступ до них.

European Commission
+1

# 4. Наступний робочий крок: аудит проєкту перед кодом

Оскільки я не бачу твій локальний репозиторій і не можу чесно стверджувати, що документація вже записана у файли, наступним кроком має бути інвентаризація поточного стану. Якщо в тебе вже є проєкт, не треба починати його заново.

Ось готова команда для AI-кодера. Вона навмисно забороняє йому одразу переписувати файли або генерувати великий обсяг коду.

# PROMPT 0 — Повний аудит і підготовка проєкту до розробки

Ти працюєш над браузерною 2D pixel-art грою про реалістичні подорожі Європою, кемпінг, побут, персонажів і пригоди.

Твоя задача на цьому етапі — не створювати нові функції, а провести аудит поточного репозиторію та підготувати безпечний план роботи.

## Концепція

- Browser-first, single-player як основний режим.
- TypeScript + Phaser — цільовий варіант технологій, якщо наявна архітектура не вимагає іншого.
- Реальна географія Європи, але вигадані NPC та сюжетні ситуації.
- Вільне переміщення в локальних 2D-сценах і окремі інтерфейси для складних систем.
- Реалістичні транспорт, бюджет, спорядження, погода й планування.
- Без смертей персонажів, графічного насильства, порнографії, незаконних дій як ігрової цілі та pay-to-win.
- Екологічні дії є необов'язковою частиною повсякденного життя, а не основною метою гри.
- Сайт, журнал, клуби та публічні рейтинги не повинні бути обов'язковими для проходження одиночної гри.
- Безпека, приватність і можливість відновлення збережень мають бути закладені з початку.

## Завдання

1. Перевір структуру репозиторію, package-файли, налаштування TypeScript, залежності, команди запуску, наявні сцени, UI, моделі даних, тести й документацію.

2. Запусти наявні безпечні перевірки, якщо це можливо: typecheck, lint, unit tests, build. Запиши точні результати. Не вигадуй результати команд, які не запускав.
3. Визнач, що вже працює, що частково реалізовано, що є заглушкою, а чого немає.
4. Знайди дублікати типів, конфлікти ID, розбіжності між документацією та кодом, небезпечні місця й невиправдано складні рішення.
5. Перевір, чи відокремлені визначення контенту від runtime-логіки.
6. Перевір, чи є стабільна система збереження, версійність, автозбереження, перевірка даних і захист від повторного нарахування нагород.
7. Перевір, чи є автоматичні перевірки посилань між квестами, предметами, NPC, діалогами й локаціями.
8. Визнач мінімальний вертикальний зріз: одна завершена подорож від створення персонажа до повернення або вибору наступної подорожі.
9. Визнач усі блокери перед реалізацією першого вертикального зрізу.
10. Запропонуй послідовний план реалізації з критеріями приймання для кожного етапу.

## Обмеження

- Не видаляй файли.
- Не змінюй публічні API або структуру проєкту без окремого погодження.
- Не встановлюй нові залежності без обґрунтування.
- Не генеруй сотні NPC, квестів або предметів до перевірки архітектури.
- Не використовуй зовнішні API, секрети чи реальні облікові записи.
- Не заявляй, що система безпечна, лише тому, що збірка успішна.
- Якщо є неоднозначність, зафіксуй її в списку рішень замість прихованого припущення.

## Результат

Підготуй звіт:

- `docs/PROJECT_STATUS.md`
- `docs/ARCHITECTURE_AUDIT.md`
- `docs/OPEN_DECISIONS.md`
- `docs/IMPLEMENTATION_PLAN.md`

Якщо ці файли вже існують, спочатку перевір їх і запропонуй зміни. Не перезаписуй корисний вміст без потреби.

Наприкінці виведи:

1. Що реально працює.
2. Що не працює або не перевірено.
3. П'ять найважливіших ризиків.
4. Найменший наступний крок.
5. Точні команди, які були запущені, та їхні результати.

На цьому етапі не реалізовуй нові ігрові функції. Спочатку потрібна достовірна картина проєкту.

Після цього ми зможемо перейти до PROMPT 1 — створення або узгодження базових контрактів даних і мінімального вертикального зрізу. Це правильніший старт, ніж одразу генерувати великий світ: він зменшує ризик, що ми отримаємо багато красивих документів і коду, які не працюють разом.

проінвестигейть те чого в нас не вистачає, продивись історію інших схожих ігор, їх релізи і що вони додавали після виходу бо баги і діри точно про це скажуть. Також як на мене ти точно ще щось можеш доробити і додати і покращити, тому даю тобі таку задачу. Розробка, тестування, також є частиною ШІ флоу тому і це треба не забувати. Ти вже написав що ще треба то і це теж дороби бо дірки нам не можна залишати взагалі. працюй

Продовжую як системну роботу над проєктом, а не просто дописуванням ігрових ідей. Нам потрібна не лише цікава гра, а проєкт, який можна послідовно реалізувати, перевірити, безпечно запустити, підтримувати після релізу й розширювати без руйнування вже створеного.

Одне важливе уточнення: абсолютну відсутність багів, вразливостей і прогалин ніхто не може гарантувати. Але ми можемо побудувати процес, у якому кожне важливе рішення має критерії приймання, кожна функція проходить перевірки, відомі ризики мають власників і план усунення, а невідомі проблеми виявляються якомога раніше.

Також я розділятиму три речі:

- Підтверджений факт — те, що можна обґрунтувати джерелами або перевіркою.
- Проєктне рішення — те, що ми свідомо обираємо для нашої гри.
- Неперевірене припущення — те, що потрібно протестувати, перш ніж вважати правильним.

Це особливо важливо, коли ми вчимося на релізах інших ігор: не кожен баг пояснюється слабкими документами, і не кожне оновлення означає, що розробники погано спланували продукт. Частина проблем виникає через масштаб, неочікувану поведінку гравців, мережеве навантаження, сторонні сервіси або технічні обмеження, які проявляються лише після запуску.

# 1. Що показує історія інших ігор

Я перевірив історії оновлень і доступні офіційні матеріали кількох ігор, близьких за окремими складовими до нашої: симуляції повсякденного життя, дослідження світу, подорожі, NPC, довготривалий розвиток і контент після релізу.

### Порівняльний аналіз

Stardew Valley — цілісність систем та дрібні взаємозв'язки

Вихід на ПК: 26 лютого 2016 року. Велике оновлення 1.6 на ПК: 19 березня 2024 року.

Історія версій показує, що навіть зріла гра потребує виправлень після великих оновлень. Серед виправлень є проблеми з пошкодженням збережень, подіями NPC, предметами, які губилися за повного інвентарю, виходом за межі карти, локалізацією та синхронізацією багатокористувацьких станів. Оновлення 1.6 також додало новий контент, нові діалоги, функції якості життя й виправлення.

Stardew Valley 1.6 Update Full Changelog
+2

Що беремо собі: тестування не лише окремих функцій, а й комбінацій станів: предмет + повний інвентар, квест + зміна локації, NPC + зміна дня, збереження + оновлення версії.

Valheim — масштаб гравців може змінити пріоритети розробки

Ранній доступ: лютий 2021 року. Версія 1.0: 9 вересня 2026 року.

Розробники прямо повідомляли, що після великого припливу гравців їм довелося відкласти попередню дорожню карту й зосередитися на виправленні багів та стабільності. Відкрита історія також показує, що виправлення потрібні й після повноцінного релізу: наприклад, у жовтні 2026 року виправляли експлойт, помилкові позначки легально отриманих предметів і зайве використання процесора сервером.

Valheim
+2

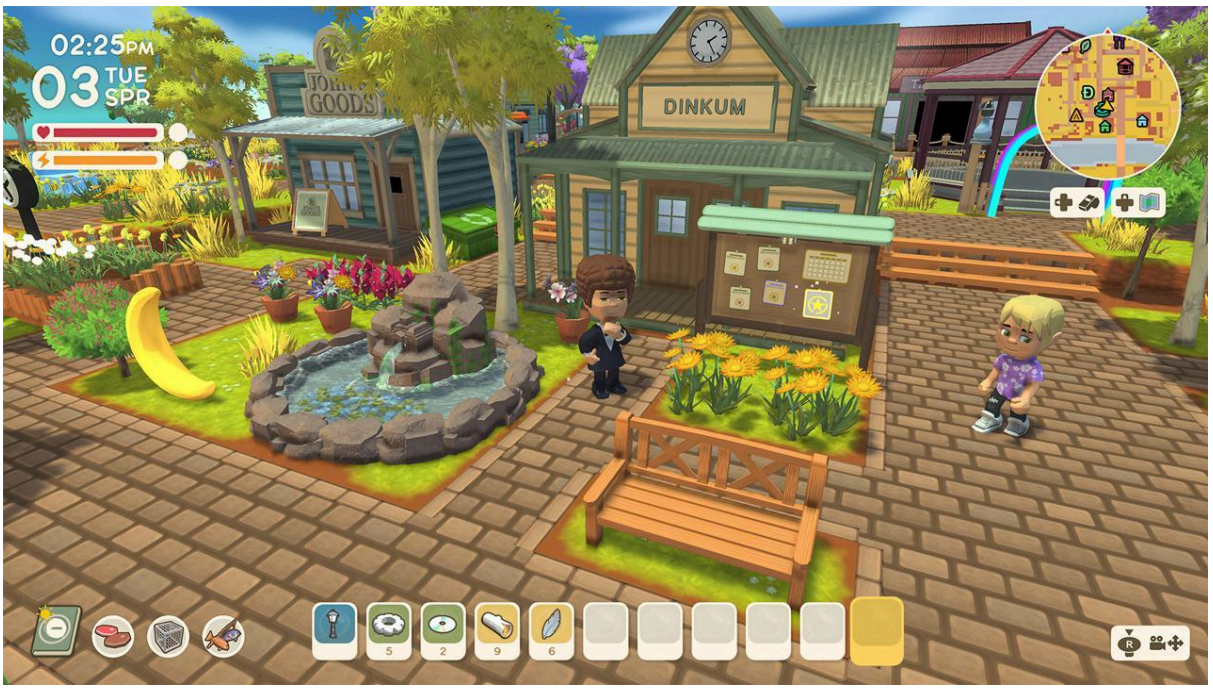
Що беремо собі: план релізу має містити резерв часу на стабілізацію; не можна обіцяти розширення світу за рахунок критичних виправлень. Навіть одиночній браузерній грі потрібні контроль навантаження, перевірки збережень та захист економіки.

No Man's Sky — після релізу змінюються не лише баги, а й самі системи

Офіційний журнал містить довгу послідовність патчів, зокрема термінові виправлення стабільності, мережі, збережень та проблем, що з'являлися після великих оновлень. Великі релізи на кшталт NEXT і Beyond розширювали ключові системи, після чого виходили додаткові виправлення.

No Man's Sky

Що беремо собі: контентні оновлення мають проходити окремий цикл перевірки; потрібні регресійні тести й можливість швидко відкликати небезпечну зміну.




Dinkum — ранній доступ як етап перевірки продукту

Повний реліз: 23 квітня 2025 року.

До виходу з раннього доступу гра отримала додаткові можливості, розширений мультиплеєр і творчий режим. Це приклад того, як великий реліз може бути результатом поступового розвитку, а не одноразової спроби випустити весь задум.

KRAFTON Press Room

Що беремо собі: рання версія має бути невеликою, але завершеною та придатною до реального тестування. Не потрібно будувати всю Європу, перш ніж перша подорож стане цікавою.

### Висновки для нашої гри

Це не повний аудит вихідного коду цих ігор — їхні внутрішні репозиторії та всі інциденти недоступні. Це аналіз опублікованих історій оновлень і задокументованих проблем. Проте вже можна визначити конкретні ризики, які варто закрити в нашому проєкті.

| Повторювана проблема | Що має бути у нас |
| --- | --- |
| Пошкоджені або несумісні збереження | Версії формату, міграції, резервна копія та відновлення |
| Квести зависають або стають недосяжними | Перевірка графа завдань і всіх умов переходів |
| Предмет зникає при отриманні нагороди | Явні правила повного інвентарю та резервного отримання |
| Гравець отримує нагороду двічі | Серверна перевірка, ідемпотентність операцій, журнал транзакцій |
| NPC повторює неактуальний діалог | Стан персонажа, пам'ять подій, тести переходів і повторної взаємодії |
| Оновлення ламає старі функції | Автоматична регресійна перевірка перед кожним релізом |
| Вихід за межі карти або застрягання | Перевірка колізій, навігації, меж та відновлення позиції |
| Відмінності між браузерами й мовами | Тести на різних розмірах екрана, браузерах і мовах |
| Експлойти економіки | Серверне підтвердження валюти, покупок і нагород |

| | |
|---|---|
| Різкий наплив користувачів | Обмеження запитів, кешування, моніторинг, план реагування |

Ключова думка: ми не повинні просто записувати баги в список. Кожен виявлений баг має перетворюватися на тест, який не дозволить непомітно повернути цю саму проблему в майбутньому.

## 2. Що ще не закрито в нашій документації

На основі вже визначеної концепції я бачу кілька систем, які потрібно додати до документації, перш ніж масштабувати розробку.

P0 — блокувальні прогалини

Мають бути закриті до створення великої кількості контенту.

- Єдина модель стану гри та правила переходів.
- Визначений власник кожного стану: клієнт, сервер або локальне збереження.
- Версійність збережень і план міграцій.
- Умови завершення квесту, відновлення після помилки й захист від повторної нагороди.
- Структура репозиторію, команди запуску, тестування та збірки.
- Мінімальна вертикальна версія: персонаж → спорядження → маршрут → кемпінг → NPC → збереження → завантаження.

P1 — якість і безпека перед відкритим тестом

- Автоматичні модульні, інтеграційні, наскрізні та регресійні тести.
- Матриця сумісності браузерів і пристроїв.
- Логи помилок без паролів, токенів і зайвих персональних даних.
- Модель загроз, резервні копії та перевірка відновлення.
- Політика модерації, скарг, блокування, апеляцій і видалення контенту.
- Відстеження продуктивності, доступності й критичних помилок.

P2 — до розширення спільноти й монетизації

- Публічний щоденник, профілі, рейтинги й антифрод.
- Експорт і видалення даних користувача.
- Модерація фотографій, текстів і зовнішніх посилань.
- Партнерські інтеграції, спонсорство та магазин мерчу.
- Відповідальність за доставку, повернення, податки та права споживачів.

- Окремі плани реагування на DDoS, витік даних і компрометацію облікових записів.

Важливо: це аудит прогалин у нашій наявній специфікації, а не перевірка реального репозиторію. Я не бачу тут файлів твого проєкту й не можу чесно заявити, що код уже перевірено. Для фактичного аудиту потрібні результати запуску агента в репозиторії або самі файли. Тому наступний крок має створити перевірюваний стан проєкту, а не удавати, що він уже готовий.

## 3. Новий обов'язковий процес: ШІ розробляє, тестує, перевіряє та документує

Це потрібно зробити частиною нашого технічного завдання. ШІ-агент не повинен отримувати завдання «зроби систему квестів» і просто повертати код. Він має працювати за контрольованим циклом.

1. Прочитати специфікацію

Контракти, обмеження, критерії приймання, залежності

2. Реалізувати мінімальну зміну

Малий обсяг, зрозумілі межі, без зайвих залежностей

3. Перевірити автоматично

Типи, lint, модульні, інтеграційні, регресійні тести, збірка

4. Перевірити негативні сценарії

Неправильні дані, подвійні кліки, перезавантаження, мережеві збої, несанкціоновані дії

5. Оновити звіт про стан

Що змінено, що протестовано, які помилки лишилися, що робити далі

Якщо перевірка не пройшла — агент виправляє проблему й запускає відповідні перевірки знову. Невдалий тест не можна позначати як успішний.

### Що має виконуватися на кожному етапі

| Перевірка | Що доводить |
| --- | --- |
| TypeScript у строгому режимі | Частина помилок типів виявляється до запуску |
| Lint і форматування | Єдині правила стилю й виявлення поширених помилок |
| Unit-тести | Окремі правила працюють на визначених прикладах |
| Integration-тести | Сумісна робота інвентарю, квестів, економіки й збережень |
| E2E-тести | Гравець справді може пройти сценарій через інтерфейс |
| Регресійні тести | Раніше виправлені проблеми не повернулися |
| Перевірки безпеки | Немає відомих порушень визначених правил доступу й валідації |
| Перевірка доступності | Основні дії доступні з клавіатури, зчитувані, не залежать лише від кольору |
| Перевірка продуктивності | Сцени й системи працюють у встановлених бюджетах часу та пам'яті |

Для браузерної гри можна оцінити Playwright для наскрізних тестів, Vitest для модульних тестів і TypeScript у строгому режимі. Це кандидати на стек, а не наказ встановити всі бібліотеки відразу. Агент має спочатку перевірити наявний репозиторій, сумісність і вартість підтримки.

Безпеку варто прив'язати до перевірюваних вимог OWASP ASVS, а не до загальної фрази «код безпечний». ASVS — це каталог вимог для перевірки технічних засобів захисту вебзастосунків.

OWASP Foundation
+1

### Обов'язкові правила для ШІ-агента

1. Не стверджувати, що тест пройдено, якщо він не запускався або результат невідомий.
2. Не видаляти файли, не змінювати контракти й не додавати залежності без обґрунтування.
3. Не маскувати помилки тестів, не послаблювати перевірки лише для зеленого результату.
4. Не переписувати велику частину системи, якщо завдання потребує малої зміни.
5. Не використовувати реальні персональні дані або секрети у тестових прикладах.
6. Для кожного виправленого дефекту додавати тест, що відтворює початкову помилку.
7. Для зміни формату збережень перевіряти як нові, так і старі тестові збереження.
8. Після зміни правил гри перевіряти не лише нову функцію, а й залежні системи.
9. Якщо тестове середовище недоступне — записувати це як блокер, а не обходити перевірку.
10. Перед публікацією вимагати явного проходження контрольного списку релізу.

Людина залишається відповідальною за схвалення архітектури, юридичних рішень, публічного релізу та ризикових змін. ШІ прискорює роботу, але самостійне повідомлення агента «все готово» не є доказом якості.

## 4. Новий документ: контроль якості та готовності до релізу

# QUALITY, QA & RELEASE ENGINEERING SPECIFICATION

Версія: 0.1
Статус: запропонована базова специфікація

## 1. Призначення

Визначити обов'язковий цикл розробки, перевірки, виправлення дефектів, випуску й підтримки браузерної гри про подорожі та кемпінг.

## 2. Класифікація дефектів

### P0 — блокувальний

- Витік або несанкціоноване розкриття даних.
- Віддалене виконання небезпечного коду або компрометація облікового запису.
- Масова втрата чи пошкодження збережень.
- Неможливість запустити гру або пройти основний сценарій.
- Несанкціоноване створення валюти, предметів чи нагород, що руйнує економіку.
- Критичне порушення доступу між користувачами.

Дія: реліз заблокований. Якщо дефект уже в production, застосовується план інциденту.

### P1 — критичний для ігрового процесу

- Квест неможливо завершити.
- Основна взаємодія або збереження регулярно ламаються.
- Прогрес втрачається за звичайного сценарію.
- Критична сцена не працює на підтримуваному браузері.
- Важливий шлях не має безпечного відновлення.

Дія: реліз блокується, якщо немає документованого й схваленого тимчасового рішення.

**P2 — значний**

- Неправильна винагорода, баланс або помітна проблема інтерфейсу.
- Побічний сценарій недоступний.
- Значне падіння продуктивності на підтримуваному пристрої.

Дія: виправити до релізу або задокументувати рішення відповідальної особи.

**P3 — незначний**

- Косметичні недоліки, дрібні неточності тексту, незначні візуальні помилки.

Дія: записати в трекер і виправити за пріоритетом.

# 3. Definition of Done для кожної задачі

Задача не завершена, поки:

1. Критерії приймання виконані.
2. Код відповідає чинним контрактам.
3. Відповідні автоматичні тести пройшли.
4. Регресійні перевірки залежних систем пройшли.
5. Немає невирішених P0/P1-дефектів, спричинених зміною.
6. Документація оновлена.
7. Результати перевірок записані з точними командами й статусами.
8. Неперевірені частини явно позначені.
9. Зміна перевірена на помилкових вхідних даних та повторних діях, якщо це релевантно.
10. Зміна не вводить несанкціонованих залежностей, секретів або сторонніх ліцензійних зобов'язань.

# 4. Обов'язкові тестові шари

### Unit

Перевіряють формули бюджету, інвентар, переходи стану, умови квестів, винагороди та правила екологічної системи.

### Integration

Перевіряють взаємодію квестів, NPC, предметів, економіки, часу, маршруту й збереження.

### E2E

Перевіряють повний шлях користувача від запуску до завершення доступного сценарію та відновлення після перезавантаження.

### Regression

Кожен підтверджений дефект отримує відтворюваний тест, якщо це технічно можливо.

### Security

Перевіряють контроль доступу, валідацію, авторизацію нагород, rate limiting, XSS, CSRF там, де це релевантно, витік секретів, небезпечні залежності й відновлення після помилок.

### Compatibility

Перевіряють підтримувані браузери, вузькі екрани, зміну розміру вікна, клавіатурне керування та локалізацію.

### Performance

Визначають вимірювані бюджети для початкового завантаження, переходів між сценами, пам'яті, частоти кадрів та серверних операцій. Значення встановлюються після прототипу й вимірювань на цільових пристроях.

## 5. Тестування збережень

- Перевіряти збереження в різних допустимих станах.
- Тестувати відновлення після перезавантаження сторінки.
- Перевіряти неповні, застарілі, пошкоджені й невідомі поля.
- Використовувати версійність формату й контрольовані міграції.
- Зберігати попередню валідну версію до завершення критичної міграції.
- Не застосовувати автоматичну міграцію без тесту на копії даних.
- Не дозволяти некоректному збереженню знищити останню відому валідну копію.
- Перевіряти, що відновлення не створює дубльовані покупки, нагороди чи предмети.

## 6. Тестування економіки та винагород

- Кожна фінансова операція має однозначний ідентифікатор.
- Повтор одного запиту не повинен повторно видавати нагороду.
- Баланс перевіряється сервером для всіх цінних онлайн-операцій.

- Транзакції мають аудитний журнал і контрольований механізм коригування.
- Валюта, предмети, досягнення й репутація перевіряються окремо.
- Усі винагороди мають умови отримання, обмеження повторення й правила відкликання.
- Виправлення помилки не повинно мовчки списувати законно отримані ресурси.

## 7. Правила роботи ШІ-агента

- Спочатку прочитати актуальну документацію й статус репозиторію.
- Скласти план змін і назвати файли, яких зміна стосується.
- Реалізовувати найменшу достатню зміну.
- Запускати перевірки, доступні в середовищі.
- Не заявляти про успіх невиконаних перевірок.
- Не видаляти чи послаблювати тести без погодження.
- Додавати регресійний тест для підтвердженого дефекту.
- Повідомляти про блокери, ризики, побічні ефекти й невідомі.
- Оновлювати PROJECT_STATUS.md та журнал змін.
- Не виконувати production-міграції, видалення даних або публікацію без явного схвалення.

## 8. Релізний шлюз

Реліз дозволено лише коли:

- Збірка успішна.
- Обов'язкові тести пройшли.
- P0/P1-дефекти відсутні або реліз заблокований.
- Міграції й відновлення перевірені.
- Резервна копія та план відкату доступні.
- Відомі обмеження описані.
- Моніторинг і контакт для інцидентів готові.
- Зміни та версія релізу задокументовані.

## 9. Післярелізний цикл

- Відстежувати помилки, падіння, втрати прогресу й продуктивність.
- Класифікувати повідомлення користувачів за впливом і відтворюваністю.
- Перевіряти виправлення на тестовому середовищі.
- Додавати регресійні тести.
- Публікувати зрозумілі примітки до випуску.

- Після критичного інциденту проводити аналіз першопричини без пошуку винних.
- Перетворювати висновки на зміни процесу, тестів або архітектури.

## 10. Статус доказів

Кожна перевірка має статус:

- PASS — перевірка виконана, результат успішний.
- FAIL — виконана, результат невдалий.
- BLOCKED — неможливо виконати через визначений блокер.
- NOT RUN — не запускалася.
- NOT APPLICABLE — обґрунтовано не стосується цієї зміни.

Статус PASS без фактичного результату виконання заборонений.

## 5. Екологічна система — додатковий розділ, який треба закріпити зараз

Твоя ідея правильна за умови, що екологічна поведінка залишається однією з частин життя мандрівника, а не перетворюється на обов'язкову роботу в кожній локації.

Я пропоную назву системи «Турбота про місце». Це краще, ніж сувора система «карми», бо вона описує конкретні дії, а не оцінює гравця як добру чи погану людину.

## Правила нарахування

### +1

Прибрав після себе

Гравець прибирає власне місце, сортує відходи за правилами цієї локації або залишає кемпінг у належному стані.

### +2

Допоміг прибрати спільне місце

Гравець добровільно прибирає безпечно доступне сміття, яке залишив хтось інший, або допомагає в погодженій локальній акції.

## −1

Залишив безлад після власної дії

Штраф застосовується лише тоді, коли гравець явно створив безлад або залишив свою ділянку в неналежному стані, маючи можливість завершити дію правильно.

Не знімаємо бали лише за те, що гравець не прибрав випадкове сміття в кожній локації. Немає покарання за пропущений день чи відмову від необов'язкової місії.

### Захист від фарму та несправедливих штрафів

- Одна й та сама купка сміття дає нагороду лише один раз.
- Повторне прибирання того самого об'єкта не створює нескінченних очок.
- Немає обов'язкової щоденної норми.
- Не можна втратити доступ до основної кампанії через низький показник турботи.
- Немає автоматичного штрафу за дію, якої гравець не робив.
- Сміття, небезпечні предмети й місця утилізації мають правила конкретної локації.
- Не пропонуємо гравцеві торкатися небезпечних відходів: гра навчає повідомити персонал або обрати безпечний варіант.
- Реальні правила сортування не вигадуємо: для кожного регіону потрібні перевірені джерела та дата перевірки.

### Що дає система

Показник турботи відкриває необов'язкові історії: довіру власника кемпінгу, запрошення до місцевої ініціативи, косметичні відзнаки, записи до особистого атласу або подяку NPC. Він не купує кращі характеристики персонажа за гроші, не визначає доступ до всієї гри й не підміняє основний цикл подорожі.

Тестові сценарії мають включати подвійний клік по сміттю, повторне входження в локацію, перезавантаження після прибирання, спробу отримати нагороду повторно, зміну правил регіону й відсутність доступного контейнера.

# 6. Додаткові прогалини: що ще необхідно передбачити

Є ще кілька напрямків, які легко забути під час створення гри, але які стають дорогими, якщо відкладати їх до релізу.

### Архітектура, контент і довготривала підтримка

- Версійність контенту. Квест має стабільний ID, версію, умови початку, переходи, винагороди та правила сумісності зі старими збереженнями.
- Граф залежностей. Якщо NPC недоступний, предмет неможливо купити або подію пропущено, необов'язковий квест не повинен блокувати кампанію назавжди.
- Ідемпотентність. Повторне надсилання тієї самої команди не має вдруге списувати кошти, видавати нагороду або завершувати завдання.
- Системний журнал. Для важливих змін стану потрібен контрольований журнал подій, але без збереження зайвих персональних даних.
- Міграції. Кожна зміна формату збереження повинна мати тест на старих версіях і шлях відновлення.
- Ліцензії контенту. Карти, шрифти, музика, звуки, спрайти, фотографії й дані про маршрути мають документоване походження та дозволене використання.
- Доступність. Керування клавіатурою, зрозумілі контрасти, читабельний текст, масштабування й альтернативи діям, які залежать від швидкої реакції.
- Резервний режим. Якщо сервер недоступний, гра має пояснити, що працює, що недоступне та чи можна безпечно продовжити локальну гру.
- Спостережуваність. Ми повинні знати, де гра зависає, скільки тривають завантаження, які переходи не завершуються і які браузери дають помилки — не збираючи при цьому більше даних, ніж потрібно.
- План припинення сервісу. Якщо онлайн-сайт колись перестане працювати, гравець має заздалегідь знати, що станеться з локальними збереженнями, обліковим записом і опублікованим щоденником.

### Безпека та законність

GDPR потрібно врахувати в архітектурі до запуску профілів, щоденника, рейтингів і спільноти: мінімізація даних, обмеження цілей, права користувачів, строки зберігання, видалення й експорт. Офіційні матеріали Єврокомісії пояснюють як принципи, так і обов'язки організацій.

European Commission
+1

Для захисту від атак потрібні різні рівні: CDN/WAF за потреби, обмеження частоти запитів, перевірка доступу на сервері, резервні копії, оновлення залежностей, моніторинг і план реагування. Це зменшує ризик, але не гарантує абсолютного захисту від DDoS чи компрометації.

Застосовність вимог щодо ШІ залежатиме від того, які саме AI-функції ми реалізуємо та як їх використовуємо. Тому не слід автоматично вважати, що гра потребує всіх можливих заходів для будь-якої ШІ-системи. Потрібно вести реєстр використання ШІ, постачальників, даних і ризиків, а юридичну застосовність вимог перевірити перед запуском. Якщо сюжет, діалоги й ігрові правила створюються наперед, а не генеруються ШІ для кожного гравця в реальному часі, це значно спрощує контроль якості та модерацію.

**Монетизація без шкоди для гравця**

Залишаємо гру без pay-to-win, платного прискорення й штучних обмежень прогресу. Потенційні джерела фінансування:

- добровільні пожертви без ігрової переваги;
- чітко позначене спонсорство;
- мерч через Shopify та партнера друку й доставки;
- гранти та партнерства з туристичними або природоохоронними організаціями;
- добровільні матеріальні внески, якщо для них є прозорий процес приймання.

Транспорт, кемпер або інші дорогі подарунки найкраще розглядати як окрему майбутню програму, а не як звичайну ігрову винагороду. Потрібно заздалегідь визначити критерії отримання, податки, страхування, оформлення власності, доставку й перевірку конфлікту інтересів. Платні пожертви не повинні впливати на місце в рейтингу або шанси отримати такий приз.

## 7. Наступна команда для AI coding agent

Не треба зараз просити агента створити десятки квестів чи малювати всю Європу. Спочатку потрібно з'ясувати реальний стан репозиторію та запровадити контроль якості. Ось конкретне наступне завдання.

# PROMPT QA-0 — Аудит репозиторію, тестового процесу та ризиків

Ти працюєш над браузерною 2D-грою про подорожі, кемпінг, транспорт, NPC, бюджет, спорядження та реальну географію Європи.

Твоє завдання — провести фактичний аудит наявного репозиторію й підготувати безпечний план розробки та тестування.

## Головні правила

1. Спочатку перевір структуру репозиторію та прочитай наявні документи.
2. Не припускай, що описані функції вже реалізовані.
3. Не вигадуй результати запуску, тестів, збірки, сканування або аудиту.
4. Не видаляй файли, не переписуй архітектуру й не додавай залежності без обґрунтування.
5. Не публікуй застосунок, не виконуй production-міграції та не видаляй дані.
6. Не виводь секрети, токени, паролі чи персональні дані у звіти.
7. Якщо якогось інструмента або доступу немає, познач це як BLOCKED.
8. Не створюй велику нову ігрову функцію на цьому етапі.

## Етап A — аудит

Перевір:

- фактичну структуру файлів;
- мову, фреймворк, систему збірки й пакетний менеджер;
- наявні скрипти запуску, lint, typecheck, test і build;
- конфігурацію TypeScript;
- наявні тести та їхнє реальне покриття;
- структуру ігрового стану й збережень, якщо вони існують;
- залежності, конфігурацію середовищ і обробку помилок;
- наявні перевірки безпеки;
- доступність, продуктивність і браузерну сумісність, наскільки це можливо перевірити;
- суперечності між кодом, документацією та фактичними командами.

Не вважай відсутність тесту доказом того, що функція несправна. Розділяй підтверджені дефекти, ризики та неперевірені припущення.

## Етап B — тестова стратегія

Запропонуй найменший достатній набір інструментів для:

- статичного аналізу та перевірки типів;
- модульних тестів;

- інтеграційних тестів;
- наскрізних браузерних тестів;
- регресійних тестів;
- перевірки залежностей і секретів;
- перевірки збережень та міграцій.

Спочатку перевір наявні інструменти. Не встановлюй альтернативні бібліотеки, якщо поточний стек уже виконує потрібну роль.

Визнач, які перевірки можна автоматизувати в CI, які потребують ручного тестування, а які поки неможливо виконати.

## Етап C — аналіз критичних ризиків

Окремо перевір ризики:

- втрати або пошкодження збережень;
- повторної видачі нагород;
- подвійного списання валюти;
- обходу умов квесту;
- доступу до чужого прогресу;
- небезпечної обробки введеного користувачем тексту;
- помилок переходу між сценами;
- незавершених квестів;
- некоректної обробки мережевих помилок;
- випадкового розкриття секретів;
- відсутності відновлення після невдалого оновлення.

Не експлуатуй production-системи й не виконуй руйнівні перевірки на реальних даних.

## Етап D — документи

Створи або онови:

- docs/PROJECT_STATUS.md
- docs/ARCHITECTURE_AUDIT.md
- docs/QA_STRATEGY.md
- docs/SECURITY_PRIVACY.md
- docs/OPEN_DECISIONS.md
- docs/IMPLEMENTATION_PLAN.md

Якщо структура проєкту інша, спочатку поясни, де ці документи логічно розмістити.

Кожен висновок познач одним зі статусів:

- VERIFIED — підтверджено фактичною перевіркою;
- INFERRED — обґрунтований висновок, але без прямого доказу;
- UNKNOWN — даних недостатньо;
- BLOCKED — перевірка неможлива через конкретну перешкоду.

## Етап E — результат

У підсумку надай:

1. Реальний стан проєкту.
2. Точні команди, які було запущено, і результати.
3. Перелік знайдених дефектів із пріоритетами P0–P3.
4. Перелік критичних прогалин у тестуванні.
5. Пропоновану архітектуру тестування.
6. Рішення, які потрібні від власника проєкту.
7. Найменший наступний крок реалізації.
8. Критерії, за якими цей крок буде прийнятий.

Не заявляй про повну безпеку чи відсутність багів. Не переходь до наступної великої функції, поки аудит не завершений і блокувальні питання не задокументовані.

Головна мета: отримати перевірюваний стан проєкту, з якого можна почати контрольовану реалізацію першої вертикальної версії гри.

# 8. Порядок роботи після аудиту

1. Аудит реального репозиторію. Фіксуємо, що вже існує, що працює та що потрібно виправити.
2. Єдині контракти даних. Узгоджуємо персонажа, інвентар, транспорт, маршрут, NPC, квести, події та збереження.
3. Мінімальна вертикальна версія. Один персонаж, одна обмежена реальна локація, спорядження, одна подорож, кемпінг, кілька NPC і збереження.
4. Автоматичне тестування. Перевіряємо нормальні, помилкові та граничні сценарії, включно з перезавантаженням і відновленням.
5. Перша кампанія. Додаємо розгалужені місії, пам'ять NPC, екологічні дії, сюрпризи та нагороди поверх стабільних систем.

6. Закритий плейтест. Збираємо добровільний зворотний зв'язок, аналізуємо дефекти й повторюємо перевірки.
7. Публічний запуск і підтримка. Лише після проходження релізного шлюзу, підготовки резервного копіювання, моніторингу та плану реагування.

Мій головний висновок: якість нашої гри залежатиме не від кількості сторінок документації й не від того, скільки коду напише ШІ. Вона залежатиме від того, чи зможемо ми перетворювати задум на точні контракти, контракти — на реалізацію, реалізацію — на перевірені результати, а кожну знайдену помилку — на захист від її повторення.

Починати потрібно з PROMPT QA-0. Він дасть нам фактичну картину, а вже на її основі можна безпечно обрати наступну зміну, не будуючи нові системи на неперевіреному фундаменті.

## 2. Що ще не закрито в нашій документації

- дороби це

## 6. Додаткові прогалини: що ще необхідно передбачити

- дороби це

## 8. Порядок роботи після аудиту

- дороби це

Починати потрібно з PROMPT QA-0. Він дасть нам фактичну картину, а вже на її основі можна безпечно обрати наступну зміну, не будуючи нові системи на неперевіреному фундаменті.

- дороби це і те чого ще не вистачає і не тільки в моєму і твоєму списку і взагалі ширше думай і дивись на те що вже написали і чого не написали, думай на всі сили і не роби собі обмежень і використовуй всю силу ЧатГПТ і ШІ і інтернету і кодінгу і всього що тобі потрібно щоб якісно виконати свої задачі заради успіху гри

# Розширення проєкту: від концепції до повноцінної системи розробки та підтримки гри

Наше завдання тепер ширше, ніж доповнити кілька розділів GDD. Потрібно зібрати єдину систему проєктування, програмування, тестування, управління контентом, безпеки, випуску та розвитку гри, у якій кожне важливе рішення має обґрунтування, критерії приймання й спосіб перевірки.

Я також переглядаю попередні напрацювання як єдиний проєкт, а не як набір незалежних текстів. Уже визначені сюжет, кемпінги, NPC, подорожі, транспорт, навички, економіка, екологічна активність, щоденник, спільнота й фінансування. Тепер потрібно перевірити, як ці частини взаємодіятимуть, що станеться при помилках і що доведеться підтримувати роками.

Принцип роботи: не намагатися написати всі можливі документи одразу, а закрити всі критичні невідомі, зафіксувати залежності й будувати перевірюваний продукт поетапно. Це дозволить уникнути двох крайнощів: передчасного програмування без архітектури та нескінченного планування без робочої гри.

# 9. Повний реєстр того, що ще потрібно проєкту

Нижче — розширення попередніх розділів. Воно перетворює список «чого бракує» на конкретні документи, правила, перевірки та критерії готовності.

## 9.1. Карта документації

Це цільова структура, яку агент має створити або узгодити з реальною структурою репозиторію. Це ще не твердження, що ці файли вже існують.

`docs/` — документи, за якими розробляємо

Специфікації, архітектура, правила й рішення

Стратегія та управління

PROJECT_CHARTER.md · GDD.md · PRODUCT_ROADMAP.md · SCOPE_AND_MILESTONES.md · DECISION_LOG.md · RISKS.md

Ігрові системи та контент

NARRATIVE_BIBLE.md · WORLD_DESIGN.md · GAMEPLAY_SYSTEMS.md · ECONOMY_BALANCE.md · CONTENT_PIPELINE.md · QUEST_AUTHORING.md

Технології та дані

TECHNICAL_SPEC.md · ARCHITECTURE.md · DATA_CONTRACTS.md · SAVE_MIGRATIONS.md · API_SPEC.md · ADR/

Якість і тестування

QA_STRATEGY.md · TEST_PLAN.md · RELEASE_CHECKLIST.md · PERFORMANCE_BUDGETS.md · ACCESSIBILITY.md

Безпека та право

THREAT_MODEL.md · SECURITY_PRIVACY.md · DATA_RETENTION.md · LICENSE_REGISTER.md · UGC_POLICY.md · INCIDENT_RESPONSE.md

Запуск та експлуатація

DEPLOYMENT.md · BACKUP_RECOVERY.md · MONITORING.md · SUPPORT_PLAYBOOK.md · CHANGELOG.md · PROJECT_STATUS.md

Важливе правило: не створювати 30 порожніх файлів лише заради красивої структури. Кожен документ має бути або заповнений мінімально необхідним змістом, або позначений як `NOT STARTED` із залежностями, причиною й наступним кроком. Якщо два документи дублюють один одного, потрібно визначити єдине джерело істини й посилатися на нього.

## 9.2. Прогалини, які попередній список не охоплював повністю

| Напрям | Що потрібно визначити | Критерій готовності |
|---|---|---|
| Продукт | Цільова аудиторія, платформа, основна обіцянка гри | Є перевірювані цілі й межі першого релізу |

| | | |
|---|---|---|
| Ігровий цикл | Що гравець робить кожні 5, 20 і 60 хвилин | Прототип підтверджує, що цикл цікавий |
| Обсяг | Що входить у перший реліз, а що відкладається | Є перелік `Must / Should / Could / Won't` |
| Стан світу | Час, погода, транспорт, NPC, предмети, завдання | Немає суперечливих власників стану |
| Контент | Унікальність ID, локалізація, версії, ліцензії | Контент можна валідовувати автоматично |
| Економіка | Баланс, джерела та витрати валюти | Немає неконтрольованих способів створення ресурсів |
| Збереження | Версії, відновлення, конфлікти запису | Старі тестові збереження проходять міграції |
| Якість | Автоматичні й ручні перевірки | Кожен реліз має доказ проходження шлюзу |
| Доступність | Клавіатура, масштабування, контраст, текст | Основні сценарії перевірені на доступність |
| Безпека | Межі довіри, доступ до даних, захист від зловживань | Критичні вимоги мають конкретні тести |

| | | |
|---|---|---|
| Інфраструктура | Розгортання, відкат, резервування | Відновлення реально перевірено на тестовому середовищі |
| Підтримка | Повідомлення про помилки, запити користувачів | Є процес приймання, класифікації та вирішення |
| Бізнес | Витрати, фінансування, відповідальність за мерч | Є прозора модель без прихованих зобов'язань |
| Спільнота | Модерація, скарги, блокування, апеляції | Кожен процес має правила й відповідального |
| Довговічність | Застарілі функції, залежності, завершення сервісу | Є план підтримки й перенесення даних |

### Чотири речі, які я вважаю особливо важливими

Перша — гра має бути цікавою до появи соціальних функцій. Рейтинги, блоги, подарунки й спонсорство не повинні компенсувати слабкий ігровий цикл.

Друга — офлайн-гра й вебсервіс мають бути розділені концептуально. Якщо сайт або сервер щоденника недоступний, це не повинно автоматично знищувати локальний прогрес одиночної гри. Онлайн-нагороди й публічні профілі можуть тимчасово бути недоступні, але користувач має розуміти наслідки.

Третя — кожна система повинна мати межі. Наприклад, система квестів не повинна самостійно змінювати гроші, інвентар і відносини без контрольованих команд. Інакше виникнуть приховані побічні ефекти, які буде важко тестувати.

Четверта — план розвитку має передбачати відмову від невдалих функцій. Якщо рейтинг не покращує досвід, а створює спам або шахрайство, ми повинні мати можливість вимкнути його, не руйнуючи збереження й основний ігровий цикл.

# 10. Архітектурні правила, які потрібно узгодити до масштабної розробки

## 10.1. Розділяємо ядро гри, контент та інтерфейс

Я рекомендую рухатися в напрямку такої архітектури:

Інтерфейс та сцени

Рух, взаємодія, меню, діалоги, карта, анімації

Команди та правила гри

Купити предмет, почати квест, змінити маршрут, завершити подію

Чисте ігрове ядро

Бюджет, інвентар, час, погода, транспорт, навички, умови переходів

Модель стану та збереження

Перевірені дані, версії, міграції, відновлення

Зовнішні адаптери

Сервер, облікові записи, щоденник, карта, аналітика, резервні копії

Це логічна модель, не вимога реалізувати все окремими сервісами. Для невеликої команди краще почати з модульного моноліту, а не розподіленої архітектури з десятками сервісів. Відокремлюємо відповідальності в коді, але не додаємо інфраструктурну складність без реальної потреби.

**Контракти команд**

Усі важливі зміни мають проходити через валідовані команди. Приклади:

- `PurchaseItem`
- `UseItem`
- `StartQuest`
- `CompleteObjective`
- `CompleteQuest`
- `ChangeRoute`

- `RepairVehicle`
- `RecordEcoAction`
- `SaveGame`
- `RestoreSave`

Для кожної команди документуємо:

1. Який стан вона читає.
2. Які умови повинні виконуватися.
3. Які зміни дозволені.
4. Які побічні ефекти можливі.
5. Що відбувається при помилці.
6. Чи можна безпечно повторити запит.
7. Які тести підтверджують поведінку.

Це не означає, що нам потрібна складна enterprise-система. Навпаки: чіткі межі на початку зменшать кількість спеціальних випадків у майбутньому.

## 10.2. Єдине джерело істини для стану

Потрібно визначити, хто є авторитетним джерелом кожного типу даних.

| Дані | Одиночна гра | Онлайн-функції |
|---|---|---|
| Позиція персонажа | Локальний ігровий стан | Сервер зберігає синхронізований прогрес, якщо він потрібний |
| Інвентар і бюджет | Локальні правила та збереження | Серверна перевірка для нагород і транзакцій |
| NPC і квести | Ігрове ядро | Профіль може зберігати підтверджені досягнення |

| | | |
|---|---|---|
| Щоденник | Чернетка локально | Сервер керує опублікованими записами й доступом |
| Публічні нагороди | Не повинні бути довільно підтверджені клієнтом | Сервер перевіряє умови, дублікати та ліміти |
| Рейтинг | Може існувати локально як особиста статистика | Сервер визначає офіційні результати |
| Налаштування | Локально, якщо це доречно | Синхронізація лише за потреби |

Не потрібно змушувати одиночну гру постійно звертатися до сервера лише для того, щоб пройти звичайну сцену. Але й не можна довіряти браузерному клієнту при видачі публічних призів або підтвердженні транзакцій.

## 10.3. Схема контенту та автоматична перевірка

Кожен квест, NPC, предмет, локація та подія має стабільний ID. Назва, текст діалогу й баланс можуть змінюватися, але ID не слід змінювати без міграції або явного правила сумісності.

Для контенту потрібні автоматичні перевірки:

- унікальність ID;
- коректність посилань на NPC, предмети й локації;
- існування всіх переходів квестів;
- відсутність недосяжних обов'язкових етапів;
- відсутність нескінченних циклів без умов виходу;
- доступність необхідних предметів;
- коректність нагород і вимог;
- наявність тексту для підтримуваних мов;
- коректність версій та ліцензійних атрибуцій;
- можливість відновити прогрес після виходу з гри на кожному важливому етапі.

Нова вимога: контентний валідатор повинен запускатися автоматично під час перевірки змін. Це дасть змогу знаходити помилки сюжету ще до того, як гравець зустріне їх у грі.

# 11. Повний план перевірки гри: від одиночного тесту до відкритого релізу

Нам потрібен не один чекліст, а кілька рівнів доказів.

## 11.1. Тестова піраміда

## Ручні перевірки та плейтести

Чи цікаво грати? Чи зрозумілі правила? Чи відчувається подорож справжньою?

**Наскрізні тести**

Чи можна пройти сценарій через реальний інтерфейс?

**Інтеграційні тести**

Чи узгоджені між собою квести, інвентар, економіка, NPC і збереження?

**Модульні тести**

Чи правильно працює кожне окреме правило?

Найбільше швидких автоматичних перевірок має бути на рівні окремих правил. Ручні плейтести залишаються незамінними, бо автоматичний тест може підтвердити, що діалог відкривається, але не визначить, чи він нудний, неприродний або не відповідає характеру персонажа.

## 11.2. Обов'язкова матриця тестування першої подорожі

Ці сценарії мають перевірятися до розширення кампанії.

| ID | Сценарій | Очікуваний результат |
|---|---|---|

| | | |
|---|---|---|
| QA-TRIP-001 | Стандартна подорож автомобілем | Кампанія проходиться без блокування |
| QA-TRIP-002 | Гравець не купив необов'язкову річ | Є альтернативний шлях |
| QA-TRIP-003 | Інвентар заповнений | Предмет не зникає без пояснення |
| QA-TRIP-004 | Гравець відмовився від соціального квесту | Основна історія продовжується |
| QA-TRIP-005 | Подорож із котом | Працюють потреби, спорядження й правила кемпінгу |
| QA-TRIP-006 | Подорож із собакою | Працюють потреби, прогулянки й обмеження місць |
| QA-TRIP-007 | Велосипедний маршрут | Немає вимог до недоступного автомобіля |
| QA-TRIP-008 | Збереження перед діалогом | Діалог і прогрес відновлюються коректно |
| QA-TRIP-009 | Збереження під час квесту | Немає повторного списання чи дублювання нагороди |

| | | |
|---|---|---|
| QA-TRIP-010 | NPC тимчасово недоступний | Квест не стає безвихідним |
| QA-TRIP-011 | Подвійний клік по покупці | Операція не виконується двічі |
| QA-TRIP-012 | Перезавантаження сторінки | Гравець повертається до валідного стану |
| QA-TRIP-013 | Старий формат збереження | Міграція працює або безпечно відмовляє |
| QA-TRIP-014 | Вихід із гри на критичному етапі | Є зрозумілий шлях відновлення |
| QA-TRIP-015 | Відмова сервера щоденника | Локальна гра не втрачає основний прогрес |
| QA-TRIP-016 | Неправильні дані квесту | Контентний валідатор виявляє помилку |
| QA-TRIP-017 | Зміна розміру вікна | Камера, меню та керування лишаються придатними |
| QA-TRIP-018 | Клавіатурне керування | Основний шлях доступний без миші, де це передбачено |

| | | |
|---|---|---|
| QA-TRIP-019 | Повторне отримання нагороди | Дубль не зараховується |
| QA-TRIP-020 | Помилка під час збереження | Остання валідна копія не знищується |

Ця матриця — початковий набір, а не повне покриття. Після реалізації кожної системи до неї додаються специфічні граничні випадки.

## 11.3. Додаткові методи, які варто запровадити

Property-based testing. Замість перевірки лише одного прикладу автоматично генерувати багато комбінацій станів. Наприклад, сума грошей не повинна змінюватися через відкриття меню; інвентар не повинен містити від'ємну кількість предметів; повтор команди не повинен дублювати нагороду.

Тестування на випадкових послідовностях дій. Генерувати послідовності на кшталт «купити — вийти — перезавантажити — відкрити квест — повернутися на карту» й перевіряти інваріанти після кожної дії.

Диференційні тести. Якщо формула або логіка змінюється, порівнювати старі й нові результати на контрольованих прикладах, щоб виявити непередбачені зміни.

Мутаційне тестування. Перевіряти, чи справді тести виявлять помилку, якщо правило навмисно змінити. Це корисно для критичних формул економіки та винагород, але не обов'язково застосовувати до всього коду від першого дня.

Тестування відмов. Перевіряти повільну мережу, недоступний сервер, неповну відповідь, повтор запиту та переривання запису. Виконувати такі експерименти в тестовому середовищі, не на живих даних.

Тестування збережень. Створити контрольний набір валідних і спеціально пошкоджених файлів, зокрема старих версій. Після кожної зміни формату перевіряти їх автоматично.

# 12. Продуктивність, доступність і сумісність

Це окремий напрям, який часто відкладають до кінця, хоча він впливає на архітектуру з першого дня.

## 12.1. Продуктивність

Потрібно визначити бюджети для:

- часу початкового завантаження;
- розміру першого завантажуваного пакета;
- переходів між локаціями;
- частоти кадрів у локальних сценах;
- використання пам'яті;
- кількості активних NPC і об'єктів;
- обробки великих журналів подій;
- часу відповіді серверних функцій.

Не встановлюємо довільні цільові числа до появи прототипу. Спочатку вимірюємо на визначених тестових пристроях, потім задаємо бюджети й перевіряємо їх у CI або на регулярних контрольних збірках.

Для вебсайту й сторінок спільноти можна використовувати Core Web Vitals як орієнтир реальної якості завантаження та взаємодії. Для самої гри потрібні також окремі ігрові метрики — наприклад, стабільність частоти кадрів і час завантаження сцен.

GitHub

## 12.2. Доступність

Варто орієнтуватися на WCAG 2.2 для вебінтерфейсів, перевіряючи застосовність конкретних критеріїв до гри та її функцій.

W3C

Мінімальні вимоги:

- основні меню доступні з клавіатури;
- фокус клавіатури помітний;
- текст читабельний і масштабується;
- колір не є єдиним способом передавання важливої інформації;
- текстові підказки не зникають надто швидко;
- керування можна перепризначати там, де це доречно;
- анімації, струс камери й інтенсивні ефекти можна обмежувати;
- діалоги, помилки й підтвердження мають зрозумілі назви;
- інтерфейс працює на вузьких екранах, якщо мобільний браузер входитиме до підтримуваних платформ.

## 12.3. Сумісність

Потрібно визначити офіційний перелік підтримуваних браузерів і мінімальні вимоги до пристрою. Не обіцяємо підтримку всіх браузерів і старих телефонів без перевірки.

Тестуємо щонайменше:

- підтримувані настільні браузери;
- мобільні браузери, якщо вони входять до цільового релізу;
- різні співвідношення сторін;
- масштабування та зміну розміру вікна;
- відновлення після втрати фокуса;
- клавіатуру й мишу;
- сенсорне керування, якщо воно підтримується;
- локалізацію, довгі слова, переноси й шрифти.

# 13. Операційна готовність: що станеться після запуску

Гра не закінчується на успішній збірці. Якщо ми додамо профілі, публічні щоденники, рейтинги й нагороди, потрібно вміти виявляти проблеми та безпечно їх усувати.

## 13.1. Моніторинг

Запровадити мінімальний набір метрик:

- частота критичних помилок;
- частка невдалих завантажень і збережень;

- невдалі міграції;
- тривалість серверних операцій;
- кількість відхилених підозрілих транзакцій;
- помилки авторизації та надмірні запити;
- доступність основних онлайн-функцій;
- помилки під час публікації щоденника;
- витрати на інфраструктуру.

OpenTelemetry надає незалежні від конкретного постачальника підходи до метрик, журналів і трасування. Це корисна основа, але впроваджувати повноцінну розподілену телеметрію для маленького прототипу не потрібно — починаємо з мінімально необхідних сигналів.

OpenTelemetry
+1

Усі логи мають бути мінімізованими: не записуємо паролі, токени, вміст приватних повідомлень або повні персональні дані без чіткої необхідності.

## 13.2. План реагування на інциденти

Для кожного серйозного інциденту потрібна однакова послідовність:

1. Виявити й підтвердити проблему.
2. Визначити масштаб і постраждалі функції.
3. Зупинити поширення шкоди — наприклад, тимчасово вимкнути вразливу функцію.
4. Зберегти необхідні технічні докази без надмірного збору персональних даних.
5. Відновити безпечну роботу або перейти в контрольований режим.
6. Перевірити виправлення в тестовому середовищі.
7. Відновити функцію лише після перевірки.
8. Задокументувати першопричину та запобіжні заходи.
9. Повідомити користувачів або відповідні органи, якщо це необхідно за законом.

Окремі плани потрібні для втрати даних, компрометації облікових записів, шахрайства з нагородами, DDoS, витоку секретів і проблем із постачальником.

## 13.3. Резервне копіювання та відновлення

Резервна копія сама по собі нічого не доводить. Потрібні регулярні перевірки, що з неї можна відновитися.

Зафіксувати:

- які дані резервуються;
- частоту копіювання;
- строк зберігання;
- шифрування й доступ;
- розділення production і тестового середовища;
- цільовий час відновлення (RTO);
- допустиму втрату даних у часі (RPO);
- порядок перевірки цілісності;
- порядок відновлення після невдалої міграції.

Конкретні RTO та RPO встановлюємо відповідно до архітектури й вартості сервісу. Для локальної одиночної гри й серверного щоденника вони можуть бути різними.

# 14. Безпека, приватність і правові питання — окремий пакет вимог

Це не розділ, який можна закрити фразою «дотримуватися GDPR». Нам потрібні реєстр даних, модель загроз, перевірки доступу й процедура реагування.

## 14.1. Реєстр даних

Для кожного поля потрібно документувати:

| Поле | Що фіксуємо |
|---|---|
| Назва даних | Наприклад, email, ігровий прогрес, публічний псевдонім |
| Мета | Для чого дані потрібні |

| | |
|---|---|
| Джерело | Користувач, гра, зовнішній сервіс |
| Місце зберігання | Пристрій, сервер, сторонній постачальник |
| Доступ | Хто й за яких умов може читати або змінювати дані |
| Строк зберігання | Коли дані видаляються або переглядаються |
| Правова підстава | Визначається відповідно до фактичної мети й застосовного законодавства |
| Видалення та експорт | Як виконується запит користувача |
| Залежності | Які функції перестануть працювати після видалення |

Не збираємо точну геолокацію, контакти, адресу, дату народження або інші персональні дані «про всяк випадок». Реальна географія гри не означає, що нам потрібна реальна геолокація гравця.

## 14.2. Мінімальна модель загроз

До загроз потрібно включити:

- крадіжку або підміну облікового запису;
- читання чи зміну чужого збереження;
- підроблення валюти, досягнень або нагород;
- XSS через профіль, щоденник чи коментар;
- автоматизований спам і масові запити;
- витік ключів доступу;
- компрометацію сторонньої бібліотеки;
- помилки налаштування хмарних ресурсів;

- DDoS та вичерпання ресурсів;
- небезпечні посилання й завантажені користувачами файли;
- зловживання модераторськими правами;
- неправильне видалення даних;
- втрату або пошкодження резервних копій.

Для вебчастини використовуємо OWASP ASVS 5.0.0 як каталог перевірюваних вимог і підбираємо відповідний рівень суворості до ризиків конкретних функцій. Це стандарт перевірки, а не автоматичний захист і не сертифікат безпеки.

OWASP Application Security Verification Standard
+1

## 14.3. Специфічні правові питання

Перед публічним запуском необхідно перевірити застосовність і виконання вимог щодо:

- GDPR і прав користувачів;
- електронних комунікацій, cookies та аналітики;
- захисту неповнолітніх і вікових обмежень;
- Digital Services Act, якщо сервіс підпадає під його вимоги;
- авторського права, ліцензій та використання відкритих географічних даних;
- споживчого законодавства, якщо продаємо фізичні товари;
- правил спонсорства, конкурсів і призів;
- податкових наслідків пожертв і натуральних внесків;
- застосовності AI Act до фактичних функцій і способу використання ШІ.

Юридичний статус залежить від реальної бізнес-моделі, аудиторії, країн і функцій. Тому потрібен список питань для професійної перевірки, а не вигаданий висновок, що одна універсальна політика автоматично закриє всі вимоги.

# 15. Порядок роботи після аудиту — тепер із контрольними воротами

Ми не повинні переходити до наступного етапу лише тому, що агент написав «готово». Для кожного етапу встановлюємо критерії приймання.

**G0**

Фактичний аудит репозиторію

Вихід: карта коду, список перевірок, ризики, невідомі, стан інструментів.

Приймання: результати мають бути підтверджені фактичними командами або позначені як неперевірені. Жодних непогоджених великих змін.

**G1**

Узгодження правил та архітектури

Вихід: контракти даних, архітектурні рішення, межі MVP, реєстр ризиків.

Приймання: кожна основна система має визначеного власника стану, залежності, помилкові сценарії й критерії приймання.

**G2**

Вертикальний прототип

Вихід: одна завершена ігрова подорож із реальними переходами між системами.

Приймання: можна почати гру, взаємодіяти, пройти маршрут, завершити перший сюжетний етап, зберегти прогрес і відновити його.

**G3**

Повноцінний перший ігровий цикл

Вихід: персонаж, спорядження, бюджет, маршрут, кемпінг, NPC, необов'язкові місії.

Приймання: основна кампанія прохідна в запланованих варіантах; відмови від необов'язкових завдань не блокують прогрес.

**G4**

Закритий плейтест

Вихід: звіт про проходження, проблеми UX, баланс, продуктивність і стабільність.

Приймання: усі блокувальні дефекти виправлені; повторні тести підтверджують виправлення; критичні ризики задокументовані.

**G5**

Публічний реліз

Вихід: збірка, політики, резервні копії, моніторинг, план інцидентів.

Приймання: обов'язкові перевірки пройшли, немає відкритих P0/P1-дефектів, підготовлено відкат і канал підтримки.

**G6**

Післярелізний розвиток

Вихід: регулярні патчі, нові історії, поліпшення, підтримка спільноти.

Приймання: кожне оновлення має регресійні тести, нотатки до випуску й перевірений план відкату.

## Що не можна робити до проходження цих етапів

- Не створювати всю карту Європи.
- Не запускати публічні рейтинги та призові програми.
- Не додавати платні функції або інтеграції магазину.
- Не будувати складну систему клубів і спільнот.
- Не вводити runtime-генерацію основного сюжету через ШІ.
- Не мігрувати реальні дані без перевіреного механізму відновлення.

Це не означає, що ці функції відкидаються. Ми просто не дозволяємо їм відволікати від доказу того, що основна гра працює.

# 16. Оновлений PROMPT QA-0 — фактичний аудит і підготовка до розробки

Це оновлена версія попереднього промпту. Вона охоплює не лише тестування, а й узгодженість документації, залежності, контент, продуктивність, безпеку та наступний перевірюваний крок.

# PROMPT QA-0 v2 — Повний аудит проєкту перед реалізацією

## Роль

Ти — AI coding agent, який працює над браузерною 2D-грою про подорожі, кемпінг, транспорт, бюджет, спорядження, NPC, реальну географію Європи та необов'язкові соціальні функції.

Твоє завдання — не просто перевірити код. Ти маєш встановити фактичний стан проєкту, виявити прогалини в документації та архітектурі, визначити критичні ризики, підготувати тестовий процес і запропонувати найменший безпечний наступний крок.

## Основні правила

1. Почни з фактичної структури репозиторію та чинних документів.
2. Не вважай попередні плани доказом того, що функція вже реалізована.
3. Не вигадуй результати команд, тестів, збірок або сканування.
4. Не видаляй файли й не переписуй архітектуру на цьому етапі.
5. Не додавай залежності без перевірки наявного стеку та обґрунтування.
6. Не запускай руйнівні перевірки на production або реальних даних.
7. Не публікуй застосунок і не виконуй незворотних міграцій.
8. Не розкривай секрети, токени, паролі або персональні дані у звітах.
9. Якщо перевірка неможлива, познач її BLOCKED або NOT RUN і поясни причину.
10. Не переходь до великої реалізації, поки результати аудиту не задокументовані.

## Етап 1. Інвентаризація

Визнач:

- фактичну структуру репозиторію;
- мову, фреймворк, систему збірки й пакетний менеджер;
- точки входу та команди запуску;

- конфігурації TypeScript, lint, форматування й тестів;
- наявні модулі та їхні залежності;
- структуру ігрового стану;
- структуру збережень і наявні міграції;
- джерела ігрових даних і контенту;
- налаштування середовищ і секретів;
- наявні CI/CD-процеси, якщо вони існують.

Для кожного висновку вказуй доказ: файл, конфігурацію, команду або фактичний результат.

## Етап 2. Аудит документації

Порівняй код і документи. Знайди:

- суперечливі правила;
- повторення одного рішення в різних місцях;
- відсутні визначення термінів;
- неузгоджені ID і схеми даних;
- функції без критеріїв приймання;
- невизначені межі MVP;
- неописані помилкові сценарії;
- невизначених власників даних;
- відсутні рішення щодо сумісності збережень;
- залежності, які блокують реалізацію.

Створи реєстр прогалин із пріоритетом, наслідком, залежностями, способом перевірки та рекомендованим наступним кроком.

## Етап 3. Архітектура та ігровий стан

Перевір:

- розділення інтерфейсу, правил гри, даних і збережень;
- джерело істини для кожного типу стану;
- межі між локальною грою й онлайн-функціями;
- обробку команд і повторних запитів;
- правила інвентарю, економіки, квестів і NPC;
- версійність контенту;
- можливість відновлення після помилок;
- циклічні залежності й надмірне зчеплення модулів.

Не впроваджуй складну архітектуру лише заради формального розділення. Пропонуй найменше достатнє рішення.

## Етап 4. Аудит якості

Перевір, що реально доступно:

- typecheck;
- lint;
- unit-тести;
- integration-тести;
- E2E-тести;
- регресійні тести;
- перевірка залежностей;
- перевірка секретів;
- перевірка контентних даних;
- аналіз продуктивності;
- перевірка доступності;
- CI-запуски.

Виконуй лише безпечні й доречні команди. Записуй точну команду, результат, код завершення та відомі обмеження.

Не встановлюй інструменти автоматично, якщо спершу не оцінено наявний стек, сумісність і вартість підтримки.

## Етап 5. Модель загроз

Оціни ризики:

- доступу до чужого прогресу;
- підміни валюти, інвентарю, досягнень і нагород;
- повторного виконання транзакцій;
- XSS та небезпечної обробки тексту;
- спаму, автоматизації та вичерпання ресурсів;
- компрометації облікових записів;
- витоку секретів;
- небезпечних сторонніх залежностей;
- втрати збережень;
- неправильного видалення даних;
- відсутності резервного відновлення;
- неконтрольованих адміністративних прав.

Відрізняй реальні дефекти від ризиків і неперевірених припущень. Не заявляй про повну безпеку.

## Етап 6. Контент і наратив

Перевір:

- унікальність ID квестів, NPC, предметів і локацій;
- валідність посилань;
- досяжність обов'язкових етапів;
- наявність альтернативних шляхів;
- доступність предметів для завдань;
- можливість відмови від необов'язкових місій;
- відновлення після виходу й перезавантаження;
- повторні нагороди;
- послідовність пам'яті NPC;
- ліцензійні джерела для зовнішніх даних.

Не створюй новий великий обсяг контенту. Спершу визнач, які валідатори й контракти потрібні.

## Етап 7. Продуктивність і доступність

Визнач:

- підтримувані браузери й пристрої, якщо вони вже задекларовані;
- наявні вимірювання часу завантаження;
- проблеми з ресурсами, сценами й пам'яттю;
- підтримку клавіатури;
- поведінку при зміні розміру екрана;
- ризики для локалізації й масштабування тексту.

Не вигадуй результати продуктивності без вимірювання.

## Етап 8. Документи

Створи або онови:

- docs/PROJECT_STATUS.md
- docs/ARCHITECTURE_AUDIT.md
- docs/QA_STRATEGY.md
- docs/THREAT_MODEL.md
- docs/OPEN_DECISIONS.md

- docs/RISK_REGISTER.md
- docs/IMPLEMENTATION_PLAN.md

Якщо є чинні аналоги, онови їх замість створення дубльованих документів.

Кожен висновок познач:

- VERIFIED — безпосередньо підтверджено;
- INFERRED — обґрунтований висновок;
- UNKNOWN — даних недостатньо;
- BLOCKED — перевірка заблокована;
- NOT RUN — перевірку не виконували.

## Етап 9. Пріоритизація

Для кожної проблеми зафіксуй:

- ID;
- опис;
- статус доказу;
- вплив;
- ймовірність;
- пріоритет;
- залежності;
- запропоноване рішення;
- критерій приймання;
- спосіб тестування;
- ризик відкладення.

Пріоритет визначай за ризиком і впливом, а не за зручністю реалізації.

## Етап 10. Фінальний звіт

Надай:

1. Фактичний стан репозиторію.
2. Команди, які реально запускалися, та їхні результати.
3. Відомі дефекти.
4. Прогалини документації.
5. Ризики архітектури та безпеки.
6. Стан тестування.
7. Невирішені питання.
8. Найменший наступний крок.

9. Точні критерії приймання цього кроку.
10. Перелік перевірок, які залишилися невиконаними.

Не стверджуй, що проєкт готовий до релізу лише на підставі успішної збірки.

Після аудиту зупинись. Не переходь до наступної великої функції без окремого завдання та зрозумілих критеріїв приймання.

Мета — отримати перевірювану картину проєкту й основу для безпечної, поступової реалізації першої вертикальної версії гри.

# 17. Що робити після QA-0: конкретний план на наступні завдання

Після отримання звіту агента не варто одразу братися за перший пункт у довгому списку. Спершу потрібно визначити блокувальні залежності. Проте подальша послідовність уже зрозуміла.

| Етап | Результат | Умова переходу |
|---|---|---|
| PROMPT QA-0 | Фактичний аудит | Звіт перевірений, блокери виділені |
| PROMPT 1 | Контракти даних і рішення архітектури | Немає критичних суперечностей |
| PROMPT 2 | Мінімальне ігрове ядро | Правила перевіряються незалежно від графіки |
| PROMPT 3 | Система збереження | Є тести версій, відновлення й помилкових даних |

| | | |
|---|---|---|
| PROMPT 4 | Перша вертикальна сцена | Рух, колізії та взаємодія працюють |
| PROMPT 5 | Інвентар, предмети та бюджет | Усі критичні операції валідовані |
| PROMPT 6 | Система квестів | Граф переходів проходить перевірку |
| PROMPT 7 | NPC та пам'ять | Повторні взаємодії й стани перевірені |
| PROMPT 8 | Перша подорож і кемпінг | Основний сценарій прохідний |
| PROMPT 9 | Екологічна активність та інші побічні системи | Немає фарму, несправедливих штрафів і блокування основного сюжету |
| PROMPT 10 | QA, доступність і баланс | Пройдено тестову матрицю |
| PROMPT 11 | Закритий плейтест | Є звіт, виправлення та регресійні перевірки |
| PROMPT 12 | Підготовка релізу | Пройдено релізний шлюз |

Це не жорстка нумерація файлів або команд, а пропонована послідовність. Якщо аудит покаже, що частина систем уже існує, не потрібно переписувати її лише для дотримання плану.

## 17.1. Як ми будемо використовувати ШІ протягом усього життєвого циклу

ШІ має виконувати не тільки написання коду, а й інші ролі:

- Аналітик: знаходить суперечності, залежності та невизначені вимоги.
- Архітектор: пропонує варіанти й пояснює компроміси.
- Розробник: реалізує невеликі, контрольовані зміни.
- Автор тестів: створює модульні, інтеграційні та регресійні перевірки.
- QA-аналітик: генерує граничні сценарії та перевіряє повноту тестової матриці.
- Редактор контенту: перевіряє послідовність діалогів, квестів і NPC.
- Помічник з безпеки: аналізує модель загроз і перевіряє захисні правила.
- Аналітик плейтесту: групує відгуки, відділяє спостереження від припущень і визначає пріоритети.
- Менеджер документації: оновлює статуси, рішення, залежності та журнал змін.

Але кожна роль повинна мати чіткий результат і межі. Наприклад, ШІ може знайти підозрілий шлях доступу до даних, але без виконання перевірок не повинен заявляти, що вразливість доведена або що система безпечна.

## 17.2. Єдиний звіт після кожної задачі

Кожна зміна має завершуватися коротким звітом такого формату:

# Implementation Report

## 1. Задача

ID та коротка назва.

## 2. Що змінено

Список файлів і короткий опис кожної зміни.

## 3. Рішення

Чому обрано цей підхід; які альтернативи розглядалися.

## 4. Перевірки

Для кожної перевірки:

- команда;
- фактичний результат;
- статус PASS / FAIL / BLOCKED / NOT RUN;
- відомі обмеження.

## 5. Регресійні ризики

Які залежні системи можуть бути зачеплені та як вони перевірялися.

## 6. Відомі проблеми

Невиправлені дефекти, технічний борг і неперевірені припущення.

## 7. Документація

Що оновлено; які рішення додані до журналу.

## 8. Критерії приймання

Для кожного критерію — виконано, не виконано або не перевірено.

## 9. Наступний крок

Одна конкретна рекомендована задача з поясненням залежностей.

## 10. Висновок

Чи можна прийняти зміну, чи потрібні виправлення, чи існує блокер.

# 18. Що я рекомендую затвердити як незмінні принципи проєкту

Ці принципи мають діяти для всіх майбутніх завдань, якщо власник проєкту явно не змінить їх через документоване рішення.

1. Спочатку робочий малий продукт, потім масштабування. Не створюємо величезний світ до перевірки першої подорожі.
2. Одиночна гра має залишатися самодостатньою. Соціальні та онлайн-функції не повинні без потреби блокувати базовий прогрес.
3. Кожна цінна онлайн-операція перевіряється на сервері. Клієнт не є джерелом істини для офіційних нагород і рейтингів.
4. Кожна важлива зміна має тест. Виправлення без регресійного захисту — незавершене виправлення.
5. Контент перевіряється як дані. Квести й NPC не можуть покладатися лише на ручне вичитування.
6. ШІ не вигадує результати перевірок. Невідоме лишається невідомим до появи доказу.
7. Реальні дані мають походження. Для географії, маршрутів, правил кемпінгів та екології ведемо джерела, ліцензії й дати перевірки.
8. Жодної монетизації через примус. Немає pay-to-win, платного прискорення, покарання за перерву чи обов'язкових соціальних публікацій.
9. Приватність закладається з початку. Збираємо лише необхідні дані, описуємо їх використання й підтримуємо належні права користувачів.
10. Реліз — це контрольований процес. Є критерії готовності, резервні копії, відкат, моніторинг і відповідальний за рішення.
11. Післярелізні проблеми змінюють процес. Кожен серйозний дефект дає не лише виправлення, а й висновок, як запобігти повторенню.
12. Не додаємо складність без доказу потреби. Кожен сервіс, інтеграція, рейтинг або нова валюта повинні мати чітку мету.

## 19. Підсумок: що вже зроблено в межах цього планування і що ще потрібно

У цій розмові ми розширили специфікацію, визначили нові вимоги до архітектури, тестування, безпеки, доступності, експлуатації, правових перевірок і процесу розробки. Підготували оновлений PROMPT QA-0 і формат звіту для кожної зміни.

Але є принципова різниця між підготовленою специфікацією і реалізованим та перевіреним проєктом. Ми ще не підтвердили фактичний стан репозиторію, не виконали його тести й не довели працездатність архітектури. Саме для цього потрібен перший аудит.

Мій конкретний порядок дій такий:

1. Передати агенту PROMPT QA-0 v2 у середовищі, де доступний репозиторій.

2. Отримати фактичний звіт, а не лише список припущень.
3. На основі результатів розділити блокери, критичні дефекти, архітектурні рішення та необов'язкові покращення.
4. Узгодити контракти й мінімальну вертикальну версію.
5. Реалізувати її з автоматичними тестами та доказами проходження критеріїв.
6. Розширювати гру лише після підтвердження стабільності попереднього етапу.

Наша мета — не створити документацію, у якій нібито немає жодної прогалини. Наша мета — створити процес, який системно виявляє прогалини, не дозволяє критичним проблемам непомітно проходити далі й дає нам змогу розвивати гру без хаотичних переписувань. Це найреалістичніший шлях до якісної, цікавої та довготривалої гри.

Щось є що додати? Якщо є то допиши і краще за всіх це зроби!

Так. Є кілька важливих речей, які варто додати, і я б зробив це не просто ще одним списком ідей, а останнім рівнем контролю якості всього проєкту.

Найбільша небезпека зараз — не брак ідей. Навпаки, ідей, механік і майбутніх можливостей уже багато. Небезпека в тому, що ми можемо створити велику документацію, але пропустити суперечності між системами, недооцінити обсяг розробки або почати будувати сайт, спільноту й економіку раніше, ніж сама гра стане цікавою.

Тому я б додав до документації ще один обов’язковий документ — `PROJECT_ASSURANCE.md` («Гарантії якості та контроль ризиків проєкту») — і кілька пов’язаних із ним процедур. Він не обіцятиме неможливого «нуль помилок», а визначатиме, як ми виявлятимемо помилки, доводитимемо готовність функцій і не дозволятимемо критичним проблемам непомітно переходити в наступний етап.

## 1. Що показує досвід інших ігор

Я перевірив історії оновлень кількох ігор, які корисні для нашого проєкту. Важливо: публічні списки виправлень показують реальні типи проблем, які розробники усували, але не є повною історією всіх їхніх помилок чи інцидентів безпеки.

Stardew Valley — складні взаємодії та збереження

У виправленнях після великого оновлення 1.6 фігурували аварійні завершення, помилки завдань і NPC, проблеми з предметами та збереженнями, а також відмінності між платформами. Окремо додавали відновлення деяких пошкоджених збережень.

Stardew Valley Wiki
+2

Що беремо собі: перевірка стану після кожного переходу, автоматичні резервні копії, контрольовані міграції збережень і тестування не лише «щасливого» проходження, а й незвичних послідовностей дій.

Dinkum — перенесення даних і поведінка інтерфейсу

В опублікованих виправленнях є проблема пошкодження збереження під час перенесення даних із демоверсії до повної гри, а також помилки керування, інтерфейсу й синхронізації мережевої гри.

steamcommunity.com

Що беремо собі: перевіряти імпорт, експорт і відновлення даних; не припускати, що інтерфейс працює однаково з мишкою, клавіатурою й контролером; визначити сумісність версій даних.

The Long Dark — критичні сюжетні блокування та сумісність

У примітках до оновлення сюжетного режиму розробники зазначали відомі проблеми, за яких можна було застрягнути в локації, зіткнутися з аварійним завершенням на певній платформі або не побачити старі збереження в новій версії застосунку.

The Long Dark
+1

Що беремо собі: перед релізом проходити сюжет від початку до кінця, перевіряти можливість виходу з кожної локації, мати аварійний шлях відновлення та відкрито публікувати список відомих проблем.

Euro Truck Simulator 2 — поступове вдосконалення основних систем

В історії оновлень є не лише виправлення, а й поступові зміни фізики, підказок, інтерфейсу, конфігурації транспорту та карт.

steamcommunity.com
+1

Що беремо собі: закладати можливість покращувати вже наявні системи без повного переписування гри. Для нас це особливо важливо для транспорту, планування маршруту, бюджету та спорядження.

### Висновки для нашої гри

Це не просто список чужих багів. Це вимоги до нашої архітектури:

- Збереження — частина ігрового дизайну. Квест, покупка, ремонт, розмова й перехід між локаціями мають залишатися коректними після перезапуску.
- Сюжет не може створювати тупики. У кожного обов’язкового завдання повинен бути досяжний наступний крок або безпечний шлях відновлення.

- Оновлення мають бути зворотно сумісними настільки, наскільки це можливо. Якщо формат змінюється, потрібна міграція та перевірка старого стану.
- Інтерфейс — це функціональна система, а не декор. Переповнений текст, недоступна кнопка, втрата фокуса або зламана навігація можуть блокувати проходження.
- Не можна тестувати лише за одним сценарієм. Потрібні різні персонажі, домашні тварини, транспорт, вибори, порядок дій і стани збереження.
- Відомі проблеми слід документувати чесно. Релізний список відомих проблем, тимчасові обмеження й інструкції відновлення кращі за мовчання.

Ще один важливий висновок: ми не повинні намагатися відразу створити масштаб гри, яка розвивалася роками. Варто спочатку побудувати маленьку, завершену подорож, перевірити її та лише потім збільшувати світ.

## 2. Новий обов’язковий документ проєкту

Нижче — готова специфікація, яку потрібно додати до документації та надалі використовувати разом із `PROJECT_STATUS.md`, `QA_STRATEGY.md`, `THREAT_MODEL.md` і `RELEASE_CHECKLIST.md`.

# PROJECT_ASSURANCE.md

## Гарантії якості, цілісності та керованості проєкту

**Статус:** запропонований нормативний документ
**Застосування:** усі етапи розробки гри, сайту, серверної частини та спільноти
**Головний принцип:** жодна функція не вважається готовою лише тому, що її реалізовано або вона один раз запрацювала.

## 1. Що означає «готово»

Кожна функція проходить п'ять перевірок:

1. **Відповідність задуму:** зрозуміло, яку проблему гравця вона вирішує.
2. **Коректність:** правила, стани та наслідки відповідають специфікації.
3. **Стійкість:** повторні дії, перезапуск, помилки й неочікувані стани не руйнують гру.
4. **Безпека:** користувач не може отримати чужі дані або підробити серверні винагороди.

5. **Докази:** є результати тестів, відомі обмеження та зрозумілі критерії приймання.

Статуси функції:

- `PROPOSED` — ідея.
- `SPECIFIED` — вимоги описані.
- `IMPLEMENTED` — код написаний.
- `TESTED` — визначені перевірки виконані.
- `ACCEPTED` — критерії приймання виконані.
- `RELEASED` — функція доступна в конкретному релізі.
- `DEPRECATED` — функція виводиться з використання.
- `BLOCKED` — є перешкода.
- `UNKNOWN` — доказів недостатньо.

Статуси не можна підвищувати автоматично лише за фактом завершення завдання програмування.

## 2. Класифікація дефектів

- **P0 — критичний:** ризик витоку чужих даних, серйозне порушення безпеки, масове пошкодження збережень або блокування запуску основного продукту. Реліз заборонений.
- **P1 — високий:** втрата прогресу, невиправний тупик обов'язкового квесту, систематичне неправильне списання ресурсів, критична недоступність основного сценарію. Реліз заборонений без виправлення або формально схваленого безпечного рішення.
- **P2 — середній:** істотна проблема окремої механіки, локації чи пристрою з доступним обходом. Потрібні відповідальний, строк і рішення щодо релізу.
- **P3 — низький:** косметичні помилки, незначні незручності, дрібні неточності тексту. Можна планувати на наступні ітерації.

Будь-який дефект, пов'язаний із безпекою, конфіденційністю або юридичними вимогами, оцінюється також за окремою процедурою ризику. Його не можна автоматично знизити до P2 чи P3 через наявність тимчасового обходу.

## 3. Обов'язкові інваріанти гри

Ці правила мають перевірятися автоматично там, де це можливо.

**Збереження**

- Завантаження збереження не створює предметів, грошей або винагород повторно.
- Невдалий запис не знищує останнє справне збереження.
- Міграція формату або завершується коректно, або зберігає початковий файл і повідомляє про проблему.
- Персонаж не може опинитися в неіснуючій локації без механізму відновлення.
- Квестові прапорці, інвентар, бюджет і стан світу після перезавантаження узгоджені.

**Економіка**

- Покупка або продаж застосовуються атомарно: немає списання грошей без відповідної операції з предметом.
- Повторне надсилання тієї самої критичної команди не дає подвійної винагороди.
- Серверні винагороди та рейтинги перевіряються сервером, а не довіряють значенням із браузера.
- Всі зміни винагород мають пояснюваний запис у журналі транзакцій.

**Квести й NPC**

- Кожна обов'язкова ціль має досяжний шлях завершення.
- Необов'язкова відмова від побічного квесту не блокує основний сюжет.
- Відсутній NPC, предмет або ресурс не спричиняє безвихідного стану.
- Повторний діалог не дублює одноразові нагороди.
- Стан NPC та його пам'ять узгоджені зі збереженим станом світу.

**Безпека й приватність**

- Користувач не має доступу до чужого приватного збереження без належної авторизації.
- Публікації та профілі не дозволяють виконувати довільний код через введений текст.
- Видалення облікового запису охоплює визначені системи зберігання, резервні копії та сторонні інтеграції відповідно до затвердженої політики.
- Секрети, ключі доступу й приватні дані не потрапляють у публічний репозиторій або звичайні журнали.

## 4. Контроль змін

Кожна зміна повинна містити:

- посилання на вимогу або проблему;

- перелік змінених файлів і даних;
- пояснення архітектурних рішень;
- автоматичні тести та їх фактичні результати;
- перевірку суміжних систем;
- оцінку впливу на збереження, контент і сумісність;
- документацію, оновлену за потреби;
- відомі обмеження та спосіб відкату.

Зміни формату збережень, економіки, авторизації, винагород, квестових контрактів і структури контенту потребують окремої перевірки впливу.

## 5. Обов'язкові перевірки перед релізом

- Основний сценарій прохідний від початку до кінця.
- Збереження та відновлення перевірені на підтримуваних версіях.
- Міграції даних перевірені на тестових копіях старих збережень.
- Відсутні відкриті P0 та P1.
- Відомі P2 мають відповідальних і документоване рішення.
- Критичні API перевірені на помилкові запити, повтори та неавторизований доступ.
- Перевірено клавіатуру, фокус, читабельність, масштабування та критичні сценарії доступності.
- Контент перевірено автоматичним валідатором.
- Резервне копіювання та відновлення перевірені практично.
- Підготовлено список відомих проблем, інструкції підтримки та план реагування.
- Реліз можна відкотити або безпечно виправити без втрати критичних даних.
- Є фактичні результати перевірок і відповідальна особа за рішення про випуск.

## 6. Правила роботи з AI-агентами

AI-агент повинен:

1. Спочатку вивчити наявний код, контракти та тести.
2. Змінювати лише необхідний обсяг системи.
3. Не створювати дублікати вже наявних механік без обґрунтування.
4. Не стверджувати, що тести пройдено, якщо вони не запускалися.
5. Відокремлювати перевірені факти від припущень.
6. Не видаляти дані, файли або збереження без дозволу.
7. Не виконувати руйнівні операції на продуктивному середовищі.
8. Повідомляти про невизначеність, регресії та конфлікти вимог.

9. Перед завершенням завдання оновлювати `PROJECT_STATUS.md` і пов'язані документи.
10. Зупинятися та запитувати рішення людини, якщо вимоги суперечать одна одній або ризик перевищує визначені межі.

## 7. Відновлення після інциденту

1. Зафіксувати симптоми та часовий проміжок.
2. Обмежити шкоду, не знищуючи докази.
3. Захистити дані користувачів і збереження.
4. Визначити масштаб і причину.
5. Виправити проблему та додати тест, який відтворює її.
6. Перевірити виправлення на ізольованому середовищі.
7. Випустити виправлення контрольовано.
8. Повідомити користувачів, якщо це потрібно.
9. Провести аналіз першопричини та змінити процес, щоб зменшити шанс повторення.

## 8. Критерій зрілості проєкту

Проєкт вважається готовим до розширення, коли основний ігровий цикл стабільний, дані відновлюються, контент перевіряється автоматично, зміни не ламають суміжні системи, а команда може пояснити, що перевірено, що невідомо та які ризики залишилися.

Відсутність відомих дефектів не є доказом відсутності дефектів. Доказом зрілості є повторюваний процес перевірки, відновлення та контрольованого випуску.

## 3. Ще шість прогалин, які потрібно закрити окремо

Це речі, які часто відкладають «на потім», а потім вони стають дорогими проблемами.

1. Версіонування контенту

Квести, NPC, предмети й локації теж змінюватимуться. Потрібно мати стабільні ID, перевірку посилань, правила перейменування та видалення, а також міграції стану, коли старий квест більше не відповідає новій версії контенту.

2. Поведінка без інтернету

Основна одиночна гра має запускатися та залишатися придатною до гри без постійного з'єднання. Якщо сайт або сервер недоступний, це не повинно блокувати локальне проходження. Онлайн-нагороди, синхронізація й спільнота можуть тимчасово бути недоступні з чітким поясненням.

3. Відновлення даних

Недостатньо написати «є резервні копії». Треба визначити, що саме копіюється, як часто, хто має доступ, скільки копії зберігаються та як часто проводиться пробне відновлення. Встановити вимірювані цілі втрати даних і часу відновлення — RPO та RTO — після отримання перших реальних вимірів.

4. Походження реальних даних

Для маршрутів, правил кемпінгів, цін, дорожніх обмежень, природних ризиків і туристичних об'єктів зберігати джерело, дату перевірки, регіон, ліцензію, рівень упевненості й дату наступної перевірки. Ігрове спрощення має бути позначене як спрощення, а не видаватися за перевірений факт.

5. Модерація та правила спільноти

До відкриття публікацій, коментарів і рейтингів потрібні скарги, блокування користувачів, обмеження частоти дій, процес оскарження модераторських рішень, правила авторських прав і видалення контенту. Не відкривати публічну спільноту раніше, ніж з'явиться мінімальний робочий процес модерації.

6. Окреме управління матеріальними винагородами

Якщо в майбутньому будуть футболки, спорядження, кемпери або великі призи, це вже не просто ігрова механіка. Потрібні правила участі, географічні обмеження, верифікація переможців, доставка, податки, гарантії, страхування та юридична перевірка. До цього моменту такі призи мають залишатися ідеєю, а не обіцянкою гравцям.

## 4. Що робити далі — правильний порядок

Я б закріпив такий порядок і не дозволяв великій кількості нових ідей відволікати нас від нього.

1. Аудит фактичного репозиторію. Запустити раніше підготовлений `QA-0`: з'ясувати, які документи, код, тести й системи реально існують. Не робити висновків про готовність за документацією в чаті.

2. Реєстр прогалин і ризиків. Для кожної прогалини записати пріоритет, причину, залежності, критерій завершення, відповідального та доказ перевірки.
3. Узгодження контрактів. Визначити єдині моделі стану світу, персонажа, інвентарю, квестів, NPC, збережень і транзакцій. Виявити суперечності до того, як вони розійдуться по коду.
4. Вертикальний зріз. Зробити одну невелику, але завершену подорож: персонаж, магазин, транспорт, маршрут, кемпінг, NPC, короткий квест, збереження та відновлення.
5. Автоматизовані перевірки. Додати тести і валідатор контенту до кожної зміни. Перевіряти не лише правильні дії, а й повтори, відмови, помилки, перезапуски та незвичні стани.
6. Закрите тестування. Дати невеликій групі людей пройти гру без підказок розробника. Записати, де вони губляться, що неправильно розуміють і що їм цікаво повторювати.
7. Контрольований реліз. Спочатку обмежений випуск, перевірка телеметрії без зайвого збору персональних даних, план відкату, резервні копії та готова підтримка.
8. Розширення за доказами. Лише після стабільного ядра додавати нові регіони, десятки NPC, ширшу економіку, спільноту, таблиці лідерів і матеріальні винагороди.

## 5. Останнє правило: не плутати повноту з обсягом

Ми можемо спроєктувати сотні подій, десятки персонажів і великий каталог спорядження. Але найбільшу цінність матиме не найбільший каталог, а гра, у якій подорож цікава, рішення мають зрозумілі наслідки, NPC поводяться послідовно, а прогрес не зникає через випадкову помилку.

Тому моє головне доповнення до всього попереднього плану таке:

Жодна нова система не отримує пріоритет лише тому, що вона цікава. Спочатку ми доводимо, що основний цикл працює, що дані захищені, що помилки можна виявляти й виправляти, а потім масштабуємо контент.

Я підготував ці доповнення як специфікацію для документації, але ще не перевіряв їх у твоєму реальному репозиторії й не запускав там тести. Наступний фактичний крок — виконати аудит у середовищі, де знаходиться код, а потім перетворити виявлені проблеми на конкретні задачі з перевірюваними критеріями приймання.

Я розумію що ти ще не все написав, допиши, думай і пиши, тут в тебе вся історія є тому згадай і допиши те що ще не зробив і не написав чи забув чи знайшов щось цікаве і важливе для нас.

Ти маєш рацію. У нашій історії вже накопичилося набагато більше, ніж один GDD: ігрові механіки, персонажі, сюжет, подорожі Європою, домашні тварини, економіка, спорядження, реальні правила кемпінгу, досягнення, спільнота, сайт, журнал мандрівника, винагороди, потенційний магазин товарів і робота через AI-агента. Частину цього ми описали окремо, але ще не звели в одну узгоджену систему.

Тому я не просто додам ще список ідей. Я доповню проєкт тим, що легко забути під час великої розробки: як усі системи взаємодіятимуть, що станеться в нестандартних ситуаціях, які рішення потрібно прийняти заздалегідь і що ми маємо перевірити, перш ніж вкладати час у масштабування гри.

Також важливо розділити три речі: уже визначені рішення, запропоновані доповнення та питання, які ще треба вирішити. Інакше через кілька місяців AI-агент може сприйняти стару ідею як затверджену вимогу або створити другу версію вже наявної системи.

Ось ще один дуже конкретний урок із першоджерел: в офіційних виправленнях `Stardew Valley` описано не тільки звичайні помилки, а й дублювання предметів, пошкодження або неправильне перенесення стану старих збережень, пропущені рецепти та проблеми, які виникають лише за певної послідовності дій.

Stardew Valley Wiki
+1

А в офіційних примітках до `The Long Dark` зустрічаються блокування проходження після завантаження контрольної точки, неправильне оновлення цілей місій, проблеми з позицією персонажа та відмінності між платформами.

steamcommunity.com
+1

З цього випливає додаткове правило саме для нас: тестувати потрібно не тільки кожну функцію окремо, а й переходи між станами. Наприклад: почати ремонт → зберегтися → перезавантажити гру → відкрити інвентар → завершити ремонт

→ повторно натиснути кнопку → перевірити гроші, предмети, квест і журнал. Саме на стиках систем часто виникають найнеприємніші помилки.

## 6. Доповнення, які потрібно внести в документацію гри

Це не нові випадкові функції. Це конкретні вимоги, які випливають із нашого задуму й допоможуть уникнути протиріч.

## A. Єдина модель світу

Ми вже описали три масштаби: карту Європи, маршрут і локальні сцени. Тепер потрібно чітко визначити, як вони обмінюються даними.

- Карта Європи відповідає за регіони, доступність напрямків, великі відстані й відкриті локації.
- Маршрут відповідає за дороги, час у дорозі, паливо, транспортні обмеження, зупинки й дорожні події.
- Локальна сцена відповідає за пересування, фізичні взаємодії, розмови, предмети та об'єкти навколо гравця.
- Єдиний стан світу зберігає дату, погоду, бюджет, спорядження, квести, NPC, транспорт і домашніх тварин.

Нова вимога: локальна сцена не повинна самостійно змінювати бюджет чи квестові прапорці в обхід ігрових правил. Вона надсилає команду, а спільна система правил перевіряє та застосовує наслідки.

## B. Планування реальної подорожі

Навчальна цінність гри залежить не від кількості реальних назв на карті, а від правильності причин і наслідків.

Для кожної реалістичної механіки потрібно визначити:

- вхідні дані та формулу;
- одиниці вимірювання;
- межі застосування;
- джерело фактичних даних;
- спрощення для ігрового процесу;
- тестові приклади;
- дату наступного перегляду, якщо інформація змінюється.

Наприклад, витрата пального має залежати від відстані, витрати на 100 км і ціни за літр. Але реальна витрата також залежить від навантаження, швидкості, рельєфу, погоди й стану транспортного засобу. Для першої версії можна залишити спрощену формулу, а інші чинники вводити лише тоді, коли вони додають цікаві рішення.

Для маршрутів Європою окремо потрібні правила платних доріг, поромів, екологічних зон, обмежень висоти й маси, сезонних закриттів та правил конкретного кемпінгу. Не варто робити вигляд, що всі ці дані незмінні або вже перевірені.

## C. Безпека без надмірного покарання

Наш принцип — реалістична подорож без смерті персонажа. Це потрібно послідовно підтримати всіма системами.

- Попереджати про небезпеку до того, як вона стане незворотною.
- Розрізняти невдале рішення, випадкову помилку керування та непередбачувану ситуацію.
- Дозволяти зупинитися, відпочити, змінити маршрут, звернутися по допомогу або скористатися альтернативним транспортом.
- Не використовувати травми як спосіб примусити гравця витрачати реальні гроші.
- Не перетворювати втому, голод, спрагу й температуру на постійний набір дрібних покарань.
- Навчати через зрозумілі наслідки, а не через приховані правила.

Потрібен також окремий баланс між реалізмом і комфортом: небезпечні події повинні бути зрозумілими, а не випадковими пастками.

## D. Домашні тварини — повноцінні супутники

Ми вже визначили, що тварину не можна просто покинути. Це має бути правилом системи, а не лише сюжетною обіцянкою.

Потрібно передбачити:

- воду, їжу, відпочинок, комфорт і безпечне перевезення;
- вимоги конкретного кемпінгу щодо тварин;
- переноску й тихе місце для кота;
- прогулянки, навантаження, повідець і відпочинок для собаки;
- перевірку документів і правила перевезення в різних країнах;

- ветеринарну допомогу та безпечний варіант, якщо запланований маршрут непридатний для тварини.

У гравця мають бути альтернативи: змінити кемпінг, обрати інший маршрут або відкласти поїздку. Відсутність потрібного предмета не повинна призводити до втрати улюбленця чи безвихідної ситуації.

## E. Персонажі та пам'ять NPC

Для Елізи, Нори, Люка, Інгрід, Мілана та інших персонажів потрібна спільна модель поведінки.

Розділити:

- незмінну біографію;
- поточний розклад і місце перебування;
- стосунки з гравцем;
- важливі факти пам'яті;
- стан квестів;
- тимчасові емоції чи реакції;
- доступні репліки й умови їх появи.

NPC не повинен забувати виконану обіцянку після завантаження гри, але й не потрібно зберігати кожну випадкову фразу назавжди. Зберігати слід значущі факти, що впливають на майбутні взаємодії.

Також потрібні перевірки розкладів: NPC не може одночасно бути в майстерні й на іншому кінці міста, а сюжетний персонаж не повинен зникнути саме тоді, коли він необхідний для обов'язкової цілі.

## F. Економіка та відсутність платної переваги

Внутрішні гроші, Traveler Miles, досягнення, репутація й матеріальні винагороди мають бути різними поняттями.

- Ігрові гроші використовуються в економіці подорожі.
- Traveler Miles — винагороди за визначені ігрові та спільнотні дії, якщо ця система буде реалізована.
- Досягнення фіксують виконані цілі.
- Репутація відображає якість внеску в спільноту, а не кількість витрачених грошей.
- Матеріальні призи — окрема система з юридичними й операційними вимогами.

Жодна покупка за реальні гроші не повинна давати приховану перевагу в рейтингах або пришвидшувати проходження. Лайки не повинні автоматично перетворюватися на валюту: це стимулювало б накрутку.

## G. Доступність, локалізація та керування

Навіть якщо перша версія підтримує лише клавіатуру й мишку, архітектура інтерфейсу має не блокувати подальше додавання контролера чи сенсорного керування.

Передбачити:

- зміну розміру тексту й достатній контраст;
- клавіатурну навігацію та помітний фокус;
- зрозумілі помилки й підтвердження небезпечних дій;
- паузу або безпечне призупинення складних взаємодій;
- локалізовані тексти без зашитих у код фраз;
- формати дат, часу, відстані, маси й валюти;
- адаптацію довжини тексту в меню та діалогах;
- субтитри, регулювання гучності й можливість вимкнути анімації, де це доречно.

Важливо: підтримка різних мов — це не тільки переклад діалогів. Це ще й форматування даних, шрифти, перенесення тексту, термінологія й тестування інтерфейсу на кожній підтримуваній мові.

# 7. Що ще потрібно вирішити до великого обсягу розробки

Ці рішення не варто вигадувати мовчки. Їх потрібно зафіксувати в `DECISION_LOG.md`, а після вибору — відобразити в технічній специфікації.

| Питання | Рекомендована початкова позиція |
| --- | --- |
| Рушій | TypeScript + Phaser як кандидат для браузерної 2D-гри; перевірити вертикальним прототипом |

| | |
|---|---|
| Збереження | Локальне збереження з версіонуванням, резервною копією та перевіркою відновлення |
| Сервер | Не потрібен для базового одиночного проходження; потрібен для майбутніх облікових записів і перевірених онлайн-винагород |
| Карта | Не намагатися відтворити всю Європу детально; почати з обмеженої кількості перевірених регіонів |
| Реальні дані | Ліцензійний реєстр і перевірка джерел до інтеграції в продукт |
| AI у грі | Спочатку авторизовані діалоги й контент; генеративний AI не повинен бути критично необхідним для проходження |
| Спільнота | Вмикати публічні публікації лише після створення модерації, скарг і правил приватності |
| Рейтинги | Добровільна участь, окремі категорії, серверна перевірка результатів |
| Монетизація | Без pay-to-win, платного пришвидшення та платних блокувань прогресу |
| Реліз | Спочатку браузерний прототип, потім закрите тестування, далі рішення щодо публічного релізу |

| | |
|---|---|
| Платформи | Зафіксувати підтримувані браузери, мінімальні пристрої й методи керування після вимірювання прототипу |
| Дистрибуція | Визначити, де зберігаються дані, як оновлюється гра і що відбувається зі старими версіями |

Це рекомендовані стартові рішення, а не твердження, що вони вже остаточно затверджені або реалізовані.

## 8. Новий документ: реєстр рішень, невідомого та залежностей

Я б додав до репозиторію окремий файл `docs/OPEN_QUESTIONS_AND_DEPENDENCIES.md`. Його роль — не допустити, щоб важливі питання загубилися серед сотень задач.

# OPEN_QUESTIONS_AND_DEPENDENCIES.md

## Призначення

Цей документ містить рішення, які потрібно прийняти, невідомі факти, зовнішні залежності та умови, без яких не можна безпечно перейти до наступного етапу.

## Правила ведення

Кожен запис повинен містити:

- ID;
- питання або невідомий факт;
- чому це важливо;
- доступні варіанти;
- рекомендований варіант і його обґрунтування;
- залежні системи;
- спосіб перевірки;
- відповідального;

- статус: OPEN, INVESTIGATING, DECIDED, VERIFIED, BLOCKED;
- дату рішення та посилання на відповідний запис у DECISION_LOG.md.

Не позначати питання як VERIFIED лише тому, що AI запропонував переконливу відповідь. Для перевірки потрібні джерела, тест або фактичний результат.

## Початковий реєстр

### ARCH-001 — Ігровий рушій

Перевірити Phaser і TypeScript на прототипі з tilemap, колізіями, переходами між сценами, збереженням і доступністю інтерфейсу.

Критерій закриття: робочий прототип, вимірювання продуктивності, перелік обмежень і зафіксоване архітектурне рішення.

### SAVE-001 — Стратегія збережень

Визначити формат, резервні копії, відновлення, версіонування та міграції.

Критерій закриття: автоматизовані тести збереження, завантаження, пошкодженого файлу, повторного завантаження та міграції старої версії.

### DATA-001 — Джерела реальної географії

Визначити джерела, ліцензії, атрибуцію, правила оновлення та межі використання.

Критерій закриття: затверджений реєстр джерел і перевірені правила інтеграції.

### GAME-001 — Межі першої версії

Визначити мінімальну завершену подорож, обов'язкові механіки й те, що відкладається.

Критерій закриття: погоджений список функцій першого релізу та критерії приймання.

### WORLD-001 — Час, погода та симуляція

Визначити, які процеси працюють під час руху, діалогу, меню, сну, переходу між локаціями та закритої вкладки браузера.

Критерій закриття: документована модель часу, детерміновані тести та відсутність суперечливих станів після завантаження.

### QUEST-001 — Життєвий цикл квестів

Визначити початок, прогрес, відмову, провал, відкладення, завершення, повторення та міграцію квесту після зміни контенту.

Критерій закриття: валідатор графа квестів і тестування обов'язкових та необов'язкових гілок.

### ECON-001 — Економіка та винагороди

Розділити внутрішні гроші, Traveler Miles, репутацію, досягнення та матеріальні винагороди.

Критерій закриття: правила нарахування, обмеження повторів, журнал транзакцій і тести проти дублювання винагород.

### PET-001 — Безпека тварин

Визначити стан, потреби, перевезення, допустимі кемпінги, ветеринарні ситуації та шляхи відновлення.

Критерій закриття: кожен критичний сценарій має безпечне продовження без покинення тварини.

### NPC-001 — Пам'ять і розклади NPC

Визначити факти пам'яті, часові правила, доступність персонажів та взаємодію з квестами.

Критерій закриття: тести узгодженості стану NPC після переходів і завантаження.

### WEB-001 — Межа між грою та сайтом

Визначити, які дані залишаються локальними, які синхронізуються, які є публічними, а які приватними.

Критерій закриття: схема даних, правила авторизації, експорту та видалення.

### SEC-001 — Модель загроз

Визначити цінні активи, довірені межі, загрози, захисні заходи та перевірки.

Критерій закриття: пріоритизований реєстр ризиків, визначені критерії релізу та задокументовані результати перевірок.

### OPS-001 — Відновлення сервісу

Визначити резервне копіювання, відновлення, сповіщення, процедуру інциденту та відповідальних.

Критерій закриття: успішна практична перевірка відновлення на ізольованому середовищі.

### COMMUNITY-001 — Безпечна спільнота

Визначити правила публікацій, скарг, блокувань, модерації, апеляцій та боротьби зі спамом.

Критерій закриття: модераційний процес перевірений до відкриття публікацій користувачів.

### COMMERCE-001 — Матеріальні товари та призи

Визначити, чи входять магазин, друк на вимогу, пожертви та фізичні призи до початкового релізу чи майбутніх етапів.

Критерій закриття: затверджена модель відповідальності, витрат, доставки, повернень, податків і юридичної перевірки.

### RELEASE-001 — Публічний реліз

Визначити мінімальні критерії якості, список відомих проблем, підтримувані середовища та план відкату.

Критерій закриття: пройдено всі обов'язкові релізні перевірки й немає відкритих блокувальних дефектів.

## Правило переходу між етапами

Етап не переходить у завершений статус, якщо його обов'язкові питання залишаються OPEN або BLOCKED. Виняток можливий лише за документованим рішенням із поясненням ризику, відповідальним і планом його закриття.

## 9. Новий рівень QA: перевірка не лише помилок, а й якості гри

Є різниця між грою, яка технічно працює, і грою, у яку хочеться повернутися.

Тому додаємо до `QA_STRATEGY.md` окремий напрям — UX та ігрове тестування.

| Напрям | Що перевіряти |
|---|---|
| Перші 10 хвилин | Чи розуміє новий гравець мету, керування та першу дію |
| Перша подорож | Чи можна завершити маршрут без допомоги розробника |
| Економіка | Чи зрозуміло, на що витрачаються гроші та як уникнути неприємного фінансового тупика |
| Інвентар | Чи очевидно, які предмети потрібні, чого бракує і що можна залишити вдома |
| Діалоги | Чи має вибір зрозумілі наслідки та чи не повторюються беззмістовні репліки |
| NPC | Чи помічає гравець, що персонажі пам'ятають важливі вчинки |
| Темп | Чи чергуються подорож, дослідження, спілкування та відпочинок |

| | |
|---|---|
| Комфорт | Чи не перетворюються голод, втома, бюджет і спорядження на нескінченну рутину |
| Повторне проходження | Чи є сенс обрати інший тип подорожі, транспорт, склад сім'ї або стиль гри |
| Доступність | Чи можна виконати основні дії без залежності від кольору, точного наведення або швидкої реакції |

Не варто відразу встановлювати довільні KPI на кшталт «усі повинні грати 30 хвилин» або «80% мають повернутися наступного дня». Спочатку потрібні реальні спостереження, добровільний зворотний зв'язок і розуміння, що саме робить гру цікавою.

Ми також не повинні збирати зайві персональні дані лише заради аналітики. Для багатьох питань достатньо добровільних відгуків тестувальників, локальних журналів помилок і знеособлених агрегованих показників.

## 10. Що я пропоную остаточно зафіксувати в нашій стратегії

Ось принципи, які мають керувати подальшою роботою над усіма вже обговореними системами:

1. Основна гра важливіша за сайт і спільноту. Спочатку цікава одиночна подорож, потім соціальні функції.
2. Немає незворотних пасток. Обов'язкові квести, домашні тварини й основний прогрес мають безпечні шляхи відновлення.
3. Реалізм має навчати, а не мучити. Симуляція повинна допомагати приймати зрозумілі рішення, а не змушувати керувати десятками дрібних показників.
4. Кожна значуща дія має узгоджені наслідки. Купівля, ремонт, подорож, діалог і винагорода мають коректно впливати на спільний стан світу.
5. Реальні факти відділені від ігрових припущень. Джерела, ліцензії та дати перевірки не губляться в контенті.
6. AI допомагає створювати й перевіряти, але не підмінює докази. Результати тестів повинні бути фактичними, а невідоме — явно позначеним.

7. Безкоштовна модель не повинна карати за відсутність. Жодних втрачених серій, штучної енергетики чи обов'язкових соціальних публікацій.
8. Винагороди не можна легко накручувати. Онлайн-результати перевіряються сервером, повторні операції контролюються, а репутація не дорівнює кількості лайків.
9. Ми не обіцяємо того, чого ще не можемо забезпечити. Це стосується фізичних призів, реальних маршрутних даних, сумісності платформ і майбутніх функцій.
10. Кожен реліз повинен бути відновлюваним. Є резервні копії, список відомих проблем, критерії відкату й план реагування.
11. Нова функція має виправдовувати свою складність. Якщо вона не покращує подорож, історію, навчання або стабільність, її можна відкласти.
12. Документація відображає фактичний стан проєкту. Не плутати задумане, написане, протестоване й випущене.

## 11. Що вже зроблено в межах нашого плану, а що ще ні

Щоб не створювати ілюзію, ніби весь проєкт уже завершено, важливо розділити результати.

Опрацьовано в нашій документації

Основну концепцію, базовий ігровий цикл, модель світу, персонажів і сюжетні приклади, системи подорожі, економіки, спорядження, домашніх тварин, NPC, квестів, прогресії, спільноти, монетизації, тестування, безпеки та релізної підготовки. Додано нові вимоги до контролю якості й реєстр відкритих питань.

Потрібно перевірити на реальному проєкті

Які файли вже є, що реалізовано, які тести проходять, чи є конфлікти в моделях даних, чи працює збереження та чи не дублюються системи. Тексти, створені тут, самі собою не змінюють файли у твоєму репозиторії.

Потрібно реалізувати й довести

Завершений вертикальний зріз, автоматичний валідатор контенту, тести переходів станів, міграції збережень, практичне відновлення резервної копії, реальне закрите тестування та перевірку безпеки перед запуском онлайн-функцій.

Мій висновок: тепер нам потрібне не безкінечне накопичення нових розділів, а один узгоджений комплект документації, який не суперечить сам собі й може

безпосередньо керувати розробкою. Найбільша наступна користь буде від фактичного аудиту репозиторію, закриття конфліктів і створення першої завершеної подорожі з перевіреним збереженням.

Абсолютно гарантувати відсутність усіх помилок неможливо. Але можна зробити так, щоб важливі помилки було значно важче пропустити, прогрес було легше відновити, а кожне наступне оновлення не перетворювалося на ризик для всієї гри.

якщо нема що більше тобі додати, але я впевнений що є то додай а потім вже фіналізуй все і зроби фінальний звіт де все буде написано і деталізовано для ШІ Клод і його аналізу. Я передам повний наш чат для його ознайомлення і він має сказати чи він може таку гру осилити чи ні, тому пиши все фіналізовано і далі в кінці вже всього напиши саме для ШІ Клод те що ми вже узгодили, що маємо і чого не вистачає і що він має зробити, він чекає як і я від тебе роботу на повну силу і без прогалин і неякісної роботи.

## 12. Фінальний пакет для Claude: як читати й оцінювати наш проєкт

Перш ніж передавати матеріали, встановімо важливе правило: Claude має не просто сказати, чи здатний він писати код. Він має оцінити, чи можна довести цей задум до якісного продукту за допомогою AI-розробки, які частини реально автоматизувати, які потребують людських рішень і що потрібно скоротити або перевірити до початку масштабної роботи.

Також я додаю в пакет не лише наші ідеї, а й те, чого ще не можна вважати готовим: немає підтвердженого аудиту репозиторію, перевіреної архітектури, фактичних результатів тестування чи затвердженої оцінки бюджету й термінів. Claude повинен перевірити ці речі, а не заповнювати прогалини вигаданими висновками.

Нижче — основний документ передачі проєкту. Він має бути для Claude робочою основою, а історія нашого чату — додатковим контекстом.

# PROJECT HANDOFF & FEASIBILITY REVIEW

## Фінальний пакет концепції, вимог, ризиків і завдання для Claude

**Версія документа:** 1.0
**Дата:** 22 жовтня 2026 року
**Мова проєкту:** робоча документація українською; мова коду та технічних ідентифікаторів — англійська
**Тип продукту:** одиночна 2D-гра про кемпінг, подорожі Європою, побут, транспорт, взаємодію з людьми й відповідальне планування
**Основна платформа:** браузер
**Можливе майбутнє розширення:** ПК-версія, вебсайт, журнал подорожей, спільнота, рейтинги та матеріальні товари
**Поточний статус:** концепція й вимоги розробляються; фактичний стан репозиторію не перевірений
**Призначення документа:** передати Claude цілісний опис задуму для незалежної оцінки здійсненності, складності, ризиків, архітектури та наступних кроків

---

# ЧАСТИНА І. ЩО МИ БУДУЄМО

## 1. Короткий опис

Ми хочемо створити оригінальну 2D-гру про кемпінг і подорожі, натхненну привабливістю затишних симуляторів на кшталт Stardew Valley, але без копіювання її світу, персонажів, мистецтва, діалогів, сюжетів або конкретних механік.

Гра має поєднувати:

- дослідження європейських локацій;
- планування маршрутів і бюджету;
- підготовку до подорожей;
- транспорт, ремонт і технічні проблеми;
- вибір спорядження;
- кемпінг, побут і відпочинок;
- знайомства з унікальними NPC;
- розгалужені історії та побічні завдання;
- сімейні подорожі, друзів і домашніх тварин;
- реалістичні, але не надмірно каральні наслідки рішень;
- розвиток навичок, особисті цілі та досягнення.

У майбутньому гру можна пов'язати з окремим сайтом, де гравці матимуть профілі, журнал подорожей, атлас місць, досягнення, добровільні рейтинги й можливість публікувати історії.

Головна відмінність продукту — не просто «гра про виживання», а **затишний симулятор самостійної подорожі, який навчає планувати реальні поїздки, приймати зважені рішення та знайомитися з людьми через історії**.

## 2. Основна філософія

1. Гра повинна бути цікавою сама по собі, без сайту, рейтингів і соціальних мереж.
2. Реалізм має підтримувати цікаві рішення, а не створювати нескінченну рутину.
3. Не передбачено смерті персонажа. Можливі травми, затримки, поломки, витрати, допомога й відновлення.
4. Обов'язковий сюжет не повинен мати невиправних тупиків.
5. Домашніх тварин не можна покинути заради прогресу.
6. Жодних pay-to-win, платного пришвидшення, платної енергії, лутбоксів або штучних блокувань прогресу.
7. Винагороди за спільнотну активність не повинні залежати від позитивних відгуків, кількості накручених лайків або примусу до публікацій.
8. AI допомагає розробляти, тестувати й підтримувати продукт, але не підміняє фактичні результати тестування вигаданими.
9. Усі реальні дані мають мати джерела, ліцензії та зрозумілі межі точності.
10. Архітектура повинна підтримувати поступове розширення без необхідності щоразу переписувати основні системи.

---

# ЧАСТИНА II. ІГРОВИЙ ДИЗАЙН

## 3. Візуальний стиль і камера

- 2D pixel art.
- Камера зверху під кутом, у стилі top-down/three-quarter, а не строго плаский вигляд зверху.
- Вільне переміщення персонажа в локальних сценах.
- Колізії, взаємодії з предметами, входи до будівель, переходи між локаціями.
- Читабельні об'єкти, персонажі й навколишнє середовище.

- Окремі UI-екрани для складних систем: інвентар, бюджет, спорядження, карта, маршрут, журнал, персонаж, квести.
- Перспектива, масштаб спрайтів, розмір тайлів, палітра, анімації й правила колізій мають бути визначені до масового створення графіки.

Не використовувати чужі спрайти або графіку без відповідної ліцензії. Не копіювати впізнавані художні рішення конкретної гри настільки, щоб продукт виглядав як її клон.

# 4. Три масштаби світу

## 4.1. Карта Європи

Відповідає за:

- країни й регіони;
- напрямки подорожей;
- доступні локації;
- відкриття нових місць;
- загальний вибір наступної подорожі.

Не потрібно одразу створювати повністю деталізовану інтерактивну карту всієї Європи.

## 4.2. Маршрут і дорога

Відповідає за:

- відстань;
- тривалість поїздки;
- дорожні події;
- платні дороги й пороми;
- витрату пального;
- обмеження транспортного засобу;
- зупинки, затримки й зміну маршруту;
- оцінку вартості та часу прибуття.

## 4.3. Локальні сцени

Приклади:

- будинок і подвір'я;
- магазин спорядження;
- майстерня;

- автодім або кемпер;
- кемпінг;
- місто;
- туристична стежка;
- поромний термінал;
- заправка;
- пункт заряджання;
- ринок;
- туристичний інформаційний центр.

Тут гравець рухається, взаємодіє з предметами, розмовляє з NPC, виконує квести та розв'язує локальні ситуації.

### 4.4. Єдиний стан світу

Усі масштаби повинні використовувати узгоджений стан гри:

- календар і час;
- місце персонажа;
- маршрут;
- бюджет;
- інвентар;
- транспорт;
- спорядження;
- квести;
- стосунки та пам'ять NPC;
- стан домашніх тварин;
- погода;
- бронювання;
- події та історія рішень.

Локальна сцена не повинна самостійно змінювати квест, гроші чи інвентар поза спільними правилами гри.

## 5. Основний ігровий цикл

1. Визначити мету подорожі.
2. Обрати напрямок.
3. Перевірити бюджет.
4. Обрати транспорт і маршрут.
5. Підготувати спорядження.
6. Перевірити транспорт і потреби супутників.
7. Вирушити в дорогу.
8. Реагувати на події, затримки та можливості.

9. Прибути до локації.
10. Розбити табір або скористатися іншою формою ночівлі.
11. Спілкуватися, досліджувати місцевість, виконувати завдання.
12. Відпочити, відновитися, полагодити речі.
13. Оцінити витрати, вивчений досвід і наслідки рішень.
14. Обрати наступний етап подорожі.

Основний цикл має бути зрозумілим уже в першій короткій подорожі.

## 6. Потреби, небезпеки й травми

Параметри персонажа:

- голод;
- спрага;
- втома;
- тепловий комфорт;
- мокрий одяг;
- стрес;
- фізична форма;
- травми.

Рівні стану: нормальний, попередження, критичний.

Потрібно уникати надмірного мікроменеджменту. Кожен показник має впливати на рішення, а не просто вимагати регулярного натискання кнопок.

Можливі ситуації:

- розтягнення;
- падіння;
- перегрівання;
- сильний вітер;
- несправність транспортного засобу;
- невідповідний маршрут;
- нестача води;
- пошкодження спорядження.

Варіанти реагування:

- зупинитися;
- відпочити;
- змінити маршрут;
- відремонтувати предмет;

- звернутися до майстерні;
- викликати допомогу;
- переночувати в безпечному місці;
- змінити плани.

Необхідні попередження до небезпечних рішень. Не можна створювати невидимі правила, які раптово роблять проходження неможливим.

## 7. Навички персонажа

Вісім основних навичок:

1. Планування та навігація.
2. Кемпінг і встановлення табору.
3. Механіка.
4. Електрика та автономне живлення.
5. Харчування й запаси.
6. Фінанси й торгівля.
7. Фізична форма й безпека.
8. Комунікація та соціальні взаємодії.

Навички покращуються через практику й навчання, із поступовим зменшенням приросту.

Висока навичка не повинна давати магічної невразливості. Інструменти, сертифікація, фізичні можливості та реальні обмеження залишаються важливими.

## 8. Створення персонажа

Гравець визначає:

- зовнішність;
- особистісні риси;
- професію;
- робочий графік;
- сім'ю та стосунки;
- стартові ресурси;
- особисті цілі;
- початковий набір навичок.

Профілі для початкової системи:

1. Самостійний дослідник.

2. Пара, що живе разом.
3. Самостійний мандрівник із друзями.
4. Сім'я з підлітком.
5. Найманий механік.
6. Самозайнятий механік або торговець транспортом.
7. Віддалений працівник.
8. Працівник зі змінним графіком.
9. Пенсіонери, що подорожують удвох.
10. Студент.
11. Один із батьків із дитиною.
12. Велотурист.

Початкові ресурси мають відрізнятися, але всі стартові профілі повинні мати життєздатний шлях до першої подорожі.

## 9. Сім'я та домашні тварини

Один із прикладів сім'ї, який уже обговорювався:

- George, 42 роки;
- Victoria, 42 роки;
- Matthew, 15 років;
- Mia, кішка породи Scottish Fold, 5 років.

Це приклад, а не обов'язковий головний герой.

Гравець може обирати:

- самостійну подорож;
- подорож із партнером;
- подорож із друзями;
- сімейну подорож;
- подорож із дитиною;
- подорож із домашньою твариною.

Домашні тварини:

- малі, середні та великі собаки;
- коти з відповідними потребами;
- харчування, вода, відпочинок, простір;
- правила перевезення;
- вимоги кемпінгів;
- повідець і прогулянки для собак;
- переноска, лоток і спокійне місце для котів;

- ветеринарні документи й допомога.

Тварина — повноцінний супутник, якого не можна покинути, щоб спростити проходження.

## 10. Транспорт і економіка

Можливі типи подорожей:

- автомобіль;
- кемпер або автодім;
- велосипед;
- комбінований маршрут із поромами та іншими видами транспорту.

Параметри транспорту:

- стан;
- пробіг;
- витрата пального;
- навантаження;
- ремонт;
- придатність до конкретного маршруту;
- обмеження висоти й маси;
- обладнання;
- автономне живлення;
- водопостачання;
- холодильно-опалювальні системи.

Приклад формули пального:

```
fuel_cost = distance_km * consumption_l_per_100km / 100 * price_per_liter
```

Приклад: 400 км, 7 л/100 км, €1,80 за літр дають €50,40.

Це базова модель. Додаткові чинники, як-от навантаження, рельєф і погода, можна вводити пізніше, якщо вони справді покращують ігрові рішення.

Бюджет подорожі:

- пальне;
- ночівлі;
- харчування;
- платні дороги;

- пороми;
- ремонти;
- покупки;
- додаткові витрати.

Вживаний транспорт має враховувати стан, вік, ринкову вартість, дефекти та витрати на ремонт. Формула вартості є ігровою моделлю, а не гарантією реальної ринкової ціни.

## 11. Спорядження й інвентар

Категорії:

- намети;
- спальники;
- килимки;
- одяг;
- дощовики;
- ліхтарі;
- кухонне спорядження;
- їжа й вода;
- аптечка;
- інструменти;
- електричне обладнання;
- велосипеди й запчастини;
- спорядження для домашніх тварин;
- ремонтні матеріали.

Потрібні:

- місткість інвентарю;
- маса;
- сумісність предметів;
- знос;
- ремонт;
- витрати ресурсів;
- покупки й продажі;
- обмеження для конкретних дій;
- перевірка предметів, необхідних для квестів.

Не можна допустити, щоб обов'язковий предмет можна було безповоротно продати або втратити без альтернативного шляху.

# 12. NPC та пам'ять

Усі NPC мають бути людьми з індивідуальними:

- іменами;
- зовнішністю;
- біографією;
- професією;
- розкладом;
- цілями;
- майном;
- характером;
- стосунками;
- пам'яттю;
- історіями та квестами.

Не можна просто перейменовувати одного персонажа й використовувати його як різних NPC.

### Приклади вже створених персонажів

**Elise Vermeulen, 51** — власниця кемпінгу.

Можливі квести:

- «Правила без дрібного шрифту»;
- «Тиха ділянка»;
- «Гість, який не повернувся»;
- «Кемпінг для різних поколінь».

**Nora De Smet, 28** — велотуристка й фотографка.

Можливі квести:

- «Кадр до дощу»;
- «Дорога без узбіччя»;
- «Фото чи попередження?»;
- «Історія за знімком».

**Luc Bernard, 46** — велосипедний механік.

Теми: проколи, звуки велосипеда, ремонт і підготовка до маршруту.

**Ingrid Mertens, 63** — досвідчена мандрівниця, що зберігає старі карти.

Можливі квести:

- «Позначки на полях»;
- «Міст, якого вже немає»;
- «Запис без дати»;
- «Що варто зберегти?».

**Milan Peeters, 31** — віддалений працівник у кемпері.

Теми: зв'язок, батарея, робочі зустрічі й баланс між роботою та подорожами.

**Sarah Lambert, 37** — місцева гідеса.

Теми: відповідальний туризм і взаємодія з місцевими жителями.

**Thomas Legrand, 44** — працівник поромного термінала.

Теми: час посадки, квитки, черги й затримки.

**Claire Dubois, 39** — ветеринарка.

Теми: безпечне перевезення тварин, документи й ветеринарна допомога.

Додаткові запропоновані персонажі:

- Marta Koval, 34 — координаторка Wayfarers' Circle;
- Benoît Richard, 55 — власник майстерні кемперів;
- Amina Haddad, 26 — сезонна працівниця туристичного інформаційного центру;
- Pavel Novak, 71 — колишній водій туристичного автобуса.

Ці персонажі та імена — контентні пропозиції, які потрібно перевірити на узгодженість і відсутність дублювання в майбутньому реєстрі NPC.

## Пам'ять NPC

Зберігати значущі факти:

- PlayerMetNpc;
- PlayerHelpedNpc;
- PlayerMadePromise;
- PlayerKeptPromise;
- PlayerBrokePromise;
- SharedEventExperienced;
- NpcQuestCompleted.

Не потрібно зберігати кожну випадкову репліку. Потрібно зберігати факти, що впливають на майбутні реакції.

Діалоги спочатку авторизовані, контрольовані й тестовані. Генеративний AI не повинен бути обов'язковим для основного проходження.

## 13. Квести й сюжет

Квестова система повинна підтримувати:

- передумови;
- обов'язкові й необов'язкові цілі;
- розгалуження;
- відкладені квести;
- перевірки предметів, часу, місця та стосунків;
- винагороди;
- відкриття наступних подій;
- одноразові та повторювані квести;
- відмову;
- завершення;
- збереження стану;
- міграцію старих квестів після оновлення контенту.

Кожен обов'язковий квест має досяжний шлях завершення. Необов'язкова відмова не блокує головну історію.

### Стартова кампанія

Робоча назва: «Вихідні, які пішли не за планом».

Цільова тривалість першого проходження: 60–90 хвилин. Це гіпотеза для тестування, а не доведена метрика.

Етапи:

1. ST-001 — створення персонажа.
2. ST-002 — ідея подорожі.
3. ST-003 — перевірка бюджету.
4. ST-004 — магазин спорядження.
5. ST-005 — перевірка перед виїздом.
6. ST-006 — перший відрізок маршруту.
7. ST-007 — невелика затримка.
8. ST-008 — прибуття до кемпінгу й зустріч з Елізою.
9. ST-009 — облаштування табору.

10. ST-010 — перша зустріч із Норою.
11. ST-011 — вечірній вибір.
12. ST-012 — мініквест із загубленою річчю.
13. ST-013 — необов'язкова місія ECO-001.
14. ST-014 — розмова перед від'їздом.
15. ST-015 — вибір наступної подорожі.

Перші 90 хвилин мають бути привітними: без каральної погоди, критичного дефіциту бюджету, непередбачуваних поломок і складного інвентарного мікроменеджменту.

Необхідні варіанти для транспорту, сімейного складу й домашніх тварин, а також перевірка збереження на різних етапах.

## 14. Події світу

Каталог, запропонований для подальшої розробки:

- POS-001–012: позитивні та побутові можливості;
- LIFE-001–012: щоденні ситуації;
- BIKE-001–012: велосипедні ситуації;
- VEH-001–012: транспорт і кемпери;
- TRIP-001–012: дорожні події;
- SOC-001–012: соціальні ситуації;
- PET-001–006: події з домашніми тваринами;
- GEAR-001–008: предмети та спорядження.

Приклади: прокол колеса, слизька дорога, зламана блискавка намету, несправність холодильника, затримка порома, переповнений кемпінг, брак води, загублений повідець, дощ, місцевий фестиваль, знайомство з мандрівником.

Це каталог тем, а не готові сценарії. Кожна реалізована подія потребує передумов, варіантів дій, наслідків, перевірок, обмежень повторення й шляху відновлення.

## 15. Екологічні та соціальні системи

Екологічні дії — необов'язкова частина гри.

Можливі механіки:

- прибирання після себе;
- сортування сміття;
- допомога кемпінгу;

- турбота про природні стежки;
- відповідальне ставлення до місцевих правил.

Не потрібно карати гравця за відсутність необов'язкової екологічної активності. Правила сортування залежать від місця, тому їх потрібно прив'язувати до перевірених локальних даних.

Система може називатися «Турбота» або «Care/Footprint», але не повинна бути каральною системою карми.

---

# ЧАСТИНА III. ПРОГРЕСІЯ, СПІЛЬНОТА Й МАЙБУТНЯ ЕКОСИСТЕМА

## 16. Прогресія й досягнення

Напрями розвитку:

- навички;
- відкриття регіонів;
- історії NPC;
- особисті цілі;
- спорядження та база;
- колекціонування;
- атлас подорожей.

Для навичок запропоновано шість рангів — від Novice до Mentor.

Категорії досягнень:

- майстерність;
- дослідження;
- людяність;
- колекціонування;
- кампанія;
- стиль життя.

Досягнення мають відображати реальні дії, а не змушувати до повторюваного гринду.

## 17. Traveler Miles та винагороди

Запропоновані категорії:

- внутрішні ігрові гроші;
- Traveler Miles;
- медалі та досягнення;
- репутація за внесок у спільноту;
- добровільні рейтинги;
- майбутні матеріальні призи.

Попередні приклади можливих винагород:

- перша історія — 100 miles;
- новий розділ історії — 50–150;
- нова локація — 10–25;
- навчальний етап — 10–40;
- перший ремонт велосипеда — 25;
- особиста ціль — 20–100;
- публікація в журналі — 0–20 після перевірки якості;
- корисна відповідь — 0–15 з обмеженнями.

Ці цифри є чернетковими, не затвердженим балансом.

Не винагороджувати простий щоденний вхід, накрутку лайків або позитивні відгуки на зовнішніх платформах. Не вимагати публікацій у соцмережах.

## 18. Сайт і журнал мандрівника

Можливі розділи:

- Home;
- Game;
- Living Journal;
- Route Atlas;
- Community;
- Profile & Achievements.

Журнал може пропонувати чернетку історії на основі фактичних подій у грі, але користувач повинен мати змогу відредагувати текст перед публікацією. AI не повинен вигадувати події чи почуття від імені користувача.

Типи публікацій:

- подорож;
- уроки та висновки;
- спорядження;
- спостереження про маршрут;
- кумедні історії;
- історії персонажів;
- запитання спільноті.

Публікації не повинні автоматично використовуватися для реклами або сторонніх цілей без належної прозорості та правової підстави.

## 19. Рейтинги та антизловживання

Можливі категорії рейтингів:

- Explorer;
- Master;
- Storyteller;
- Good Neighbor;
- Road Expert;
- Seasonal Traveler.

Участь добровільна. Необхідно:

- перевіряти онлайн-винагороди на сервері;
- обмежувати повторні операції;
- виявляти накрутку й автоматизовані зловживання;
- не використовувати кількість лайків як єдину міру якості;
- забезпечити скарги, блокування й модерацію;
- не публікувати приватні дані без дозволу.

## 20. Майбутні матеріальні товари

Можливий майбутній напрям:

- Shopify або інша платформа магазину;
- друк на вимогу;
- одяг і сувеніри;
- товари, пов'язані з грою;
- добровільні пожертви;
- спонсорство;
- матеріальні призи за окремими правилами.

Необхідно окремо розглянути доставку, повернення, гарантії, податки, правила конкурсів, страхування, права споживачів і юридичні вимоги в країнах учасників.

Не обіцяти транспортні засоби, кемпери або великі призи до юридичної, фінансової та операційної перевірки.

---

# ЧАСТИНА IV. ТЕХНІЧНА АРХІТЕКТУРА

## 21. Рекомендована стартова архітектура

Попередня рекомендація: TypeScript і Phaser для браузерної 2D-гри. Це кандидат на перевірку, а не остаточно затверджений вибір.

Потрібно перевірити:

- роботу tilemap;
- колізії;
- переміщення;
- анімації;
- інтерфейс;
- переходи між сценами;
- збереження;
- підтримувані браузери;
- продуктивність на мінімальних цільових пристроях;
- доступність;
- потенційний шлях до ПК-версії.

Починати з модульного моноліту, а не з мікросервісів.

### Логічні шари

1. UI та сцени.
2. Команди й правила гри.
3. Ядро ігрової логіки.
4. Стан і збереження.
5. Зовнішні адаптери: сервер, сайт, аналітика, синхронізація.

Приклади команд:

- PurchaseItem;
- UseItem;

- StartQuest;
- CompleteObjective;
- CompleteQuest;
- ChangeRoute;
- RepairVehicle;
- RecordEcoAction;
- SaveGame;
- RestoreSave.

Кожна команда повинна мати визначені передумови, зміни стану, побічні ефекти, помилки, ідемпотентність і тести.

## 22. Структура репозиторію — цільовий варіант

```
project/
  apps/
    game/
    website/
    server/
  packages/
    game-data/
    game-rules/
    shared-types/
  content/
    quests/
    dialogue/
    npcs/
    locations/
    items/
    events/
    achievements/
  docs/
    GDD.md
    NARRATIVE_BIBLE.md
    TECHNICAL_SPEC.md
    ARCHITECTURE.md
    DATA_CONTRACTS.md
    SAVE_MIGRATIONS.md
    QA_STRATEGY.md
    TEST_PLAN.md
    SECURITY_PRIVACY.md
    THREAT_MODEL.md
    LICENSE_REGISTER.md
    UGC_POLICY.md
```

```
  PROJECT_ASSURANCE.md
  OPEN_QUESTIONS_AND_DEPENDENCIES.md
  PROJECT_STATUS.md
  DECISION_LOG.md
  RELEASE_CHECKLIST.md
  INCIDENT_RESPONSE.md
  BACKUP_RECOVERY.md
tests/
  unit/
  integration/
  regression/
```

Це пропозиція щодо структури, а не твердження, що відповідні файли вже створені.

Не потрібно механічно створювати кожен файл. Документи повинні бути корисними, мати власника й не дублювати один одного.

## 23. Джерела істини

Потрібно розділити:

- статичні визначення контенту;
- стан проходження;
- локальні дані збереження;
- дані облікового запису;
- серверні винагороди;
- публічний профіль;
- публікації журналу;
- репутацію;
- модерацію.

Одиночна гра не повинна залежати від постійного доступу до сервера. Сервер потрібен для функцій, де потрібна довіра: онлайн-винагород, рейтингів, облікових записів і спільноти.

## 24. Контракти даних

Передбачити типізовані моделі:

- PlayerCharacter;
- Household;
- Employment;

- BankAccount;
- Inventory;
- CampingItem;
- Vehicle;
- Trip;
- Location;
- WeatherState;
- Incident;
- SkillProfile;
- WorldCalendar;
- Pet;
- NPCDefinition;
- NPCState;
- QuestDefinition;
- QuestState;
- EventDefinition;
- EventState;
- Reservation;
- SaveGame;
- RewardLedgerEntry;
- JournalPost;
- Account;
- PlayerProfile.

Визначення контенту повинні бути відокремлені від змінного стану гри.

Кожен запис має стабільний ID. Не використовувати назви, перекладені рядки або індекси масивів як єдині ідентифікатори.

## 25. Збереження й міграції

Обов'язкові вимоги:

- версія формату;
- перевірка структури;
- контрольовані міграції;
- резервна копія перед небезпечною міграцією;
- відновлення після пошкодженого файлу;
- атомарний запис там, де це можливо;
- тестування старих форматів;
- відсутність дублювання предметів і винагород;
- документований механізм відновлення.

Потрібно визначити, що відбувається при перезапуску браузера, закритті вкладки, втраті інтернету, помилці запису та зміні версії гри.

## 26. Валідація контенту

Автоматичний валідатор має перевіряти:

- унікальність ID;
- посилання між сутностями;
- граф квестів;
- досяжність обов'язкових цілей;
- доступність предметів;
- відсутність недосяжних NPC;
- коректність локалізації;
- наявність джерел і ліцензій;
- допустимі значення;
- цикли, що можуть породжувати нескінченні винагороди;
- міграцію стану після зміни контенту.

Валідатор повинен запускатися в CI перед злиттям змін.

---

# ЧАСТИНА V. QA, БЕЗПЕКА ТА ЕКСПЛУАТАЦІЯ

## 27. Тестування

Рекомендована піраміда:

1. Unit-тести для правил, формул та переходів станів.
2. Integration-тести для інвентарю, квестів, збереження, NPC і економіки.
3. End-to-end-тести основного проходження.
4. Ручні ігрові тести з реальними людьми.

Додатково:

- property-based testing;
- випадкові послідовності дій;
- fault injection;
- тести міграцій;

- перевірки відновлення;
- регресійні тести;
- mutation testing там, де це виправдано.

**Початкові сценарії QA**

- QA-TRIP-001 — звичайна подорож автомобілем.
- QA-TRIP-002 — відсутній необов'язковий предмет.
- QA-TRIP-003 — переповнений інвентар.
- QA-TRIP-004 — відмова від соціального квесту.
- QA-TRIP-005 — подорож із котом.
- QA-TRIP-006 — подорож із собакою.
- QA-TRIP-007 — подорож велосипедом.
- QA-TRIP-008 — збереження перед діалогом.
- QA-TRIP-009 — збереження під час квесту.
- QA-TRIP-010 — NPC недоступний.
- QA-TRIP-011 — подвійне натискання покупки.
- QA-TRIP-012 — перезавантаження сторінки.
- QA-TRIP-013 — старий формат збереження.
- QA-TRIP-014 — вихід під час критичного етапу.
- QA-TRIP-015 — сервер журналу недоступний.
- QA-TRIP-016 — невалідні дані квесту.
- QA-TRIP-017 — зміна розміру вікна.
- QA-TRIP-018 — клавіатурна навігація.
- QA-TRIP-019 — повторне отримання винагороди.
- QA-TRIP-020 — помилка збереження.

Також перевіряти повністю укомплектовану подорож, відсутність додаткового спорядження, повернення додому, повторні діалоги, різні транспортні засоби й різні типи сім'ї.

## 28. Класифікація дефектів

- P0 — критичний дефект, серйозний ризик безпеки, витоку або масової втрати даних. Реліз заборонений.
- P1 — втрата прогресу, блокування обов'язкового сюжету або критичної функції. Реліз заборонений без виправлення чи формально схваленого безпечного рішення.
- P2 — істотна проблема з обходом. Потрібні відповідальний і задокументоване рішення.
- P3 — косметична або незначна проблема.

Питання безпеки й приватності додатково оцінюються за моделлю ризику. Їх не можна автоматично знижувати лише через тимчасовий обхід.

## 29. Безпека вебсервісу

Загрози, які потрібно розглянути:

- викрадення облікового запису;
- доступ до чужого збереження;
- підробка валют і винагород;
- повторні запити для дублювання нагород;
- XSS у профілях і публікаціях;
- спам та автоматизовані зловживання;
- витік секретів;
- компрометація залежностей;
- неправильні налаштування хмарних ресурсів;
- DDoS;
- небезпечні завантаження;
- зловживання правами модератора;
- помилки видалення;
- втрата резервних копій.

Для перевірюваних вимог безпеки розглянути OWASP ASVS 5.0.0 як один із стандартів, що застосовуються до вебзастосунків і сервісів. Це рамка для перевірки, а не гарантія безпеки сама по собі.

Офіційний ресурс: https://asvs.owasp.org/

## 30. Приватність і право

Потрібно окремо перевірити застосовність вимог:

- GDPR;
- правил щодо cookies та електронної приватності;
- вимог до захисту даних дітей;
- Digital Services Act — залежно від моделі спільноти й застосовності;
- авторських прав і ліцензій;
- правил споживчих товарів і повернень;
- вимог до конкурсів, спонсорства та оподаткування;
- регулювання AI, якщо в продукті з'являться відповідні функції.

Це не юридичний висновок. Перед публічним запуском потрібна перевірка фахівцем відповідно до реальної моделі продукту та аудиторії.

Для реальних картографічних даних OpenStreetMap потрібно враховувати ліцензію та атрибуцію даних. Ліцензія даних і умови використання тайлів — не одне й те саме.

Джерело: https://www.openstreetmap.org/copyright

Для доступності вебінтерфейсу орієнтиром може бути WCAG 2.2, з урахуванням застосовних юридичних вимог.

Джерело: https://www.w3.org/WAI/standards-guidelines/wcag/

## 31. Моніторинг та відновлення

Потрібно контролювати:

- критичні помилки;
- помилки завантаження й збереження;
- невдалі міграції;
- час відповіді сервісів;
- дублювання винагород;
- підозрілі запити;
- помилки журналу;
- витрати інфраструктури.

Для невеликого прототипу не обов'язково відразу впроваджувати складну систему спостережуваності. OpenTelemetry можна розглянути пізніше, коли це буде виправдано.

Офіційний ресурс: https://opentelemetry.io/docs/

План інциденту:

1. Зафіксувати симптоми.
2. Обмежити шкоду.
3. Захистити дані.
4. Визначити масштаб.
5. Знайти першопричину.
6. Додати тест, що відтворює дефект.
7. Перевірити виправлення.
8. Випустити контрольоване оновлення.
9. Повідомити користувачів, якщо необхідно.
10. Оновити документацію та процеси.

Резервне копіювання повинно включати реальну перевірку відновлення, а не лише підтвердження, що копії створюються.

---

# ЧАСТИНА VI. РОЗРОБКА ЗА ДОПОМОГОЮ AI

## 32. Ролі AI-агента

Claude або інший агент може допомагати в ролях:

- аналітик вимог;
- архітектор;
- розробник;
- автор тестів;
- QA-аналітик;
- редактор контенту;
- помічник з безпеки;
- аналітик плейтестів;
- менеджер документації.

Одна роль не повинна автоматично вважатися незалежною перевіркою іншої. AI може написати код і тест, який пропустить ту саму помилку. Для критичних систем потрібні незалежні перевірки, інші сценарії й фактичні докази.

## 33. Стандарт роботи над задачею

Кожна задача має містити:

1. Мету.
2. Контекст і відповідні файли.
3. Обмеження.
4. Критерії приймання.
5. Перелік тестів.
6. Очікувані зміни документації.
7. Ризики й залежності.

Після виконання агент повинен надати:

- перелік змінених файлів;
- пояснення рішень;
- результати реально запущених перевірок;
- відомі проблеми;
- ризики регресії;
- оновлені документи;

- рекомендацію щодо наступної задачі.

Агент не має права стверджувати, що перевірку пройдено, якщо вона не запускалася.

## 34. Запропонований порядок розробки

**QA-0. Аудит репозиторію**

Інвентаризація файлів, коду, документації, тестів, контенту, архітектури, збережень і безпеки. Без масштабних змін коду.

**Етап 1. Контракти та архітектура**

Узгодити моделі даних, стан світу, команди, помилки й залежності.

**Етап 2. Мінімальне ядро**

Реалізувати базові правила й тестовані переходи станів.

**Етап 3. Збереження**

Реалізувати версіонування, відновлення й тести міграції.

**Етап 4. Вертикальна сцена**

Персонаж, пересування, колізії, інтерактивні об'єкти й перехід між локаціями.

**Етап 5. Інвентар та бюджет**

Покупки, предмети, обмеження й економічні інваріанти.

**Етап 6. Квести**

Почати з трьох невеликих завдань із перевіреними гілками.

**Етап 7. NPC та пам'ять**

Розклади, факти пам'яті, стосунки й контрольовані діалоги.

**Етап 8. Перша подорож**

Зібрати основний цикл і перевірити всі переходи між його системами.

**Етап 9. Додаткові механіки**

Екологічні активності, додатковий контент, навички та досягнення.

**Етап 10. QA й доступність**

Регресія, випадкові послідовності дій, помилки, інтерфейс і продуктивність.

**Етап 11. Закритий плейтест**

Спостереження за реальними гравцями, виправлення незрозумілих механік і проблем темпу.

**Етап 12. Підготовка релізу**

Відомі проблеми, відновлення, підтримка, план виправлень і рішення щодо публічного запуску.

Не потрібно переписувати вже справні системи лише заради формального проходження цього списку.

---

# ЧАСТИНА VII. ПРИКЛАДИ З ІСТОРІЇ ІНШИХ ІГОР

## 35. Чого вчитися з релізів і оновлень

Публічні історії оновлень не показують усіх внутрішніх дефектів, але допомагають побачити типові ризики.

### Stardew Valley

Важливі уроки:

- сумісність старих збережень;
- проблеми, що виникають після великих оновлень;
- дублювання предметів;
- помилки квестів та NPC;
- відмінності між платформами.

Історія версій: https://wiki.stardewvalley.net/Version_History

### Dinkum

Важливі уроки:

- перевірка перенесення даних;
- поведінка інтерфейсу в різних сценаріях;
- взаємодія мережевих функцій;
- тестування крайових випадків.

Для конкретних висновків потрібно перевіряти офіційні примітки до відповідних версій, а не покладатися лише на сторонні перекази.

### The Long Dark

Важливі уроки:

- контроль проходження сюжетних місій;
- перевірка контрольних точок;
- відсутність блокувань після завантаження;
- відмінності між платформами.

### Euro Truck Simulator 2

Важливі уроки:

- поступове вдосконалення складних симуляцій;
- зміни карти й транспортних систем;
- підтримка сумісності між оновленнями;
- збереження стабільності основного циклу.

### Загальний висновок

Не копіювати обсяг цих ігор. Перенести їхні уроки у власний процес: тестувати стан, переходи, сумісність, відновлення, продуктивність і реальну поведінку гравців.

---

# ЧАСТИНА VIII. РИЗИКИ, ВІДКРИТІ ПИТАННЯ Й КРИТЕРІЇ ГОТОВНОСТІ

## 36. Головні ризики

1. Надмірний масштаб для першого релізу.

2. Суперечності між документами.
3. Втрата прогресу через збереження або міграції.
4. Неправильний стан квестів після переходів.
5. Дублювання винагород і помилки економіки.
6. Недоступність NPC у потрібний момент.
7. Непослідовна пам'ять персонажів.
8. Невідповідність реальних даних фактичним умовам.
9. Неправильні ліцензії на карти й графіку.
10. Передчасна реалізація сайту та соціальних функцій.
11. Витік даних або зловживання онлайн-винагородами.
12. Нестача реального плейтестування.
13. Надмірна залежність від AI без незалежної перевірки.
14. Відсутність реалістичної оцінки часу, бюджету та обсягу.
15. Розростання технічної складності без доведеної потреби.

## 37. Питання, які Claude має розглянути

- Чи підходить Phaser для всіх заявлених вимог?
- Чи потрібна інша технологія?
- Які системи мають бути в MVP?
- Що слід виключити з першого релізу?
- Чи достатньо модульного моноліту?
- Який формат даних найкраще підходить для контенту?
- Як тестувати міграції та відновлення?
- Які системи можна безпечно доручити AI?
- Які задачі потребують людського рішення або зовнішнього фахівця?
- Які вимоги до продуктивності потрібно виміряти на прототипі?
- Які дані можна залишити локальними?
- Коли справді знадобляться сервер, акаунти й спільнота?
- Які ліцензійні й юридичні питання блокують публічний запуск?
- Який мінімальний бюджет і час потрібні для прототипу, MVP та повнішої версії?
- Які невідомі факти не дозволяють зараз давати надійну оцінку?

## 38. Критерії приймання вертикального зрізу

Вертикальний зріз вважається готовим лише після перевірки такого сценарію:

1. Створити персонажа.
2. Обрати транспорт.
3. Скласти простий бюджет.
4. Купити необхідне спорядження.
5. Перевірити інвентар.

6. Вирушити в дорогу.
7. Прибути до першої локації.
8. Взаємодіяти з NPC.
9. Виконати невеликий квест.
10. Зберегти гру.
11. Перезапустити її.
12. Відновити стан.
13. Перевірити інвентар, гроші, квест і NPC.
14. Завершити подорож.
15. Переконатися, що повторні дії не дублюють винагороди.

Додатково перевірити відмову від побічного завдання, відсутній необов'язковий предмет, помилку запису, зміну розміру вікна та некоректний вхідний стан.

## 39. Що вважати успішним першим релізом

Не максимальну кількість NPC, предметів або локацій.

Успішний перший реліз:

- має завершену подорож;
- дає гравцеві зрозумілу мету;
- підтримує основні рішення щодо бюджету й спорядження;
- має кілька унікальних NPC;
- містить завершені квести;
- надійно зберігає прогрес;
- не має відомих блокувальних дефектів;
- працює в заявлених середовищах;
- має контрольований процес оновлення;
- дає можливість розширювати контент без переписування ядра.

---

# ЧАСТИНА IX. ЗАВДАННЯ ДЛЯ CLAUDE

## 40. Роль і мета аналізу

Claude має виступити незалежним технічним і продуктовим рецензентом.

Мета — не похвалити концепцію і не почати відразу генерувати тисячі рядків коду. Потрібно дати чесну відповідь, чи реально реалізувати задум, якими етапами, за яких обмежень і з якими ризиками.

**Claude повинен відокремити:**

- вимоги, які вже описані;
- пропозиції, які ще не затверджені;
- факти, підтверджені репозиторієм або тестами;
- припущення;
- невідомі питання;
- рішення, які потрібно прийняти людині.

# 41. Інструкція для Claude

Прочитай весь переданий чат разом із цим документом. Використовуй історію як додатковий контекст, а цей документ — як узагальнений опис проєкту. Якщо між ними є суперечності, не приховуй їх: наведи обидва варіанти й запропонуй рішення.

Не починай із безумовної згоди. Твоє завдання — незалежна оцінка.

### Завдання A. Оцінка здійсненності

Дай чітку відповідь:

1. Чи можна реалізувати задум технічно?
2. Чи може AI-агент бути основним помічником у розробці?
3. Які частини можна автоматизувати майже повністю?
4. Які частини потребують людського контролю?
5. Які частини надто складні для першого релізу?
6. Які функції варто відкласти?
7. Які ризики можуть зірвати проєкт?
8. Яких даних бракує для надійної оцінки?
9. Чи рекомендуєш продовжувати саме з цією архітектурою?
10. Що ти змінив би, якби відповідав за результат?

Не відповідай просто «так, усе можливо». Поясни умови, межі й компроміси.

### Завдання B. Аудит вимог

Побудуй таблицю:

- ID;
- вимога;
- джерело в документації;
- статус: описано / частково описано / не визначено;
- залежності;

- ризик;
- критерій приймання;
- рекомендація.

Знайди:

- суперечності;
- дублювання;
- невизначені терміни;
- приховані залежності;
- недосяжні вимоги;
- неперевірювані твердження;
- функції, що не відповідають основній концепції.

### Завдання C. Перевірка технічної архітектури

Оціни:

- TypeScript + Phaser;
- альтернативні рушії, якщо вони дають суттєву перевагу;
- структуру репозиторію;
- моделі даних;
- стан світу;
- квестову систему;
- NPC та пам'ять;
- збереження;
- міграції;
- інвентар та економіку;
- контентну валідацію;
- серверні функції;
- межу між грою та сайтом.

Не пропонуй складнішу архітектуру без чіткого обґрунтування.

### Завдання D. Межі MVP

Склади реалістичний MVP, який дає завершений ігровий досвід.

Визнач:

- що обов'язково має увійти;
- що бажано, але не критично;
- що не входить у перший реліз;
- що може бути технічно підготовлено без повної реалізації;
- які критерії дозволять перейти до наступного етапу.

Не намагайся втиснути всі задумки в MVP.

### Завдання E. Технічний план

Побудуй поетапний план із залежностями.

Для кожного етапу вкажи:

- мету;
- вхідні дані;
- конкретні результати;
- файли або модулі, які потрібно створити чи змінити;
- тести;
- критерії завершення;
- ризики;
- що має бути готове перед наступним етапом.

Якщо маєш доступ до репозиторію, спочатку досліди його. Не припускай, що запропонована структура відповідає реальним файлам.

### Завдання F. QA та безпека

Перевір, чи достатньо визначені:

- збереження й відновлення;
- міграції;
- квести;
- NPC;
- економіка;
- повторні операції;
- домашні тварини;
- помилки браузера;
- доступність;
- ліцензії;
- онлайн-винагороди;
- захист персональних даних;
- резервні копії;
- відкат релізу.

Додай відсутні критичні сценарії тестування.

Не заявляй, що безпеку перевірено, якщо ти лише прочитав документацію.

### Завдання G. Контент і наратив

Оціни:

- чи має гра виразну ідентичність;
- чи не розпорошено концепцію;
- чи достатньо сильний перший ігровий цикл;
- чи узгоджуються NPC, квести, навички й економіка;
- чи можна підтримувати великий контентний каталог без хаосу;
- які персонажі та події мають увійти до першої версії;
- які сценарії можуть створювати сюжетні тупики.

Не копіюй Stardew Valley або інші ігри. Допоможи сформувати оригінальну ідентичність.

### Завдання H. Ресурси, терміни та складність

Оціни окремо:

- технічний прототип;
- вертикальний зріз;
- MVP;
- контентно насичену версію;
- сайт і спільноту;
- матеріальні винагороди.

Якщо немає інформації про команду, бюджет, робочий час, досвід і стан репозиторію, не вигадуй точних строків. Надай умовні сценарії, припущення й діапазони, чітко позначивши невизначеність.

### Завдання I. Остаточний висновок

У кінці дай один із варіантів:

- **GO** — можна починати реалізацію визначеного MVP;
- **GO WITH CONDITIONS** — можна починати після виконання конкретних умов;
- **NO-GO FOR NOW** — спочатку потрібно вирішити блокувальні питання.

Обґрунтуй висновок.

Вкажи п'ять найважливіших ризиків, п'ять найбільш цінних наступних дій і список рішень, які має прийняти власник проєкту.

## 42. Правила чесності для аналізу

- Не стверджуй, що читав файли репозиторію, якщо доступу до них не було.
- Не вигадуй результати тестів.
- Не перетворюй запропоновані ідеї на затверджені рішення.
- Не давай безумовних гарантій відсутності дефектів.
- Не пропонуй великі платформи, мікросервіси або генеративний AI лише тому, що це модно.
- Не створюй повний код гри до завершення оцінки вимог та MVP.
- Якщо потрібно більше інформації, спочатку сформулюй найважливіші питання й поясни, чому вони блокують рішення.
- Розділяй обов'язкові вимоги та майбутні можливості.
- Пріоритизуй завершений і стабільний продукт над максимальною кількістю функцій.

## 43. Формат звіту Claude

Поверни звіт із такими розділами:

1. Короткий вердикт.
2. Що саме ми будуємо.
3. Що вже добре визначено.
4. Що суперечливе або недостатньо визначене.
5. Найважливіші технічні ризики.
6. Рекомендована архітектура.
7. Реалістичні межі MVP.
8. План розробки.
9. QA, безпека та приватність.
10. Оцінка складності й ресурсів.
11. Рішення, які повинен прийняти власник.
12. Перші конкретні задачі.
13. Остаточний GO / GO WITH CONDITIONS / NO-GO FOR NOW.

Наприкінці додай короткий перелік того, що можна починати робити негайно, і того, що не можна починати до закриття блокувальних питань.

# ЧАСТИНА X. ЩО Я ВВАЖАЮ НАЙВАЖЛИВІШИМ

Моя рекомендація до Claude — не починати з усієї Європи, сотень NPC або вебсайту.

Почати з однієї невеликої, якісно реалізованої подорожі. У ній мають працювати персонаж, інвентар, бюджет, транспорт, локальна сцена, один кемпінг, кілька NPC, один завершений квест і збереження. Потім потрібно довести, що цю подорож можна пройти, зберегти, завантажити й завершити без втрати стану.

Якщо це вдасться, у нас буде доказ, що основна архітектура здатна підтримувати більший продукт. Якщо не вдасться — ми дізнаємося про проблеми на маленькому прототипі, а не після місяців створення контенту.

## Джерела для технічної перевірки

Ці джерела варто передати Claude для незалежної перевірки технологій і стандартів:

- Phaser — офіційна документація: https://docs.phaser.io/
- TypeScript — офіційний довідник: https://www.typescriptlang.org/docs/
- OWASP ASVS — вимоги до перевірки безпеки вебзастосунків: https://asvs.owasp.org/
- WCAG — стандарти доступності: https://www.w3.org/WAI/standards-guidelines/wcag/
- OpenStreetMap — авторське право та атрибуція: https://www.openstreetmap.org/copyright
- OpenTelemetry — документація спостережуваності: https://opentelemetry.io/docs/
- Stardew Valley — історія версій: https://wiki.stardewvalley.net/Version_History

## Фінальний статус передачі

Цей пакет є узагальненою специфікацією концепції та плану перевірки, а не доказом готової гри. Він охоплює ключові рішення й пропозиції з нашої історії, але не замінює фактичний аудит коду, перевірку всіх джерел, юридичний висновок, плейтестування або тестування безпеки.

Claude має отримати повний чат разом із цим документом. Якщо його висновки суперечитимуть моїм рекомендаціям, потрібно розглянути аргументи й докази, а не автоматично віддавати перевагу будь-якій із двох оцінок.

Головна мета: не довести, що задум величезний і красивий, а визначити, як перетворити його на оригінальну, стабільну, цікаву та реально здійсненну гру, яку можна розвивати роками без втрати керованості.

Кінець пакета передачі для Claude.

Ти все розписав що мав і що хотів чи є ще щось що можеш додати?

Так. Після повторного перегляду всієї історії я бачу ще кілька важливих речей, які варто додати до пакета для Claude. Попередній документ добре описує задум, ігрові системи, архітектуру та процес розробки, але ще не є повністю закритою специфікацією для виробництва гри.

Найбільші прогалини — не в кількості ідей. Вони в тому, як ми доведемо, що гра цікава, як керуватимемо обсягом роботи, як перевірятимемо реалістичність механік, як зберігатимемо сумісність між версіями та як уникнемо ситуації, коли AI швидко створює багато коду, але проєкт стає дедалі важче підтримувати.

Я б додав наступні розділи до фінального пакета. Вони доповнюють попередній документ, а не замінюють його.

## 44. Додаткові уроки з реальних ігор: не лише функції, а й помилки

Є корисне підтвердження з офіційних історій оновлень інших ігор: проблеми часто виникають не всередині окремої механіки, а на стиках між системами.

- У примітках до оновлень The Long Dark описані проблеми з контрольними точками, відновленням після завантаження, сюжетними блокуваннями, аварійними завершеннями й сумісністю старих збережень. Це обґрунтовує окрему матрицю «збереження × квестовий етап × перехід між локаціями».

  Steam Community Announcements
  +1
- Офіційні оновлення Dinkum включають виправлення пошкоджених збережень під час перенесення між версіями, проблеми взаємодії з інтерфейсом, синхронізації та станів, які могли заблокувати гравця. Звідси урок: перевіряти не лише нормальний сценарій, а й переходи між версіями, режими роботи та нетипові послідовності дій.

  steamcommunity.com
- В історії версій Stardew Valley є тривала робота над сумісністю платформ, продуктивністю, поведінкою NPC та крайовими випадками. Це показує,

чому реліз не можна вважати кінцем QA.

Stardew Valley Wiki
+1

Це не доводить, що ми обов'язково матимемо ті самі дефекти. Це підстава закласти відповідні перевірки заздалегідь.

Додатково варто запровадити модель зрілості процесу безпеки, наприклад OWASP SAMM: вона допомагає оцінювати не лише окремі вразливості, а й те, наскільки системно організовані вимоги, реалізація, перевірка та експлуатація.

OWASP SAMM
+1

# ДОДАТОК А. КРИТИЧНІ ДОПОВНЕННЯ ДО СПЕЦИФІКАЦІЇ

## Додаткова версія для незалежного аналізу Claude

Цей додаток доповнює основний документ PROJECT HANDOFF & FEASIBILITY REVIEW. Він не змінює затверджених принципів проєкту, а додає відсутні процеси, перевірки та рішення, які необхідно закрити до масштабної розробки.

---

## A1. Зафіксувати ідентичність гри та її межі

До розробки потрібно створити документ `docs/PRODUCT_PILLARS.md`.

У ньому мають бути п'ять незмінних принципів:

1. **Подорож — головна діяльність.** Економіка, транспорт, кемпінг, NPC та квести підтримують подорож, а не відволікають від неї.

2. **Реалізм із людяністю.** Реальні проблеми мають зрозумілі причини, попередження й варіанти відновлення.
3. **Люди мають значення.** NPC — це не лише джерела завдань і винагород, а персонажі з власними цілями й пам'яттю.
4. **Свобода без покарання за відсутність.** Немає обов'язкових щоденних входів, втрачених через відсутність нагород або штучних таймерів.
5. **Стабільність важливіша за кількість.** Менше якісних систем краще, ніж багато недороблених.

Для кожної нової функції ставити питання: який принцип вона посилює? Якщо відповідь відсутня, функція не повинна автоматично потрапляти в план.

**Критерій приймання**

Будь-яку функцію можна класифікувати як обов'язкову, бажану, майбутню або відхилену з документованим обґрунтуванням.

## A2. Визначити, що означає «гра цікава»

Технічна працездатність не гарантує цікавого ігрового досвіду.

Створити `docs/PLAYTEST_PLAN.md` із перевірками:

- чи розуміє гравець, що робити далі;
- чи має він значущий вибір;
- чи зрозумілі причини витрат і наслідків;
- чи цікаво спілкуватися з NPC;
- чи не перетворюються потреби на рутину;
- чи можна грати короткими сесіями;
- чи відчувається прогрес;
- чи виникає бажання здійснити ще одну подорож.

Проводити плейтести з людьми, які не брали участі в розробці. Не пояснювати механіку до того, як стане зрозуміло, чи може гравець розібратися самостійно.

Не вигадувати універсальний показник «цікавості». Визначити конкретні запитання, спостережувані проблеми й критерії ухвалення рішень.

## A3. Запровадити контроль обсягу

Створити `docs/SCOPE_AND_CHANGE_CONTROL.md`.

Кожна нова функція повинна містити:

- користь для гравця;
- вартість реалізації;
- залежності;
- ризик регресії;
- вимоги до тестування;
- вплив на контент;
- вплив на збереження;
- вплив на приватність або безпеку;
- рішення про включення в поточний етап.

Застосовувати правило заміщення: якщо нова функція істотно збільшує обсяг MVP, визначити, що можна перенести на наступний етап.

Не додавати систему лише тому, що AI здатний швидко згенерувати її початковий код.

## A4. Формалізувати симуляцію часу та подій

Створити `docs/SIMULATION_RULES.md`.

Потрібно визначити:

- що відбувається в ігрову хвилину, годину й день;
- коли зменшуються потреби;
- коли змінюється погода;
- коли NPC переходять між локаціями;
- як обробляються запізнення;
- як розв'язуються одночасні події;
- що відбувається, коли гравець відкрив меню;
- чи продовжується час під час діалогів;
- як обробляється сон;
- що відбувається після перезавантаження браузера.

Рекомендація для початку: детерміновані правила й контрольовані переходи часу. Випадковість повинна мати визначені межі та можливість тестування через фіксовані seed-значення.

Не дозволяти, щоб різні сцени самостійно змінювали час і потреби незалежно одна від одної.

## A5. Визначити політику випадковості та повторюваності

Для випадкових подій необхідно встановити:

- умови запуску;
- частоту;
- вагу ймовірності;
- період повторення;
- взаємовиключні події;
- мінімальні умови безпеки;
- вплив попередніх рішень;
- поведінку після завантаження збереження.

Тест повинен мати змогу повторити конкретний сценарій. Якщо подія зламалася, розробник має відтворити її за заданим seed і станом світу.

Випадковість не повинна бути причиною втрати єдиного предмета, потрібного для обов'язкового квесту, або появи нездоланної ситуації.

## A6. Додати специфікацію інтерфейсу та UX

Створити `docs/UI_UX_SPEC.md`.

Визначити:

- основні екрани;
- навігацію;
- пріоритет інформації;
- керування клавіатурою;
- поведінку миші;
- фокус і повернення фокуса;
- масштабування інтерфейсу;
- довгі тексти й переповнення;
- підказки;
- підтвердження незворотних дій;
- помилки та способи відновлення;
- поведінку під час втрати фокуса вікна.

Окремо перевірити:

- чи видно, скільки грошей залишилося;
- чи зрозуміло, чому предмет недоступний;
- чи показано ризик перед небезпечним маршрутом;
- чи зрозуміло, чому квест не можна завершити;
- чи може гравець скасувати випадкову дію;
- чи не закриває UI важливу частину локальної сцени.

## A7. Додати аудіо та звуковий дизайн

Створити `docs/AUDIO_SPEC.md`.

Для MVP визначити мінімальний набір:

- звуки кроків;
- відкриття й закриття дверей;
- взаємодії з предметами;
- інвентарю;
- покупок;
- ремонту;
- табору;
- дощу й вітру;
- навколишнього середовища;
- діалогів і переходів.

Потрібно передбачити:

- окреме регулювання музики та ефектів;
- вимкнення звуку;
- відсутність різких непередбачуваних піків гучності;
- ліцензії на музику й ефекти;
- оптимізацію розміру аудіофайлів;
- повагу до налаштувань користувача.

Повноцінну музичну бібліотеку можна створювати після підтвердження вертикального зрізу.

## A8. Визначити локалізацію та культурну адаптацію

Створити `docs/LOCALIZATION.md`.

Навіть якщо перша версія буде однією мовою, архітектура не повинна прив'язувати ігрові правила до текстових рядків.

Передбачити:

- стабільні ключі локалізації;
- зовнішні файли текстів;
- підтримку довших перекладів;
- формати дат, часу, чисел і валют;
- одиниці вимірювання;
- правила перенесення рядків;

- відсутність тексту, вбудованого безпосередньо в спрайти;
- контекст для перекладачів;
- перевірку діалогів у реальному UI.

Для реальних європейських локацій не можна вважати, що правила кемпінгу, сортування, дорожні збори чи перевезення тварин однакові в усіх країнах.

## A9. Додати реєстр достовірності реальних даних

Створити `docs/REAL_WORLD_DATA_POLICY.md`.

Для кожного факту визначати:

- ID;
- твердження;
- країну або регіон;
- джерело;
- дату перевірки;
- ліцензію;
- рівень упевненості;
- строк актуальності;
- відповідального за повторну перевірку;
- ігрове спрощення;
- тест або правило, яке використовує цей факт.

Розділити три категорії:

1. Перевірений факт.
2. Ігрове наближення.
3. Вигаданий елемент світу.

Не подавати вигадану механіку як реальну пораду щодо безпеки.

Для небезпечних або юридично чутливих тем не покладатися на неперевірені AI-висновки.

## A10. Визначити політику генеративного AI в самому продукті

Створити `docs/AI_POLICY.md`.

Початкова рекомендація: гра не повинна залежати від генеративного AI під час звичайного проходження.

Якщо пізніше AI використовуватиметься для діалогів, журналу або рекомендацій, потрібно визначити:

- які дані йому дозволено передавати;
- чи надсилаються персональні дані;
- чи можна грати без AI;
- як обмежуються вигадані факти;
- як обробляються небезпечні поради;
- як контролюється вартість запитів;
- як обробляються помилки сервісу;
- як захищаються секрети;
- як повідомляється користувачеві про використання AI;
- чи можна вимкнути функцію.

Ключові квести, прогрес і винагороди повинні залишатися працездатними, навіть якщо зовнішній AI-сервіс недоступний.

## A11. Додати політику модифікацій і користувацького контенту

Створити `docs/MODDING_AND_UGC_POLICY.md`.

До публічного запуску вирішити:

- чи дозволяються модифікації;
- які типи файлів можна імпортувати;
- чи дозволяються користувацькі квести;
- як перевіряються сторонні пакети контенту;
- як запобігати виконанню небезпечного коду;
- які правила діють для публікацій;
- як працюють скарги й апеляції;
- хто відповідає за порушення авторських прав;
- як видаляється контент.

Не обіцяти повну підтримку модифікацій, якщо архітектура ще не має ізольованого й безпечного механізму їх завантаження.

## A12. Додати політику оновлень і сумісності

Створити `docs/VERSIONING_AND_COMPATIBILITY.md`.

Кожен реліз повинен мати:

- версію;
- список змін;
- зміни формату збереження;
- міграційні правила;
- відомі проблеми;
- перевірені платформи;
- можливість відкату веброзгортання;
- план відновлення даних;
- порядок повідомлення про критичні дефекти.

Потрібно визначити, які старі збереження підтримуються та як довго.

Не видаляти старі поля зі збереження без міграційної стратегії.

Не змінювати ID квестів, NPC або предметів без перевірки посилань і сумісності.

## A13. Додати керування залежностями та ланцюгом постачання

Створити `docs/DEPENDENCY_POLICY.md`.

Необхідно:

- зафіксувати версії залежностей;
- перевіряти ліцензії;
- регулярно перевіряти відомі вразливості;
- мінімізувати непотрібні пакети;
- не встановлювати залежності лише заради дрібної зручності;
- не передавати секрети в репозиторій;
- переглядати зміни lock-файлів;
- перевіряти походження графічних і аудіоактивів.

Для відкритих залежностей можна оцінити користь інструментів на кшталт OpenSSF Scorecard, але не сприймати один рейтинг як повний аудит безпеки.

## A14. Визначити метрики якості

Створити `docs/QUALITY_METRICS.md`.

На прототипі вимірювати:

- частку успішних запусків;

- помилки збереження й відновлення;
- критичні помилки;
- блокування проходження;
- тривалість першої подорожі;
- частоту незрозумілих дій;
- місця, де гравці застрягають;
- частку завершення першої подорожі;
- продуктивність;
- розмір завантаження;
- проблеми доступності.

Цільові значення встановлювати після перших вимірювань і визначення підтримуваних пристроїв. Не вигадувати відсотки успішності без базових даних.

Телеметрія має бути мінімально необхідною, прозорою та узгодженою з вимогами приватності. Основна одиночна гра не повинна вимагати відправлення повної історії дій на сервер.

## A15. Визначити правила випуску

Створити `docs/RELEASE_GATES.md`.

Реліз не допускається, якщо:

- є невирішені блокувальні дефекти;
- не перевірено збереження;
- не перевірено критичні міграції;
- немає відомого способу відновлення;
- не перевірено основний ігровий цикл;
- є невідомі ліцензії на обов'язкові активи;
- залишилися відомі критичні проблеми безпеки;
- відсутній список відомих проблем;
- збірка не відтворюється з контрольованого стану репозиторію.

Потрібно також визначити, хто має право схвалити реліз і хто відповідає за повернення до попередньої версії.

## A16. Додати політику підтримки після релізу

Створити `docs/POST_RELEASE_SUPPORT.md`.

Визначити:

- як користувач повідомляє про помилку;
- які дані потрібно надати для відтворення;
- як захищаються збереження;
- як сортуються повідомлення;
- як визначається пріоритет;
- як публікуються відомі проблеми;
- як повідомляється про виправлення;
- як відбувається розбір першопричини;
- як запобігати повторенню дефекту.

Кожен критичний дефект повинен приводити не лише до виправлення коду, а й до тесту, який не дозволить помилці непомітно повернутися.

---

# ДОДАТОК В. НОВІ КРИТЕРІЇ ГОТОВНОСТІ

До вже визначених етапів додати такі умови.

## B1. До написання великого обсягу коду

- підтверджено фактичний стан репозиторію;
- визначено MVP;
- затверджено основні архітектурні рішення;
- узгоджено модель стану;
- визначено формат збереження;
- зафіксовано критерії приймання;
- сформовано список блокувальних невідомих.

## B2. До масштабного створення контенту

- працює контентний валідатор;
- квестова система має тести;
- існує стабільна модель NPC;
- збереження відновлюється;
- визначені правила часу й випадковості;
- перевірено, що зміна контенту не руйнує наявні проходження.

## B3. До закритого плейтесту

- перша подорож прохідна від початку до кінця;
- є зрозумілий шлях відновлення після помилки;
- перевірено збереження;
- доступні базові налаштування;
- визначено збір зворотного зв'язку;
- немає відомих блокувальних дефектів.

## B4. До публічного запуску

- перевірено ліцензії;
- визначено підтримувані браузери й пристрої;
- перевірено резервне копіювання та відновлення;
- визначено процес інцидентів;
- виконано перевірки приватності й безпеки;
- є план оновлень;
- є відповідальний за підтримку;
- опубліковано правила спільноти, якщо вона входить у реліз.

---

# ДОДАТОК C. ДОДАТКОВІ ЗАВДАННЯ CLAUDE

До основного аудиту додати ще десять завдань.

1. Знайти системи, що створюють циклічні залежності.
2. Знайти рішення, які передчасно ускладнюють MVP.
3. Перевірити, чи не суперечать одне одному економіка, час, потреби й квести.
4. Перевірити поведінку при повторному виконанні кожної важливої команди.
5. Перевірити всі переходи через збереження, завантаження та зміну версії.
6. Перевірити доступність усіх обов'язкових предметів і NPC.
7. Визначити мінімальні показники якості, які реально можна виміряти.
8. Перевірити, які функції потребують зовнішніх сервісів і які можна залишити локальними.
9. Визначити, які рішення повинні бути прийняті людиною, а не AI.
10. Запропонувати короткий список ризиків, усунення яких дасть найбільше зниження ймовірності провалу.

Для кожного виявленого ризику зазначити ймовірність, вплив, спосіб виявлення, спосіб пом'якшення, відповідального та критерій закриття.

Не заявляти, що прогалину закрито, лише тому, що для неї створено документ. Вона закрита лише тоді, коли вимога реалізована або свідомо відкладена, рішення задокументовано, а відповідні перевірки пройдено.

---

# ДОДАТОК D. ПІДСУМКОВИЙ ПРИНЦИП

Проєкт має розвиватися через цикл:

**Вимога → рішення → реалізація → тест → перевірка результату → документація → контрольована зміна.**

Необхідно зберігати баланс між трьома речами:

- якістю ігрового досвіду;
- технічною надійністю;
- реальним обсягом роботи.

Мета не в тому, щоб передбачити абсолютно кожну помилку до першого запуску. Це неможливо. Мета — побудувати процес, у якому важливі помилки виявляються якомога раніше, виправляються контрольовано й не повертаються непомітно в наступних версіях.

**Кінець додаткового пакета.**

## Мій остаточний висновок

Тепер документація охоплює значно більше, ніж ідею гри: дизайн, наратив, системи, архітектуру, контент, QA, безпеку, приватність, випуск і супровід.

Але я не буду стверджувати, що прогалин більше немає. Без доступу до фактичного репозиторію, прототипу й результатів тестів це було б неправдою. Наступний найбільш цінний крок — попросити Claude провести незалежний аудит основного документа разом із цим додатком, а потім перевірити реальні файли проєкту, якщо вони доступні.

Саме так ми перейдемо від великого плану до доказово перевіреного плану розробки.

Значно більше але не все і пеіно ти можеш доробити ще те чого немає, дороби все те чого нам не вистачає для старту і аналізу гри, дожми все що ще не зробив

Так. Тоді доводимо підготовку до стану, коли Claude зможе не просто прочитати ідею, а системно перевірити задум, виявити суперечності, визначити реалістичний обсяг першої версії та почати створення гри без хаотичного переписування всього проєкту.

Але є важлива межа: я можу завершити специфікації, правила, шаблони, сценарії перевірки й план аналізу. Я не буду стверджувати, що вже перевірив код, протестував збірку або підтвердив працездатність систем, яких фактично не запускав.

Найбільша проблема зараз не в тому, що бракує ще десятка великих документів. Нам потрібно закрити прогалини між задумом, конкретними правилами гри, перевірними вимогами та рішенням про те, що саме будувати першим.

Нижче — додатковий пакет, який закриває критично важливі питання, не охоплені попередніми матеріалами. Його можна передати Claude разом з основним пакетом і попередніми додатками.

## Що ще потрібно закрити перед стартом

Я додаю ще сім блоків, які мають практичну цінність саме для нашої гри:

1. Фіксація рішень і припущень. Щоб Claude не вигадав непогоджені правила й не перетворив припущення на нібито затверджені вимоги.
2. Повна модель ігрового циклу. Що гравець робить, чому це цікаво, які рішення приймає, що змінюється у світі та за що він отримує винагороду.
3. Правила симуляції. Час, погода, гроші, транспорт, спорядження, втома, домашні тварини, NPC, завдання й наслідки — щоб системи не суперечили одна одній.
4. Межі першої версії. Що обов’язково має працювати в першому прототипі, що відкладаємо, а що взагалі не створюємо без підтвердженої потреби.
5. Критерії приймання. Як перевірити кожну функцію й зрозуміти, чи справді її завершено.
6. Перевірка досвіду гравця. Як з’ясувати, чи гра захоплює, чи зрозуміла, чи не перевантажена системами й чи вчить планувати подорожі.
7. Єдиний протокол роботи Claude. Що агент повинен дослідити, які докази надати, де зупинитися й коли має право переходити до реалізації.

**Додаток до пакета передачі проєкту**

# ДОДАТОК B. ПОВНЕ ЗАКРИТТЯ ПРОГАЛИН ПЕРЕД СТАРТОМ ГРИ

**Проєкт:** реалістична, доброзичлива 2D indie-гра про кемпінг, подорожі, людей, транспорт і життя в дорозі.

**Статус документа:** додаткові вимоги та план перевірки. Це не підтвердження того, що гра вже реалізована або протестована.

## B1. Правила достовірності проєкту

Кожне суттєве твердження в документації, аудиті та звітах повинно належати до однієї з категорій:

- **APPROVED** — рішення явно погоджено.
- **EXISTING** — підтверджено в наявних файлах або коді.
- **VERIFIED** — перевірено тестом, запуском або надійним джерелом відповідно до типу твердження.
- **PROPOSED** — рекомендація, яку ще потрібно погодити.
- **ASSUMED** — робоче припущення.
- **UNKNOWN** — інформації недостатньо.
- **CONFLICT** — два або більше джерел суперечать одне одному.
- **BLOCKED** — перевірка неможлива через відсутність доступу, інструментів або залежностей.
- **NOT TESTED** — реалізація є, але відповідну поведінку не перевірено.

Заборонено видавати запропоновану архітектуру за наявну, запланований тест за виконаний або неперевірений факт за підтверджений.

Для кожного важливого рішення вести запис:

- ID рішення;
- питання;
- варіанти;
- рекомендований варіант;
- аргументи;
- наслідки;
- залежності;
- статус;

- дата;
- критерій перегляду рішення.

# B2. Продуктова модель

### Головна обіцянка гравцеві

Гравець може побудувати власну історію подорожей Європою: підготуватися, обрати маршрут, розрахувати бюджет, організувати спорядження, взаємодіяти з людьми, розв'язувати побутові проблеми, відкривати місця й повертатися до світу, який пам'ятає його рішення.

Гра повинна поєднувати три цінності:

1. **Приємна подорож:** цікаві місця, відкриття, атмосфера, свобода вибору.
2. **Змістовні рішення:** маршрут, гроші, погода, транспорт, час, спорядження та потреби супутників впливають одне на одного.
3. **Людська історія:** персонажі мають характер, потреби, пам'ять, власні справи й реакції на вчинки гравця.

Реалістичність не повинна перетворювати гру на роботу з таблицями. Деталізація потрібна лише тоді, коли створює цікаве рішення, зрозумілий наслідок або корисне навчання.

### П'ять головних принципів

- Подорож і дослідження — основа гри.
- Помилки створюють цікаві ситуації, а не безвихідь.
- Складність зростає поступово.
- Гравець може грати у власному темпі без покарання за перерву.
- Жодна другорядна система не повинна робити основний цикл недоступним.

### Що не можна вважати успіхом саме по собі

- Велика карта без цікавих рішень.
- Сотні предметів без корисних відмінностей.
- Багато NPC без відчутної індивідуальності.
- Десятки характеристик, які не впливають на вибір.
- Реалістична формула, яку гравець не може зрозуміти.
- Велика кількість квестів, які відрізняються лише текстом.
- Соціальна система, що змушує публікувати контент або запрошувати друзів.
- Генерація контенту за допомогою AI без перевірки цілісності.

# B3. Повний цикл гри

Кожна велика ігрова активність повинна підтримувати один або кілька етапів цього циклу:

1. **Намір:** куди хоче вирушити гравець і навіщо.
2. **Планування:** маршрут, погода, бюджет, тривалість, транспорт, супутники.
3. **Підготовка:** перевірка спорядження, покупка необхідного, пакування.
4. **Вирушення:** вибір часу й способу пересування.
5. **Подорож:** навігація, відкриття, події, витрати та можливі зміни планів.
6. **Прибуття:** пошук місця, реєстрація, розміщення, облаштування табору.
7. **Життя в локації:** відпочинок, прогулянки, знайомства, побут, квести.
8. **Рішення:** залишитися, змінити план, витратити ресурси, попросити допомогу або продовжити.
9. **Підсумок:** витрати, здобуті навички, відносини, відкриття, записи в щоденнику.
10. **Наступний намір:** нова подорож, повернення, покращення спорядження або продовження історії.

Не кожна ігрова сесія повинна проходити всі десять етапів. Гравець має отримувати маленькі завершені результати навіть за коротку сесію.

### Обов'язкові питання для кожної системи

- Яку потребу гравця вона задовольняє?
- Яке рішення дозволяє прийняти?
- Які дані читає?
- Який стан змінює?
- Який результат бачить гравець?
- Що станеться, якщо гравець відмовиться?
- Що станеться, якщо умови зміняться?
- Чи можна відновитися після помилки?
- Чи зберігається стан після перезапуску?
- Як перевірити правильність поведінки?

Якщо система не має чіткої відповіді на ці запитання, її потрібно спростити або відкласти.

# B4. Контракт ігрових систем

Для кожної системи створити короткий контракт із такими розділами:

1. Призначення.
2. Стан і дані.

3. Дозволені дії.
4. Передумови дій.
5. Зміни стану.
6. Наслідки для інших систем.
7. Відображення в інтерфейсі.
8. Збереження та відновлення.
9. Помилки й крайові випадки.
10. Автоматичні тести.
11. Критерії приймання.
12. Відомі обмеження.

### Мінімальний перелік контрактів

- Персонаж і домогосподарство.
- Календар і час.
- Погода та її вплив.
- Карта, маршрут і переміщення.
- Транспорт і технічний стан.
- Бюджет, витрати та покупки.
- Інвентар, екіпірування й вага.
- Потреби, втома, температура й травми.
- Домашні тварини.
- NPC, їхні розклади та пам'ять.
- Квести, цілі, розгалуження й винагороди.
- Випадкові та заплановані події.
- Кемпінг і його інфраструктура.
- Навички та прогрес.
- Щоденник і відкриття.
- Досягнення та внутрішні винагороди.
- Збереження й міграція даних.
- Налаштування, доступність та керування.
- Онлайн-функції, якщо вони будуть включені до відповідної версії.

## B5. Узгодженість симуляції

До реалізації потрібно створити документ `SIMULATION_RULES.md`, який стане єдиним джерелом правил для всіх систем.

### Час

Визначити:

- ігровий календар і часовий пояс;

- тривалість ігрової хвилини відносно реального часу;
- швидкість часу під час ходьби, їзди, діалогів, відпочинку й меню;
- які меню зупиняють час;
- що відбувається, коли гравець виходить із гри;
- як працюють сон, втома, розклади NPC, бронювання й дедлайни;
- чи можуть події відбуватися одночасно;
- як обробляються переходи через північ і зміну сезону.

Рекомендація: на початку меню, діалоги та планування не повинні непомітно витрачати час. Відсутність гравця не повинна створювати критичні втрати.

### Погода

Визначити:

- джерело погодних даних: авторизовані реальні дані, сценарна погода або процедурна модель;
- регіональні відмінності;
- період прогнозу;
- точність прогнозу;
- вплив на швидкість, комфорт, видимість, спорядження та ризики;
- правила попереджень;
- поведінку, якщо прогноз невідомий;
- межу між правдоподібністю та ігровим балансом.

Погода не повинна змінюватися довільно лише для покарання гравця.

### Бюджет

Єдиний розрахунок поїздки повинен враховувати відповідні категорії:

- паливо або заряджання;
- платні дороги й пороми;
- проживання;
- харчування та воду;
- паркування;
- обслуговування й ремонт;
- придбання спорядження;
- резерв на непередбачені витрати.

Потрібно визначити правила округлення, валюту, відображення оцінок і фактичних витрат, повернення платежів, відмови в оплаті та недостатнього балансу.

Розрахунки повинні мати автоматичні тести на нульові, граничні, від'ємні та некоректні значення.

### Інвентар і спорядження

Визначити:

- кількість слотів, вагу або комбіновану модель обмежень;
- екіпіровані предмети й предмети в сховищі;
- стан, знос і ремонт;
- сумісність із транспортом і типом подорожі;
- витратні предмети;
- предмети квестів;
- втрату, передачу, продаж і повернення предметів;
- поведінку повного інвентарю;
- збереження унікальних предметів, потрібних для прогресу.

Не можна допустити, щоб обов'язкове завдання стало непрохідним через необоротний продаж або втрату ключового предмета.

### Ризики й травми

Кожна небезпечна ситуація повинна мати:

- зрозумілу причину;
- попередження, якщо ризик можна передбачити;
- доступні способи реагування;
- ціну помилки;
- варіанти допомоги або відновлення;
- правила, що не допускають нескінченного циклу покарань.

Основна філософія — безпечне навчання через наслідки, а не жорстке покарання. Смерть персонажа не є необхідною умовою реалістичності.

### Домашні тварини

Для кожного типу тварини описати:

- потреби у воді, їжі, відпочинку й активності;
- реакцію на транспорт і незнайомі місця;
- необхідні предмети;
- обмеження кемпінгів;
- правила безпеки;
- варіанти допомоги;
- відновлення після стресу;

- взаємодію з іншими персонажами.

Тварини — не декоративні предмети. Їхні потреби повинні мати значення, але система не повинна вимагати безперервного мікрокерування.

### NPC та пам'ять

Пам'ять NPC повинна бути структурованою, а не залежати лише від тексту діалогу.

Мінімальні типи фактів:

- персонажі познайомилися;
- гравець допоміг NPC;
- NPC допоміг гравцеві;
- гравець дав обіцянку;
- обіцянку виконано або порушено;
- персонажі разом пережили подію;
- квест завершено;
- змінилися відносини;
- NPC переїхав або змінив роботу.

Для кожного факту визначити автора події, учасників, час, контекст, тривалість актуальності й правила впливу на діалоги.

### Випадкові події

Випадковість має бути контрольованою:

- події повинні мати передумови й виключення;
- повтори повинні обмежуватися cooldown-правилами;
- сюжетні події не повинні губитися через невдалий випадковий вибір;
- важливі рішення мають відтворюватися в тестах;
- гравець не повинен отримувати безперервний потік проблем;
- кілька подій не повинні блокувати одна одну без визначеного пріоритету.

Для тестів використовувати фіксовані seed або інший механізм відтворення випадкових послідовностей.

## B6. Обов'язковий обсяг першого прототипу

Не намагатися одразу реалізувати всю Європу, всі професії, сотні предметів, повний соціальний сайт, мерчандайзинг і десятки транспортних систем.

Прототип повинен довести, що основа гри працює.

### Вертикальний зріз

Рекомендований сценарій:

1. Гравець створює персонажа або вибирає готовий профіль.
2. Перебуває вдома та переглядає ресурси.
3. Планує коротку поїздку.
4. Перевіряє бюджет, прогноз і спорядження.
5. Вирушає в дорогу.
6. Переміщується між двома або більше вузлами маршруту.
7. Прибуває до кемпінгу.
8. Облаштовує місце й взаємодіє з інфраструктурою.
9. Знайомиться з одним NPC.
10. Виконує невелике завдання з вибором.
11. Отримує наслідок у бюджеті, відносинах або прогресі.
12. Зберігає гру, перезапускає її та продовжує з коректного стану.
13. Переглядає підсумок подорожі в журналі.

### Умови приймання вертикального зрізу

- Сценарій можна завершити від початку до кінця.
- Основні дії зрозумілі без читання документації розробника.
- Витрати й зміни ресурсів пояснюються гравцеві.
- Стан зберігається після перезапуску.
- Помилки збереження не призводять до тихої втрати прогресу.
- Немає критичного softlock, який блокує продовження.
- Ключові правила мають автоматичні тести.
- Інтерфейс підтримує основний спосіб керування та зміну розміру вікна.
- Тестувальник може відтворити помилку за записаними кроками.
- Після гри людина може пояснити, що було цікаво, що незрозуміло та чого вона хотіла б далі.

Якщо якась функція не потрібна для цього сценарію, її не потрібно додавати до першої версії лише тому, що вона згадана в GDD.

## B7. Система пріоритетів

Кожну вимогу позначати одним із пріоритетів:

- **P0 — блокер:** без цього не можна безпечно або коректно продовжувати.
- **P1 — необхідно для вертикального зрізу:** без цього не перевірити основний цикл.

- **P2 — необхідно для повноцінного MVP:** важливо для запланованого першого релізу, але не для найменшого прототипу.
- **P3 — майбутнє розширення:** додається лише після підтвердження цінності.
- **P4 — відхилено або відкладено без визначеної дати:** не реалізовувати зараз.

Для кожної вимоги записувати цінність, складність, ризик, залежності та критерій приймання.

Високий пріоритет не означає, що функцію треба реалізовувати до перевірки її залежностей.

## B8. Економіка й прогрес без примусу

Описати всі джерела та витрати ігрових ресурсів:

- стартові гроші;
- заробіток;
- покупки;
- ремонт;
- проживання;
- нагороди за завдання;
- обмін і продаж;
- безкоштовні або бюджетні альтернативи;
- повернення й помилки транзакцій.

Для кожної валюти визначити призначення, способи отримання, способи витрачання, обмеження та правила відновлення після помилок.

Якщо пізніше з'являться Traveler Miles або інші мережеві винагороди, вони мають бути відокремлені від локального ігрового балансу. Цінні онлайн-операції повинні перевірятися сервером, а не лише клієнтським кодом.

Не використовувати:

- оплату за прискорення;
- плату за відновлення прогресу;
- механіки pay-to-win;
- обов'язкові щоденні входи;
- покарання за відсутність;
- штучні таймери, що примушують повернутися;
- винагороди за позитивні відгуки або маніпулятивну активність.

## B9. Вимірювання цікавості

До закритого тестування сформувати гіпотези, а не заздалегідь оголошувати гру цікавою.

Приклади гіпотез:

- гравець розуміє, навіщо планувати маршрут;
- підготовка створює змістовний вибір, але не перевантажує;
- зміна погоди впливає на рішення зрозумілим способом;
- NPC запам'ятовують важливі дії;
- гравець розуміє, як виправити помилку;
- після короткої сесії залишається бажання продовжити подорож.

### Мінімальний протокол тестування

1. Запросити людей, які не брали участі в розробці.
2. Дати їм завдання без підказок розробника.
3. Спостерігати за помилками, затримками та незрозумілими рішеннями.
4. Записувати, де гравець не розуміє мету, правила або наслідки.
5. Після тесту запитати, що запам'яталося, що дратувало та що хотілося б зробити далі.
6. Розділяти фактичну поведінку й думки учасника.
7. Пріоритизувати проблеми за впливом, частотою та складністю виправлення.
8. Повторити тест після змін.

Не вигадувати універсальні порогові значення для утримання чи тривалості сесії. Початкові цілі потрібно перевірити на прототипі та скоригувати за результатами тестів.

## B10. Повнота контенту

Створити єдиний реєстр контенту для локацій, NPC, предметів, транспорту, квестів, подій, діалогів, досягнень і звукових ресурсів.

Кожен запис має містити:

- стабільний ID;
- тип;
- назву;
- короткий опис;
- статус;
- залежності;

- джерело фактів, якщо вони використовуються;
- ліцензію, якщо потрібна;
- посилання на пов'язані записи;
- локалізаційний статус;
- тести валідації;
- дату останньої перевірки.

**Автоматичні перевірки контенту**

- унікальність ID;
- коректність посилань;
- відсутність недосяжних обов'язкових квестів;
- відсутність неіснуючих предметів у вимогах;
- відсутність недоступних винагород;
- відсутність тупикових гілок без визначеного завершення;
- коректність локалізаційних ключів;
- відсутність порушень схеми;
- перевірка ліцензій і атрибуції;
- перевірка сумісності з поточними версіями схем збереження.

Контент, який не пройшов валідацію, не повинен потрапляти до релізної збірки непоміченим.

## B11. Технічні рішення без передчасного ускладнення

До реалізації зафіксувати архітектурні рішення та їхні причини.

Початкові рекомендації для перевірки:

- TypeScript як основна мова;
- Phaser як кандидат для 2D-ігрового рушія;
- модульний моноліт замість мікросервісів;
- чисті правила гри, відокремлені від інтерфейсу;
- типізовані контракти між системами;
- дані контенту окремо від поточного стану проходження;
- версіоновані збереження;
- автоматичні перевірки в CI;
- відсутність обов'язкової серверної залежності для базового одиночного режиму.

Це рекомендації, а не догма. Claude повинен перевірити їх проти фактичного репозиторію, цільових браузерів, потреб у збереженнях, ліцензій і можливого майбутнього перенесення на PC.

Для кожної залежності перевірити ліцензію, активність підтримки, відомі ризики, сумісність, розмір і потребу в ній. Не додавати пакет лише тому, що він популярний.

## B12. Межі між грою, сайтом і сервером

Якщо плануються облікові записи, щоденники, спільнота, рейтинги та винагороди, заздалегідь розділити:

**Локальна гра:** поточне проходження, локальний інвентар, події світу, налаштування, локальний щоденник.

**Обліковий запис:** ідентифікація користувача та налаштування профілю.

**Онлайн-сервіс:** публікації, модерація, рейтинги, мережеві винагороди, синхронізація й резервні копії, якщо їх включено до конкретної версії.

Визначити:

- які дані залишаються локальними;
- які дані передаються серверу;
- хто має право читати й змінювати дані;
- як працюють конфлікти синхронізації;
- як захищаються приватні збереження;
- як обробляються дублікати запитів;
- як експортувати й видаляти персональні дані;
- як працюють блокування, скарги та модерація;
- як відновлюватися після недоступності сервера.

Не переносити весь одиночний режим на сервер без конкретної потреби.

## B13. Доступність і комфорт

Мінімальні вимоги до перевірки:

- керування клавіатурою;
- видимий фокус;
- читабельний текст і масштабування;
- зрозумілі повідомлення про помилки;
- колір не є єдиним способом передати значення;
- налаштування гучності;
- можливість зменшити інтенсивні анімації, де це доречно;
- достатній контраст;
- зрозумілі піктограми або текстові пояснення;

- відсутність обов'язкової залежності від точних швидких натискань, якщо це не є свідомою механікою;
- коректне відображення на цільових розмірах екрана;
- передбачувана поведінка фокуса в діалогах та інтерфейсних меню.

Перевірити доступність не лише в головному меню, а й у діалогах, інвентарі, налаштуваннях, картах і повідомленнях про помилки.

## B14. Відновлення після помилок

Для критичних операцій визначити поведінку за принципом: **або операція завершується коректно, або стан залишається узгодженим і гравець отримує зрозуміле повідомлення**.

Перевірити:

- збереження під час активного завдання;
- перезапуск після покупки;
- помилку під час збереження;
- пошкоджений файл збереження;
- застарілу версію формату;
- повторний клік по дії;
- переривання мережевого запиту;
- недоступність сервера;
- часткове оновлення стану;
- невідповідність між журналом, інвентарем і квестом;
- повторне отримання нагороди;
- перехід між локаціями під час активної події.

Для локальних даних передбачити резервну копію або інший механізм відновлення, якщо це технічно виправдано. Міграції збережень мають бути протестовані на реальних прикладах старих форматів.

## B15. Безпека, право та походження даних

До публічного релізу перевірити:

- ліцензії коду, шрифтів, музики, звуків, зображень і карт;
- умови використання картографічних даних і тайлів;
- приватність і мінімізацію даних;
- правила зберігання, експорту та видалення;
- захист облікових записів і приватних збережень;
- захист від підробки онлайн-винагород;

- захист від XSS і зловживань користувацьким контентом;
- модерацію та оскарження рішень;
- правила конкурсів і призів, якщо вони з'являться;
- податкові, споживчі та договірні питання для мерчу;
- застосовність норм щодо неповнолітніх, приватності та цифрових сервісів.

Не робити висновок про повну юридичну відповідність лише на підставі документації. Для реального запуску потрібна перевірка актуальних вимог у відповідних юрисдикціях.

## B16. Випуск і підтримка

Визначити перед релізом:

- підтримувані браузери й пристрої;
- версію гри та схеми збережень;
- спосіб резервного копіювання;
- процедуру випуску й відкату;
- журнал змін;
- канали повідомлення про помилки;
- мінімальну діагностичну інформацію;
- процедуру реагування на втрату даних;
- правила сумісності зі старими збереженнями;
- власника кожної критичної проблеми;
- критерії блокування релізу.

Публічний реліз не повинен починатися лише тому, що гра запускається на комп'ютері розробника.

## B17. Єдині критерії готовності

**Готовність документації:**

- немає непомічених суперечностей у критичних вимогах;
- відкриті питання мають власників або визначений наступний крок;
- припущення відокремлені від погоджених рішень;
- ключові системи мають контракти;
- межі MVP зафіксовані.

**Готовність прототипу:**

- вертикальний зріз проходиться від початку до кінця;

- збереження й відновлення перевірено;
- критичні сценарії мають автоматичні тести;
- відомі помилки задокументовані;
- немає невирішених блокерів, що руйнують основний цикл.

**Готовність MVP:**

- погоджений обсяг реалізований;
- основні сценарії перевірені;
- контент проходить валідацію;
- доступність і продуктивність перевірені на цільових пристроях;
- відомі ризики прийняті свідомо;
- є процедура випуску й підтримки.

**Готовність публічного сервісу:**

- додатково перевірені безпека, приватність, модерація, відновлення, ліцензії та юридичні вимоги, актуальні для реалізованих функцій.

## B18. Підсумкове правило

Для кожної суттєвої вимоги простежується повний ланцюг:

**Потреба → рішення → специфікація → реалізація → тест → доказ результату → документація → контрольована зміна.**

Якщо ланцюг обривається, відповідний пункт не можна оголошувати повністю завершеним.

## Головний промпт для Claude: знайти залишкові прогалини й підготувати проєкт до старту

Цей промпт важливіший за ще один окремий опис механіки. Він змушує Claude зіставити всі передані матеріали, усунути дублювання, знайти конфлікти та сформувати єдиний план дій, а не просто написати ще один довгий документ.

# MASTER PROMPT — PROJECT COMPLETENESS, FEASIBILITY & START READINESS AUDIT

# 1. Твоя роль

Ти дієш як міждисциплінарна команда:

- Lead Game Designer;
- Technical Game Architect;
- Gameplay Systems Designer;
- Narrative Designer;
- QA Lead;
- UX and Accessibility Specialist;
- Security and Privacy Reviewer;
- Content Pipeline Engineer;
- Production Manager;
- Release and Operations Reviewer.

Твоє завдання — не просто оцінити концепцію, а визначити, чи достатньо специфікації для контрольованого початку розробки реальної гри.

# 2. Матеріали для аналізу

Прочитай усі доступні матеріали цього проєкту, включно з:

- основним описом гри;
- GDD та narrative bible, якщо вони існують;
- технічною специфікацією;
- попередніми додатками;
- сценарієм першої подорожі;
- описами персонажів, NPC, квестів, подій і предметів;
- архітектурними пропозиціями;
- правилами збереження;
- QA-планом;
- вимогами безпеки й приватності;
- додатками A і B;
- поточними файлами репозиторію, якщо вони доступні;
- фактичним кодом, тестами, конфігурацією та історією змін, якщо вони доступні.

Не припускай, що кожен документ існує. Спочатку склади інвентар доступних матеріалів.

Якщо частина історії розмови, вкладень або репозиторію недоступна, явно зафіксуй це. Не вигадуй прочитані документи.

## 3. Головна мета

Підготувати проєкт до першого контрольованого етапу розробки:

**створення перевірного вертикального зрізу, який доводить, що основний ігровий цикл цікавий, технічно здійсненний, зберігає стан і може розширюватися без архітектурного хаосу.**

Не намагайся реалізувати всі задумки одночасно.

## 4. Етап A — повний аудит матеріалів

Створи реєстр усіх вимог і рішень.

Для кожного пункту вкажи:

- ID;
- опис;
- джерело;
- категорію;
- пріоритет P0–P4;
- статус;
- залежності;
- суперечності;
- критерій приймання;
- відповідний тест;
- рекомендовану дію.

Використовуй статуси:

- APPROVED;
- EXISTING;
- VERIFIED;
- PROPOSED;
- ASSUMED;
- UNKNOWN;
- CONFLICT;
- BLOCKED;
- NOT TESTED.

Знайди:

1. суперечності між документами;
2. дублювання;

3. нечіткі формулювання;
4. правила без конкретних значень;
5. системи без визначених входів і виходів;
6. функції без критеріїв приймання;
7. вимоги без тестів;
8. рішення, що передчасно ускладнюють архітектуру;
9. непідтверджені припущення;
10. вимоги, які не потрібні для першого прототипу.

Не створюй новий документ для кожної дрібниці. Визнач єдине канонічне місце для кожного виду інформації.

## 5. Етап B — перевірка цілісності дизайну

Перевір взаємозалежності:

- час ↔ розклади NPC ↔ квести;
- погода ↔ маршрут ↔ транспорт ↔ безпека;
- бюджет ↔ покупки ↔ ремонт ↔ проживання;
- інвентар ↔ спорядження ↔ квести;
- персонаж ↔ домогосподарство ↔ тварини;
- локації ↔ переходи ↔ доступність послуг;
- NPC ↔ пам'ять ↔ відносини ↔ діалоги;
- події ↔ умови ↔ наслідки ↔ повторюваність;
- збереження ↔ версії даних ↔ міграції;
- прогрес ↔ винагороди ↔ онлайн-сервіс.

Для кожної критичної взаємодії визнач:

- власника стану;
- дозволені команди;
- передумови;
- порядок змін;
- побічні ефекти;
- правила відновлення;
- тести;
- можливі тупики.

Окремо перевір, чи може гравець продовжити гру після відмови від квесту, пропуску дня, нестачі грошей, поломки, недоступності NPC, повного інвентарю, пропущеного транспорту або перезапуску.

## 6. Етап C — оцінка технічної здійсненності

Якщо доступний репозиторій:

- перевір фактичну структуру;
- визнач мову, рушій і залежності;
- перевір команди встановлення, запуску, тестування й збірки;
- запусти безпечні перевірки, які підтримує середовище;
- зафіксуй точні результати;
- відокрем помилки середовища від помилок проєкту;
- не виконуй руйнівних операцій;
- не видаляй і не переписуй файли без необхідності та погодження.

Якщо репозиторій відсутній, виконай архітектурний аналіз на рівні специфікації, але не стверджуй, що код перевірений.

Оціни придатність TypeScript і Phaser як початкового варіанта, враховуючи:

- 2D-графіку з перспективою 3/4;
- локальні сцени та переходи;
- навігацію між вузлами маршруту;
- діалоги й інтерфейс;
- збереження;
- браузерний запуск;
- потенційний майбутній PC-порт;
- доступність;
- тестованість;
- ліцензії;
- складність підтримки.

Порівнюй альтернативи лише там, де вони можуть змінити рішення. Не проводь нескінченне порівняння рушіїв.

# 7. Етап D — визначення першого продукту

Сформуй три чітко відокремлені обсяги:

### D1. Технічний прототип

Мінімальна перевірка руху, сцени, переходів, взаємодій і збереження.

### D2. Вертикальний зріз

Завершений короткий ігровий досвід із плануванням, подорожжю, кемпінгом, NPC, невеликим завданням, наслідками та відновленням збереження.

### D3. MVP

Найменший продукт, який можна надати цільовим гравцям для змістовного тестування. Його обсяг потрібно визначити після аудиту, а не прирівнювати автоматично до всіх описаних систем.

Для кожного рівня вкажи:

- включені функції;
- виключені функції;
- критерії приймання;
- залежності;
- основні ризики;
- тестові сценарії;
- умови переходу до наступного етапу.

## 8. Етап E — перевірка першої подорожі

Перевір усі 15 запланованих етапів першої історії, якщо вони доступні.

Для кожного етапу перевір:

- передумови;
- доступні дії;
- повний текст і варіанти діалогу;
- перевірку умов;
- зміни часу, грошей, предметів, відносин і стану світу;
- наступні етапи;
- альтернативні завершення;
- збереження та відновлення;
- можливість відмовитися;
- поведінку з різними типами транспорту, супутниками й тваринами.

Побудуй граф переходів.

Знайди недосяжні етапи, цикли без виходу, тупики, недоступні предмети, неіснуючих NPC, суперечливі передумови та гілки, що втрачають прогрес після перезапуску.

Не вигадуй відсутні сцени як завершені. Запропонуй відсутній контент окремо зі статусом PROPOSED.

## 9. Етап F — ризики та якість

Сформуй пріоритетний реєстр ризиків.

Для кожного ризику вкажи:

- ймовірність;
- вплив;
- ранній індикатор;
- профілактику;
- спосіб виявлення;
- план реагування;
- критерій закриття.

Обов'язково розглянь:

- надмірний обсяг;
- втрату або пошкодження збережень;
- логічні тупики;
- складну для підтримки архітектуру;
- неперевірений контент;
- нецікаву основну петлю;
- слабку читабельність інтерфейсу;
- проблеми продуктивності;
- порушення ліцензій;
- приватність і безпеку;
- онлайн-шахрайство, якщо є мережеві винагороди;
- залежність від зовнішніх сервісів;
- недоступність підтримки після релізу.

## 10. Етап G — сформуй мінімально необхідну документацію

Визнач, які документи потрібно створити або оновити, а які вже достатні.

Переважний мінімальний набір:

- `PROJECT_CHARTER.md`;
- `GDD.md`;
- `MVP_SCOPE.md`;
- `DECISION_LOG.md`;
- `PROJECT_STATUS.md`;
- `ARCHITECTURE.md`;
- `DATA_CONTRACTS.md`;

- `SIMULATION_RULES.md`;
- `SAVE_MIGRATIONS.md`;
- `NARRATIVE_BIBLE.md`;
- `CONTENT_PIPELINE.md`;
- `QA_STRATEGY.md`;
- `TEST_PLAN.md`;
- `SECURITY_PRIVACY.md`;
- `LICENSE_REGISTER.md`;
- `RELEASE_CHECKLIST.md`.

Це не вимога створювати всі файли незалежно від потреби. Об'єднуй близькі документи, якщо так легше підтримувати єдине джерело правди.

Для кожного документа вкажи:

- чи він уже існує;
- що в ньому не вистачає;
- чи потрібно створити або оновити;
- які джерела він використовує;
- які інші документи залежать від нього.

Не створюй порожніх шаблонів лише для того, щоб дерево файлів виглядало повним.

## 11. Етап H — план реалізації

Сформуй короткі, послідовні етапи з конкретними результатами.

Кожен етап повинен містити:

- мету;
- передумови;
- файли або модулі, яких він стосується;
- очікувану поведінку;
- автоматичні тести;
- ручну перевірку;
- критерії приймання;
- ризики;
- умови завершення;
- процедуру відкату, якщо вона потрібна.

Початковий порядок для перевірки:

1. Аудит.
2. Уточнення критичних рішень.
3. Контракти даних.
4. Мінімальне ядро правил.
5. Збереження та відновлення.
6. Одна ігрова сцена.
7. Переміщення й взаємодії.
8. Один завершений цикл подорожі.
9. NPC і короткий квест.
10. Перевірка вертикального зрізу з гравцями.
11. Виправлення проблем.
12. Рішення про розширення MVP.

Зміни порядок, якщо фактичний репозиторій або результати тестів цього вимагають.

## 12. Етап I — незалежна перевірка

Після підготовки результатів повторно перевір:

- чи залишилися суперечності;
- чи всі P0 мають рішення;
- чи всі P1 мають критерії приймання;
- чи є невирішені залежності;
- чи можна відтворити тести;
- чи відповідає план фактичним можливостям проєкту;
- чи не з'явилися зайві системи;
- чи не видано припущення за факти.

Проведи також аналіз протилежної гіпотези: чому цей задум може не спрацювати, що може бути надто дорогим, нудним або складним для підтримки.

## 13. Формат результату

Надай результат у такому порядку:

1. Короткий вердикт: READY, READY WITH CONDITIONS або NOT READY.
2. Що фактично перевірено.
3. Які матеріали були недоступні.
4. Найважливіші блокери.
5. Таблиця суперечностей і прогалин.
6. Реєстр рішень та невідомих.
7. Рекомендована архітектура з аргументами.

8. Точний обсяг вертикального зрізу.
9. План тестування.
10. План виправлення блокерів.
11. Перелік документів для створення або оновлення.
12. Послідовний план реалізації.
13. Критерії переходу до наступного етапу.
14. Перелік того, що ще не перевірено.

Для кожного висновку вкажи джерело або поясни, що це рекомендація.

Не вигадуй оцінки строків без обґрунтованих припущень про команду, досвід, інструменти й обсяг роботи.

## 14. Межі повноважень

На етапі аудиту:

- не переписуй весь проєкт;
- не видаляй наявні системи;
- не додавай залежності без аргументів;
- не запускай руйнівні команди;
- не роби непогоджених публічних або платних дій;
- не оголошуй функцію завершеною без перевірки;
- не приховуй невизначеність;
- не вигадуй результати тестів.

Якщо маєш доступ до файлів і можеш безпечно створити звіт, збережи його у відповідному місці репозиторію. Якщо доступу немає, надай звіт у відповіді.

Завершуй аудит чіткою відповіддю:

**Що конкретно потрібно зробити наступним, щоб ми отримали перший перевірний ігровий результат із найменшим ризиком марно витратити час?**

## Що тепер вважати завершеним

Не потрібно нескінченно розширювати документацію. Після цього додатка та аудиту Claude має бути здатен визначити, чого справді бракує, замість того щоб ми постійно здогадувалися.

Є три різні рівні готовності:

- Готовність концепції: зрозуміло, яку гру ми створюємо і для кого.

- Готовність до розробки: правила, обсяг, архітектурні рішення й критерії перевірки достатньо визначені.
- Готовність продукту до гравців: гра реалізована, перевірена на цільових пристроях, має прийнятну якість і готова до підтримки.

Ми можемо значно наблизити перші два рівні документацією та аудитом. Третій неможливо чесно підтвердити, поки немає реалізації й доказів її перевірки.

Моя рекомендація: передай Claude основний пакет, попередні додатки A і B та новий Master Prompt. Попроси спочатку провести аудит, а не одразу генерувати код. Після звіту потрібно закрити реальні блокери й лише тоді створювати перший вертикальний зріз.

Саме це буде справжнім «дотисканням» підготовки: не максимальна кількість тексту, а мінімум невідомих, контрольований обсяг і перевірний шлях від ідеї до робочої гри.